



Oracle AI Database: SQL Workshop

Student Guide – Volume II
S1106065GC10

Copyright © 2025, Oracle and/or its affiliates.

Disclaimer

This document contains proprietary information and is protected by copyright and other intellectual property laws. The document may not be modified or altered in any way. Except where your use constitutes "fair use" under copyright law, you may not use, share, download, upload, copy, print, display, perform, reproduce, publish, license, post, transmit, or distribute this document in whole or in part without the express authorization of Oracle.

The information contained in this document is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

Restricted Rights Notice

If this documentation is delivered to the United States Government or anyone using the documentation on behalf of the United States Government, the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

Trademark Notice

Oracle®, Java, MySQL, and NetSuite are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

Third-Party Content, Products, and Services Disclaimer

This documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

1011042025

For Instructor Use Only.
This document should not be distributed.

Contents

----- Important Note -----

Oracle AI Database 26ai has replaced **Oracle Database 23ai**. This change was announced at Oracle AI World in October 2025. The architecture, concepts, and features presented in this course remain fully relevant to 26ai.

O Course Overview

Course Outline O-2

Target Audience O-4

Learning Outcomes O-5

What's Next? I-7

1 Introduction

Objectives 1-2

Overview of Oracle Database 23c and Related Products 1-3

Oracle Database 23c: Focus Areas 1-4

Oracle Database 23c 1-5

Overview of Relational Database Management Concepts and Terminologies 1-7

Relational and Object Relational Database Management Systems 1-8

Data Storage on Different Media 1-9

Relational Database Concept 1-10

Definition of a Relational Database 1-11

Data Models 1-12

Entity Relationship Model 1-13

Entity Relationship Modeling Conventions 1-14

Relating Multiple Tables 1-16

Relational Database Terminology 1-17

Human Resources (HR) Schema and the Tables Used in This Course 1-18

Human Resources (HR) Application 1-19

Tables Used in This Course 1-20

Tables Used in the Course 1-21

Introduction to SQL and Its Development Environments 1-22

Using SQL to Query Your Database 1-23

How SQL Works 1-24

SQL Statements Used in the Course 1-25

Development Environments for SQL in Oracle 1-26

Introduction to Oracle Live SQL 1-27

For Instructor Use Only.
This document should not be distributed.

Oracle Database 23c SQL Documentation and Additional Resources	1-28
Oracle Database Documentation	1-29
Additional Resources for Oracle	1-30
Oracle University: Oracle SQL Training	1-31
Oracle SQL Certification	1-32
Summary	1-33

2 Retrieving Data Using the SQL SELECT Statement

Course Roadmap	2-2
Objectives	2-3
Capabilities of SQL SELECT Statements	2-4
HR Application Scenario	2-5
Writing SQL Statements	2-6
Basic SELECT Statement	2-7
Selecting All Columns	2-8
Executing SQL Statements with Oracle SQL Developer and SQL*Plus	2-9
Column Heading Defaults in SQL Developer and SQL*Plus	2-10
Selecting Specific Columns	2-11
Selecting from dual with Oracle Database	2-12
New Feature in 23c: SELECT Without FROM Clause	2-13
Arithmetic Expressions and NULL Values in the SELECT statement	2-14
Arithmetic Expressions	2-15
Using Arithmetic Operators	2-16
Operator Precedence	2-17
Defining a Null Value	2-18
Null Values in Arithmetic Expressions	2-19
Column Aliases	2-20
Defining a Column Alias	2-21
Using Column Aliases	2-22
Use of the Concatenation Operator, Literal Character Strings, Alternative Quote Operator, and the DISTINCT Keyword	2-23
Concatenation Operator in Oracle	2-24
Literal Character Strings	2-25
Using Literal Character Strings in Oracle	2-26
Alternative Quote (q) Operator in Oracle	2-27
Duplicate Rows	2-28
DESCRIBE Command	2-29
Displaying Table Structure by Using the DESCRIBE Command	2-30
Displaying Table Structure by Using Oracle SQL Developer	2-31
Summary	2-32

3 Restricting and Sorting Data

Course Roadmap 3-2

Objectives 3-3

Limiting Rows 3-4

Limiting Rows by Using a Selection 3-5

Limiting Rows That Are Selected 3-6

Using the WHERE Clause 3-7

Character Strings and Dates 3-8

Comparison Operators 3-9

Using Comparison Operators 3-10

Range Conditions Using the BETWEEN Operator 3-11

Using the IN Operator 3-12

Pattern Matching Using the LIKE Operator 3-13

Combining Wildcard Symbols 3-14

Using NULL Conditions 3-15

Defining Conditions Using Logical Operators 3-16

Using the AND Operator 3-17

Using the OR Operator 3-18

Using the NOT Operator 3-19

Rules of Precedence for Operators in an Expression 3-20

Rules of Precedence 3-21

Sorting Rows Using the ORDER BY Clause 3-23

Using the ORDER BY Clause 3-24

Sorting 3-25

SQL Row Limiting Clause in a Query 3-27

SQL Row Limiting Clause 3-28

Using SQL Row Limiting Clause in a Query in Oracle 3-29

SQL Row Limiting Clause: Example in Oracle 3-30

Substitution Variables in Oracle 3-31

Using the Single-Ampersand Substitution Variable 3-34

Character and Date Values with Substitution Variables 3-36

Specifying Column Names, Expressions, and Text 3-37

Using the Double-Ampersand Substitution Variable 3-38

Using the Ampersand Substitution Variable in SQL*Plus 3-39

Assigning Values to Variables 3-40

Using the DEFINE Command in Oracle 3-41

Using the VERIFY Command in Oracle 3-42

Summary 3-43

4 Using Single-Row Functions to Customize Output

Course Roadmap 4-2

Objectives 4-3

HR Application Scenario 4-4

Single-Row SQL Functions 4-5

SQL Functions 4-6

Two Types of SQL Functions 4-7

Single-Row Functions 4-8

Character Functions 4-10

Case-Conversion Functions 4-13

Using Case-Conversion Functions in WHERE Clauses in Oracle 4-14

Character-Manipulation Functions 4-15

Using Character-Manipulation Functions 4-16

Nesting Functions 4-17

Nesting Functions: Example 4-19

Number Functions 4-20

Numeric Functions 4-21

Using the ROUND Function 4-22

Using the TRUNC Function in Oracle 4-23

Using the MOD Function 4-24

Working with Dates in Oracle Databases 4-25

RR Date Format in Oracle 4-27

Using the SYSDATE Function in Oracle 4-29

Using the CURRENT_DATE and CURRENT_TIMESTAMP Functions
in Oracle 4-30

Arithmetic with Dates in Oracle 4-31

Using Arithmetic Operators with Dates in Oracle 4-32

Date Functions 4-33

Date-Manipulation Functions in Oracle 4-34

Using Date Functions in Oracle 4-35

Using ROUND and TRUNC Functions with Dates in Oracle 4-36

Summary 4-37

5 Using Conversion Functions and Conditional Expressions

Course Roadmap 5-2

Objectives 5-3

Implicit and Explicit Data Type Conversion 5-4

Conversion Functions 5-5

Implicit Data Type Conversion of Strings to Numbers 5-6

Implicit Data Type Conversion of Numbers to Strings 5-7

TO_CHAR, TO_DATE, and TO_NUMBER Functions in Oracle	5-8
Using the TO_CHAR Function with Dates	5-9
Elements of the Date Format Model	5-10
Using the TO_CHAR Function with Dates	5-14
Using the TO_CHAR Function with Numbers	5-15
Using the TO_NUMBER and TO_DATE Functions	5-18
Using TO_CHAR and TO_DATE Functions with the RR Date Format	5-20
General Functions	5-21
NVL Function	5-23
Using the NVL Function in Oracle	5-24
Using the NVL2 Function in Oracle	5-25
Using the NULLIF Function	5-26
Using the COALESCE Function	5-27
Conditional Expressions	5-29
CASE Expression	5-31
Using the CASE Expression	5-32
Searched CASE Expression	5-33
DECODE Function in Oracle	5-34
Using the DECODE Function	5-35
JSON Functions	5-37
JSON_QUERY Function	5-38
JSON_TABLE Function	5-39
JSON_VALUE Function	5-40
Summary	5-41

6 Reporting Aggregated Data Using the Group Functions

Course Roadmap	6-2
Objectives	6-3
Group Functions	6-4
Types of Group Functions	6-6
Group Functions: Syntax	6-7
Using the AVG and SUM Functions	6-8
Using the MIN and MAX Functions	6-9
Using the COUNT Function	6-10
Using the DISTINCT Keyword	6-11
Group Functions and Null Values in Oracle	6-12
Grouping Rows	6-13
Creating Groups of Data	6-14
Creating Groups of Data: GROUP BY Clause Syntax	6-15
Using the GROUP BY Clause	6-16
Using the GROUP BY Column Position	6-17

- Using the GROUP BY Clause 6-18
- Grouping by More Than One Column 6-19
- Using the GROUP BY Clause on Multiple Columns 6-20
- Illegal Queries Using Group Functions 6-21
- Illegal Queries Using Group Functions in a WHERE Clause 6-22
- Restricting Group Results 6-23
- Restricting Group Results with the HAVING Clause 6-24
- Using the HAVING Clause 6-25
- Nesting Group Functions 6-27
 - Nesting Group Functions in Oracle 6-28
- Summary 6-29

7 Displaying Data from Multiple Tables Using Joins

- Course Roadmap 7-2
- Objectives 7-3
- Types of Joins and Their Syntax 7-4
 - Why Join? 7-5
 - Obtaining Data from Multiple Tables 7-6
 - Types of Joins 7-7
 - Joining Tables Using SQL Syntax 7-8
- Natural Join 7-9
 - Creating Natural Joins 7-10
 - Retrieving Records with Natural Joins 7-11
- Join with the USING Clause 7-12
 - Creating Joins with the USING Clause 7-13
 - Joining Column Names 7-14
 - Retrieving Records with the USING Clause 7-15
 - Qualifying Ambiguous Column Names 7-16
 - Using Table Aliases with the USING Clause in Oracle 7-17
- Join with the ON Clause 7-18
 - Creating Joins with the ON Clause 7-19
 - Retrieving Records with the ON Clause 7-20
 - Creating Three-Way Joins 7-21
 - Applying Additional Conditions to a Join 7-22
- Self-Join 7-23
 - Joining a Table to Itself 7-24
 - Self-Joins Using the ON Clause 7-25

- Nonequijoins 7-26
 - Retrieving Records with Nonequijoins 7-28
- Outer Join 7-29
 - Returning Records with No Direct Match Using OUTER Joins 7-30
 - INNER Versus OUTER Joins 7-31
 - LEFT OUTER JOIN 7-32
 - RIGHT OUTER JOIN 7-33
 - FULL OUTER JOIN in Oracle 7-34
- Cartesian Product 7-35
 - Cartesian Products 7-36
 - Generating a Cartesian Product 7-37
 - Creating Cross Joins 7-38
- Summary 7-39

8 Using Subqueries to Solve Queries

- Course Roadmap 8-2
- Objectives 8-3
- Subquery: Types, Syntax, and Guidelines 8-4
 - Using a Subquery to Solve a Problem 8-5
 - Subquery Syntax 8-6
 - Using a Subquery 8-7
 - Rules and Guidelines for Using Subqueries 8-8
 - Types of Subqueries 8-9
- Single-Row Subqueries 8-10
 - Executing Single-Row Subqueries 8-12
 - Using Group Functions in a Subquery 8-13
 - HAVING Clause with Subqueries 8-14
 - What is wrong with this statement? 8-15
 - No Rows Returned by the Inner Query 8-16
- Multiple-Row Subqueries 8-17
 - Using the ANY Operator in Multiple-Row Subqueries 8-19
 - Using the ALL Operator in Multiple-Row Subqueries 8-20
- Multiple-Column Subqueries 8-21
 - Multiple-Column Subquery: Example 8-23
- Null Values in a Subquery 8-24
- Summary 8-26

9 Using Set Operators

- Course Roadmap 9-2
- Objectives 9-3
- Set Operators: Types and Guidelines 9-4

Set Operators	9-5
Set Operator Rules	9-6
Oracle Server and Set Operators	9-7
Tables Used in This Lesson	9-8
UNION and UNION ALL Operators	9-13
UNION Operator	9-14
Using the UNION Operator	9-15
UNION ALL Operator	9-16
Using the UNION ALL Operator	9-17
INTERSECT Operator	9-18
Using the INTERSECT Operator	9-20
MINUS Operator	9-21
Using the MINUS Operator	9-23
Matching SELECT Statements	9-24
Matching SELECT Statements in Oracle	9-25
Matching the SELECT Statement: Example in Oracle	9-26
Using the ORDER BY Clause in Set Operations	9-27
Using the ORDER BY Clause in Set Operations in Oracle	9-28
Using the ORDER BY Clause in Set Operations in Oracle: Example	9-29
Summary	9-30

10 Managing Tables Using DML Statements

Course Roadmap	10-2
Objectives	10-3
HR Application Scenario	10-4
Adding New Rows in a Table	10-5
Data Manipulation Language	10-6
Adding a New Row to a Table	10-7
INSERT Statement Syntax	10-8
Inserting New Rows	10-9
INSERT Statement Syntax	10-10
Inserting Rows with Null Values	10-11
Inserting Special Values	10-12
Inserting Specific Date and Time Values	10-13
Creating a Script	10-14
Copying Rows from Another Table	10-15
Changing Data in a Table	10-16
UPDATE Statement Syntax	10-18
Updating Rows in a Table	10-19
Updating Two Columns with a Subquery	10-20
Updating Rows Based on Another Table	10-21

Enhancement in 23c	10-22
RETURNING Clause in 23c: Example	10-23
Removing Rows from a Table	10-24
Removing a Row from a Table	10-25
DELETE Statement	10-26
Deleting Rows from a Table	10-27
Deleting Rows Based on Another Table	10-28
Enhancements to UPDATE and DELETE Statements in 23c	10-29
UPDATE: Example	10-30
DELETE: Example	10-31
TRUNCATE Statement	10-32
Database Transaction Control	10-33
Database Transactions	10-34
Database Transactions: Start and End	10-35
Advantages of the COMMIT and ROLLBACK Statements	10-36
Explicit Transaction Control Statements	10-37
Rolling Back Changes to a Marker	10-38
Implicit Transaction Processing	10-39
State of Data Before COMMIT or ROLLBACK	10-40
State of Data After COMMIT	10-41
Committing Data	10-42
State of Data After ROLLBACK	10-43
State of Data After ROLLBACK: Example	10-44
Statement-Level Rollback	10-45
Read Consistency	10-46
Implementing Read Consistency	10-48
Manual Data Locking	10-49
FOR UPDATE Clause in a SELECT Statement	10-50
FOR UPDATE Clause: Examples	10-51
LOCK TABLE Statement	10-52
Summary	10-53

11 Introduction to Data Definition Language

Course Roadmap	11-2
Objectives	11-3
HR Application Scenario	11-4
Database Objects	11-5
Naming Rules for Tables and Columns	11-7
CREATE TABLE Statement	11-8
Creating Tables	11-10
Staging Tables	11-11

Creating Staging Tables: Methods	11-12
Using Staging Tables: Example	11-13
Data Types	11-14
Datetime Data Types	11-17
SQL*Plus Support for Boolean Data Type in 23c	11-18
Creating a Table with Boolean Type and Inserting Data: Example	11-19
Querying the Table Containing BOOLEAN Values in SQL Developer:	
Example	11-20
Default Output in SQL*Plus: Example	11-21
Using the COLUMN Command to Customize Output: Example	11-22
DEFAULT Option	11-23
23c Enhancement to SQL Statements	11-24
UPDATE Option to the Existing DEFAULT ON NULL Clause: Example	11-25
Updating demo_t and Setting ename to the 'missing name' Value:	
Example	11-26
DEFAULT ON NULL with IDENTITY Columns and Update Column c2	
to <sequence>.nextval: Example	11-27
Constraints: Overview	11-28
Including Constraints	11-29
Constraint Guidelines	11-30
Defining Constraints	11-31
Defining Constraints: Example	11-32
NOT NULL Constraint	11-33
UNIQUE Constraint	11-34
PRIMARY KEY Constraint	11-36
FOREIGN KEY Constraint	11-37
FOREIGN KEY Constraint: Keywords	11-39
CHECK Constraint	11-40
CREATE TABLE: Example	11-41
IF (NOT) EXISTS Enhancement	11-42
IF NOT EXISTS Clause: Example	11-43
Violating Constraints	11-44
Creating a Table by Using a Subquery	11-46
ALTER TABLE Statement	11-49
Adding a Column	11-52
Modifying a Column	11-53
Dropping a Column	11-54
SET UNUSED Option	11-55
Read-Only Tables	11-57

DROP TABLE Statement 11-58
 Dropping a Table 11-59
 Dropping the DEMO Table If It Exists: Example 11-60
Summary 11-61

12 Introduction to Data Dictionary Views

Objectives 12-2
Introduction to Data Dictionary 12-3
 Why data dictionary? 12-4
 Data Dictionary 12-5
 Data Dictionary Structure 12-6
 How to Use Dictionary Views 12-8
 USER_OBJECTS and ALL_OBJECTS Views 12-9
 USER_OBJECTS View 12-10
Querying the Dictionary Views 12-11
 Table Information 12-12
 Column Information 12-13
 Constraint Information 12-15
 USER_CONSTRAINTS: Example 12-16
 Querying USER_CONS_COLUMNS 12-17
Adding a Comment to a Table and Querying the Dictionary Views 12-18
 Adding Comments to a Table 12-19
Summary 12-20

13 Creating Sequences, Synonyms, and Indexes

Objectives 13-2
Create, Maintain, and Use Sequences 13-3
 E-Commerce Scenario 13-4
 Database Objects 13-5
 Referencing Another User's Tables 13-6
 Sequences 13-7
 CREATE SEQUENCE Statement: Syntax 13-8
 Creating a Sequence 13-10
 NEXTVAL and CURRVAL Pseudocolumns 13-11
 Using a Sequence 13-13
 SQL Column Defaulting Using a Sequence 13-14
 Caching Sequence Values 13-15
 Modifying a Sequence 13-16
 Guidelines for Modifying a Sequence 13-17
 Sequence Information 13-18

Create Private and Public Synonyms	13-19
Synonyms	13-20
Creating a Synonym for an Object	13-21
Creating and Removing Synonyms	13-22
Synonym Information	13-23
Create and Maintain Indexes	13-24
Indexes	13-25
How are indexes created?	13-26
Creating an Index	13-27
CREATE INDEX with the CREATE TABLE Statement	13-28
Function-Based Indexes	13-30
Creating Multiple Indexes on the Same Set of Columns	13-31
Creating Multiple Indexes on the Same Set of Columns: Example	13-32
Index Information	13-33
USER_INDEXES: Examples	13-34
Querying USER_IND_COLUMNS	13-35
Removing an Index	13-36
Summary	13-37

14 Creating Views

Objectives	14-2
Overview of Views	14-3
Why views?	14-4
Database Objects	14-5
What is a view?	14-6
Advantages of Views	14-7
Simple Views and Complex Views	14-8
Creating, Modifying, and Retrieving Data from a View	14-9
Creating a View	14-10
Retrieving Data from a View	14-13
Modifying a View	14-14
Creating a Complex View	14-15
View Information	14-16
Data Manipulation Language (DML) Operations on a View	14-17
Rules for Performing DML Operations on a View	14-18
Rules for Performing Modify Operations on a View	14-19
Rules for Performing Insert Operations Through a View	14-20
Using the WITH CHECK OPTION Clause	14-21
Denying DML Operations	14-22

Dropping a View 14-24
Removing a View 14-25
Summary 14-26

15 Managing Schema Objects

Objectives 15-2
Manage Constraints 15-3
Adding a Constraint: Syntax 15-4
Adding a Constraint 15-5
Dropping a Constraint 15-6
Dropping a Constraint: ONLINE 15-7
ON DELETE Clause 15-8
Cascading Constraints 15-9
Cascading Constraints: Example 15-10
Renaming Table Columns and Constraints 15-11
Disabling Constraints 15-12
Enabling Constraints 15-13
Constraint States 15-14
Deferring Constraints 15-15
Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE 15-16
DROP TABLE ... PURGE 15-18
Create and Use Temporary Tables 15-19
Using Temporary Tables 15-20
Temporary Table 15-21
Temporary Table: Characteristics 15-22
Creating a Global Temporary Table 15-23
Creating a Private Temporary Table 15-24
Creating and Using External Tables 15-25
External Tables 15-26
Creating a Directory for the External Table 15-27
Creating an External Table 15-29
Creating an External Table by Using ORACLE_LOADER 15-31
Querying External Tables 15-32
Creating an External Table by Using ORACLE_DATAPUMP: Example 15-33
Summary 15-34

16 Retrieving Data by Using Subqueries

Objectives 16-2
Retrieving Data by Using a Subquery as a Source 16-3
Writing a Multiple - Column Subquery 16-6
Multiple-Column Subqueries 16-7

Column Comparisons	16-8
Pairwise Comparison Subquery	16-9
Nonpairwise Comparison Subquery	16-10
Using Scalar Subqueries in SQL	16-11
Scalar Subquery Expressions	16-12
Scalar Subqueries: Examples	16-13
Solving Problems with Correlated Subqueries	16-14
Correlated Subqueries	16-15
Using Correlated Subqueries: Example 1	16-17
Using Correlated Subqueries: Example 2	16-18
Using the EXISTS and NOT EXISTS Operators	16-19
Using the EXISTS Operator	16-20
Find All Departments That Do Not Have Employees: Example	16-22
Using the WITH Clause	16-23
WITH Clause	16-24
WITH Clause: Example	16-25
Recursive WITH Clause	16-26
Recursive WITH Clause: Example	16-27
Summary	16-28

17 Manipulating Data by Using Subqueries

Objectives	17-3
Using Subqueries to Manipulate Data	17-3
Inserting Values by Using a Subquery as a Target	17-5
Inserting by Using a Subquery as a Target (1/2)	17-6
Inserting by Using a Subquery as a Target (2/2)	17-7
Using the WITH CHECK OPTION keyword on DML statements	17-8
Using Correlated Subqueries to Update and Delete Rows	17-11
Correlated UPDATE	17-12
Using Correlated UPDATE	17-13
Correlated DELETE	17-15
Using Correlated DELETE	17-16
Summary	17-17

18 Controlling User Access

Objectives	18-2
Differentiating System Privileges from Object Privileges	18-3
Controlling User Access	18-4
Privileges	18-5
System Privileges	18-6
Creating Users	18-7

User System Privileges	18-8
Granting System Privileges	18-9
Creating a Role	18-10
What is a role?	18-11
Creating and Granting Privileges to a Role	18-12
Changing Your Password	18-13
Object Privileges	18-14
Granting Object Privileges	18-17
Passing On Your Privileges	18-18
Confirming Granted Privileges	18-19
Revoking Object Privileges	18-20
Summary	18-23
Practice 18: Overview	18-24

19 Manipulating Data Using Advanced Queries

Course Roadmap	19-2
Objectives	19-3
Specifying Explicit Default Values in INSERT and UPDATE Statements	19-4
Explicit Default Feature: Overview	19-5
Using Explicit Default Values	19-6
Multitable INSERTs	19-7
E-Commerce Scenario	19-8
Multitable INSERT Statements: Overview	19-9
Types of Multitable INSERT Statements	19-11
Multitable INSERT Statements	19-12
Unconditional INSERT ALL	19-14
Conditional INSERT ALL: Example	19-15
Conditional INSERT ALL	19-16
Conditional INSERT FIRST: Example	19-18
Conditional INSERT FIRST	19-19
Pivoting INSERT	19-20
Merging Rows in a Table	19-23
MERGE Statement	19-24
MERGE Statement Syntax	19-25
Merging Rows: Example	19-26
Performing Flashback Operations	19-29
FLASHBACK TABLE Statement	19-30
Using the FLASHBACK TABLE Statement	19-32

Tracking the Changes in Data Over a Period of Time 19-33

Tracking Changes in Data 19-34

Flashback Query: Example 19-35

Flashback Version Query: Example 19-36

VERSIONS BETWEEN Clause 19-37

Summary 19-38

20 Managing Data in Different Time Zones

Objectives 20-2

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP 20-3

E-Commerce Scenario 20-4

Time Zones 20-5

TIME_ZONE Session Parameter 20-6

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP 20-7

Comparing Date and Time in a Session's Time Zone 20-8

DBTIMEZONE and SESSIONTIMEZONE 20-10

TIMESTAMP Data Types 20-11

TIMESTAMP Fields 20-12

Difference Between DATE and TIMESTAMP 20-13

Comparing TIMESTAMP Data Types 20-14

INTERVAL Data Types 20-15

INTERVAL Fields 20-17

INTERVAL YEAR TO MONTH: Example 20-18

INTERVAL DAY TO SECOND Data Type: Example 20-20

Additional Date Functions 20-21

EXTRACT 20-22

TZ_OFFSET 20-23

FROM_TZ 20-25

TO_TIMESTAMP 20-26

TO_YMINTERVAL 20-27

TO_DSINTERVAL 20-28

Daylight Saving Time (DST) 20-29

SYSDATE and SYSTIMESTAMP Data Handling 20-31

Lesson Agenda 20-32

Date and Time in Oracle Databases 20-33

Database Time Versus OS System Time 20-34

Database-Specific Time in Oracle Database 23c 20-35

Database Administrator Configures Database-Specific Time 20-36

Summary 20-37

A Table Descriptions

B Using SQL Developer

Objectives	B-2
What is Oracle SQL Developer?	B-4
Specifications of SQL Developer	B-5
SQL Developer 17.4.1 Interface	B-6
Creating a Database Connection	B-8
Browsing Database Objects	B-11
Displaying the Table Structure	B-12
Browsing Files	B-13
Creating a Schema Object	B-14
Creating a New Table: Example	B-15
Using SQL Worksheet	B-16
Executing SQL Statements	B-19
Saving SQL Scripts	B-20
Executing Saved Script Files: Method 1	B-21
Executing Saved Script Files: Method 2	B-22
Formatting the SQL Code	B-23
Using Snippets	B-24
Using Snippets: Example	B-25
Using the Recycle Bin	B-26
Debugging Procedures and Functions	B-27
Database Reporting	B-28
Creating a User-Defined Report	B-29
External Tools	B-30
Setting Preferences	B-31
Data Modeler in SQL Developer	B-32
Summary	B-33

C Using SQL*Plus

Objectives	C-2
SQL and SQL*Plus Interaction	C-3
SQL Statements Versus SQL*Plus Commands	C-4
SQL*Plus: Overview	C-5
Logging In to SQL*Plus	C-6
Displaying the Table Structure	C-7
SQL*Plus Editing Commands	C-9
Using LIST, n, and APPEND	C-11
Using the CHANGE Command	C-12
SQL*Plus File Commands	C-13

Using the SAVE and START Commands C-14
SERVEROUTPUT Command C-15
Using the SQL*Plus SPOOL Command C-16
Using the AUTOTRACE Command C-17
Summary C-18

D Commonly Used SQL Commands

Objectives D-2
Basic SELECT Statement D-3
SELECT Statement D-4
WHERE Clause D-5
ORDER BY Clause D-6
GROUP BY Clause D-7
Data Definition Language D-8
CREATE TABLE Statement D-9
ALTER TABLE Statement D-10
DROP TABLE Statement D-11
GRANT Statement D-12
Privilege Types D-13
REVOKE Statement D-14
TRUNCATE TABLE Statement D-15
Data Manipulation Language D-16
INSERT Statement D-17
UPDATE Statement Syntax D-18
DELETE Statement D-19
Transaction Control Statements D-20
COMMIT Statement D-21
ROLLBACK Statement D-22
SAVEPOINT Statement D-23
Joins D-24
Types of Joins D-25
Qualifying Ambiguous Column Names D-26
Natural Join D-27
Equijoins D-28
Retrieving Records with Equijoins D-29
Add Additional Search Conditions to the ON clause Using AND or WHERE D-30
Retrieving Records with Nonequijoins D-31
Retrieving Records by Using the USING Clause D-32
Retrieving Records by Using the ON Clause D-33
Left Outer Join D-34
Right Outer Join D-35

Full Outer Join D-36
Self-Join: Example D-37
Cross Join D-38
Summary D-39

E Generating Reports by Grouping Related Data

Objectives E-2
Review of Group Functions E-3
Review of the GROUP BY Clause E-4
Review of the HAVING Clause E-5
GROUP BY with ROLLUP and CUBE Operators E-6
ROLLUP Operator E-7
ROLLUP Operator: Example E-8
CUBE Operator E-9
CUBE Operator: Example E-10
GROUPING Function E-11
GROUPING Function: Example E-12
GROUPING SETS E-13
GROUPING SETS: Example E-15
Composite Columns E-17
Composite Columns: Example E-19
Concatenated Groupings E-21
Concatenated Groupings: Example E-22
Summary E-23

F Hierarchical Retrieval

Objectives F-2
Sample Data from the EMPLOYEES Table F-3
Natural Tree Structure F-4
Hierarchical Queries F-5
Walking the Tree F-6
Walking the Tree: From the Bottom Up F-8
Walking the Tree: From the Top Down F-9
Ranking Rows with the LEVEL Pseudocolumn F-10
Formatting Hierarchical Reports Using LEVEL and LPAD F-11
Pruning Branches F-13
Summary F-14

G Writing Advanced Scripts

Objectives G-2
Using SQL to Generate SQL G-3

Creating a Basic Script G-4
Controlling the Environment G-5
The Complete Picture G-6
Dumping the Contents of a Table to a File G-7
Generating a Dynamic Predicate G-9
Summary G-11

H Oracle Database Architectural Components

Objectives H-2
Oracle Database Architecture: Overview H-3
Oracle Database Server Structures H-4
Connecting to the Database H-5
Interacting with an Oracle Database H-6
Oracle Memory Architecture H-8
Process Architecture H-10
Database Writer Process H-12
Log Writer Process H-13
Checkpoint Process H-14
System Monitor Process H-15
Process Monitor Process H-16
Oracle Database Storage Architecture H-17
Logical and Physical Database Structures H-19
Processing a SQL Statement H-21
Processing a Query H-22
Shared Pool H-23
Database Buffer Cache H-25
Program Global Area H-26
Processing a DML Statement H-27
Redo Log Buffer H-29
Rollback Segment H-30
COMMIT Processing H-31
Summary of the Oracle Database Architecture H-33
Summary H-34

I Regular Expression Support

Objectives I-2
What Are Regular Expressions? I-3
Benefits of Using Regular Expressions I-4
Using Regular Expressions Functions and Conditions in SQL and PL/SQL I-5
What are metacharacters? I-6
Using Metacharacters with Regular Expressions I-7

Regular Expressions Functions and Conditions: Syntax I-9
Performing a Basic Search by Using the REGEXP_LIKE Condition I-10
Replacing Patterns by Using the REGEXP_REPLACE Function I-11
Finding Patterns by Using the REGEXP_INSTR Function I-12
Extracting Substrings by Using the REGEXP_SUBSTR Function I-13
Subexpressions I-14
Using Subexpressions with Regular Expression Support I-15
Why access the nth subexpression? I-16
REGEXP_SUBSTR: Example I-17
Using the REGEXP_COUNT Function I-18
Regular Expressions and Check Constraints: Examples I-19
Quiz I-20
Summary I-21

J MySQL Syntax

Objectives J-2
Icons Used in This Course J-3
MySQL: A Modern Database for the Digital Age J-4
High Scalability with MySQL J-5
MySQL-Supported Operating Systems J-6
MySQL Enterprise Edition J-7
Why MySQL Enterprise Edition? J-8
Oracle Premier Support for MySQL J-9
MySQL and Oracle Integration J-10
Development Environments for SQL in MySQL J-11
MySQL Websites J-12
MySQL Community Resources J-13
Oracle University: MySQL Training J-14
MySQL Certification J-15
Selecting All Columns J-16
Executing SQL Statements in MySQL Workbench J-17
Executing SQL Statements in mysql Command-Line Client J-18
Selecting Specific Columns J-19
Selecting Constant Expressions in MySQL J-20
Using Arithmetic Operators J-21
Operator Precedence J-22
Defining a Null Value J-23
Null Values in Arithmetic Expressions J-24
Using Column Aliases J-25
Concatenation Function in MySQL: CONCAT() J-26
Using Literal Character Strings in MySQL J-27

Including a Single Quotation Mark in a String with an Escape Sequence
in MySQL J-28

Duplicate Rows J-29

Displaying Table Structure by Using the DESCRIBE Command J-30

Displaying Table Structure by Using MySQL Workbench J-31

Limiting Rows by Using a Selection J-32

Using the WHERE Clause J-33

Character Strings and Dates J-34

Using Comparison Operators J-35

Range Conditions Using the BETWEEN Operator J-36

Using the IN Operator J-37

Pattern Matching Using the LIKE Operator J-38

Combining Wildcard Symbols J-39

Using NULL Conditions J-40

Using the AND Operator J-41

Using the OR Operator J-42

Using the NOT Operator J-43

Rules of Precedence J-44

Using the ORDER BY Clause J-45

Using SQL Row Limiting Clause in a Query in MySQL J-46

SQL Row Limiting Clause: Example in MySQL J-47

Using the SET Statement in MySQL J-48

Case-Conversion Functions J-49

Case-Insensitive Queries in MySQL J-50

Using Character-Manipulation Functions J-51

Nesting Functions: Example J-52

Numeric Functions J-53

Using the ROUND Function J-54

Using the TRUNCATE Function in MySQL J-55

Using the MOD Function J-56

Working with Dates in MySQL Databases J-57

Displaying the Current Date in MySQL J-58

Date-Manipulation Functions in MySQL J-59

Using Date Functions in MySQL J-60

Extracting the Month or Year Portion of Dates in MySQL J-61

Implicit Data Type Conversion of Strings to Numbers J-62

Implicit Data Type Conversion of Numbers to Strings J-63

Using the CAST() Function in Oracle J-64

Explicit Data Type Conversion of Strings to Numbers in MySQL J-65

Explicit Data Type Conversion of Numbers to Strings in MySQL J-66

General Functions J-67

NVL Function (Oracle) and IFNULL() Function (MySQL) J-68
Using the IFNULL Function in MySQL J-69
Using the NULLIF Function J-70
Using the COALESCE Function J-71
Using the CASE Expression J-72
Using the AVG and SUM Functions J-73
Using the MIN and MAX Functions J-74
Using the COUNT Function J-75
Using the DISTINCT Keyword J-76
Group Functions and Null Values in MySQL J-77
Using the GROUP BY Clause J-78
Using the GROUP BY Clause on Multiple Columns J-80
Illegal Queries Using Group Functions J-81
Illegal Queries Using Group Functions in a WHERE Clause J-82
Using the HAVING Clause J-83
Retrieving Records with Natural Joins J-85
Retrieving Records with the USING Clause J-86
Retrieving Records with the ON Clause J-87
Creating Three-Way Joins J-88
Self-Joins Using the ON Clause J-89
Retrieving Records with Nonequijoins J-90
LEFT OUTER JOIN J-91
RIGHT OUTER JOIN J-92
Creating Cross Joins J-93
Executing Single-Row Subqueries J-94
Using Group Functions in a Subquery J-95
HAVING Clause with Subqueries J-96
What is wrong with this statement? J-97
No Rows Returned by the Inner Query J-98
Using the ANY Operator in Multiple-Row Subqueries J-99
Using the ALL Operator in Multiple-Row Subqueries J-100
Multiple-Column Subquery: Example J-101
Null Values in a Subquery J-102
Using the UNION Operator J-103
Using the UNION ALL Operator J-104
Matching SELECT Statements in MySQL J-105
Matching the SELECT Statement: Example in MySQL J-106
Using the ORDER BY Clause with UNION in MySQL J-107
Using the ORDER BY Clause with UNION: Example in MySQL J-108
Data Manipulation Language J-109
Adding a New Row to a Table J-110

INSERT Statement Syntax J-111
Inserting New Rows: Listing Column Names J-112
Inserting New Rows: Omitting Column Names J-113
Inserting Rows with Null Values J-114
Inserting Special Values in MySQL J-115
Inserting Specific Date and Time Values in MySQL J-116
Inserting and Reformatting Specific Date and Time Values in MySQL J-117
Copying Rows from Another Table J-118
Changing Data in a Table J-119
UPDATE Statement Syntax J-120
Updating Rows in a Table J-121
Updating Rows Based on Another Table J-122
Removing a Row from a Table J-123
DELETE Statement J-124
Deleting Rows from a Table J-125
Deleting Rows Based on Another Table J-126
TRUNCATE Statement J-127
Multiple-Statement Transactions J-128
Transaction Diagram J-129
AUTOCOMMIT and Transaction Control Statements J-130
Committing Data in a Transaction J-131
Rolling Back Changes J-132
Rolling Back Changes to a Marker J-133
Consistent Reads J-134
FOR UPDATE Clause in a SELECT Statement J-135
FOR UPDATE Clause: Examples J-136
Creating a Database: Syntax J-137
MySQL Naming Conventions J-138
CREATE TABLE Statement J-139
Data Types: Overview J-140
Numeric Data Types J-141
Date and Time Data Types J-142
String Data Types J-143
Indexes, Keys, and Constraints J-144
Table Indexes J-145
Primary Keys J-146
Unique Key Constraints J-147
Foreign Key Constraints J-148
Foreign Key Constraint: Example Tables J-149
FOREIGN KEY Constraint: Example Statement J-150
FOREIGN KEY Constraint: Referential Actions J-151

Secondary Indexes J-152
Column Options J-153
Creating a Table Using a Subquery J-154
Creating a Table Using a Subquery: Example J-155
ALTER TABLE Statement J-156
ALTER TABLE Statement: Add, Modify, or Drop Columns J-157
Adding a Column J-158
Modifying a Column J-159
Dropping a Column J-160
ALTER TABLE Statement: Add an Index or Constraint J-161
ALTER TABLE to Add a Constraint or Index: Example J-162
Creating Indexes by Using the CREATE INDEX Statement J-163
Viewing Index Definitions by Using the SHOW INDEX Statement J-164
Showing How a Table Was Created with the SHOW CREATE
TABLE Statement J-165
Dropping a Table J-166
Summary J-167

For Instructor Use Only.
This document should not be distributed.

Managing Schema Objects



For Instructor Use Only.

This document should not be distributed.

Objectives



Manage constraints

Create and use temporary tables

Create and use external tables

In this lesson, you learn about constraints and altering existing objects. You will also learn about external tables and the provision to name the index at the time of creating a `PRIMARY KEY` constraint.



Manage Constraints

For Instructor Use Only.

This document should not be distributed.

Adding a Constraint: Syntax

Use the `ALTER TABLE` statement to:

- Add or drop a constraint, but not to modify its structure
- Enable or disable constraints
- Add a `NOT NULL` constraint by using the `MODIFY` clause

```
ALTER TABLE <table_name>
ADD [CONSTRAINT <constraint_name>]
type (<column_name>);
```

You can add a constraint for existing tables by using the `ALTER TABLE` statement with the `ADD` clause.

In the syntax:

<i>table_name</i>	Name of the table
<i>constraint_name</i>	Name of the constraint
<i>type</i>	Constraint type
<i>column_name</i>	Name of the column affected by the constraint

The constraint name syntax is optional, although recommended. If you do not name your constraints, the system generates constraint names.

Guidelines

- You can add, drop, enable, or disable a constraint, but you cannot modify its structure.
- You can add a `NOT NULL` constraint to an existing column by using the `MODIFY` clause of the `ALTER TABLE` statement.

Note: You can define a `NOT NULL` column only if the table is empty or if the column has a value for every row.

Adding a Constraint

Add a FOREIGN KEY constraint to the EMP2 table

```
ALTER TABLE emp2  
MODIFY employee_id PRIMARY KEY;
```

Table EMP2 altered.

```
ALTER TABLE emp2  
ADD CONSTRAINT emp_mgr_fk  
FOREIGN KEY (manager_id)  
REFERENCES emp2 (employee_id);
```

Table EMP2 altered.

The first example in the slide modifies the EMP2 table to add a PRIMARY KEY constraint on the EMPLOYEE_ID column. Note that because no constraint name is provided, the constraint is automatically named by the Oracle server.

The second example in the slide creates a FOREIGN KEY constraint on the EMP2 table. The constraint ensures that a manager exists as a valid employee in the EMP2 table.

Note: The EMP2 table is created with the following statement:

```
CREATE TABLE emp2 as  
SELECT * FROM employees;
```

Dropping a Constraint

- The `drop constraint` clause enables you to drop an integrity constraint from a database.
- Remove the manager constraint from the `EMP2` table:

```
ALTER TABLE emp2  
DROP CONSTRAINT emp_mgr_fk;
```

Table EMP2 altered.

- Remove the `PRIMARY KEY` constraint on the `EMP2` table and drop the associated `FOREIGN KEY` constraint on the `EMP2.MANAGER_ID` column:

```
ALTER TABLE emp2  
DROP PRIMARY KEY CASCADE;
```

Table EMP2 altered.

To drop a constraint, you can identify the constraint name from the `USER_CONSTRAINTS` and `USER_CONS_COLUMNS` data dictionary views. Then use the `ALTER TABLE` statement with the `DROP` clause. The `CASCADE` option of the `DROP` clause causes any dependent constraints also to be dropped.

Syntax

```
ALTER TABLE      table  
DROP  PRIMARY KEY | UNIQUE (column) |  
      CONSTRAINT  constraint [CASCADE];
```

In the syntax:

<i>table</i>	Name of the table
<i>column</i>	Name of the column affected by the constraint
<i>constraint</i>	Name of the constraint

When you drop an integrity constraint, that constraint is no longer enforced by the Oracle server and is no longer available in the data dictionary.

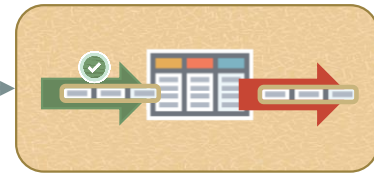
Dropping a Constraint: ONLINE

The `ONLINE` keyword indicates that DML operations on the table are allowed while dropping the constraint.

```
ALTER TABLE myemp2  
DROP CONSTRAINT emp_name_pk ONLINE;
```

Table MYEMP2 altered.

DML operations allowed while
dropping a constraint



You can also drop a constraint by using an `ONLINE` keyword:

```
ALTER TABLE myemp2  
DROP CONSTRAINT emp_id_pk ONLINE;
```

Use the `ALTER TABLE` statement with the `DROP` clause. The `ONLINE` option of the `DROP` clause indicates that DML operations are allowed on the table while dropping the constraint.

Note: The `myemp2` table is created by using the following statement:

```
CREATE TABLE myemp2  
(emp_id NUMBER(6) CONSTRAINT emp_name_pk PRIMARY KEY ,  
emp_name VARCHAR2(20));
```


ON DELETE Clause

- ON DELETE CASCADE clause deletes child rows when a parent key is deleted:

```
ALTER TABLE dept2 ADD CONSTRAINT dept_lc_fk  
FOREIGN KEY (location_id)  
REFERENCES locations(location_id) ON DELETE CASCADE;
```

Table DEPT2 altered.

- ON DELETE SET NULL clause sets the child rows value to null when a parent key is deleted:

```
ALTER TABLE emp2 ADD CONSTRAINT emp_dt_fk  
FOREIGN KEY (department_id)  
REFERENCES departments(department_id) ON DELETE SET NULL;
```

Table EMP2 altered.

ON DELETE

By using the ON DELETE clause, you can determine how Oracle Database handles referential integrity if you remove a referenced primary or unique key value.

ON DELETE CASCADE

The ON DELETE CASCADE clause allows parent key data that is referenced from the child table to be deleted, but not updated. When data in the parent key is deleted, all the rows in the child table that depend on the deleted parent key values are also deleted. To specify this referential action, include the ON DELETE CASCADE clause in the definition of the FOREIGN KEY constraint.

ON DELETE SET NULL

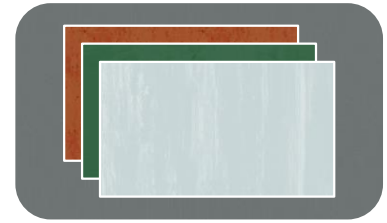
When data in the parent key is deleted, the ON DELETE SET NULL clause causes all the rows in the child table that depend on the deleted parent key value to be converted to null.

If you omit this clause, Oracle does not allow you to delete referenced key values in the parent table that have dependent rows in the child table.

Cascading Constraints

The CASCADE CONSTRAINTS clause:

- Is used along with the DROP COLUMN clause
- Drops all referential integrity constraints that refer to the PRIMARY and UNIQUE keys defined on the dropped columns
- Drops all multicolumn constraints defined on the dropped columns



This statement illustrates the usage of the CASCADE CONSTRAINTS clause. Assume that the TEST1 table is created as follows:

```
CREATE TABLE test1 (  
    col1_pk NUMBER PRIMARY KEY,  
    col2_fk NUMBER,  
    col1 NUMBER,  
    col2 NUMBER,  
    CONSTRAINT fk_constraint FOREIGN KEY (col2_fk) REFERENCES  
        test1,  
    CONSTRAINT ck1 CHECK (col1_pk > 0 and col1 > 0),  
    CONSTRAINT ck2 CHECK (col2_fk > 0));
```

An error is returned for the following statements:

- ALTER TABLE test1 DROP (col1_pk);
ERROR: col1_pk is a parent key.
- ALTER TABLE test1 DROP (col1);
ERROR: col1 is referenced by the multicolumn constraint, ck1.

Cascading Constraints: Example

```
ALTER TABLE emp2  
DROP COLUMN employee_id CASCADE CONSTRAINTS;
```

Table EMP2 altered.

```
ALTER TABLE test1  
DROP (col1_pk, col2_fk, col1) CASCADE CONSTRAINTS;
```

Table TEST1 altered.

Submitting the following statement drops the `EMPLOYEE_ID` column, the `PRIMARY KEY` constraint, and any `FOREIGN KEY` constraints referencing the `PRIMARY KEY` constraint for the `EMP2` table:

```
ALTER TABLE emp2 DROP COLUMN employee_id CASCADE CONSTRAINTS;
```

If all columns referenced by the constraints defined on the dropped columns are also dropped, `CASCADE CONSTRAINTS` is not required. For example, assuming that no other referential constraints from other tables refer to the `COL1_PK` column, it is valid to submit the following statement without the `CASCADE CONSTRAINTS` clause for the `TEST1` table created on the previous page:

```
ALTER TABLE test1 DROP (col1_pk, col2_fk, col1);
```

Guidelines

- Enabling a `PRIMARY KEY` constraint that was disabled with the `CASCADE` option does not enable any `FOREIGN KEYs` that are dependent on `PRIMARY KEY`.
- To enable a `UNIQUE` or `PRIMARY KEY` constraint, you must have the privileges necessary to create an index on the table.

Renaming Table Columns and Constraints

- Use the `RENAME` table clause of the `ALTER TABLE` statement to rename tables.

```
ALTER TABLE marketing RENAME TO new_marketing;
```

1

- Use the `RENAME COLUMN` clause of the `ALTER TABLE` statement to rename table columns.

```
ALTER TABLE new_marketing RENAME COLUMN team_id  
TO id;
```

2

- Use the `RENAME CONSTRAINT` clause of the `ALTER TABLE` statement to rename any existing constraint for a table.

```
ALTER TABLE new_marketing RENAME CONSTRAINT mktg_pk  
TO new_mktg_pk;
```

3

The `RENAME` table clause enables you to rename an existing table in any schema (except the `SYS` schema). To rename a table, you must either be the database owner or the table owner.

When you rename a table column, the new name must not conflict with the name of any existing column in the table. You cannot use any other clauses in conjunction with the `RENAME COLUMN` clause.

The examples in the slide use the `marketing` table with the `PRIMARY KEY mktg_pk` defined on the `id` column.

```
CREATE TABLE marketing (team_id NUMBER(10),  
                        target VARCHAR2(50),  
                        CONSTRAINT mktg_pk PRIMARY KEY(team_id));
```

Example 1 shows that the `marketing` table is renamed `new_marketing`.

Example 2 shows that the `team_id` column of the `new_marketing` table is renamed `id`.

Example 3 shows that the `mktg_pk` constraint is renamed `new_mktg_pk`.

When you rename any existing constraint for a table, the new name must not conflict with any of your existing constraint names. You can use the `RENAME CONSTRAINT` clause to rename system-generated constraint names.

Disabling Constraints

- Execute the `DISABLE` clause of the `ALTER TABLE` statement to deactivate an integrity constraint.

```
ALTER TABLE emp2  
DISABLE CONSTRAINT emp_dt_fk;
```

Table EMP2 altered.

- Apply the `CASCADE` option to disable the primary key.

```
ALTER TABLE dept2  
DISABLE primary key CASCADE;
```

Table DEPT2 altered.

You can disable a constraint, without dropping it or re-creating it, by using the `ALTER TABLE` statement with the `DISABLE` clause. You can also disable the primary key or unique key by using the `CASCADE` option.

Syntax

```
ALTER TABLE table  
DISABLE CONSTRAINT constraint [CASCADE];
```

In the syntax:

<i>table</i>	Name of the table
<i>constraint</i>	Name of the constraint

Guidelines

- You can use the `DISABLE` clause in both the `CREATE TABLE` statement and the `ALTER TABLE` statement.
- The `CASCADE` clause disables dependent integrity constraints.
- Disabling a `UNIQUE` or `PRIMARY KEY` constraint removes the unique index.
- Apply the `CASCADE` option to disable the primary key. It will also disable all dependent `FOREIGN KEY` constraints automatically.

Enabling Constraints

- Activate an integrity constraint that is currently disabled in the table definition by using the `ENABLE` clause.

```
ALTER TABLE emp2  
ENABLE CONSTRAINT emp_dt_fk;
```

Table EMP2 altered.

- A **UNIQUE index** is automatically created if you enable a `UNIQUE` key or a `PRIMARY KEY` constraint.

You can enable a constraint without dropping it or re-creating it by using the `ALTER TABLE` statement with the `ENABLE` clause.

Syntax

```
ALTER    TABLE    table  
ENABLE  CONSTRAINT constraint;
```

In the syntax:

<i>table</i>	Name of the table
<i>constraint</i>	Name of the constraint

Guidelines

- If you enable a constraint, that constraint applies to all the data in the table. All the data in the table must comply with the constraint.
- If you enable a `UNIQUE` key or a `PRIMARY KEY` constraint, a `UNIQUE` or `PRIMARY KEY` index is created automatically. If an index already exists, it can be used by these keys.
- You can use the `ENABLE` clause in both the `CREATE TABLE` statement and the `ALTER TABLE` statement.

Constraint States

An integrity constraint defined on a table can be in one of the following states:

- `ENABLE VALIDATE`
- `ENABLE NOVALIDATE`
- `DISABLE VALIDATE`
- `DISABLE NOVALIDATE`

```
ALTER TABLE dept2  
ENABLE NOVALIDATE PRIMARY KEY;
```

```
Table DEPT2 altered.
```

You can enable or disable integrity constraints at the table level by using the `CREATE TABLE` or `ALTER TABLE` statement. You can also set constraints to `VALIDATE` or `NOVALIDATE`, in any combination with `ENABLE` or `DISABLE`, where:

- `ENABLE` ensures that all incoming data conforms to the constraint
- `DISABLE` allows incoming data, regardless of whether it conforms to the constraint
- `VALIDATE` ensures that existing data conforms to the constraint
- `NOVALIDATE` means that some existing data may not conform to the constraint

`ENABLE VALIDATE` is the same as `ENABLE`. The constraint is checked and is guaranteed to hold for all rows.

`ENABLE NOVALIDATE` means that the constraint is checked, but it does not have to be true for all rows. This allows existing rows to violate the constraint, while ensuring that all new or modified rows are valid. In an `ALTER TABLE` statement, `ENABLE NOVALIDATE` resumes constraint checking on disabled constraints without first validating all data in the table.

`DISABLE NOVALIDATE` is the same as `DISABLE`. The constraint is not checked and is not necessarily true.

`DISABLE VALIDATE` disables the constraint, drops the index on the constraint, and disallows any modification of the constrained columns.

Deferring Constraints

Constraints can have the following attributes:

- DEFERRABLE or NOT DEFERRABLE
- INITIALLY DEFERRED or INITIALLY IMMEDIATE

```
ALTER TABLE dept2
ADD CONSTRAINT dept2_id_pk
PRIMARY KEY (department_id)
DEFERRABLE INITIALLY DEFERRED;
```

Deferring constraint on creation

```
SET CONSTRAINTS dept2_id_pk IMMEDIATE;
```

Changing a specific constraint attribute

```
ALTER SESSION
SET CONSTRAINTS= IMMEDIATE;
```

Changing all constraints for a session

You can defer checking constraints for validity until the end of the transaction.

- A constraint is **deferred** if the system does not check whether the constraint is satisfied until a COMMIT statement is submitted. If a deferred constraint is violated, the database returns an error, and the transaction is not committed and it is rolled back.
- If a constraint is **immediate** (not deferred), it is checked at the end of each statement. If it is violated, the statement is rolled back immediately.
- If a constraint causes an action (for example, DELETE CASCADE), that action is always taken as part of the statement that caused it, whether the constraint is deferred or immediate.
- Use the SET CONSTRAINTS statement to specify, for a particular transaction, whether a deferrable constraint is checked following each data manipulation language (DML) statement or when the transaction is committed. To create deferrable constraints, you must create a nonunique index for that constraint.
- You can define constraints as either DEFERRABLE or NOT DEFERRABLE (default) and either INITIALLY DEFERRED or INITIALLY IMMEDIATE (default). These attributes can be different for each constraint.

Usage scenario: The company policy dictates that department number 40 should be changed to 45. Changing the DEPARTMENT_ID column affects employees assigned to this department. Therefore, you make the PRIMARY KEY and FOREIGN KEYS deferrable and initially deferred. You update both department and employee information, and at the time of commit, all the rows are validated.

Difference Between INITIALLY DEFERRED and INITIALLY IMMEDIATE

INITIALLY DEFERRED	Waits until the transaction ends to check the constraint
INITIALLY IMMEDIATE	Checks the constraint at the end of the statement execution

```
CREATE TABLE emp_new_sal (salary NUMBER
    CONSTRAINT sal_ck CHECK (salary > 100)
    DEFERRABLE INITIALLY IMMEDIATE,
    bonus NUMBER
    CONSTRAINT bonus_ck CHECK (bonus > 0 )
    DEFERRABLE INITIALLY DEFERRED );
```

Table EMP_NEW_SAL created.

A constraint that is defined as deferrable can be specified as either `INITIALLY DEFERRED` or `INITIALLY IMMEDIATE`. The `INITIALLY IMMEDIATE` clause is the default.

In the example in the slide:

- `sal_ck` constraint is created as `DEFERRABLE INITIALLY IMMEDIATE`
- `bonus_ck` constraint is created as `DEFERRABLE INITIALLY DEFERRED`

After creating the `emp_new_sal` table, as shown in the slide, you attempt to insert values into the table and observe the results. When both the `sal_ck` and `bonus_ck` constraints are satisfied, the rows are inserted without an error.

Example 1: Insert a row that violates `sal_ck`. In the `CREATE TABLE` statement, `sal_ck` is specified as an initially immediate constraint. This means that the constraint is verified immediately after the `INSERT` statement, and you observe an error.

```
INSERT INTO emp_new_sal VALUES (90, 5);
```

```
SQL Error: ORA-02290: check constraint (TEACH_B.SAL_CK) violated
02290. 00000 - "check constraint (%s.%s) violated"
```

1 row inserted.

Example 2: Insert a row that violates `bonus_ck`. In the `CREATE TABLE` statement, `bonus_ck` is specified as deferrable and also initially deferred. Therefore, the constraint is not verified until you `COMMIT` or set the constraint state back to immediate.

```
INSERT INTO emp_new_sal VALUES (110, -1);
```

The row insertion is successful. But you observe an error when you commit the transaction.

```
COMMIT;
```

```
SQL Error: ORA-02091: transaction rolled back
ORA-02290: check constraint (TEACH_B.BONUS_CK) violated
02091. 00000 - "transaction rolled back"
```

The commit failed due to constraint violation. Therefore, at this point, the transaction is rolled back by the database.

Example 3: Set the `DEFERRED` status to all constraints that can be deferred. Note that you can also set the `DEFERRED` status to a single constraint if required.

```
ALTER SESSION SET CONSTRAINTS = DEFERRED;
```

```
Session altered.
```

Now, if you attempt to insert a row that violates the `sal_ck` constraint, the statement is executed successfully.

```
INSERT INTO emp_new_sal VALUES (90, 5);
```

```
1 row inserted.
```

However, you observe an error when you commit the transaction. The transaction fails and is rolled back. This is because both the constraints are checked upon `COMMIT`.

```
COMMIT;
```

```
SQL Error: ORA-02091: transaction rolled back
ORA-02290: check constraint (TEACH_B.SAL_CK) violated
02091. 00000 - "transaction rolled back"
```

Example 4: Set the `IMMEDIATE` status to both the constraints that were set as `DEFERRED` in the previous example.

```
ALTER SESSION SET CONSTRAINTS = IMMEDIATE;
```

```
Session altered.
```

You observe an error if you attempt to insert a row that violates either `sal_ck` or `bonus_ck`.

```
INSERT INTO emp_new_sal VALUES (110, -1);
```

```
SQL Error: ORA-02290: check constraint (TEACH_B.BONUS_CK) violated
02290. 00000 - "check constraint (%s.%s) violated"
```

Note: If you create a table without specifying constraint deferability, the constraint is checked immediately at the end of each statement. For example, with the `CREATE TABLE` statement of the `newemp_details` table, if you do not specify the `newemp_det_pk` constraint deferability, the constraint is checked immediately.

```
CREATE TABLE newemp_details(emp_id NUMBER,
emp_name VARCHAR2(20),
CONSTRAINT newemp_det_pk PRIMARY KEY(emp_id));
```

When you attempt to defer the `newemp_det_pk` constraint that is not deferrable, you observe the following error:

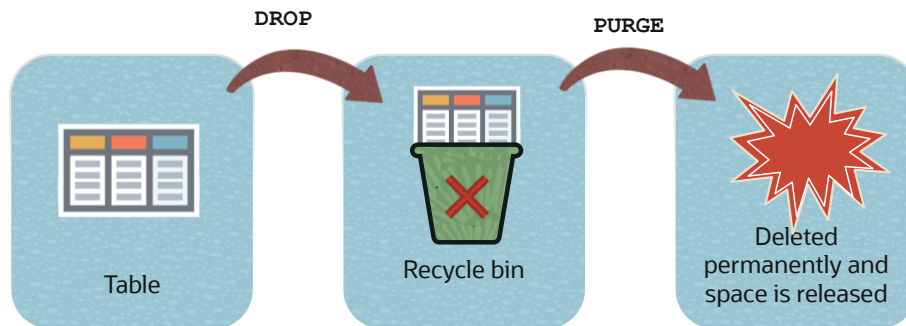
```
SET CONSTRAINT newemp_det_pk DEFERRED;
```

```
SQL Error: ORA-02447: cannot defer a constraint that is not deferrable
```

DROP TABLE ... PURGE

```
DROP TABLE emp_new_sal PURGE;
```

```
Table EMP_NEW_SAL dropped.
```



Oracle Database provides a feature for dropping tables. When you drop a table, the database does not immediately release the space associated with the table. Rather, the database renames the table and places it in a recycle bin, where it can later be recovered with the `FLASHBACK TABLE` statement if you find that you dropped the table in error. If you want to immediately release the space associated with the table at the time you issue the `DROP TABLE` statement, then include the `PURGE` clause as shown in the statement in the slide.

Specify `PURGE` only if you want to drop the table and release the space associated with it in a single step. If you specify `PURGE`, the database does not place the table and its dependent objects into the recycle bin.

Using this clause is equivalent to first dropping the table and then purging it from the recycle bin. This clause saves you one step in the process. It also provides enhanced security if you want to prevent sensitive material from appearing in the recycle bin.

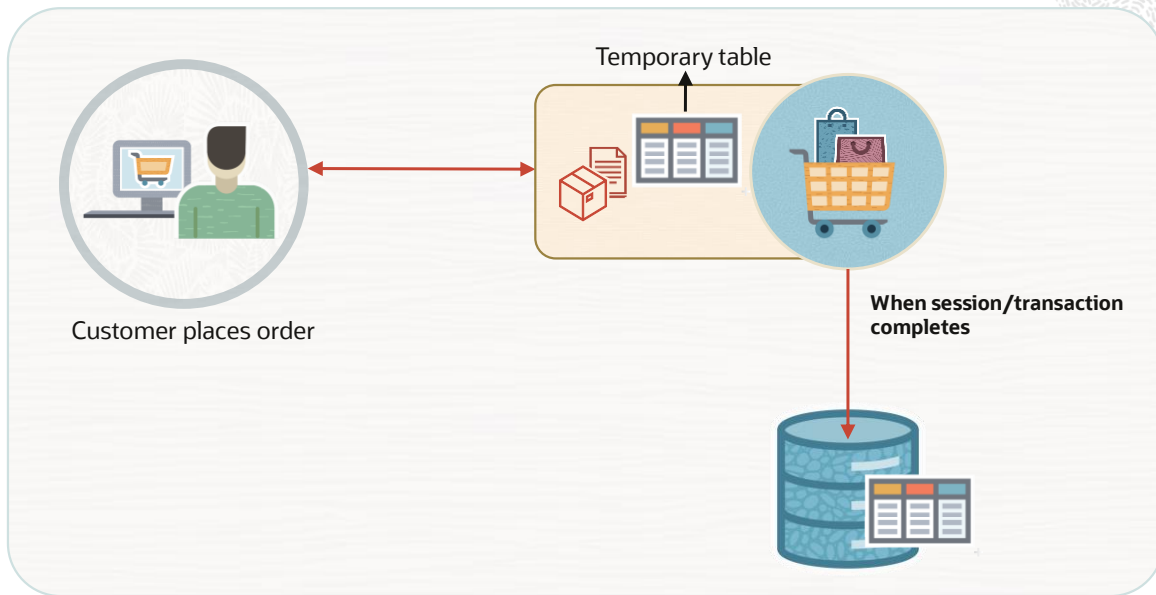


Create and Use Temporary Tables

For Instructor Use Only.

This document should not be distributed.

Using Temporary Tables



Brian is designing the OracleKart application. When customers place an order, the orders are to be stored in an `ORDER_DETAILS` table. Brian wants to enable customers, before they finalize their order, to use a “shopping cart” in which they can list items they wish to purchase. They should be able to add and remove items from this shopping cart. Storing this information in the `ORDER_DETAILS` table is not feasible because this table is not accessible to customers. Moreover, this data is not required to be stored permanently but only for the duration of a transaction. SQL enables you to store such information in temporary tables.

The shopping cart can be a temporary table. Each item is represented by a row in the temporary table. James continues to add and remove items from his cart before he finalizes on the items to buy. During this session, the cart data is private and the data in the temporary table keeps changing. After James finalizes his shopping and makes the payment, the application moves the rows from his cart to a permanent table, such as the `ORDER_DETAILS` table. At the end of the session, the data in the temporary table is automatically dropped.

Therefore, temporary tables are useful in applications where a result set must be buffered.

Temporary Table

- Holds data that exists only for the duration of a transaction or session
 - Data is private to the session.
- Can be either a **global** temporary table or a **private** temporary table

A temporary table holds data that exists only for the duration of a transaction or session.

Data in a temporary table is private to the session. Each session can see and modify only its own data.

You can create either a global temporary table or a private temporary table.

For Instructor Use Only.
This document should not be distributed.

Temporary Table: Characteristics

Characteristic	Global	Private
Naming rules	Same as for permanent tables	Must be prefixed with ORA\$PTT_
Visibility of table definition	All sessions	Only the session that created the table
Storage of table definition	Disk	Memory only
Types	Transaction-specific (ON COMMIT DELETE ROWS) or session-specific (ON COMMIT PRESERVE ROWS)	Transaction-specific (ON COMMIT DROP DEFINITION) or session-specific (ON COMMIT PRESERVE DEFINITION)

For Instructor Use Only.

This document should not be distributed.

Creating a Global Temporary Table

```
CREATE GLOBAL TEMPORARY TABLE cart(n NUMBER,d DATE)
ON COMMIT DELETE ROWS;
```

```
CREATE GLOBAL TEMPORARY TABLE emp_details
ON COMMIT PRESERVE ROWS;
```

A global temporary table is visible to all sessions, but the data in a global temporary table is visible only to the session that inserts the data into the table.

To create a global temporary table, use the following command:

```
CREATE GLOBAL TEMPORARY TABLE tablename(
    column_definition,
    ...)
ON COMMIT [PRESERVE | DELETE] ROWS;
```

The **ON COMMIT** clause indicates if the data in the table is transaction-specific (the default) or session-specific, the implications of which are:

- **PRESERVE ROWS:** This creates a global temporary table that is session-specific. A session gets bound to the global temporary table with the first insert into the table in the session. This binding goes away at the end of the session or by issuing a **TRUNCATE** of the table in the session. The database truncates the table when you terminate the session.
- **DELETE ROWS:** This creates a global temporary table that is transaction-specific. A session becomes bound to the global temporary table with a transaction's first insert into the table. The binding goes away at the end of the transaction. The database truncates the table (delete all rows) after each commit.

When you create a global temporary table in an Oracle database, you create a static table definition. Like permanent tables, global temporary tables are defined in the data dictionary. However, global temporary tables and their indexes do not automatically allocate a segment when created. Instead, temporary segments are allocated when data is first inserted. Until data is loaded in a session, the table appears empty.

Creating a Private Temporary Table

1

```
CREATE PRIVATE TEMPORARY TABLE ORA$PTT_sales_trans
(time_id      DATE,
 amt_sold    NUMBER(8,2))
ON COMMIT DROP DEFINITION;
```

2

```
CREATE PRIVATE TEMPORARY TABLE ORA$PTT_sales_sess
(time_id      DATE,
 amt_sold    NUMBER(8,2))
ON COMMIT PRESERVE DEFINITION;
```

Private temporary tables are temporary database objects that are dropped at the end of a transaction or session. Private temporary tables are stored in memory and the metadata and content of a private temporary table is visible only within the session that created it.

To create a private temporary table, use the following command:

```
CREATE PRIVATE TEMPORARY TABLE tablename(
    column_definition,
    ...)
ON COMMIT [DROP | PRESERVE] DEFINITION;
```

The **ON COMMIT** clause indicates if the data in the table is transaction-specific (the default) or session-specific, the implications of which are as follows:

- **DROP DEFINITION:** This creates a private temporary table that is transaction-specific. All data in the table is lost, and the table is dropped at the end of transaction.
- **PRESERVE DEFINITION:** This creates a private temporary table that is session-specific. All data in the table is lost, and the table is dropped at the end of the session that created the table.

Note: Names of private temporary tables must be prefixed according to the initialization parameter `PRIVATE_TEMP_TABLE_PREFIX`.

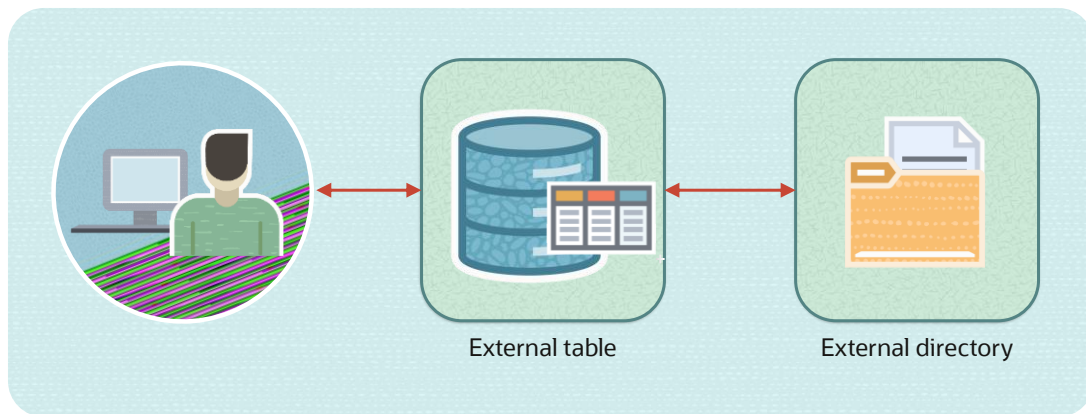


Creating and Using External Tables

For Instructor Use Only.

This document should not be distributed.

External Tables



In an external table, you store the metadata in the database and the actual data outside the database. This external table can be thought of as a view that is used for running any SQL query against external data without requiring that the external data first be loaded into the database. The external table data can be queried and joined directly and in parallel without requiring that the external data first be loaded in the database.

You can use SQL, PL/SQL, and Java to query the data in an external table.

The main difference between external tables and regular tables is that externally organized tables are read-only. No data manipulation language (DML) operations are possible, and no indexes can be created on them. However, you can create an external table, and thus unload data, by using the `CREATE TABLE AS SELECT` command.

The Oracle Server provides two major access drivers for external tables:

- The `ORACLE_LOADER` access driver is used for reading data from external files whose format can be interpreted by the `SQL*Loader` utility. Note that not all `SQL*Loader` functionality is supported with external tables.
- The `ORACLE_DATAPUMP` access driver can be used to both import and export data by using a platform-independent format. The rows from a `SELECT` statement to be loaded into an external table are written as part of a `CREATE TABLE ... ORGANIZATION EXTERNAL ... AS SELECT` statement by the access driver. You can then use `SELECT` to read data out of that data file. You can also create an external table definition on another system and use that data file. This allows data to be moved between Oracle databases.

Creating a Directory for the External Table

Create a **DIRECTORY** object that corresponds to the directory on the file system where the external data source resides.

```
CREATE OR REPLACE DIRECTORY emp_dir  
AS '/.../emp_dir';  
  
GRANT READ ON DIRECTORY emp_dir TO ora_21;
```

Use the `CREATE DIRECTORY` statement to create a directory object. A directory object specifies an alias for a directory on the server's file system where it resides. You can use directory names when referring to an external data source, rather than hard-code the operating system path name, for greater file management flexibility.

You must have the `CREATE ANY DIRECTORY` system privileges to create directories. When you create a directory, you are automatically granted the `READ` and `WRITE` object privileges and can grant `READ` and `WRITE` privileges to other users and roles. The DBA can also grant these privileges to other users and roles.

A user needs `READ` privileges for all directories used in external tables for access and `WRITE` privileges for the log, bad, and discard file locations being used.

In addition, a `WRITE` privilege is necessary when the external table framework is being used to unload data.

Oracle also provides the `ORACLE_DATAPUMP` type, with which you can unload data (that is, read data from a table in the database and insert it into an external table) and then reload it into an Oracle database. This is a one-time operation that can be done when the table is created. After the creation and initial population is done, you cannot update, insert, or delete any rows.

Syntax

```
CREATE [OR REPLACE] DIRECTORY directory AS 'path_name';
```

In the syntax:

OR REPLACE	Specify OR REPLACE to re-create the directory database object if it already exists. You can use this clause to change the definition of an existing directory without dropping, re-creating, and regrating database object privileges previously granted on the directory. Users who were previously granted privileges on a redefined directory can continue to access the directory without requiring that the privileges be regranted.
directory	Specify the name of the directory object to be created. The maximum length of the directory name is 30 bytes. You cannot qualify a directory object with a schema name.
'path_name'	Specify the full path name of the operating system directory to be accessed. The path name is case-sensitive.

For Instructor Use Only.
This document should not be distributed.

Creating an External Table

Syntax:

```
CREATE TABLE <table_name>
  ( <col_name> <datatype>, ... )
  ORGANIZATION EXTERNAL
  (TYPE <access_driver_type>
  DEFAULT DIRECTORY <directory_name>
  ACCESS PARAMETERS
  (... ) )
  LOCATION ('<location_specifier>')
  REJECT LIMIT [0 | <number> | UNLIMITED];
```

You create external tables by using the `ORGANIZATION EXTERNAL` clause of the `CREATE TABLE` statement.

You are not, in fact, creating a table. Rather, you are creating metadata in the data dictionary that you can use to access external data. You use the `ORGANIZATION` clause to specify the order in which the data rows of the table are stored. By specifying `EXTERNAL` in the `ORGANIZATION` clause, you indicate that the table is a read-only table located outside the database. Note that the external files must already exist outside the database.

`TYPE <access_driver_type>` indicates the access driver of the external table. The access driver is the application programming interface (API) that interprets the external data for the database. If you do not specify `TYPE`, Oracle uses the default access driver, `ORACLE_LOADER`. The other option is `ORACLE_DATAPUMP`.

You use the `DEFAULT DIRECTORY` clause to specify one or more Oracle database directory objects that correspond to directories on the file system where the external data sources may reside.

The optional `ACCESS PARAMETERS` clause enables you to assign values to the parameters of the specific access driver for this external table.

Use the `LOCATION` clause to specify one external locator for each external data source. Usually, `<location_specifier>` is a file, but it need not be.

The `REJECT LIMIT` clause enables you to specify how many conversion errors can occur during a query of the external data before an Oracle error is returned and the query is aborted. The default value is 0.

The syntax for using the ORACLE_DATAPUMP access driver is as follows:

```
CREATE TABLE extract_emps  
  
ORGANIZATION EXTERNAL (TYPE ORACLE_DATAPUMP  
                        DEFAULT DIRECTORY ...  
                        ACCESS PARAMETERS (... )  
                        LOCATION (...)  
  
                        PARALLEL 4  
  
                        REJECT LIMIT UNLIMITED  
  
AS  
  
SELECT * FROM ...;
```

For Instructor Use Only.
This document should not be distributed.

Creating an External Table by Using ORACLE_LOADER

```
CREATE TABLE oldemp (fname char(25), lname CHAR(25))
ORGANIZATION EXTERNAL
(TYPE ORACLE_LOADER
DEFAULT DIRECTORY emp_dir
ACCESS PARAMETERS
(RECORDS DELIMITED BY NEWLINE
FIELDS(fname POSITION ( 1:20) CHAR,
lname POSITION (22:41) CHAR))
LOCATION ('emp.dat'));
```

Table OLDEMP created.

Assume that there is a flat file that has records in the following format:

10,jones,11-Dec-1934

20,smith,12-Jun-1972

Records are delimited by new lines. The file is located at
/home/oracle/labs/sql2/emp_dir/emp.dat.

To convert this file as the data source for an external table, whose metadata will reside in the database, you must perform the following steps:

1. Create a directory object, emp_dir, as follows:

```
CREATE DIRECTORY emp_dir AS '/home/oracle/labs/sql2/emp_dir' ;
```

2. Run the CREATE TABLE command shown in the slide.

The example in the slide illustrates the table specification to create an external table for the file:

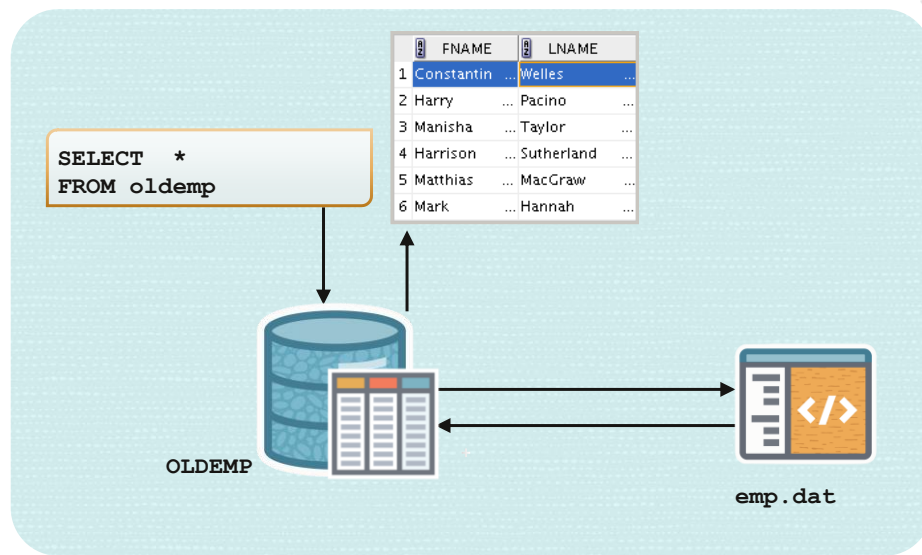
/home/oracle/labs/sql2/emp_dir/emp.dat

In the example, the TYPE specification is given only to illustrate its use. ORACLE_LOADER is the default access driver if not specified. The ACCESS PARAMETERS option provides values to parameters of the specific access driver, which are interpreted by the access driver, not by the Oracle Server.

After the CREATE TABLE command executes successfully, the OLDEMP external table can be described and queried in the same way as a relational table.

Note: The emp_dir directory on the file system must have write permission for the user and others for the external table to work successfully.

Querying External Tables



An external table does not tell you how data is stored in the database. It also does not tell you how data is stored in the external source. Instead, it tells you how the external table layer must present the data to the server. It is the responsibility of the access driver and the external table layer to do the necessary transformations required on the data in the data file so that it matches the external table definition.

When the database server accesses data in an external source, it calls the appropriate access driver to get the data from an external source in a form that the database server expects.

Remember that the description of the data in the data source is separate from the definition of the external table. The source file can contain more or fewer fields than there are columns in the table. Also, the data types for fields in the data source can be different from the columns in the table. The access driver takes care of ensuring that the data from the data source is processed so that it matches the definition of the external table.

Creating an External Table by Using ORACLE_DATAPUMP: Example

```
CREATE TABLE emp_ext
(employee_id, first_name, last_name)
ORGANIZATION EXTERNAL
(
  TYPE ORACLE_DATAPUMP
  DEFAULT DIRECTORY emp_dir
  LOCATION
    ('emp1.exp', 'emp2.exp')
)
PARALLEL
AS
SELECT employee_id, first_name, last_name
FROM   employees;
```

Table EMP_EXT created.

You can perform the unload and reload operations with external tables by using the ORACLE_DATAPUMP access driver.

Note: In the context of external tables, loading data refers to the act of data being read from an external table and loaded into a table in the database. Unloading data refers to the act of reading data from a table and inserting it into an external table.

The example in the slide illustrates the table specification to create an external table by using the ORACLE_DATAPUMP access driver. Data is then populated into the two files: emp1.exp and emp2.exp.

To populate data read from the EMPLOYEES table into an external table, you must perform the following steps:

1. Create a directory object, emp_dir, as follows:
`CREATE OR REPLACE DIRECTORY emp_dir AS '/stage/Labs/sql2/emp_dir';`
2. Run the CREATE TABLE command shown in the slide.

Note: The emp_dir directory is the same as created in the previous example of using ORACLE_LOADER.

You can query the external table by executing the following code:

```
SELECT * FROM emp_ext;
```

Summary



Manage constraints

Create and use temporary tables

Create and use external tables

In this lesson, you learned how to perform the following tasks for schema object management:

- Alter tables to add or modify columns or constraints.
- Use the `ORGANIZATION EXTERNAL` clause of the `CREATE TABLE` statement to create an external table. An external table is a read-only table whose metadata is stored in the database but whose data is stored outside the database.
- Use external tables to query data without first loading it into the database.

Retrieving Data by Using Subqueries



For Instructor Use Only.

This document should not be distributed.

Objectives



Retrieve data by using a subquery as a source

Write a multiple-column subquery

Use scalar subqueries in SQL

Solve problems with correlated subqueries

Use the `EXISTS` and `NOT EXISTS` operators

Use the `WITH` clause

In this lesson, you learn how to write multiple-column subqueries and subqueries in the `FROM` clause of a `SELECT` statement. You also learn to solve problems by using scalar, correlated subqueries and to use the `WITH` clause.



Retrieving Data by Using a Subquery as a Source

For Instructor Use Only.

This document should not be distributed.

Retrieving Data by Using a Subquery as a Source

```
SELECT department_name, city
FROM departments
NATURAL JOIN (SELECT l.location_id, l.city, l.country_id
               FROM locations l
               JOIN countries c
               ON (l.country_id = c.country_id)
               JOIN regions
               USING (region_id)
               WHERE region_name = 'Europe');
```

	DEPARTMENT_NAME	CITY
1	Human Resources	London
2	Sales	Oxford
3	Public Relations	Munich

You can use a subquery in the `FROM` clause of a `SELECT` statement, which is very similar to how views are used. This form of a subquery is also called an *inline view*.

A subquery in the `FROM` clause of a `SELECT` statement defines a data source for that particular `SELECT` statement only. Just like a database view, the `SELECT` statement in the subquery can be as simple or as complex as you like.

When a database view is created, the associated `SELECT` statement is stored in the data dictionary. In situations where you do not have the necessary privileges to create database views, or when you would like to test the suitability of a `SELECT` statement to become a view, you can use an inline view.

With inline views, you can have all the code needed to support the query in one place. This means that you can avoid the complexity of creating a separate database view.

The example in the slide shows how to use an inline view to display the department name and the city in Europe. The subquery in the `FROM` clause fetches the location ID, city name, and the country by joining three different tables. The output of the inner query is considered as a table for the outer query. The inner query is similar to that of a database view but does not have any physical name.

You can display the same output as in the example in the slide by performing the following two steps:

1. Create a database view:

```
CREATE OR REPLACE VIEW european_cities
AS
SELECT l.location_id, l.city, l.country_id
FROM   locations l
JOIN   countries c
ON(l.country_id = c.country_id)
JOIN regions USING(region_id)
WHERE region_name = 'Europe';
```

2. Join the EUROPEAN_CITIES view with the DEPARTMENTS table:

```
SELECT department_name, city
FROM   departments
NATURAL JOIN european_cities;
```

Note: You learned how to create database views in the lesson titled “Creating Views.”

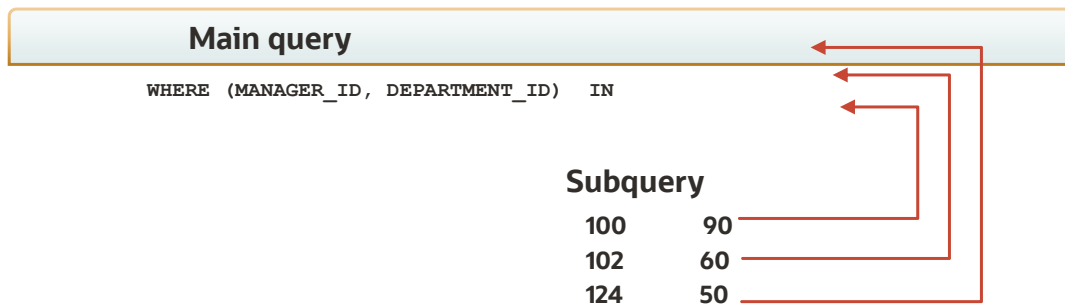


Writing a Multiple - Column Subquery

For Instructor Use Only.

This document should not be distributed.

Multiple-Column Subqueries



Each row of the main query is compared to values from a multiple-row and multiple-column subquery.

So far, you have written single-row subqueries and multiple-row subqueries where only one column is returned by the inner `SELECT` statement, and this is used to evaluate the expression in the parent `SELECT` statement.

If you want to compare two or more columns, you must write a compound `WHERE` clause by using logical operators. Using multiple-column subqueries, you can combine duplicate `WHERE` conditions into a single `WHERE` clause.

Syntax

```
SELECT      column, column, ...
FROM        table
WHERE       (column, column, ...) IN
            (SELECT column, column, ...
             FROM    table
             WHERE   condition);
```

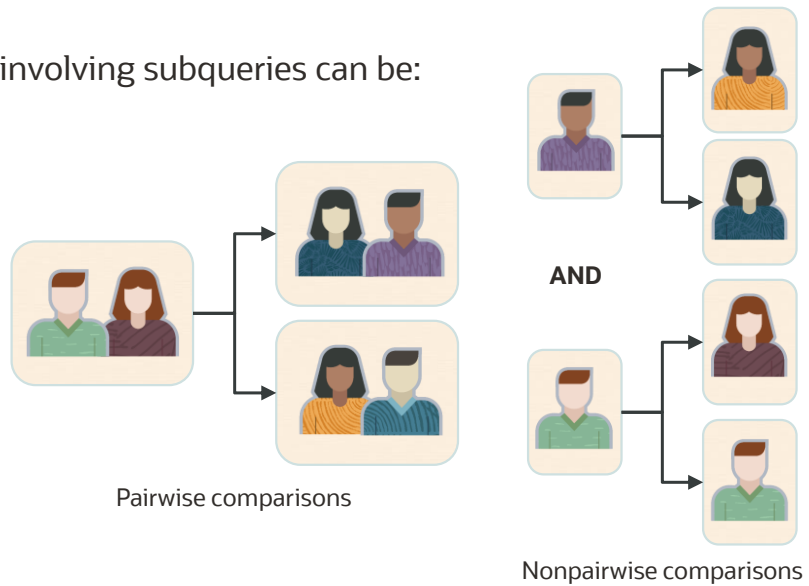
The graphic in the slide illustrates that the values of `MANAGER_ID` and `DEPARTMENT_ID` from the main query are being compared with the `MANAGER_ID` and `DEPARTMENT_ID` values retrieved by the subquery. Because the number of columns that are being compared is more than one, the example qualifies as a multiple-column subquery.

Note: Before you run the examples in the next few slides, you need to create the `empl_demo` table and populate data into it by using the `lab_06_insert_empdata.sql` file.

Column Comparisons

Multiple-column comparisons involving subqueries can be:

- Pairwise comparisons
- Nonpairwise comparisons



Pairwise Versus Nonpairwise Comparisons

Multiple-column comparisons involving subqueries can be nonpairwise comparisons or pairwise comparisons. If you consider the example “Display the details of the employees who work in the same department, and have the same manager, as ‘Daniel?’” then you get the correct result with the following statement:

```
SELECT first_name, last_name, manager_id, department_id
FROM empl_demo
WHERE manager_id IN (SELECT manager_id
                     FROM empl_demo
                     WHERE first_name = 'Daniel')
AND department_id IN (SELECT department_id
                      FROM empl_demo
                      WHERE first_name = 'Daniel');
```

There is only one “Daniel” in the `EMPL_DEMO` table (Daniel Faviat, who is managed by employee 108 and works in department 100). However, if the subqueries return more than one row, the result might not be correct. For example, if you run the same query but substitute “John” for “Daniel,” you get an incorrect result. This is because the combination of `department_id` and `manager_id` is important. To get the correct result for this query, you need a pairwise comparison.

Pairwise Comparison Subquery

Display the details of the employees who are managed by the same manager and work in the same department as the employees with `EMPLOYEE_ID` 199 or 174.

```
SELECT employee_id, manager_id, department_id
FROM employees
WHERE (manager_id, department_id) IN
      (SELECT manager_id, department_id
       FROM employees
       WHERE employee_id IN (174, 199))
AND employee_id NOT IN (174, 199);
```

	EMPLOYEE_ID	MANAGER_ID	DEPARTMENT_ID
1	141	124	50
2	142	124	50
3	143	124	50
4	144	124	50

...

The example shows a pairwise comparison of the columns. It compares the values in the `MANAGER_ID` column and the `DEPARTMENT_ID` column of each row in the `EMPLOYEES` table with the values in the `MANAGER_ID` column and the `DEPARTMENT_ID` column for the employees with the `EMPLOYEE_ID` 199 or 174.

- First, the subquery to retrieve the `MANAGER_ID` and `DEPARTMENT_ID` values for the employees with the `EMPLOYEE_ID` 199 or 174 is executed.
- These values are compared with the `MANAGER_ID` column and the `DEPARTMENT_ID` column of each row in the `EMPLOYEES` table. If the values match, the row is displayed.
- In the output, the records of the employees with the `EMPLOYEE_ID` 199 or 174 will not be displayed. The output of the query is shown in the slide.

Nonpairwise Comparison Subquery

Display the details of the employees who are managed by the same manager as the employees with `EMPLOYEE_ID` 174 or 141 and work in the same department as the employees with `EMPLOYEE_ID` 174 or 141.

```
SELECT employee_id, manager_id, department_id
FROM employees
WHERE manager_id IN
    (SELECT manager_id
     FROM employees
     WHERE employee_id IN (174,141))
AND department_id IN
    (SELECT department_id
     FROM employees
     WHERE employee_id IN (174,141))
AND employee_id NOT IN(174,141);
```

	EMPLOYEE_ID	MANAGER_ID	DEPARTMENT_ID
1	142	124	50
2	143	124	50

The example in the slide shows a nonpairwise comparison of the columns. It displays the `EMPLOYEE_ID`, `MANAGER_ID`, and `DEPARTMENT_ID` of an employee whose manager ID matches with any of the manager IDs of employees whose employee IDs are either 174 or 141 and `DEPARTMENT_ID` match any of the department IDs of employees whose employee IDs are either 174 or 141.

- First, the subquery to retrieve the `MANAGER_ID` values for the employees with the `EMPLOYEE_ID` 174 or 141 is executed.
- Similarly, the second subquery to retrieve the `DEPARTMENT_ID` values for the employees with the `EMPLOYEE_ID` 174 or 141 is executed.
- The retrieved values of the `MANAGER_ID` and `DEPARTMENT_ID` columns are compared with the `MANAGER_ID` and `DEPARTMENT_ID` column for each row in the `EMPLOYEES` table.
- If the `MANAGER_ID` column of the row in the `EMPLOYEES` table matches with any of the values of the `MANAGER_ID` retrieved by the inner subquery and if the `DEPARTMENT_ID` column of the row in the `EMPLOYEES` table matches with any of the values of the `DEPARTMENT_ID` retrieved by the second subquery, the record is displayed.



Using Scalar Subqueries in SQL

For Instructor Use Only.

This document should not be distributed.

Scalar Subquery Expressions

- A scalar subquery is a subquery that returns exactly one column value from one row.
- Scalar subqueries can be used in:
 - Condition and expression part of `DECODE` and `CASE`
 - All clauses of `SELECT` except `GROUP BY`
 - `SET` clause and `WHERE` clause of an `UPDATE` statement



A subquery that returns exactly one column value from one row is referred to as a scalar subquery. Multiple-column subqueries that are written to compare two or more columns, using a compound `WHERE` clause and logical operators, do not qualify as scalar subqueries.

The value of the scalar subquery expression is the value of the select list item of the subquery. If the subquery returns 0 rows, the value of the scalar subquery expression is `NULL`. If the subquery returns more than one row, the Oracle Server returns an error. The Oracle Server has always supported the usage of a scalar subquery in a `SELECT` statement.

You can use scalar subqueries in:

- The condition and expression part of `DECODE` and `CASE`
- All clauses of `SELECT` except `GROUP BY`
- The `SET` clause and `WHERE` clause of an `UPDATE` statement

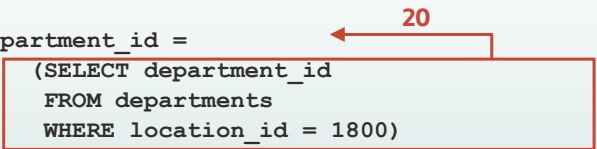
However, scalar subqueries are not valid expressions in the following places:

- In the `RETURNING` clause of data manipulation language (DML) statements
- As the basis of a function-based index
- In `GROUP BY` clauses and `CHECK` constraints
- In `CONNECT BY` clauses
- In statements that are unrelated to queries, such as `CREATE PROFILE`

Scalar Subqueries: Examples

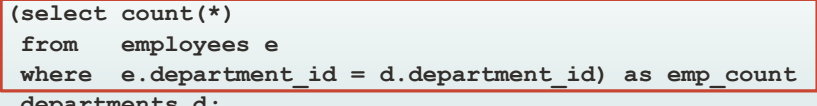
- Scalar subqueries in CASE expressions:

```
SELECT employee_id, last_name,  
       (CASE  
         WHEN department_id =  
           (SELECT department_id  
            FROM departments  
            WHERE location_id = 1800)  
         THEN 'Canada' ELSE 'USA' END) location  
FROM   employees;
```

A red box highlights the inner subquery: `(SELECT department_id FROM departments WHERE location_id = 1800)`. A red arrow points from the number `20` to the `department_id` column of this subquery, indicating its result is used in the `WHEN` clause of the outer `CASE` expression.

- Scalar subqueries in the SELECT statement:

```
select department_id, department_name,  
       (select count(*)  
        from employees e  
        where e.department_id = d.department_id) as emp_count  
from   departments d;
```

A red box highlights the scalar subquery: `(select count(*) from employees e where e.department_id = d.department_id) as emp_count`, which is used as a column value in the `SELECT` statement.

The first example in the slide demonstrates that scalar subqueries can be used in CASE expressions.

The inner query returns the value 20, which is the department ID of the department whose location ID is 1800.

The CASE expression in the outer query uses the result of the inner query to display the employee ID, last names, and a value of Canada or USA, depending on whether the department ID of the record retrieved by the outer query is 20.

The second example in the slide demonstrates that scalar subqueries can be used in SELECT statements.



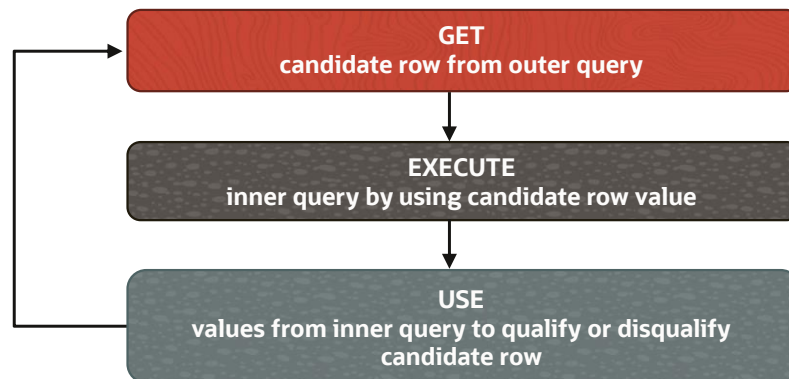
Solving Problems with Correlated Subqueries

For Instructor Use Only.

This document should not be distributed.

Correlated Subqueries

Correlated subqueries are used for row-by-row processing. Each subquery is executed once for every row of the outer query.



The Oracle Server performs a correlated subquery when the subquery references a column from a table referred to in the parent statement. Whenever the parent statement is processed for a row, the correlated subquery is evaluated once for that row. The parent statement can be a `SELECT`, an `UPDATE`, or a `DELETE` statement.

Nested Subqueries Versus Correlated Subqueries

- With a normal **nested subquery**, the inner `SELECT` query runs first and executes once, returning values to be used by the main query.
- A **correlated subquery**, however, executes once for each candidate row considered by the outer query. That is, the inner query is driven by the outer query.

Nested Subquery Execution

- Inner query executes first and finds a value.
- Outer query executes once, using the value from the inner query.

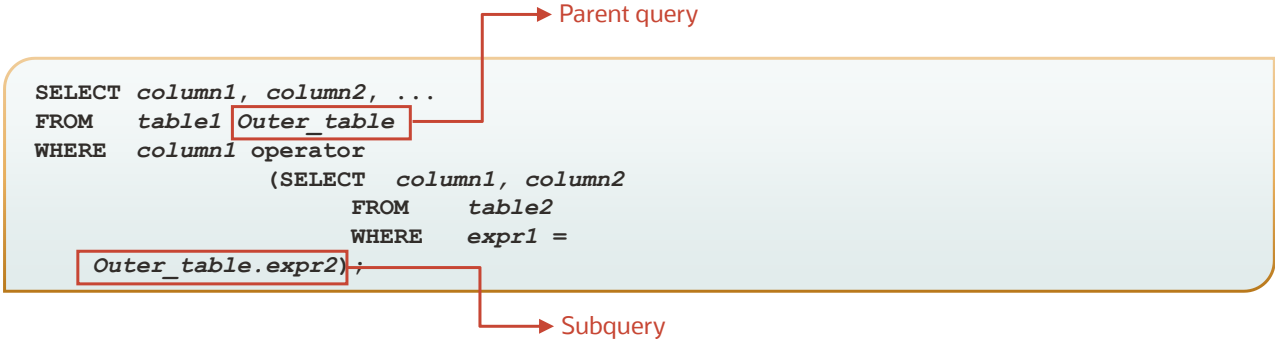
Correlated Subquery Execution

- Get a candidate row (fetched by the outer query).
- Execute the inner query by using the value of the candidate row.
- Use the values resulting from the inner query to qualify or disqualify the candidate.
- Repeat until no candidate row remains.

Correlated Subqueries

The subquery references a column from a table in the parent query.

```
SELECT column1, column2, ...  
FROM   table1 Outer_table  
WHERE  column1 operator  
        (SELECT column1, column2  
          FROM   table2  
          WHERE  expr1 =  
                Outer_table.expr2);
```



A correlated subquery is one way of reading every row in a table and comparing values in each row against related data. It is used whenever a subquery must return a different result or set of results for each candidate row considered by the main query. That is, you use a correlated subquery to answer a multipart question whose answer depends on the value in each row processed by the parent statement.

The Oracle server performs a correlated subquery when the subquery references a column from a table in the parent query.

Note: You can use the `ANY` and `ALL` operators in a correlated subquery.

Using Correlated Subqueries: Example 1

Find all employees who earn more than the average salary in their department.

```
SELECT last_name, salary, department_id
FROM   employees outer_table
WHERE  salary >
      (SELECT AVG(salary)
       FROM   employees inner_table
       WHERE  inner_table.department_id =
              outer_table.department_id);
```

	LAST_NAME	SALARY	DEPARTMENT_ID
1	King	24000	90
2	Hunold	9000	60
3	Ernst	6000	60
4	Greenberg	12008	100
5	Faviet	9000	100
6	Raphaely	11000	30

Each time a row from the outer query is processed, the inner query is evaluated.

The example in the slide finds employees who earn more than the average salary of their department. In this case, the correlated subquery specifically computes the average salary for each department.

Because both the outer query and inner query use the `EMPLOYEES` table in the `FROM` clause, an alias is given to `EMPLOYEES` in the outer `SELECT` statement for clarity. The alias makes the entire `SELECT` statement more readable. Without the alias, the query would not work properly because the inner statement would not be able to distinguish the inner table column from the outer table column.

The correlated subquery performs the following steps for each row of the `EMPLOYEES` table:

1. The `department_id` of the row is determined.
2. The `department_id` is then used to evaluate the subquery.
3. If the salary in that row is greater than the average salary in that department, then the row is returned.

The subquery is evaluated once for each row of the `EMPLOYEES` table.

Using Correlated Subqueries: Example 2

Display the details of the highest earning employees in each department.

```
SELECT department_id, employee_id, salary
FROM EMPLOYEES e
WHERE salary =
    (SELECT MAX(DISTINCT salary)
     FROM EMPLOYEES
     WHERE e.department_id = department_id);
```

	DEPARTMENT_ID	EMPLOYEE_ID	SALARY
1	90	100	24000
2	60	103	9000
3	100	108	12008
4	30	114	11000

....

The example in the slide displays the details of the highest earning employees in each department. The Oracle Server evaluates a correlated subquery as follows:

1. Selects a row from the table specified in the outer query. This will be the current candidate row.
2. Stores the value of the column referenced in the subquery from this candidate row. In the example in the slide, the column referenced in the subquery is `e.department_id`.
3. Performs the subquery with its condition referencing the value from the outer query's candidate row. In the example in the slide, the `MAX(DISTINCT salary)` group function is evaluated based on the value of the `E.DEPARTMENT_ID` column obtained in step 2.
4. Evaluates the `WHERE` clause of the outer query on the basis of results of the subquery performed in step 3. This determines whether the candidate row is selected for output. In the example, the highest salary earned by employees in a department is evaluated by the subquery and is compared with the salary of the candidate row in the `WHERE` clause of the outer query. If the condition is satisfied, the candidate row's employee record is displayed.
5. Repeats the procedure for the next candidate row of the table, and so on, until all the rows in the table have been processed.

The correlation is established by using an element from the outer query in the subquery. In this example, you compare `DEPARTMENT_ID` from the table in the subquery with `DEPARTMENT_ID` from the table in the outer query.



Using the EXISTS and NOT EXISTS Operators

For Instructor Use Only.

This document should not be distributed.

Using the EXISTS Operator

- EXISTS operator tests for existence of rows in the result set of the subquery.
- If a subquery row value is found:
 - Search does not continue in the inner query
 - Condition is flagged TRUE
- If a subquery row value is not found:
 - Condition is flagged FALSE
 - Search continues in the inner query

Although with nesting SELECT statements all logical operators are valid, you can use the EXISTS operator.

The EXISTS operator is frequently used with correlated subqueries to test whether a value retrieved by the outer query exists in the results set of the values retrieved by the inner query.

The EXISTS operator logic is as follows:

- If the subquery returns at least one row, the operator returns TRUE.
- If the value does not exist, it returns FALSE.

Accordingly, you can use the NOT EXISTS operator to test whether a value retrieved by the outer query is not a part of the result set of the values retrieved by the inner query.

Using the EXISTS Operator

```
SELECT employee_id, last_name, job_id, department_id
FROM   employees outer
WHERE  EXISTS ( SELECT NULL
                  FROM   employees
                  WHERE  manager_id =
                        outer.employee_id );
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90
4	103	Hunold	IT_PROG	60
5	108	Greenberg	FI_MGR	100
6	114	Raphaely	PU_MAN	30
7	120	Weiss	ST_MAN	50
8	121	Fripp	ST_MAN	50

...

Observe the example in the slide, which displays all the managers in the `EMPLOYEES` table using the `EXISTS` operator.

The `EXISTS` operator ensures that the search in the inner query does not continue when at least one match is found for the manager and employee number by the condition:

```
WHERE manager_id = outer.employee_id
```

Note that the inner `SELECT` query does not need to return a specific value, so a constant can be selected.

Find All Departments That Do Not Have Employees: Example

```
SELECT department_id, department_name
FROM departments d
WHERE NOT EXISTS (SELECT NULL
                  FROM employees
                  WHERE department_id = d.department_id);
```

	DEPARTMENT_ID	DEPARTMENT_NAME
1	120	Treasury
2	130	Corporate Tax
3	140	Control And Credit
4	150	Shareholder Services
5	160	Benefits
6	170	Manufacturing
7	180	Construction
8	190	Contracting
9	200	Operations
10	210	IT Support

...

All Rows Fetched: 16



Using the NOT EXISTS Operator

Alternative Solution

A NOT IN construct can be used as an alternative for a NOT EXISTS operator, as shown in the following example:

```
SELECT department_id, department_name
FROM departments
WHERE department_id NOT IN (SELECT department_id
                           FROM employees);
```

However, NOT IN evaluates to FALSE if any member of the set is a NULL value. Therefore, your query will not return any rows even if there are rows in the departments table that satisfy the WHERE condition.



Using the WITH Clause

For Instructor Use Only.

This document should not be distributed.

WITH Clause

- Using the **WITH** clause, you can use the same query block in a `SELECT` statement when it occurs more than once within a complex query.
- It retrieves the results of a query block and stores them in the user's temporary tablespace.
- It can improve performance.



Using the `WITH` clause, you can define a query block before using it in a query.

The `WITH` clause (formally known as `subquery_factoring_clause`) enables you to reuse the same query block in a `SELECT` statement when it occurs more than once within a complex query. This is particularly useful when a query has many references to the same query block and there are joins and aggregations. This also helps in reducing the cost to evaluate the query block multiple times.

Using the `WITH` clause, the Oracle server retrieves the results of a query block and stores them in the user's temporary tablespace. This can improve performance.

WITH Clause: Benefits

- Makes the query easy to read
- Evaluates a clause only once, even if it appears multiple times in the query
- In most cases, may improve performance for large queries

For Instructor Use Only.

This document should not be distributed.

This document should not be distributed.

This document should not be distributed.

This document should not be distributed.

This document should not be distributed.

This document should not be distributed.

- This document should not be distributed.

Recursive WITH Clause

The Recursive `WITH` clause:

- Enables formulation of recursive queries
- Creates a query with a name, called the Recursive `WITH` element name
- Contains two types of query block members: an anchor and a recursive
- Is `ANSI` compatible



Using the extension of the `WITH` clause, you can also formulate recursive queries.

- Recursive `WITH` defines a recursive query with a name, the Recursive `WITH` element name.
- The Recursive `WITH` element definition must contain at least two query blocks: an anchor member and a recursive member. There can be multiple anchor members, but there can be only a single recursive member.
- The anchor member must appear before the recursive member, and it cannot reference *query_name*. The anchor member can be composed of one or more query blocks combined by the set operators, for example, `UNION ALL`, `UNION`, `INTERSECT`, or `MINUS`.
- The recursive member must follow the anchor member and must reference *query_name* exactly once. You must combine the recursive member with the anchor member by using the `UNION ALL` set operator.
- The Recursive `WITH` clause complies with the American National Standards Institute (ANSI) standard.
- Recursive `WITH` can be used to query hierarchical data, such as organization charts.

Recursive WITH Clause: Example

FLIGHTS table

	SOURCE	DESTIN	FLIGHT_TIME
1	San Jose	Los Angeles	1.3
2	New York	Boston	1.1
3	Los Angeles	New York	5.8

1

```
WITH Reachable_From (Source, Destin, TotalFlightTime) AS
(
    SELECT Source, Destin, Flight_time
    FROM Flights
    UNION ALL
    SELECT incoming.Source, outgoing.Destin,
           incoming.TotalFlightTime+outgoing.Flight_time
    FROM Reachable_From incoming, Flights outgoing
    WHERE incoming.Destin = outgoing.Source
)
SELECT Source, Destin, TotalFlightTime
FROM Reachable_From;
```

2

3

	SOURCE	DESTIN	TOTALFLIGHTTIME
1	San Jose	Los Angeles	1.3
2	New York	Boston	1.1
3	Los Angeles	New York	5.8
4	Los Angeles	Boston	6.9
5	San Jose	New York	7.1
6	San Jose	Boston	8.2

Example 1 in the slide displays records from a `FLIGHTS` table describing flights between two cities.

Using the query in example 2, you query the `FLIGHTS` table to display the total flight time between any source and destination.

The `WITH` clause in the query, which is named `Reachable_From`, has a `UNION ALL` query with two branches:

- The first branch is the *anchor* branch, which selects all the rows from the `Flights` table.
- The second branch is the recursive branch. It joins the contents of `Reachable_From` to the `Flights` table to find other cities that can be reached and adds these to the content of `Reachable_From`. The operation will finish when no more rows are found by the recursive branch.

Example 3 displays the result of the query that selects everything from the `WITH` clause element `Reachable_From`.

For details, see:

- *Oracle Database SQL Language Reference 23c*.
- *Oracle Database Data Warehousing Guide 23c*.

Summary



Retrieve data by using a subquery as a source

Write a multiple-column subquery

Use scalar subqueries in SQL

Solve problems with correlated subqueries

Use the `EXISTS` and `NOT EXISTS` operators

Use the `WITH` clause

In this lesson, you have learned how to write multiple-column subqueries and subqueries in the `FROM` clause of a `SELECT` statement. You also learn to solve problems by using scalar, correlated subqueries and to use the `WITH` clause.

Manipulating Data by Using Subqueries



For Instructor Use Only.

This document should not be distributed.

Objectives



Use subqueries to manipulate data

Insert values by using a subquery as a target

Use the `WITH CHECK OPTION` keyword on DML statements

Use correlated subqueries to update and delete rows

In this lesson, you learn how to manipulate data in the Oracle database by using subqueries. You also learn how to solve problems by using correlated subqueries.



Using Subqueries to Manipulate Data

For Instructor Use Only.
This document should not be distributed.

Using Subqueries to Manipulate Data

You can use subqueries in data manipulation language (DML) statements to:

- Retrieve data by using an inline view
- Copy data from one table to another
- Update data in one table based on the values of another table
- Delete rows from one table based on rows in another table



Subqueries can be used to retrieve data from a table that you can use as input to `INSERT` into a different table. In this way, you can easily copy large volumes of data from one table to another with one single `SELECT` statement.

Similarly, you can use subqueries to perform mass updates and deletes by using them in the `WHERE` clause of the `UPDATE` and `DELETE` statements. You can also use subqueries in the `FROM` clause of a `SELECT` statement. This is called an inline view.

Note: You learned how to update and delete rows based on another table in the course titled *Oracle Database: SQL Workshop I*.



Inserting Values by Using a Subquery as a Target

For Instructor Use Only.

This document should not be distributed.

Inserting by Using a Subquery as a Target (1/2)

```
INSERT INTO (SELECT l.location_id, l.city, l.country_id
FROM   loc l
JOIN   countries c
ON(l.country_id = c.country_id)
JOIN regions USING(region_id)
WHERE region name = 'Europe')
VALUES (3300, 'Cardiff', 'UK');
```

1 row inserted.

You can use a subquery in place of the table name in the `INTO` clause of the `INSERT` statement. The `SELECT` list of this subquery must have the same number of columns as the column list of the `VALUES` clause.

Any rules on the columns of the base table must be followed for the `INSERT` statement to work successfully. For example, you cannot put in a duplicate location ID or leave out a value for a mandatory `NOT NULL` column.

By using subqueries in this way, you can avoid having to create a view just for performing an `INSERT`.

The example in the slide uses a subquery in the place of `LOC` to create a record for a new European city.

Note: You can also perform the `INSERT` operation on the `EUROPEAN_CITIES` view by using the following code:

```
INSERT INTO european_cities
VALUES (3300, 'Cardiff', 'UK');
```

In the example in the slide, the `loc` table is created by running the following statement:

```
CREATE TABLE loc AS SELECT * FROM locations;
```

Inserting by Using a Subquery as a Target (2/2)

Verify the results from the `INSERT` statement in the previous slide.

```
SELECT location_id, city, country_id
FROM   loc;
```

...

20	2900 Geneva	CH
21	3000 Bern	CH
22	3100 Utrecht	NL
23	3200 Mexico City	MX
24	3300 Cardiff	UK

The example in the slide shows that the insert via the inline view created a new record in the base table `LOC`.

The following example shows the results of the subquery that was used to identify the table for the `INSERT` statement.

```
SELECT l.location_id, l.city, l.country_id
FROM   loc l
JOIN   countries c
ON(l.country_id = c.country_id)
JOIN   regions USING(region_id)
WHERE region_name = 'Europe';
```



Using the **WITH CHECK OPTION** keyword on DML statements

For Instructor Use Only.

This document should not be distributed.

Using the WITH CHECK OPTION Keyword on DML Statements

The WITH CHECK OPTION keyword prohibits you from changing rows that are not in the subquery.

```
INSERT INTO ( SELECT location_id, city, country_id
              FROM   loc
              WHERE  country_id IN
                    (SELECT country_id
                     FROM   countries
                     NATURAL JOIN regions
                     WHERE  region_name = 'Europe')
              WITH CHECK OPTION )
VALUES (3600, 'Washington', 'US');
```

Error report:

SQL Error: ORA-01402: view WITH CHECK OPTION where-clause violation
01402. 00000 - "view WITH CHECK OPTION where-clause violation"

***Cause:**

***Action:**

When you specify the WITH CHECK OPTION keyword, it indicates that if the subquery is used in place of a table in an INSERT, UPDATE, or DELETE statement, changes are permitted to that table only if those changes will produce rows that are included in the subquery.

The example in the slide shows how to use an inline view with WITH CHECK OPTION. The INSERT statement prevents the creation of records in the LOC table for a city that is not in Europe.

The following example executes successfully because of the changes in the VALUES list.

```
INSERT INTO (SELECT location_id, city, country_id
            FROM   loc
            WHERE  country_id IN
                  (SELECT country_id
                   FROM   countries
                   NATURAL JOIN regions
                   WHERE  region_name = 'Europe')
            WITH CHECK OPTION)
VALUES (3500, 'Berlin', 'DE');
```


The use of an inline view with `WITH CHECK OPTION` provides an easy method to prevent changes to the table.

To prevent the creation of a non-European city, you can also use a database view by performing the following steps:

1. Create a database view:

```
CREATE OR REPLACE VIEW european_cities
AS
SELECT location_id, city, country_id
FROM   locations
WHERE  country_id in
      (SELECT country_id
       FROM countries
       NATURAL JOIN regions
       WHERE region_name = 'Europe')
WITH CHECK OPTION;
```

2. Verify the results by inserting data:

```
INSERT INTO european_cities
VALUES (3400, 'New York', 'US');
```

The second step produces the same error as shown in the slide.



Using Correlated Subqueries to Update and Delete Rows

For Instructor Use Only.

This document should not be distributed.

Correlated UPDATE

- Use a correlated subquery to update rows in one table based on rows from another table.
- Syntax:

```
UPDATE table1 alias1
SET    column = (SELECT expression
                  FROM    table2 alias2
                  WHERE    alias1.column =
                          alias2.column);
```



In the case of the `UPDATE` statement, you can use a correlated subquery to update rows in one table based on rows from another table.

Using Correlated UPDATE

- Denormalize the `EMPL6` table by adding a column to store the department name.
- Populate the table by using a correlated update.

```
ALTER TABLE empl6  
ADD (department_name VARCHAR2(25));
```

Table EMPL6 altered.

```
UPDATE empl6 e  
SET    department_name =  
      (SELECT department_name  
       FROM   departments d  
       WHERE  e.department_id = d.department_id);
```

107 rows updated.

The example in the slide denormalizes the `EMPL6` table by adding a column to store the department name and then populates the table by using a correlated update.

The following block is another example for a correlated update:

Problem Statement

The `REWARDS` table has a list of employees who have exceeded expectations in their performance. Use a correlated subquery to update rows in the `EMPL6` table based on rows from the `REWARDS` table:

```
UPDATE empl6  
  
SET    salary = (SELECT empl6.salary + rewards.pay_raise  
                 FROM   rewards  
                 WHERE  employee_id =  
                        empl6.employee_id  
                 AND   payraise_date =  
                        (SELECT MAX(payraise_date)  
                         FROM   rewards  
                         WHERE  employee_id = empl6.employee_id))  
  
WHERE  empl6.employee_id  
IN     (SELECT employee_id FROM rewards);
```

This example uses the `REWARDS` table. The `REWARDS` table has the following columns: `EMPLOYEE_ID`, `PAY_RAISE`, and `PAYRAISE_DATE`.

Every time an employee gets a pay raise, a record with details such as the employee `ID`, the amount of the pay raise, and the date of receipt of the pay raise is inserted into the `REWARDS` table. The `REWARDS` table can contain more than one record for an employee. The `PAYRAISE_DATE` column is used to identify the most recent pay raise received by an employee.

In the example, the `SALARY` column in the `EMPL6` table is updated to reflect the latest pay raise received by the employee. This is done by adding the current salary of the employee with the corresponding pay raise from the `REWARDS` table.

For Instructor Use Only.
This document should not be distributed.

Correlated DELETE

- Use a correlated subquery to delete rows in one table based on rows from another table.
- Syntax:

```
DELETE FROM table1 alias1
WHERE column operator
      (SELECT expression
       FROM table2 alias2
       WHERE alias1.column = alias2.column);
```

In the case of a `DELETE` statement, you can use a correlated subquery to delete only those rows that also exist in another table.

For example, if you decide that you will maintain only the last four job history records in the `JOB_HISTORY` table, when an employee transfers to a fifth job, you delete the oldest `JOB_HISTORY` row by looking up the `JOB_HISTORY` table for `MIN(START_DATE)` for the employee. The following code illustrates how the preceding operation can be performed by using a correlated `DELETE`:

```
DELETE FROM job_history JH
WHERE employee_id =
      (SELECT employee_id
       FROM employees E
       WHERE JH.employee_id = E.employee_id
       AND START_DATE =
            (SELECT MIN(start_date)
             FROM job_history JH
             WHERE JH.employee_id = E.employee_id)
       AND 5 > (SELECT COUNT(*)
                FROM job_history JH
                WHERE JH.employee_id = E.employee_id
                GROUP BY EMPLOYEE_ID
                HAVING COUNT(*) >= 4));
```

Using Correlated DELETE

Use a correlated subquery to delete only those rows from the `EMPL6` table that also exist in the `EMP_HISTORY` table.

```
DELETE FROM empl6 E
WHERE employee_id =
  (SELECT employee_id
   FROM   employee_history
   WHERE  employee_id = E.employee_id);
```

Two tables are used in this example. They are:

- `EMPL6` table, which provides details of all the current employees
- `EMPLOYEE_HISTORY` table, which provides details of previous employees

`EMPLOYEE_HISTORY` contains data regarding previous employees, so it would be erroneous if the same employee's record existed in both the `EMPL6` and `EMPLOYEE_HISTORY` tables. You can delete such erroneous records by using the correlated subquery shown in the slide.

Summary



Use subqueries to manipulate data

Insert values by using a subquery as a target

Use the `WITH CHECK OPTION` keyword on DML statements

Use correlated subqueries with `UPDATE` and `DELETE` statements

In this lesson, you should have learned how to manipulate data in the Oracle database by using subqueries. You also learned how to use the `WITH CHECK OPTION` keyword on DML statements and use correlated subqueries with `UPDATE` and `DELETE` statements.

For Instructor Use Only.
This document should not be distributed.

Controlling User Access



For Instructor Use Only.

This document should not be distributed.

Objectives



Differentiate system privileges from object privileges

Create a role

Describe object privileges

Revoke an object privilege

In this lesson, you learn how to control database access to specific objects and add new users with different levels of access privileges.

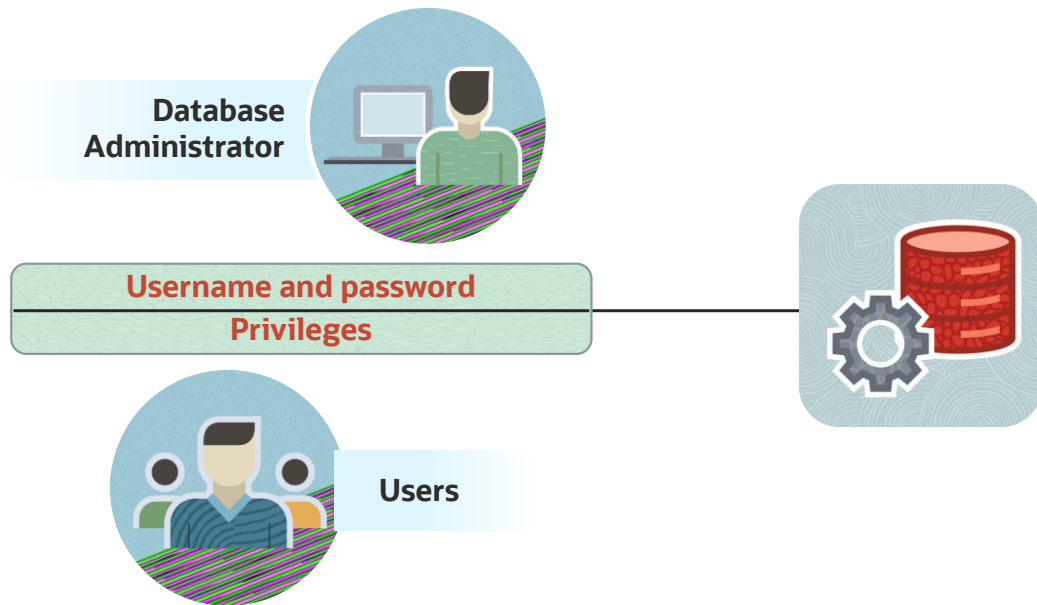


Differentiating System Privileges from Object Privileges

For Instructor Use Only.

This document should not be distributed.

Controlling User Access



In a multiple-user environment, you want to maintain security of database access and use.

With Oracle Server database security, you can do the following:

- Control database access
- Give access to specific objects in the database
- Confirm given and received privileges with the Oracle Data Dictionary

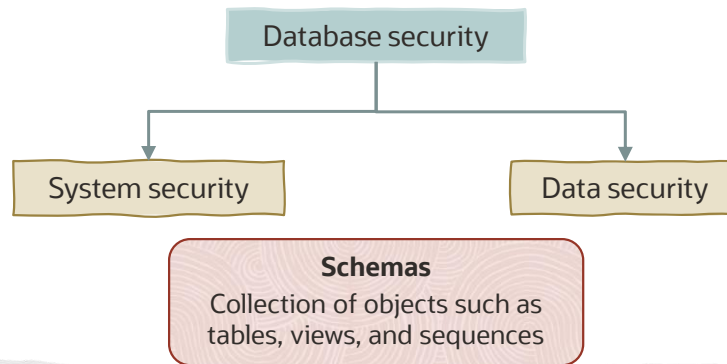
Database security can be classified into two categories:

- **System security** covers access and use of the database at the system level, such as the username and password, the disk space allocated to users, and the system operations that users can perform.
- **Data security** covers access and use of the database objects and the actions that users can perform on the objects.

Privileges

There are two types of privileges:

- System privileges: Performing a particular action within the database
- Object privileges: Manipulating the content of the database objects



What is a **privilege**?

A privilege is the right to execute particular SQL statements.

The Database Administrator (DBA) is a high-level user who can create users and grant users access to the database and its objects.

You as a user will require system privileges to gain access to the database and object privileges to manipulate the content of the objects in the database. You can also be given the privilege to grant additional privileges to other users or to roles, which are named groups of related privileges.

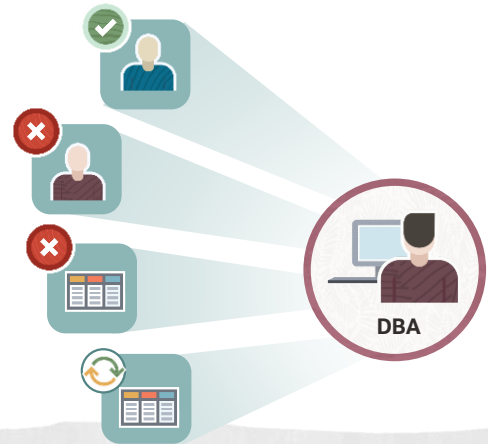
Schemas

A schema is a collection of objects such as tables, views, and sequences. The schema is owned by a database user and has the same name as the user.

A system privilege is the right to perform a particular action, or to perform an action on any schema objects of a particular type, whereas an object privilege provides the user the ability to perform a particular action on a specific schema object.

System Privileges

- More than 200 privileges are available.
- The DBA has high-level system privileges for tasks such as:
 - Creating new users
 - Removing users
 - Removing tables
 - Backing up tables



More than 200 distinct system privileges are available for users and roles. Typically, system privileges are provided by the DBA.

The table `SYSTEM_PRIVILEGE_MAP` contains all the system privileges available, based on the version release. This table is also used to map privilege type numbers to type names.

Typical DBA Privileges

System Privilege	Operations Authorized
CREATE USER	Grantee can create other Oracle users.
DROP USER	Grantee can drop another user.
DROP ANY TABLE	Grantee can drop a table in any schema.
BACKUP ANY TABLE	Grantee can back up any table in any schema with the export utility.
SELECT ANY TABLE	Grantee can query tables, views, or materialized views in any schema.
CREATE ANY TABLE	Grantee can create tables in any schema.

Creating Users

The DBA creates users with the `CREATE USER` statement.

```
CREATE USER user  
IDENTIFIED BY password;
```

```
CREATE USER demo  
IDENTIFIED BY demo;
```

```
User DEMO created.
```

The DBA creates the user by executing the `CREATE USER` statement. The user does not have any privileges at this point. The DBA can then grant privileges to that user. These privileges determine what the user can do at the database level.

The slide gives the abridged syntax for creating a user.

In the syntax:

`user` Is the name of the user to be created
`password` Specifies that the user must log in with this password

For more information, see the *Oracle Database SQL Language Reference* for Oracle Database23c.

Note: Starting with Oracle Database 11g, passwords are case-sensitive.

For Instructor Use Only.
This document should not be distributed.

User System Privileges

- After a user is created, the DBA can grant specific system privileges to that user.

```
GRANT privilege [, privilege...]  
TO user [, user| role, PUBLIC...];
```

- An application developer, for example, may have the following system privileges:
 - CREATE SESSION
 - CREATE TABLE
 - CREATE SEQUENCE
 - CREATE VIEW
 - CREATE PROCEDURE



Typical User Privileges

After the DBA creates a user, the DBA can assign privileges to that user.

System Privilege	Operations Authorized
CREATE SESSION	Connect to the database.
CREATE TABLE	Create tables in the user's schema.
CREATE SEQUENCE	Create a sequence in the user's schema.
CREATE VIEW	Create a view in the user's schema.
CREATE PROCEDURE	Create a stored procedure, function, or package in the user's schema.

In the syntax:

privilege

System privilege to be granted

user | *role* | *PUBLIC*

Name of the user, name of the role, or *PUBLIC*
(which designates that every user is granted the privilege)

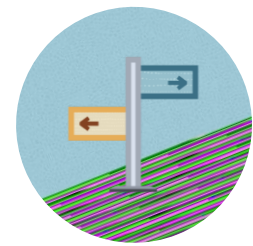
Note: You can find the current system privileges in the *SESSION_PRIVS* dictionary view. As you are already aware, data dictionary is a collection of tables and views created and maintained by the Oracle server. They contain information about the database.

Granting System Privileges

The DBA can grant specific system privileges to a user.

```
GRANT create session, create table,  
      create sequence, create view  
TO    demo;
```

Grant succeeded.



The DBA uses the `GRANT` statement to allocate system privileges to the user. After the user has been granted the privileges, the user can immediately use those privileges.

In the example in the slide, the `demo` user has been assigned the privileges to create sessions, tables, sequences, and views.

For Instructor Use Only.

This document should not be distributed.

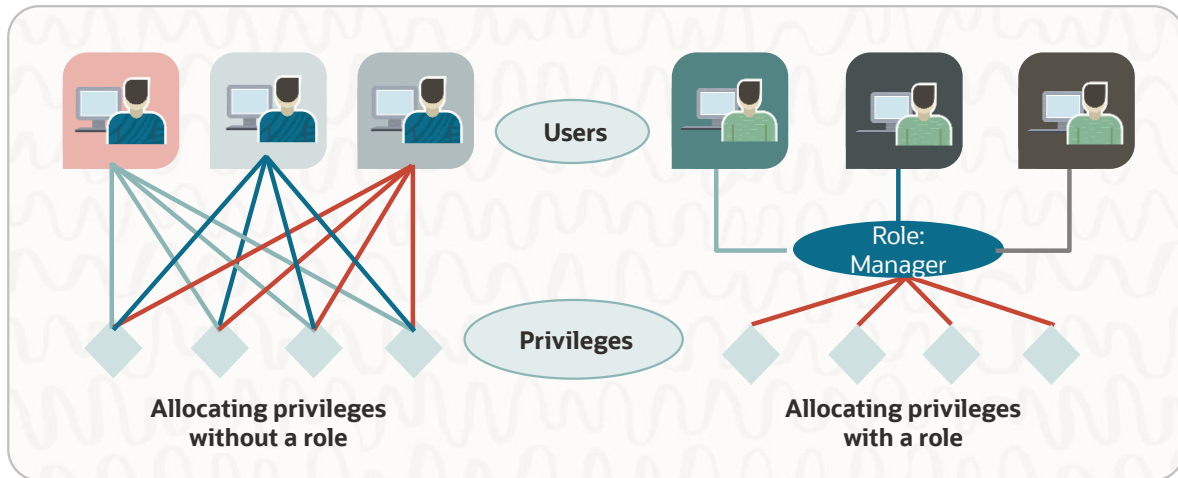


Creating a Role

For Instructor Use Only.

This document should not be distributed.

What is a role?



A role is a named group of related privileges that can be granted to a user. This method makes it easier to revoke and maintain privileges.

A user can have access to several roles, and several users can be assigned the same role. Roles are typically created for a database application.

Creating and Assigning a Role

First, the DBA must create the role. Then the DBA can assign privileges to the role and assign the role to users.

Syntax

```
CREATE ROLE role;
```

In the syntax:

role Name of the role to be created

After the role is created, the DBA can use the `GRANT` statement to assign the role to users as well as assign privileges to the role.

Note: A role is not a schema object; therefore, any user can add privileges to a role.

Creating and Granting Privileges to a Role

- Create a role:

```
CREATE ROLE manager;
```

- Grant privileges to a role:

```
GRANT create table, create view  
TO manager;
```

- Grant a role to users:

```
GRANT manager TO alice;
```



Creating a Role

- The example in the slide creates a `manager` role. The manager is then enabled to create tables and views by granting `create table` and `create view` privileges. It then grants the user `alice` the role of a Manager. Now `alice` can create tables and views.
- If users have multiple roles granted to them, they receive all the privileges associated with all the roles.

Changing Your Password

- The DBA creates your user account and initializes your password.
- User password can be changed by using the `ALTER USER` statement.

```
ALTER USER demo  
IDENTIFIED BY employ;
```



The DBA creates an account and initializes a password for every user. You can change your password by using the `ALTER USER` statement.

The example in the slide shows that the `demo` user changes the password to `employ` by using the `ALTER USER` statement.

Syntax

```
ALTER USER user IDENTIFIED BY password;
```

In the syntax:

<i>user</i>	Is the name of the user
<i>password</i>	Specifies the new password

Although this statement can be used to change your password, there are many other options. Remember that you must have the `ALTER USER` privilege to change any other option.

For more information, see the *Oracle Database SQL Language Reference* for Oracle Database 23c.

Note: SQL*Plus has a `PASSWORD` command (`PASSW`) that can be used to change the password of a user when the user is logged in. This command is not available in Oracle SQL Developer.



Object Privileges

For Instructor Use Only.

This document should not be distributed.

Object Privileges

Object Privilege	Table	View	Sequence
ALTER	✓		✓
DELETE	✓	✓	
INDEX	✓		
INSERT	✓	✓	
REFERENCES	✓		
SELECT	✓	✓	✓
UPDATE	✓	✓	

When an object privilege is given to you, it means that you have a privilege or right to perform a particular action on a specific table, view, sequence, or procedure.

Each object has a particular set of grantable privileges. The table in the slide lists the privileges for various objects. Note that the only privileges that apply to a sequence are `SELECT` and `ALTER`.

`UPDATE`, `REFERENCES`, and `INSERT` can be restricted by specifying a subset of updatable columns.

A `SELECT` privilege can be restricted by creating a view with a subset of columns and granting the `SELECT` privilege only on the view. A privilege granted on a synonym is converted to a privilege on the base table referenced by the synonym.

Note: With the `REFERENCES` privilege, you can ensure that other users can create `FOREIGN KEY` constraints that reference your table.

Object Privileges

- Object privileges vary from object to object.
- An owner:
 - Has all the privileges on the object
 - Can give specific privileges on the objects they own
- Syntax:

```
GRANT      object_priv [(columns)] ALL
ON         object
TO         {user|role|PUBLIC}
[WITH GRANT OPTION];
```

Granting Object Privileges

You can grant different object privileges for different types of schema objects.

A user automatically has all object privileges for schema objects contained in their schema. A user can grant any object privilege on any schema object that they own to any other user or role.

If the grant includes `WITH GRANT OPTION`, the grantee can further grant the object privilege to other users; otherwise, they can use the privilege but cannot grant it to other users.

In the syntax:

<i>object_priv</i>	Is the object privilege to be granted
ALL	Specifies all object privileges
<i>columns</i>	Specifies the column from a table or view on which privileges are granted
ON <i>object</i>	Is the object on which the privileges are granted
TO	Identifies to whom the privilege is granted
PUBLIC	Grants object privileges to all users
WITH GRANT OPTION	Enables the grantee to grant the object privileges to other users and roles

Note: In the syntax, *schema* is the same as the owner's name.

Granting Object Privileges

- Grant query privileges on the `EMPLOYEES` table:

```
GRANT select
ON employees
TO demo;
```

- Grant privileges to update specific columns to users and roles:

```
GRANT update (department name, location id)
ON departments
TO demo, manager;
```

Guidelines

- To grant privileges on an object, the object must be in your own schema, or you must have been granted the object privileges `WITH GRANT OPTION`.
- An object owner can grant any object privilege on the object to any other user or role of the database.
- The owner of an object automatically acquires all object privileges on that object.

The first example grants the `demo` user the privilege to query your `EMPLOYEES` table.

The second example grants `UPDATE` privileges on specific columns in the `DEPARTMENTS` table to `demo` and to the `manager` role.

For example, if your schema is `oraxx`, and the `demo` user now wants to use a `SELECT` statement to obtain data from your `EMPLOYEES` table, the syntax they must use is:

```
SELECT * FROM oraxx.employees;
```

Alternatively, the `demo` user can create a synonym for the table and issue a `SELECT` statement from the synonym:

```
CREATE SYNONYM emp FOR oraxx.employees;

SELECT * FROM emp;
```

Note: DBAs generally allocate system privileges; any user who owns an object can grant object privileges.

Passing On Your Privileges

- Give a user authority to pass along privileges:

```
GRANT  select, insert
ON     departments
TO     demo
WITH GRANT OPTION;
```



- Allow all users on the system to query data from the DEPARTMENTS table:

```
GRANT  select
ON     departments
TO     PUBLIC;
```

WITH GRANT OPTION Keyword

A privilege that is granted with the `WITH GRANT OPTION` clause can be passed on to other users and roles by the grantee. Object privileges granted with the `WITH GRANT OPTION` clause are revoked when the grantor's privilege is revoked.

You can specify `WITH GRANT OPTION` only when granting to a user or to `PUBLIC`, not when granting to a role. The grantor must meet one or more of the following criteria:

- Must be the object owner; otherwise, the grantor must have object access with `GRANT OPTION` from the user
- Must have the `GRANT ANY OBJECT PRIVILEGE` system privilege and an object privilege on the object

The example in the slide gives the `demo` user access to your `DEPARTMENTS` table with the privileges to query the table and add rows to the table. You can also observe that `demo` can give others these privileges.

PUBLIC Keyword

An owner of a table can grant access to all users by using the `PUBLIC` keyword. The second example allows all users on the system to query data from the `DEPARTMENTS` table.

Confirming Granted Privileges

Data Dictionary View	Description
ROLE_SYS_PRIVS	System privileges granted to roles
ROLE_TAB_PRIVS	Table privileges granted to roles
USER_ROLE_PRIVS	Roles accessible by the user
USER_SYS_PRIVS	System privileges granted to the user
USER_TAB_PRIVS_MADE	Object privileges granted on the user's objects
USER_TAB_PRIVS_RECD	Object privileges granted to the user
USER_COL_PRIVS_MADE	Object privileges granted on the columns of the user's objects
USER_COL_PRIVS_RECD	Object privileges granted to the user on specific columns

If you attempt to perform an unauthorized operation, such as deleting a row from a table for which you do not have the `DELETE` privilege, the Oracle server does not permit the operation to take place.

If you receive the Oracle server error message "Table or view does not exist," you have done either of the following:

- Named a table or view that does not exist
- Attempted to perform an operation on a table or view for which you do not have the appropriate privilege

The data dictionary is organized in tables and views and contains information about the database. You can access the data dictionary to view the privileges that you have. The table in the slide describes various data dictionary views.

You learn about data dictionary views in the lesson titled "Introduction to Data Dictionary Views."

Note: The `ALL_TAB_PRIVS_MADE` dictionary view describes all the object grants made by the user or made on the objects owned by the user.



Revoking Object Privileges

For Instructor Use Only.

This document should not be distributed.

Revoking Object Privileges

Syntax:

```
REVOKE {privilege [, privilege...]|ALL}
ON    object
FROM  {user[, user...]|role|PUBLIC}
[CASCADE CONSTRAINTS];
```

You can remove privileges granted to other users by using the `REVOKE` statement.

When you use the `REVOKE` statement, the privileges that you specify are revoked from the users you name and from any other users to whom those privileges were granted by the revoked user.

In the syntax:

`CASCADE` Is required to remove any referential integrity constraints made to the `CONSTRAINTS` object by means of the `REFERENCES` privilege

For more information, see the *Oracle Database SQL Language Reference* for Oracle Database 23c.

Note: If a user leaves the company and you revoke their privileges, you must re-grant any privileges that this user granted to other users. If you drop the user account without revoking privileges from it, the system privileges granted by this user to other users are not affected by this action.

Revoking Object Privileges

Revoke the `SELECT` and `INSERT` privileges

```
REVOKE select, insert
ON      departments
FROM    demo;
```

Revoke succeeded.



The example in the slide revokes `SELECT` and `INSERT` privileges given to the `demo` user on the `DEPARTMENTS` table.

Note: If a user is granted a privilege with the `WITH GRANT OPTION` clause, that user can also grant the privilege with the `WITH GRANT OPTION` clause, so that a long chain of grantees is possible, but no circular grants (granting to a grant ancestor) are permitted. If the owner revokes a privilege from a user who granted the privilege to other users, the revoking cascades to all the privileges granted.

For example, if user `A` grants a `SELECT` privilege on a table to user `B` including the `WITH GRANT OPTION` clause, user `B` can grant to user `C` the `SELECT` privilege with the `WITH GRANT OPTION` clause as well, and user `C` can then grant to user `D` the `SELECT` privilege. If user `A` revokes privileges from user `B`, the privileges granted to users `C` and `D` are also revoked.

Summary



Differentiate system privileges from object privileges

Create a role

Describe object privileges

Revoke an object privilege

DBAs establish initial database security for users by assigning privileges to the users.

- The DBA creates users who must have a password. The DBA is also responsible for establishing the initial system privileges for a user.
- After the user has created an object, the user can pass along any of the available object privileges to other users or to all users by using the `GRANT` statement.
- A DBA can create roles by using the `CREATE ROLE` statement to pass along a collection of system or object privileges to multiple users. Roles make granting and revoking privileges easier to maintain.
- Users can change their passwords by using the `ALTER USER` statement.
- You can remove privileges from users by using the `REVOKE` statement.
- With data dictionary views, users can view the privileges granted to them and those that are granted on their objects.

For Instructor Use Only.
This document should not be distributed.

Manipulating Data Using Advanced Queries



For Instructor Use Only.

This document should not be distributed.

Course Roadmap

Unit 4: Views, Sequences, Synonyms and Indexes

Unit 5: Managing Database Objects and Subqueries

Unit 6: User Access

Unit 7: Advanced Queries



Lesson 19: Manipulating Data Using Advanced Queries

You are here!



Lesson 20: Managing Data in Different Time Zones

In Unit 7, you will learn advanced SQL statements to manipulate data. You will also learn to manage data in different time zones.

Objectives



Specify explicit default values in the `INSERT` and `UPDATE` statements

Describe multitable `INSERTS`

Merge rows in a table

Perform flashback operations

Track the changes made to data over time

In this lesson, you learn how to use the `DEFAULT` keyword in `INSERT` and `UPDATE` statements to identify a default column value. You also learn about multitable `INSERT` statements, the `MERGE` statement, performing flashback operations, and tracking changes in the database.



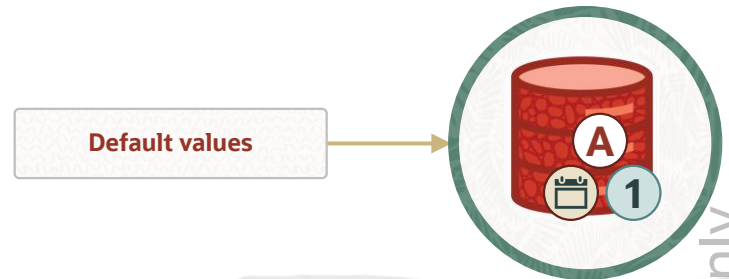
Specifying Explicit Default Values in INSERT and UPDATE Statements

For Instructor Use Only.

This document should not be distributed.

Explicit Default Feature: Overview

- Use the `DEFAULT` keyword as a column value where the default column value is desired.
- This allows the user to control where and when the default value should be applied to data.
- Explicit defaults can be used in `INSERT` and `UPDATE` statements.



You can use the `DEFAULT` keyword in `INSERT` and `UPDATE` statements to identify a default column value. If no default value exists, a null value is used.

The `DEFAULT` option saves you from having to hard-code the default value in your programs or query the dictionary to find it, as was done before this feature was introduced. Hard-coding the default is a problem if the default changes, because the code consequently needs changing. Accessing the dictionary is not usually done in an application; therefore, this is a very important feature.

Using Explicit Default Values

- DEFAULT with INSERT:

```
INSERT INTO deptm3  
  (department_id, department_name, manager_id)  
VALUES (300, 'Engineering', DEFAULT);
```

- DEFAULT with UPDATE:

```
UPDATE deptm3  
SET manager_id = DEFAULT  
WHERE department_id = 10;
```

Specify `DEFAULT` to set the column to the value previously specified as the default value for the column. If no default value for the corresponding column has been specified, the Oracle server sets the column to null.

In the first example in the slide, the `INSERT` statement uses a default value for the `MANAGER_ID` column. If there is no default value defined for the column, a null value is inserted.

The second example uses the `UPDATE` statement to set the `MANAGER_ID` column to a default value for department 10. If no default value is defined for the column, it changes the value to null.

Note: When creating a table, you can specify a default value for a column. This is discussed in the lesson titled “Introduction to Data Definition Language.”

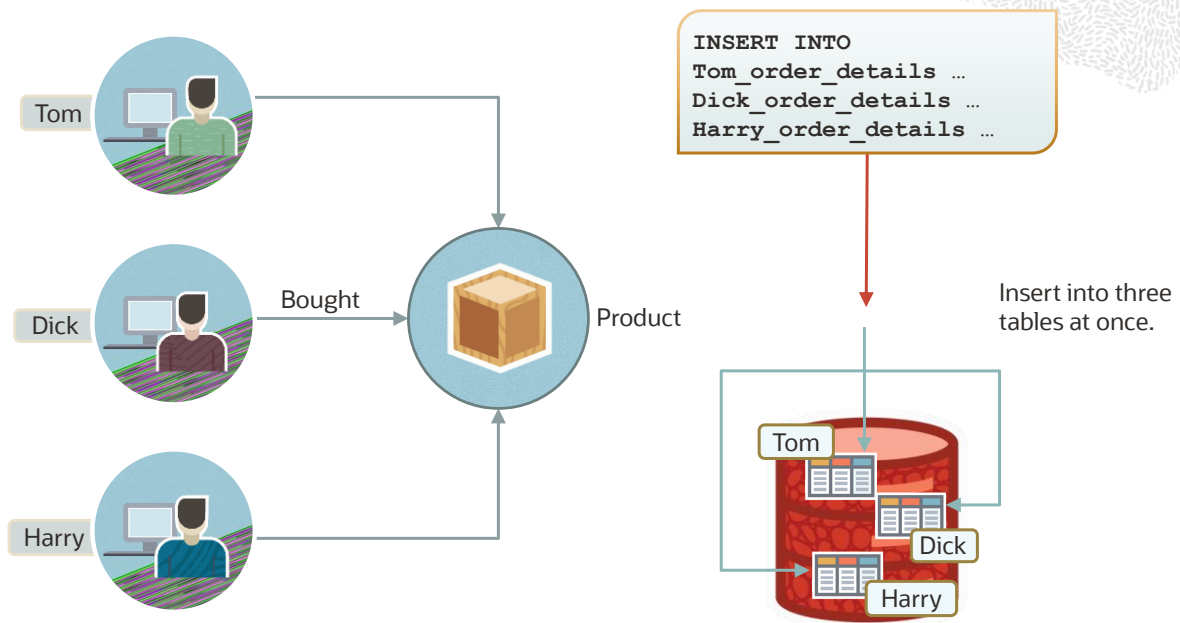


Multitable INSERTs

For Instructor Use Only.

This document should not be distributed.

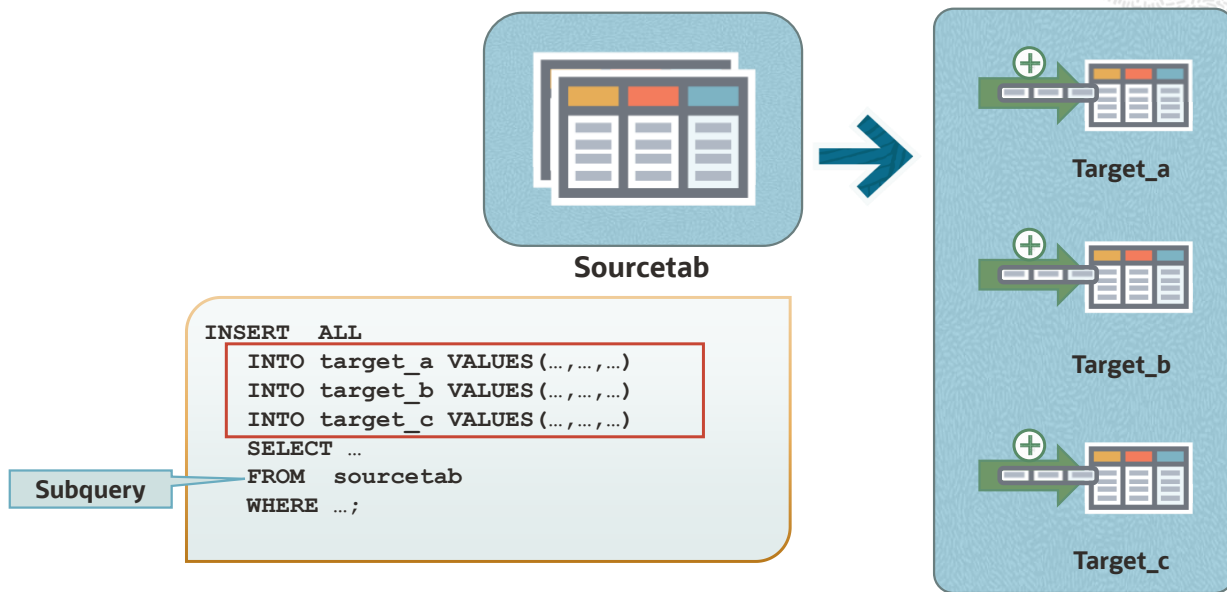
E-Commerce Scenario



Recall the e-commerce company called OracleKart. Imagine Tom, Dick, and Harry are three customers who are shopping simultaneously from different parts of the globe. Coincidentally, the three of them add the same item into their shopping carts. They check out and make the final payment for the product. At this point, an entry has to be made for the order details in each of their customer accounts.

Executing three different `INSERT` statements will hinder the performance of the database. In such a scenario, we can use a multitable `INSERT` statement to simultaneously make an entry in three of the `CUSTOMER` tables.

Multitable INSERT Statements: Overview



In a multitable `INSERT` statement, you insert computed rows into one or more tables. These computed rows are derived from the rows returned from the evaluation of a subquery.

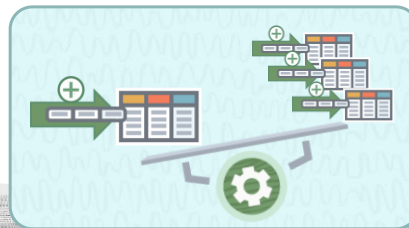
Multitable `INSERT` statements are useful in a data warehouse scenario. You need to load your data warehouse regularly so that it can serve its purpose of facilitating business analysis. To do this, data from one or more operational systems must be extracted and copied into the warehouse. The process of extracting data from the source system and bringing it into the data warehouse is commonly called ETL, which stands for extraction, transformation, and loading.

During extraction, the desired data must be identified and extracted from many different sources, such as database systems and applications. After extraction, the data must be physically transported to the target system or an intermediate system for further processing. Depending on the chosen means of transportation, some transformations can be done during this process. For example, a SQL statement that directly accesses a remote target through a gateway can concatenate two columns as part of the `SELECT` statement.

After data is loaded into the Oracle database, data transformations can be executed by using SQL operations. A multitable `INSERT` statement is one of the techniques for implementing SQL data transformations.

Multitable `INSERT` Statements: Overview

- Use the `INSERT...SELECT` statement to insert rows into multiple tables as a part of a single DML statement.
- Multitable `INSERT` statements are used in data warehousing systems to transfer data from one or more operational sources to a set of target tables.
- They provide significant performance improvement over:
 - Single DML versus multiple `INSERT...SELECT` statements
 - Single DML versus a procedure to perform multiple inserts by using the `IF...THEN` syntax



Multitable `INSERT` statements offer the benefits of the `INSERT...SELECT` statement when multiple tables are involved as targets.

Without multitable `INSERT`, you had to deal with n independent `INSERT...SELECT` statements, thus processing the same source data n times and increasing the transformation workload n times.

As with the existing `INSERT...SELECT` statement, you can use the new statement in parallel with direct load mechanism for faster performance.

Each record from any input stream, such as a nonrelational database table, can now be converted into multiple records for a more relational database table environment. To alternatively implement this functionality, you were required to write multiple `INSERT` statements.

Types of Multitable INSERT Statements

Different types of multitable INSERT statements:

- Unconditional INSERT
- Conditional INSERT ALL
- Conditional INSERT FIRST
- Pivoting INSERT



You use different clauses to indicate the type of INSERT to be executed. The types of multitable INSERT statements are:

- **Unconditional INSERT:** For each row returned by the subquery, a row is inserted into each of the target tables.
- **Conditional INSERT ALL:** For each row returned by the subquery, a row is inserted into each target table if the specified condition is met.
- **Conditional INSERT FIRST:** For each row returned by the subquery, a row is inserted into the very first target table in which the condition is met.
- **Pivoting INSERT:** This is a special case of the unconditional INSERT ALL.

Multitable INSERT Statements

- Syntax for multitable INSERT:

```
INSERT [ ALL
{ insert_into_clause [ values_clause ] }...
| conditional_insert_clause
] subquery
```

- Conditional_insert_clause:

```
INSERT [ ALL | FIRST ]
  WHEN condition THEN insert_into_clause
                        [ values_clause ]
                        [ insert_into_clause [ values_clause ] ]...
  [ ELSE insert_into_clause
                        [ values_clause ]
                        [ insert_into_clause [ values_clause ] ]...
  ]
```

The slide displays the generic format for multitable INSERT statements.

Unconditional INSERT: ALL into_clause

Specify ALL followed by multiple insert_into_clauses to perform an unconditional multitable INSERT. The Oracle Server executes each insert_into_clause once for each row returned by the subquery.

Conditional INSERT: conditional_insert_clause

Specify the conditional_insert_clause to perform a conditional multitable INSERT. The Oracle server filters each insert_into_clause through the corresponding WHEN condition, which determines whether that insert_into_clause is executed. A single multitable INSERT statement can contain up to 127 WHEN clauses.

Conditional INSERT: ALL

If you specify ALL, the Oracle server evaluates each WHEN clause regardless of the results of the evaluation of any other WHEN clause. For each WHEN clause whose condition evaluates to true, the Oracle Server executes the corresponding INTO clause list.

Conditional INSERT: FIRST

If you specify FIRST, the Oracle Server evaluates each WHEN clause in the order in which it appears in the statement. If the first WHEN clause evaluates to true, the Oracle server executes the corresponding INTO clause and skips subsequent WHEN clauses for the given row.

Conditional **INSERT**: **ELSE** Clause

For a given row, if no **WHEN** clause evaluates to true:

- If you have specified an **ELSE** clause, the Oracle server executes the **INTO** clause list associated with the **ELSE** clause
- If you did not specify an **ELSE** clause, the Oracle server takes no action for that row

Restrictions on Multitable **INSERT** Statements

- You can perform multitable **INSERT** statements only on tables, and not on views or materialized views.
- You cannot perform a multitable **INSERT** on a remote table.
- You cannot specify a table collection expression when performing a multitable **INSERT**.
- In a multitable **INSERT**, all `insert_into_clauses` cannot combine to specify more than 999 target columns.

For Instructor Use Only.
This document should not be distributed.

Unconditional INSERT ALL

- Select the `EMPLOYEE_ID`, `HIRE_DATE`, `SALARY`, and `MANAGER_ID` values from the `EMPLOYEES` table for those employees whose `EMPLOYEE_ID` is greater than 200.
- Insert these values into the `SAL_HISTORY` and `MGR_HISTORY` tables by using a multitable `INSERT`.

```
INSERT ALL
  INTO sal_history VALUES (EMPID, HIREDATE, SAL)
  INTO mgr_history VALUES (EMPID, MGR, SAL)
SELECT employee_id EMPID, hire_date HIREDATE,
       salary SAL, manager_id MGR
FROM employees
WHERE employee_id > 200;
```

12 rows inserted



The example in the slide inserts rows into both the `SAL_HISTORY` and the `MGR_HISTORY` tables.

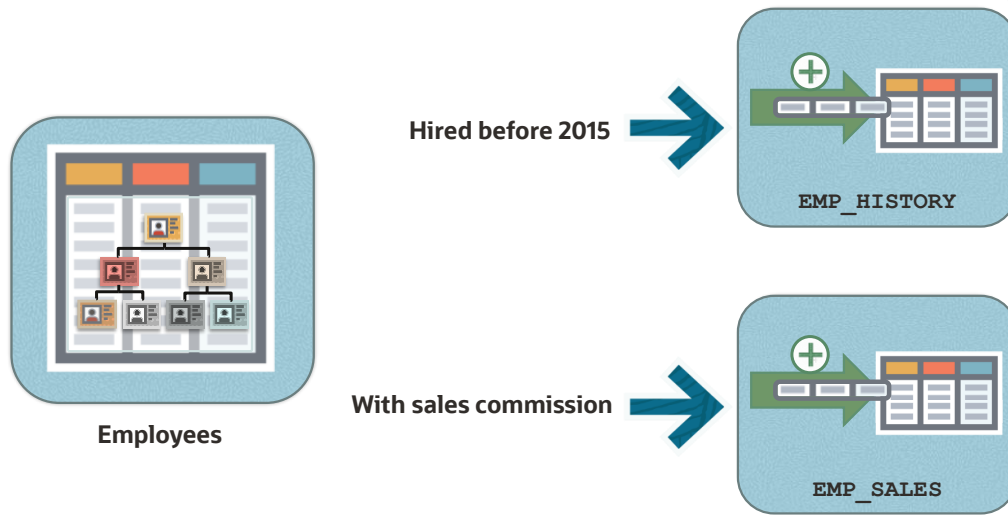
- `SELECT` statement retrieves the details of employee `ID`, hire date, salary, and manager `ID` of those employees whose employee `ID` is greater than 200 from the `EMPLOYEES` table.
- Details of the employee `ID`, hire date, and salary are inserted into the `SAL_HISTORY` table.
- Details of employee `ID`, manager `ID`, and salary are inserted into the `MGR_HISTORY` table.

This `INSERT` statement is referred to as an unconditional `INSERT` because no further restriction is applied to the rows that are retrieved by the `SELECT` statement. All the rows retrieved by the `SELECT` statement are inserted into the two tables: `SAL_HISTORY` and `MGR_HISTORY`. The `VALUES` clause in the `INSERT` statements specifies the columns from the `SELECT` statement that must be inserted into each of the tables. Each row returned by the `SELECT` statement results in two insertions: one for the `SAL_HISTORY` table and one for the `MGR_HISTORY` table.

A total of 12 rows were inserted:

```
SELECT COUNT(*) total_in_sal FROM sal_history;
SELECT COUNT(*) total_in_mgr FROM mgr_history;
```

Conditional INSERT ALL: Example



For all employees in the `employees` tables, if the employee was hired before 2015, insert that employee record into employee history. If the employee earns a sales commission, insert the record information into the `EMP_SALES` table. The SQL statement is shown in the next slide.

Conditional INSERT ALL

```
INSERT ALL
  WHEN HIREDATE < '01-JAN-15' THEN
    INTO emp_history VALUES (EMPID, HIREDATE, SAL)
  WHEN COMM IS NOT NULL THEN
    INTO emp_sales VALUES (EMPID, COMM, SAL)
  SELECT employee_id EMPID, hire_date HIREDATE,
         salary SAL, commission_pct COMM
  FROM employees;
```

112 rows inserted.



The example in the slide is similar to the example in the previous slide because it inserts rows into both EMP_HISTORY and EMP_SALES tables.

The SELECT statement retrieves details such as employee ID, hire date, salary, and commission percentage for all employees from the EMPLOYEES table. Details such as employee ID, hire date, and salary are inserted into the EMP_HISTORY table. Details such as employee ID, commission percentage, and salary are inserted into the EMP_SALES table.

This INSERT statement is referred to as a conditional INSERT ALL because a further restriction is applied to the rows that are retrieved by the SELECT statement. From the rows that are retrieved by the SELECT statement, only those rows in which the hire date was prior to 2015 are inserted in the EMP_HISTORY table. Similarly, only those rows where the value of commission percentage is not null are inserted in the EMP_SALES table.

```
SELECT count(*) FROM emp_history;
```

Result: 77 rows fetched.

```
SELECT count(*) FROM emp_sales;
```

Result: 35 rows fetched.

You can also optionally use the `ELSE` clause with the `INSERT ALL` statement.

Example:

```
INSERT ALL
  WHEN job_id IN
    (select job_id FROM jobs WHERE job_title LIKE '%Manager%') THEN
    INTO managers2(last_name,job_id,SALARY)
    VALUES (last_name,job_id,SALARY)
  WHEN SALARY>10000 THEN
    INTO richpeople(last_name,job_id,SALARY)
    VALUES (last_name,job_id,SALARY)
  ELSE
    INTO poorpeople VALUES (last_name,job_id,SALARY)
SELECT * FROM employees;
```

Result:

```
116 rows inserted
```

For Instructor Use Only.
This document should not be distributed.

Conditional INSERT FIRST: Example

Scenario: If an employee salary is 2,000, the record is inserted into the SAL_LOW table only.



Employees

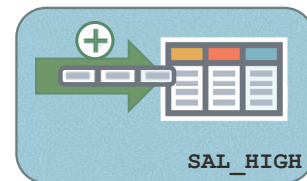
Salary < 5,000



5000 <= Salary <= 10,000



Otherwise

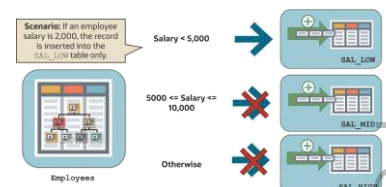


For all employees in the `EMPLOYEES` table, insert the employee information into the first target table that meets the condition. In the example, if an employee has a salary of 2,000, the record is inserted into the `SAL_LOW` table only. The SQL statement is shown in the next slide.

Conditional INSERT FIRST

```
INSERT FIRST
WHEN salary < 5000 THEN
  INTO sal_low VALUES (employee_id, last_name, salary)
WHEN salary between 5000 and 10000 THEN
  INTO sal_mid VALUES (employee_id, last_name, salary)
ELSE
  INTO sal_high VALUES (employee_id, last_name, salary)
SELECT employee_id, last_name, salary
FROM employees;
```

107 rows inserted



The `SELECT` statement retrieves details such as employee ID, last name, and salary for every employee in the `EMPLOYEES` table. For each employee record, it is inserted into the very first target table that meets the condition.

This `INSERT` statement is referred to as a conditional `INSERT FIRST`. The `WHEN salary < 5000` condition is evaluated first. If this first `WHEN` clause evaluates to true, the Oracle server executes the corresponding `INTO` clause and inserts the record into the `SAL_LOW` table. It skips subsequent `WHEN` clauses for this row.

If the row does not satisfy the first `WHEN` condition (`WHEN salary < 5000`), the next condition (`WHEN salary between 5000 and 10000`) is evaluated. If this condition evaluates to true, the record is inserted into the `SAL_MID` table, and the last condition is skipped.

If neither the first condition (`WHEN salary < 5000`) nor the second condition (`WHEN salary between 5000 and 10000`) is evaluated to true, the Oracle Server executes the corresponding `INTO` clause for the `ELSE` clause.

A total of 107 rows were inserted:

```
SELECT count(*) low FROM sal_low;
49 rows fetched.

SELECT count(*) mid FROM sal_mid;
43 rows fetched.

SELECT count(*) high FROM sal_high;
15 rows fetched.
```

Pivoting INSERT

Convert the set of sales records from the nonrelational database table to the relational format.

Emp_ID	Week_ID	MON	TUES	WED	THUR	FRI
176	6	2000	3000	4000	5000	6000



Employee_ID	WEEK	SALES
176	6	2000
176	6	3000
176	6	4000
176	6	5000
176	6	6000

Pivoting is an operation in which you must build a transformation such that each record from any input stream, such as a nonrelational database table, must be converted into multiple records for a more relational database table environment.

Suppose you receive a set of sales records from a nonrelational database table: `SALES_SOURCE_DATA`, in the following format:

```
EMPLOYEE_ID, WEEK_ID, SALES_MON, SALES_TUE, SALES_WED, SALES_THUR,
SALES_FRI
```

You want to store these records in the `SALES_INFO` table in a more typical relational format:

```
EMPLOYEE_ID, WEEK, SALES
```

To solve this problem, you must build a transformation such that each record from the original nonrelational database table, `SALES_SOURCE_DATA`, is converted into five records for the data warehouse's `SALES_INFO` table. This operation is commonly referred to as *pivoting*.

The solution to this problem is shown in the next slide.

Pivoting INSERT

```
INSERT ALL
  INTO sales_info VALUES (employee_id, week_id, sales_MON)
  INTO sales_info VALUES (employee_id, week_id, sales_TUE)
  INTO sales_info VALUES (employee_id, week_id, sales_WED)
  INTO sales_info VALUES (employee_id, week_id, sales_THUR)
  INTO sales_info VALUES (employee_id, week_id, sales_FRI)
SELECT EMPLOYEE_ID, week_id, sales_MON, sales_TUE,
       sales_WED, sales_THUR, sales_FRI
FROM sales_source_data;
```

5 rows inserted

In the example, the sales data is received from the nonrelational database table `SALES_SOURCE_DATA`, which has the details of the sales performed by a sales representative on each day of a week, for a week with a particular week ID.

```
DESC SALES_SOURCE_DATA
```

DESC SALES_SOURCE_DATA	
Name	Null Type
EMPLOYEE_ID	NUMBER(6)
WEEK_ID	NUMBER(2)
SALES_MON	NUMBER(8,2)
SALES_TUE	NUMBER(8,2)
SALES_WED	NUMBER(8,2)
SALES_THUR	NUMBER(8,2)
SALES_FRI	NUMBER(8,2)

```
SELECT * FROM SALES_SOURCE_DATA;
```

	EMPLOYEE_ID	WEEK_ID	SALES_MON	SALES_TUE	SALES_WED	SALES_THUR	SALES_FRI
1	178	6	1750	2200	1500	1500	3000

```
DESC SALES_INFO
```

```
desc sales_info
Name          Null Type
-----
EMPLOYEE_ID    NUMBER(6)
WEEK           NUMBER(2)
SALES          NUMBER(8,2)
```

```
SELECT * FROM sales_info;
```

	EMPLOYEE_ID	WEEK	SALES
1	178	6	1750
2	178	6	2200
3	178	6	1500
4	178	6	1500
5	178	6	3000

Observe in the preceding example that by using a pivoting `INSERT`, one row from the `SALES_SOURCE_DATA` table is converted into five records for the relational table, `SALES_INFO`.



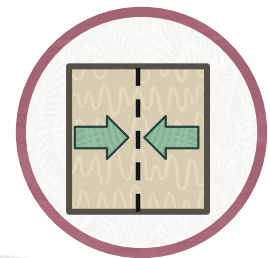
Merging Rows in a Table

For Instructor Use Only.

This document should not be distributed.

MERGE Statement

- Provides the ability to conditionally update, insert, or delete data into a database table
- Performs an `UPDATE` if the row exists and an `INSERT` if it is a new row:
 - Avoids separate updates
 - Increases performance and ease of use
 - Is useful in data warehousing applications



The `MERGE` statement is suitable in a number of data warehousing applications. For example, in a data warehousing application, you may need to work with data coming from multiple sources, some of which may be duplicates. With the `MERGE` statement, you can conditionally add or modify rows.

The Oracle server supports the `MERGE` statement for `INSERT`, `UPDATE`, and `DELETE` operations. By using this statement, you can update, insert, or delete a row conditionally into a table, thus avoiding multiple DML statements. The decision whether to update, insert, or delete into the target table is based on a condition in the `ON` clause.

You must have the `INSERT` and `UPDATE` object privileges on the target table and the `SELECT` object privilege on the source table. To specify the `DELETE` clause of `merge_update_clause`, you must also have the `DELETE` object privilege on the target table.

The `MERGE` statement is deterministic. You cannot update the same row of the target table multiple times in the same `MERGE` statement.

An alternative approach is to use PL/SQL loops and multiple DML statements. The `MERGE` statement, however, is easy to use and more simply expressed as a single SQL statement.

MERGE Statement Syntax

You can conditionally insert, update, or delete rows in a table by using the `MERGE` statement.

```
MERGE INTO table_name table_alias
  USING (table|view|sub_query) alias
  ON (join_condition)
  WHEN MATCHED THEN
    UPDATE SET
      col1 = col1_val,
      col2 = col2_val
  WHEN NOT MATCHED THEN
    INSERT (column_list)
    VALUES (column_values);
```

Merging Rows

You can update existing rows and insert new rows conditionally by using the `MERGE` statement. Using the `MERGE` statement, you can delete obsolete rows at the same time as you update rows in a table. To do this, you include a `DELETE` clause with its own `WHERE` clause in the syntax of the `MERGE` statement.

In the syntax:

<code>INTO</code> clause	Specifies the target table you are updating or inserting into
<code>USING</code> clause	Identifies the source of the data to be updated or inserted; can be a table, view, or subquery
<code>ON</code> clause	Condition on which the <code>MERGE</code> operation either updates or inserts
<code>WHEN MATCHED</code>	Instructs the server how to respond to the results of the join condition
<code>WHEN NOT MATCHED</code>	

Note: For more information, see *Oracle Database SQL Language Reference* for Oracle Database 23c.

Merging Rows: Example

Insert or update rows in the COPY_EMP3 table to match the EMPLOYEES table.

```
MERGE INTO copy_emp3 c
USING (SELECT * FROM EMPLOYEES ) e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = e.first_name,
c.last_name = e.last_name,
...
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)
WHEN NOT MATCHED THEN
INSERT VALUES(e.employee_id, e.first_name, e.last_name,
e.email, e.phone_number, e.hire_date, e.job_id,
e.salary, e.commission_pct, e.manager_id,
e.department_id);
```

107 rows merged.

```
MERGE INTO copy_emp3 c
USING (SELECT * FROM EMPLOYEES ) e
ON (c.employee_id = e.employee_id)
WHEN MATCHED THEN
UPDATE SET
c.first_name = e.first_name,
c.last_name = e.last_name,
c.email = e.email,
c.phone_number = e.phone_number,
c.hire_date = e.hire_date,
c.job_id = e.job_id,
c.salary = e.salary*2,
c.commission_pct = e.commission_pct,
c.manager_id = e.manager_id,
c.department_id = e.department_id
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)
WHEN NOT MATCHED THEN
```

```
INSERT VALUES (e.employee_id, e.first_name, e.last_name,  
e.email, e.phone_number, e.hire_date, e.job_id,  
e.salary, e.commission_pct, e.manager_id,  
e.department_id);
```

The COPY_EMP3 table is created by using the following code:

```
CREATE TABLE COPY_EMP3 AS SELECT * FROM EMPLOYEES  
WHERE SALARY<10000;
```

Then query the COPY_EMP3 table.

```
SELECT employee_id, salary, commission_pct FROM COPY_EMP3;
```

Observe that in the output, there are some employees with SALARY < 10000 and there are two employees with COMMISSION_PCT.

The example in the slide matches the EMPLOYEE_ID in the COPY_EMP3 table to the EMPLOYEE_ID in the EMPLOYEES table. If a match is found, the row in the COPY_EMP3 table is updated to match the row in the EMPLOYEES table and the salary of the employee is doubled. The records of the two employees with values in the COMMISSION_PCT column are deleted. If the match is not found, rows are inserted into the COPY_EMP3 table.

For Instructor Use Only.
This document should not be distributed.

Merging Rows: Example

1

```
TRUNCATE TABLE copy_emp3;  
SELECT * FROM copy_emp3;  
no rows selected
```

2

```
MERGE INTO copy_emp3 c  
USING (SELECT * FROM EMPLOYEES ) e  
ON (c.employee_id = e.employee_id)  
WHEN MATCHED THEN  
UPDATE SET  
  c.first_name = e.first_name,  
  c.last_name = e.last_name,  
  ...  
DELETE WHERE (E.COMMISSION_PCT IS NOT NULL)  
WHEN NOT MATCHED THEN  
INSERT VALUES(e.employee_id, e.first_name, ...
```

3

```
SELECT * FROM copy_emp3;  
107 rows selected.
```

The examples in the slide show that the `COPY_EMP3` table is empty. The `c.employee_id = e.employee_id` condition is evaluated. The condition returns false—there are no matches. The logic falls into the `WHEN NOT MATCHED` clause, and the `MERGE` command inserts the rows of the `EMPLOYEES` table into the `COPY_EMP3` table. This means that the `COPY_EMP3` table now has exactly the same data as in the `EMPLOYEES` table.

```
SELECT employee_id, salary, commission_pct from copy_emp3;
```



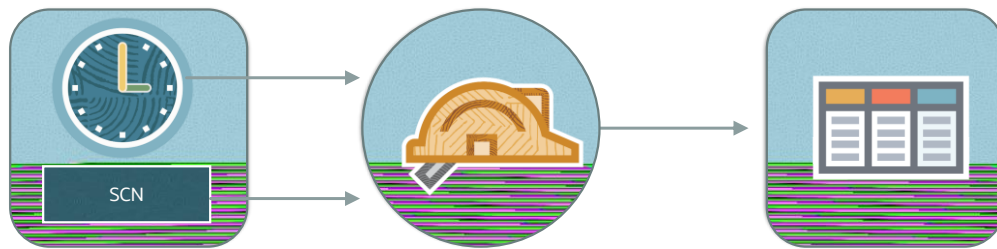
Performing Flashback Operations

For Instructor Use Only.

This document should not be distributed.

FLASHBACK TABLE Statement

- Enables you to recover tables to a specified point in time with a single statement
- Restores table data along with associated indexes and constraints
- Enables you to revert the table and its contents to a certain point in time or system change number (SCN)



Oracle Flashback Table enables you to recover tables to a specified point in time with a single statement. You can restore table data along with associated indexes and constraints while the database is online, undoing changes to only the specified tables.

The Flashback Table feature is similar to a self-service repair tool. For example, if a user accidentally deletes important rows from a table and then wants to recover the deleted rows, you can use the `FLASHBACK TABLE` statement to restore the table to the time before the deletion and see the missing rows in the table.

When using the `FLASHBACK TABLE` statement, you can revert the table and its contents to a certain time or to an SCN.

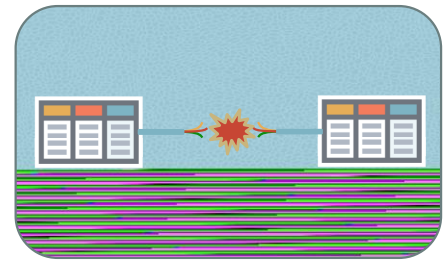
Note: The SCN is an integer value associated with each change to the database. It is a unique incremental number in the database. Every time you commit a transaction, a new SCN is recorded.

FLASHBACK TABLE Statement

- Is a repair tool for accidental table modifications
- Restores a table to an earlier point in time
- Offers ease of use, availability, and fast execution
- Is performed in place

Syntax:

```
FLASHBACK TABLE [ schema. ] table [, [ schema. ] table ]... TO { { {  
SCN | TIMESTAMP } expr | RESTORE POINT restore_point } [ { ENABLE |  
DISABLE } TRIGGERS ] | BEFORE DROP [ RENAME TO table ] } ;
```



Self-Service Repair Facility

You can use the SQL data definition language (DDL) command, `FLASHBACK TABLE`, provided by Oracle Database to restore the state of a table to an earlier point in time, in case it is inadvertently deleted or modified.

The `FLASHBACK TABLE` command is a self-service repair tool to restore data in a table along with associated attributes such as indexes or views. This is done, while the database is online, by rolling back only the subsequent changes to the given table. Compared to traditional recovery mechanisms, this feature offers significant benefits such as ease of use, availability, and faster restoration. It also takes the burden off the DBA to find and restore application-specific properties. The flashback table feature does not address physical corruption caused because of a bad disk.

Syntax

You can invoke a `FLASHBACK TABLE` operation on one or more tables, even on tables in different schemas. You specify the point in time to which you want to revert by providing a valid time stamp. By default, database triggers are disabled during the flashback operation for all tables involved. You can override this default behavior by specifying the `ENABLE TRIGGERS` clause.

Note: For more information about recycle bin and flashback semantics, refer to *Oracle Database Administrator's Guide* for Oracle Database 23c.

Using the FLASHBACK TABLE Statement

1

```
DROP TABLE emp3;
```

Table EMP3 dropped.

2

```
SELECT original_name, operation, droptime FROM recyclebin;
```

ORIGINAL_NAME	OPERATION	DROPTIME
EMP3	DROP	2016-08-26:01:53:10

3

```
FLASHBACK TABLE emp3 TO BEFORE DROP;
```

Flashback succeeded.

Syntax and Examples

The example restores the EMP3 table to a state before a DROP statement.

You can query the recycle bin data dictionary table to fetch information about dropped objects. Dropped tables and any associated objects—such as, indexes, constraints, nested tables, and so on—are not removed and still occupy space. They continue to count against user space quotas until you specifically purge from the recycle bin, or until they must be purged by the database because of tablespace space constraints.

Each user can be thought of as an owner of a recycle bin because, unless a user has the SYSDBA privilege, the only objects that the user has access to in the recycle bin are those that the user owns. A user can view their objects in the recycle bin by using the following statement:

```
SELECT * FROM RECYCLEBIN;
```

When you drop a user, objects belonging to that user are not placed in the recycle bin and those in the recycle bin are purged.

You can purge the recycle bin with the following statement:

```
PURGE RECYCLEBIN;
```

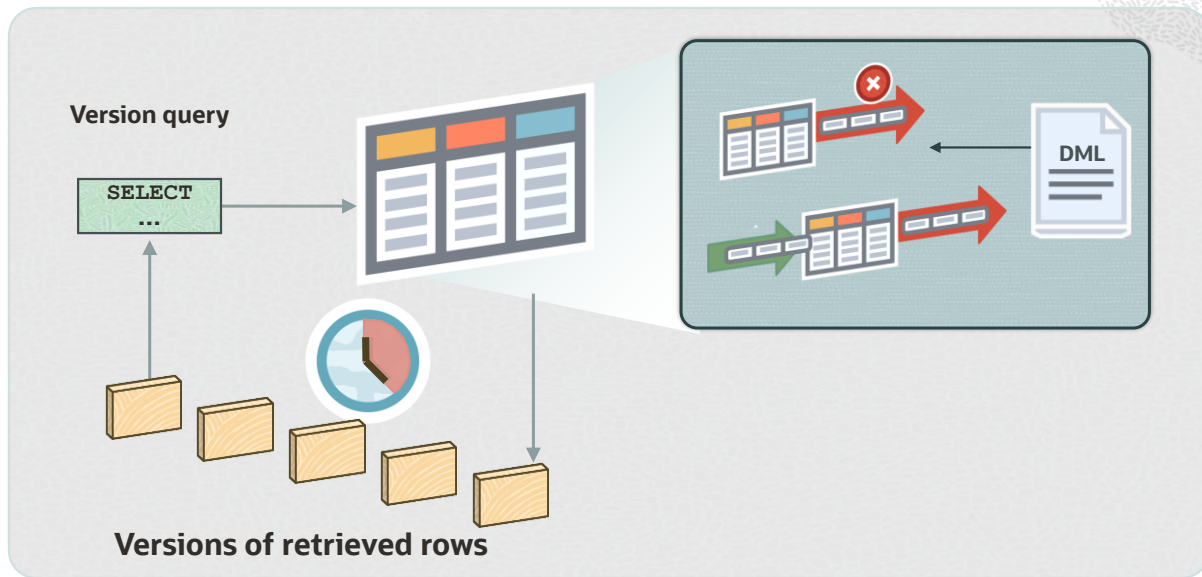


Tracking the Changes in Data Over a Period of Time

For Instructor Use Only.

This document should not be distributed.

Tracking Changes in Data



You may discover that, somehow, data in a table has been inappropriately changed. To research this, you can use multiple flashback queries to view row data at specific points in time. You can use Oracle Flashback Query to retrieve data as it existed at an earlier time.

More efficiently, you can use the Flashback Version Query feature to view all the changes to a row over a period of time. This feature enables you to append a `VERSIONS` clause to a `SELECT` statement that specifies an SCN or the timestamp range within which you want to view changes to row values. The query can also return associated metadata, such as the transaction responsible for the change.

Further, after you identify an erroneous transaction, you can use the Flashback Transaction Query feature to identify other changes that were done by the transaction. You then have the option of using the Flashback Table feature to restore the table to a state before the changes were made.

You can use a query on a table with a `VERSIONS` clause to produce all the versions of all the rows that exist, or ever existed, between the time the query was issued and the `undo_retention` seconds before the current time. `undo_retention` is an initialization parameter, which is an auto-tuned parameter.

A query that includes a `VERSIONS` clause is referred to as a version query. The results of a version query behave as though the `WHERE` clause were applied to the versions of the rows. The version query returns versions of the rows only across transactions.

SCN: The Oracle server assigns an SCN to identify the redo records for each committed transaction.

Flashback Query: Example

```
SELECT salary FROM employees3  
WHERE last_name = 'Chung';
```

1

	SALARY
1	3800

```
UPDATE employees3 SET salary = 4000  
WHERE last_name = 'Chung';
```

```
SELECT salary FROM employees3  
WHERE last_name = 'Chung';
```

2

	SALARY
1	4000

```
SELECT salary FROM employees3  
AS OF TIMESTAMP (SYSTIMESTAMP - INTERVAL '1' MINUTE)  
WHERE last_name = 'Chung';
```

3

	SALARY
1	3800

To use Oracle Flashback Query, use a `SELECT` statement with an `AS OF` clause. Oracle Flashback Query retrieves data as it existed at a particular time in the past. The query explicitly references a past time through a time stamp or SCN. It returns committed data that was current at that point in time.

In the example in the slide:

1. The salary for employee `Chung` is retrieved.
2. The salary for employee `Chung` is increased to 4000.
3. To learn what the value was before the update, you can use the Flashback Query.

Oracle Flashback Query can be used in the following scenarios:

- To recover lost data or to undo incorrect, committed changes. For example, if you mistakenly delete or update rows and then commit them, you can immediately undo the mistake.
- To compare current data with the corresponding data at some time in the past. For example, you can run a daily report that shows the change in data from yesterday. You can compare individual rows of table data or find intersections or unions of sets of rows.
- To check the state of transactional data at a particular time

Flashback Version Query: Example

```
SELECT salary FROM employees3  
WHERE employee_id = 107;
```

1

	SALARY
1	4200

```
UPDATE employees3 SET salary = salary * 1.30  
WHERE employee_id = 107;
```

```
COMMIT;
```

2

```
SELECT salary FROM employees3  
  VERSIONS BETWEEN SCN MINVALUE AND MAXVALUE  
WHERE employee_id = 107;
```

3

	SALARY
1	5460
2	4200

In the example in the slide:

1. The salary for employee 107 is retrieved.
2. The salary for employee 107 is increased by 30 percent and this change is committed.
3. The different versions of salary are displayed.

The `VERSIONS` clause does not change the plan of the query. For example, if you run a query on a table that uses the index access method, the same query on the same table with a `VERSIONS` clause continues to use the index access method.

The versions of the rows returned by the version query are versions of the rows across transactions. The `VERSIONS` clause has no effect on the transactional behavior of a query. This means that a query on a table with a `VERSIONS` clause still inherits the query environment of the ongoing transaction.

The default `VERSIONS` clause can be specified as `VERSIONS BETWEEN {SCN|TIMESTAMP} MINVALUE AND MAXVALUE`. The `VERSIONS` clause is a SQL extension only for queries. You can have DML and DDL operations that use a `VERSIONS` clause within subqueries. The row version query retrieves all the committed versions of the selected rows. Changes made by the current active transaction are not returned. The version query retrieves all incarnations of the rows. This essentially means that versions returned include deleted and subsequent reinserted versions of the rows. The row access for a version query can be defined in one of the following two categories:

- **ROWID-based row access:** In case of `ROWID`-based access, all versions of the specified `ROWID` are returned irrespective of the row content. This essentially means that all versions of the slot in the block indicated by the `ROWID` are returned.
- **All other row access:** For all other row access, all versions of the rows are returned.

VERSIONS BETWEEN Clause

```
SELECT versions_starttime "START_DATE",
       versions_endtime  "END_DATE",
       salary
FROM   employees
       VERSIONS BETWEEN SCN MINVALUE
       AND MAXVALUE
WHERE  last_name = 'Lorentz';
```

	START_DATE	END_DATE	SALARY
1	26-AUG-16 03.00.07.000000000 AM (null)	(null)	5460
2	(null)	26-AUG-16 03.00.07.000000000 AM	4200

You can use the `VERSIONS BETWEEN` clause to retrieve all the versions of the rows that exist or have ever existed between the time the query was issued and a point back in time.

If the undo retention time is less than the lower bound time or the SCN of the `BETWEEN` clause, the query retrieves versions up to the undo retention time only. The time interval of the `BETWEEN` clause can be specified as an SCN interval or a wall-clock interval. This time interval is closed at both the lower and the upper bounds.

In the example, Lorentz's salary changes are retrieved. The `NULL` value for `END_DATE` for the first version indicates that this was the existing version at the time of the query. The `NULL` value for `START_DATE` for the last version indicates that this version was created at a time before the undo retention time.

Note: The `START_DATE` and `END_DATE` values in the screenshot vary according to when the query was executed.

Summary



Specify explicit default values in the `INSERT` and `UPDATE` statements

Describe multitable `INSERTs`

Merge rows in a table

Perform flashback operations

Track the changes in data over a period of time

In this lesson, you also should have learned about multitable `INSERT` statements, the `MERGE` statement, and tracking changes in the database.

For Instructor Use Only.
This document should not be distributed.

Managing Data in Different Time Zones



For Instructor Use Only.

This document should not be distributed.

Objectives



`CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP`

Interval data types

Additional date functions

`SYSDATE` and `SYSTIMESTAMP` data handling

In this lesson, you learn how to use data types similar to `DATE` that store fractional seconds and track time zones. This lesson addresses some of the datetime functions available in the Oracle database.

For Instructor Use Only.
This document should not be distributed.

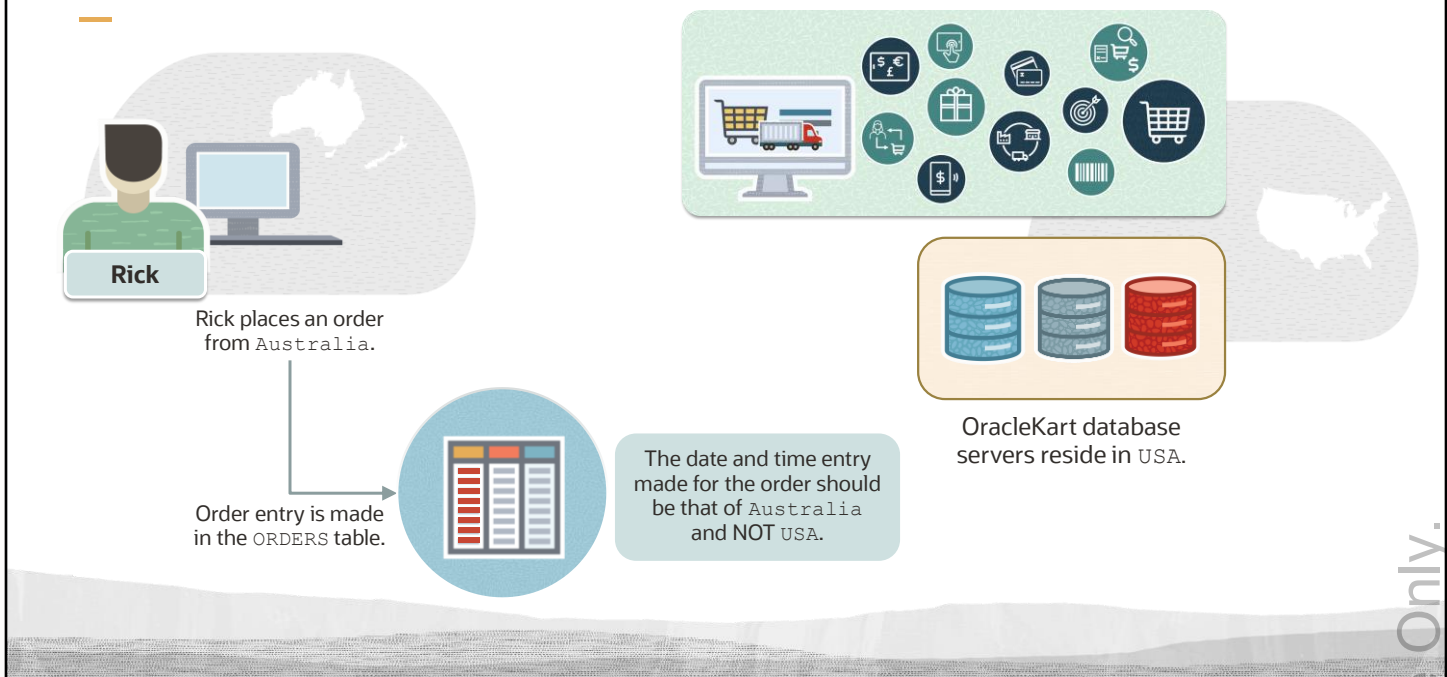


**CURRENT_DATE,
CURRENT_TIMESTAMP, and
LOCALTIMESTAMP**
—

For Instructor Use Only.

This document should not be distributed.

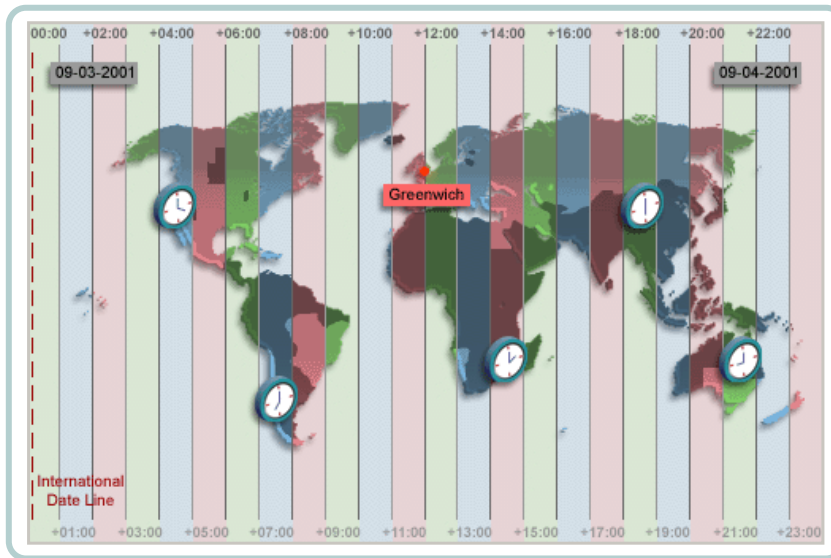
E-Commerce Scenario



Recall the OracleKart e-commerce company that is a global online shopping website. Their data center is located in the US. The customers of this site are spread out all around the globe. When a customer, Rick, who lives in Australia, places an order, the date and time of the order is recorded as per the US time zone. Due to this, the order delivery date is miscalculated, and the order is delayed. How can this be avoided?

Ideally, the order date and time should be the local time of the place from which the customer placed the order (in this case, Australia). Nonetheless, if the time zone is not set, the date and time will be set to that of the server where the database resides. For example, if the database is in the US, then the date and time entry will be in the US time zone. This will cause problems while calculating the delivery time and returns for the customer. Hence, time zones are very useful for the e-commerce industry. Let us learn more about the different time zones and how to use them.

Time Zones



The image represents the time for each time zone when Greenwich time is 12:00.

Let's look into some of the basics of time zones:

The hours of the day are measured by the turning of the earth. The time of day at any particular moment depends on where you are. When it is noon in Greenwich, England, it is midnight along the International Date Line.

The earth is divided into 24 time zones, one for each hour of the day. The time along the prime meridian in Greenwich, England, is known as Greenwich Mean Time (GMT). GMT is now known as Coordinated Universal Time (UTC).

UTC is the time standard against which all other time zones in the world are referenced. It is the same all the year round and is not affected by summertime or daylight-saving time.

The meridian line is an imaginary line that runs from the North Pole to the South Pole. It is known as zero longitude, and it is the line from which all other lines of longitude are measured. All time is measured relative to UTC and all places have a latitude (their distance north or south of the equator) and a longitude (their distance east or west of the Greenwich meridian).

TIME_ZONE Session Parameter

TIME_ZONE may be set to:

- An absolute offset
- Database time zone
- OS local time zone
- A named region

```
ALTER SESSION SET TIME_ZONE = '-05:00';  
ALTER SESSION SET TIME_ZONE = dbtimezone;  
ALTER SESSION SET TIME_ZONE = local;  
ALTER SESSION SET TIME_ZONE = 'America/New_York';
```

The Oracle database supports storing the time zone in your date and time data, as well as fractional seconds.

The ALTER SESSION command can be used to change time zone values in a user's session. The time zone values can be set to an absolute offset, a named time zone, a database time zone, or the local time zone. Observe the examples shown in the slide.

CURRENT_DATE, CURRENT_TIMESTAMP, and LOCALTIMESTAMP

- `CURRENT_DATE`:
 - Returns the current date from the user session
 - Has a data type of `DATE`
- `CURRENT_TIMESTAMP`:
 - Returns the current date and time from the user session
 - Has a data type of `TIMESTAMP WITH TIME ZONE`
- `LOCALTIMESTAMP`:
 - Returns the current date and time from the user session
 - Has a data type of `TIMESTAMP`



The `CURRENT_DATE` and `CURRENT_TIMESTAMP` functions return the current date and current time stamp, respectively. The data type of `CURRENT_DATE` is `DATE`. The data type of `CURRENT_TIMESTAMP` is `TIMESTAMP WITH TIME ZONE`.

The values returned display the time zone displacement of the SQL session executing the functions. Here, the time zone displacement is the difference (in hours and minutes) between local time and UTC. The `TIMESTAMP WITH TIME ZONE` data type has the format:

`TIMESTAMP [ (fractional_seconds_precision) ] WITH TIME ZONE`

where `fractional_seconds_precision` optionally specifies the number of digits in the fractional part of the `SECOND` datetime field and can be a number in the range 0 through 9. The default is 6.

The `LOCALTIMESTAMP` function returns the current date and time in the session time zone. The difference between `LOCALTIMESTAMP` and `CURRENT_TIMESTAMP` is that `LOCALTIMESTAMP` returns a `TIMESTAMP` value, whereas `CURRENT_TIMESTAMP` returns a `TIMESTAMP WITH TIME ZONE` value.

These functions are national language support (NLS)–sensitive—that is, the results will be in the current NLS calendar and datetime formats.

Note: The `SYSDATE` function returns the current date and time as a `DATE` data type. You learned how to use the `SYSDATE` function in the course titled *Oracle Database: SQL Workshop*.

Comparing Date and Time in a Session's Time Zone

The `TIME_ZONE` parameter is set to `-5:00`, and then `SELECT` statements for each date and time are executed to compare differences.

```
ALTER SESSION
SET NLS_DATE_FORMAT = 'DD-MON-YYYY HH24:MI:SS';
ALTER SESSION SET TIME_ZONE = '-5:00';

SELECT SESSIONTIMEZONE, CURRENT_DATE FROM DUAL;

SELECT SESSIONTIMEZONE, CURRENT_TIMESTAMP FROM DUAL;

SELECT SESSIONTIMEZONE, LOCALTIMESTAMP FROM DUAL;
```

1

2

3

The `ALTER SESSION` command sets the date format of the session to `'DD-MON-YYYY HH24:MI:SS'` that is:

day of month (1–31) – abbreviated name of month – 4-digit year hour of day (0–23):minute (0–59):second (0–59).

The example in the slide illustrates that the session is altered to set the `TIME_ZONE` parameter to `-5:00`. Then the `SELECT` statement for `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP` is executed to observe the differences in format.

Note: The `TIME_ZONE` parameter specifies the default local time zone displacement for the current SQL session. `TIME_ZONE` is only a session parameter and not an initialization parameter. The `TIME_ZONE` parameter is set as follows:

```
TIME_ZONE = '[+ | -] hh:mm'
```

The format mask `([+ | -] hh:mm)` indicates the hours and minutes before or after UTC.

Comparing Date and Time in a Session's Time Zone

Results of queries:

Session altered.

SESSIONTIMEZONE	CURRENT_DATE
1 -05:00	28-AUG-2016 18:14:49

SESSIONTIMEZONE	CURRENT_TIMESTAMP
1 -05:00	28-AUG-16 06.15.59.648595000 PM -05:00

SESSIONTIMEZONE	LOCALTIMESTAMP
1 -05:00	28-AUG-16 06.16.59.026930000 PM

Observe the results of the queries run from the previous slide:

1. The `CURRENT_DATE` function returns the current date in the session's time zone.
2. The `CURRENT_TIMESTAMP` function returns the current date and time in the session's time zone as a value of the data type `TIMESTAMP WITH TIME ZONE`.
3. The `LOCALTIMESTAMP` function returns the current date and time in the session's time zone.

Note: The code example output may vary depending on when the command is run.

DBTIMEZONE and SESSIONTIMEZONE

- Display the value of the database time zone:

```
SELECT DBTIMEZONE [FROM DUAL];
```

	DBTIMEZONE
1	+00:00

- Display the value of the session's time zone:

```
SELECT SESSIONTIMEZONE [FROM DUAL];
```

	SESSIONTIMEZONE
1	-05:00

If you are the DBA, you can set the database's default time zone by specifying the `SET TIME_ZONE` clause of the `CREATE DATABASE` statement. If you omit, the default database time zone is the operating system time zone. Note that the database time zone cannot be changed for a session with an `ALTER SESSION` statement.

The `DBTIMEZONE` function returns the value of the database time zone. The return type is a time zone offset (a character type in the format: '`[+|-] TZh:TzM`') or a time zone region name, depending on how the user specified the database time zone value in the most recent `CREATE DATABASE` or `ALTER DATABASE` statement. The example in the slide shows that the database time zone is set to "+00:00," as the `TIME_ZONE` parameter is in the format:

```
TIME_ZONE = '[+ | -] hh:mm'
```

The `SESSIONTIMEZONE` function returns the value of the current session's time zone. The return type is a time zone offset (a character type in the format '`[+|-] TZh:TzM`') or a time zone region name, depending on how the user specified the session time zone value in the most recent `ALTER SESSION` statement. The example in the slide shows that the session time zone is offset to UTC by '-5:00' hours.

Observe that the database time zone is different from the current session's time zone.

TIMESTAMP Data Types

Data Type	Fields
TIMESTAMP	Year, Month, Day, Hour, Minute, Second with fractional seconds
TIMESTAMP WITH TIME ZONE	Same as the TIMESTAMP data type; also includes: <code>TIMEZONE_HOUR</code> , and <code>TIMEZONE_MINUTE</code> or <code>TIMEZONE_REGION</code>
TIMESTAMP WITH LOCAL TIME ZONE	Same as the TIMESTAMP data type; also includes a time zone offset in its value

The **TIMESTAMP** data type is an extension of the **DATE** data type.

TIMESTAMP (fractional_seconds_precision)

This data type contains the year, month, and day values of date, as well as hour, minute, and second values of time, where `fractional_seconds_precision` is the number of digits in the fractional part of the **SECOND** datetime field. The accepted values of significant `fractional_seconds_precision` are 0 through 9. The default is 6.

TIMESTAMP (fractional_seconds_precision) WITH TIME ZONE

This data type contains all values of **TIMESTAMP** as well as time zone displacement value.

TIMESTAMP (fractional_seconds_precision) WITH LOCAL TIME ZONE

This data type contains all values of **TIMESTAMP**, with the following exceptions:

- Data is normalized to the database time zone when it is stored in the database.
- When the data is retrieved, users see the data in the session time zone.

TIMESTAMP Fields

Datetime Field	Valid Values
YEAR	–4712 to 9999 (excluding year 0)
MONTH	01 to 12
DAY	01 to 31
HOURL	00 to 23
MINUTE	00 to 59
SECOND	00 to 59.9(N) where 9(N) is precision
TIMEZONE_HOUR	–12 to 14
TIMEZONE_MINUTE	00 to 59

Each datetime data type is composed of several of these fields. You can mutually compare datetimes and assign them only if they have the same datetime fields.

Difference Between DATE and TIMESTAMP

A

```
-- when hire_date is of  
type DATE
```

```
SELECT hire_date  
FROM emp4;
```

	HIRE_DATE
1	17-JUN-11
2	21-SEP-09
3	13-JAN-09
4	03-JAN-14
5	21-MAY-15
6	25-JUN-13
7	05-FEB-14
8	07-FEB-15
9	17-AUG-10
10	16-AUG-10

...

B

```
ALTER TABLE emp4  
MODIFY hire_date TIMESTAMP;
```

```
SELECT hire_date  
FROM emp4;
```

	HIRE_DATE
1	17-JUN-11 12.00.00.000000000 AM
2	21-SEP-09 12.00.00.000000000 AM
3	13-JAN-09 12.00.00.000000000 AM
4	03-JAN-14 12.00.00.000000000 AM
5	21-MAY-15 12.00.00.000000000 AM
6	25-JUN-13 12.00.00.000000000 AM
7	05-FEB-14 12.00.00.000000000 AM
8	07-FEB-15 12.00.00.000000000 AM
9	17-AUG-10 12.00.00.000000000 AM
10	16-AUG-10 12.00.00.000000000 AM

...

TIMESTAMP Data Type: Example

Example A shows the data from the hire_date column of the EMP4 table when the data type of the column is DATE.

In example B, the table is altered and the data type of the hire_date column is made into TIMESTAMP. The output shows the differences in display. You can convert from DATE to TIMESTAMP when the column has data, but you cannot convert from DATE or TIMESTAMP to TIMESTAMP WITH TIME ZONE unless the column is blank.

You can specify the fractional seconds precision for time stamp. If none is specified, as in this example, it defaults to 6.

For example, the following statement sets the fractional seconds precision as 7:

```
ALTER TABLE emp4  
MODIFY hire_date TIMESTAMP(7);
```

The Oracle DATE data type by default looks like what is shown in this example. However, the date data type also contains additional information such as hours, minutes, seconds, AM, and PM. To obtain the date in this format, you can apply a format mask or a function to the date value.

Note: Remember to change the NLS_DATE_FORMAT to 'DD-MON-YY' before running the queries in the slide.

Comparing TIMESTAMP Data Types

```
CREATE TABLE web_orders  
  (order_date TIMESTAMP WITH TIME ZONE,  
   delivery_time TIMESTAMP WITH LOCAL TIME ZONE);
```

```
INSERT INTO web_orders values  
  (current_date, current_timestamp + 2);
```

```
SELECT * FROM web_orders;
```

	ORDER_DATE	DELIVERY_TIME
1	28-AUG-16 06.49.27.000000000 PM -05:00	30-AUG-16 06.49.27.000000000 PM

In the example in the slide, a new table `web_orders` is created with a column of data type `TIMESTAMP WITH TIME ZONE` and a column of data type `TIMESTAMP WITH LOCAL TIME ZONE`.

This table is populated whenever a `web_order` is placed. The time stamp and time zone for the user placing the order are inserted based on the `CURRENT_DATE` value. The delivery time is populated by inserting two days from the `CURRENT_TIMESTAMP` value into it every time an order is placed. When a web-based company guarantees shipping, it can estimate its delivery time based on the time zone of the person placing the order.

Note: The code example output may vary based on the time when the command is run.



INTERVAL Data Types

For Instructor Use Only.

This document should not be distributed.

INTERVAL Data Types

- The data type `INTERVAL` can store the difference between two datetime values.
- There are two classes of intervals:
 - Year-month
 - Day-time
- The precision of the interval is:
 - The actual subset of fields that constitutes an interval
 - Specified in the interval qualifier

Data Type	Fields
<code>INTERVAL YEAR TO MONTH</code>	Year, Month
<code>INTERVAL DAY TO SECOND</code>	Days, Hour, Minute, Second with fractional seconds

`INTERVAL` data types are used to store the difference between two datetime values. There are two classes of intervals: year-month intervals and day-time intervals.

A year-month interval is made up of a contiguous subset of fields of `YEAR` and `MONTH`, whereas a day-time interval is made up of a contiguous subset of fields consisting of `DAY`, `HOUR`, `MINUTE`, and `SECOND`. The actual subset of fields that constitute an interval is called the precision of the interval and is specified in the interval qualifier. Because the number of days in a year is calendar-dependent, the year-month interval is NLS-dependent, whereas day-time interval is NLS-independent.

The interval qualifier contains a leading field and may contain a trailing field. In case the trailing field is `SECOND`, it may also specify the fractional seconds precision, which is the number of digits in the fractional part of the `SECOND` value. If not specified, the default value for leading field precision is 2 digits and the default value for fractional seconds precision is 6 digits.

`INTERVAL YEAR (year_precision) TO MONTH`

This data type stores a period of time in years and months, where `year_precision` is the number of digits in the `YEAR` datetime field. The accepted values are 0 through 9. The default is 6.

`INTERVAL DAY (day_precision) TO SECOND (fractional_seconds_precision)`

This data type stores a period of time in days, hours, minutes, and seconds, where `day_precision` is the maximum number of digits in the `DAY` datetime field (accepted values are 0 through 9; the default is 2) and `fractional_seconds_precision` is the number of digits in the fractional part of the `SECOND` field (accepted values are 0 through 9; the default is 6).

INTERVAL Fields

INTERVAL Field	Valid Values for Interval
YEAR	Any positive or negative integer
MONTH	01 to 11
DAY	Any positive or negative integer
HOURL	00 to 23
MINUTE	00 to 59
SECOND	00 to 59.9(N) where 9(N) is precision

INTERVAL YEAR TO MONTH can have fields of YEAR and MONTH.

INTERVAL DAY TO SECOND can have fields of DAY, HOUR, MINUTE, and SECOND.

The actual subset of fields that constitute an item of either type of interval is defined by an interval qualifier, and this subset is known as the precision of the item.

Year-month intervals are mutually comparable and assignable only with other year-month intervals. Day-time intervals are mutually comparable and assignable only with other day-time intervals.

INTERVAL YEAR TO MONTH: Example

```
CREATE TABLE warranty
(prod_id number, warranty_time INTERVAL YEAR(3) TO MONTH);
INSERT INTO warranty VALUES (123, INTERVAL '8' MONTH);
INSERT INTO warranty VALUES (155, INTERVAL '200' YEAR(3));
INSERT INTO warranty VALUES (678, '200-11');
SELECT * FROM warranty;
```

	PROD_ID	WARRANTY_TIME
1	123	0-8
2	155	200-0
3	678	200-11

INTERVAL YEAR TO MONTH stores a period of time by using the YEAR and MONTH datetime fields. Specify INTERVAL YEAR TO MONTH as follows:

```
INTERVAL YEAR [(year_precision)] TO MONTH
```

where `year_precision` is the number of digits in the YEAR datetime field. The default value of `year_precision` is 2.

Restriction: The leading field must be more significant than the trailing field. For example, INTERVAL '0-1' MONTH TO YEAR is not valid.

Examples:

- INTERVAL '123-2' YEAR(3) TO MONTH
Indicates an interval of 123 years, 2 months
- INTERVAL '123' YEAR(3)
Indicates an interval of 123 years, 0 months
- INTERVAL '300' MONTH(3)
Indicates an interval of 300 months
- INTERVAL '123' YEAR
Returns an error because the default precision is 2, and '123' has 3

The Oracle database supports two interval data types: `INTERVAL YEAR TO MONTH` and `INTERVAL DAY TO SECOND`. For interval literals, the system also recognizes other American National Standards Institute (ANSI) interval types, such as `INTERVAL '2' YEAR` or `INTERVAL '10' HOUR`. In these cases, each interval is converted to one of the two supported types.

In the example in the slide, a `WARRANTY` table is created, which contains a `warranty_time` column that takes the `INTERVAL YEAR(3) TO MONTH` data type. Different values are inserted into it to indicate years and months for various products. When these rows are retrieved from the table, you see a year value separated from the month value by a (-).

For Instructor Use Only.
This document should not be distributed.

INTERVAL DAY TO SECOND Data Type: Example

```
CREATE TABLE lab  
( exp_id number, test_time INTERVAL DAY(2) TO SECOND);  
  
INSERT INTO lab VALUES (100012, '90 00:00:00');  
INSERT INTO lab VALUES (56098,  
INTERVAL '6 03:30:16' DAY TO SECOND);
```

```
SELECT * FROM lab;
```

	EXP_ID	TEST_TIME
1	10001290	0:0:0.0
2	560986	3:30:16.0

In the example in the slide:

- You create the `lab` table with a `test_time` column of the `INTERVAL DAY TO SECOND` data type.
- You then insert into it the value `'90 00:00:00'` to indicate 90 days and 0 hours, 0 minutes, and 0 seconds.
- You insert another row with the value `INTERVAL '6 03:30:16' DAY TO SECOND` to indicate 6 days, 3 hours, 30 minutes, and 16 seconds.
- The `SELECT` statement shows how this data is displayed in the database.



Additional Date Functions

For Instructor Use Only.

This document should not be distributed.

EXTRACT

- Display all employees who were hired after 2007.

```
SELECT last_name, employee_id, hire_date
FROM employees
WHERE EXTRACT(YEAR FROM TO_DATE(hire_date, 'DD-MON-RR')) > 2007
ORDER BY hire_date;
```

- Display the MONTH component from the HIRE_DATE for those employees whose MANAGER_ID is 100.

```
SELECT last_name, hire_date,
       EXTRACT (MONTH FROM HIRE_DATE)
FROM employees
WHERE manager_id = 100;
```

LAST_NAME	HIRE_DATE	EXTRACT(MONTH FROM HIRE_DATE)
1 Kochhar	21-SEP-09	9
2 De Haan	13-JAN-09	1
3 Raphaely	07-DEC-10	12
4 Weiss	18-JUL-12	7
5 Fripp	10-APR-13	4
...		

The EXTRACT expression extracts and returns the value of a specified datetime field from a datetime or interval value expression. You can extract any of the components mentioned in the following syntax by using the EXTRACT function. The syntax of the EXTRACT function is:

```
SELECT EXTRACT( { YEAR | MONTH | DAY | HOUR | MINUTE | SECOND
                | TIMEZONE_HOUR
                | TIMEZONE_MINUTE
                | TIMEZONE_REGION
                | TIMEZONE_ABBR }
FROM { expr } )
```

When you extract a TIMEZONE_REGION or TIMEZONE_ABBR (abbreviation), the value returned is a string containing the appropriate time zone name or abbreviation. When you extract any of the other values, the value returned is a date in the Gregorian calendar. When extracting from a datetime with a time zone value, the value returned is in UTC.

In the first example in the slide, the EXTRACT function is used to select all employees who were hired after 2007.

In the second example, the EXTRACT function is used to extract the MONTH from the HIRE_DATE column of the EMPLOYEES table for those employees who report to the manager whose EMPLOYEE_ID is 100.

TZ_OFFSET

Display the time zone offset for the 'US/Eastern', 'Canada/Yukon', and 'Europe/London' time zones:

```
SELECT TZ_OFFSET('US/Eastern'),  
       TZ_OFFSET('Canada/Yukon'),  
       TZ_OFFSET('Europe/London')  
FROM DUAL;
```

	TZ_OFFSET('US/EASTERN')	TZ_OFFSET('CANADA/YUKON')	TZ_OFFSET('EUROPE/LONDON')
1	-04:00	-07:00	+01:00

The `TZ_OFFSET` function returns the time zone offset corresponding to the value entered. The return value is dependent on the time when the statement is executed. For example, if the `TZ_OFFSET` function returns a value `-08:00`, this value indicates that the time zone for which the command was executed is eight hours behind UTC. You can enter a valid time zone name, a time zone offset from UTC (which simply returns itself), or the keyword `SESSIONTIMEZONE` or `DBTIMEZONE`. The syntax of the `TZ_OFFSET` function is:

```
TZ_OFFSET({ 'time_zone_name' | '{ + | - } hh : mi'  
          | SESSIONTIMEZONE  
          | DBTIMEZONE  
          })
```

The Fold Motor Company has its headquarters in Michigan, USA, which is in the US/Eastern time zone. The company president, Mr. Fold, wants to conduct a conference call with the vice president of the Canadian operations and the vice president of the European operations, who are in the Canada/Yukon and Europe/London time zones, respectively. Mr. Fold wants to find out the time in each of these places to make sure that his senior management will be available to attend the meeting. His secretary, Mr. Scott, helps by issuing the queries shown in the example and gets the following results:

- The 'US/Eastern' time zone is four hours behind UTC.
- The 'Canada/Yukon' time zone is seven hours behind UTC.
- The 'Europe/London' time zone is one hour ahead of UTC.

For a listing of valid time zone name values, you can query the V\$TIMEZONE_NAMES dynamic performance view.

```
SELECT * FROM V$TIMEZONE_NAMES;
```

	AZ TZNAME	AZ TZABBREV	AZ CON_ID
1	Africa/Abidjan	LMT	0
2	Africa/Abidjan	GMT	0
3	Africa/Accra	LMT	0
4	Africa/Accra	GMT	0
5	Africa/Accra	GHST	0

...

FROM_TZ

Display the `TIMESTAMP` value '2000-07-12 08:00:00' as a `TIMESTAMP WITH TIME ZONE` value for the 'Australia/North' time zone region.

```
SELECT FROM_TZ(TIMESTAMP
               '2000-07-12 08:00:00', 'Australia/North')
FROM DUAL;
```

```
FROM_TZ(TIMESTAMP'2000-07-12 08:00:00','AUSTRALIA/NORTH')
1 12-JUL-00 08.00.00.000000000 AM AUSTRALIA/NORTH
```

The `FROM_TZ` function converts a `TIMESTAMP` value to a `TIMESTAMP WITH TIME ZONE` value.

The syntax of the `FROM_TZ` function is as follows:

FROM_TZ(timestamp_value, time_zone_value)

where `time_zone_value` is a character string in the format 'TZH:TZM' or a character expression that returns a string in `TZR` (time zone region) with an optional `TZD` format. `TZD` is an abbreviated time zone string with daylight saving information. `TZR` represents the time zone region in datetime input strings. Examples are 'Australia/North', 'PST' for US/Pacific standard time, 'PDT' for US/Pacific daylight time, and so on.

The example in the slide converts a `TIMESTAMP` value to `TIMESTAMP WITH TIME ZONE`.

Note: To see a listing of valid values for the `TZR` and `TZD` format elements, query the `V$TIMEZONE_NAMES` dynamic performance view.

TO_TIMESTAMP

Display the character string '2016-03-06 11:00:00' as a `TIMESTAMP` value:

```
SELECT TO_TIMESTAMP ('2016-03-06 11:00:00',  
                     'YYYY-MM-DD HH:MI:SS')  
FROM DUAL;
```

```
TO_TIMESTAMP('2016-03-0611:00:00','YYYY-MM-DDHH:MI:SS')  
1 06-MAR-16 11.00.00.000000000 AM
```

The `TO_TIMESTAMP` function converts a string of `CHAR`, `VARCHAR2`, `NCHAR`, or `NVARCHAR2` data type to a value of the `TIMESTAMP` data type. The syntax of the `TO_TIMESTAMP` function is:

```
TO_TIMESTAMP(char [, fmt [, 'nlsparam' ] ])
```

The optional `fmt` specifies the format of `char`. If you omit `fmt`, the string must be in the default format of the `TIMESTAMP` data type. The optional `nlsparam` specifies the language in which month and day names, and abbreviations, are returned. This argument can have this form:

```
'NLS_DATE_LANGUAGE = language'
```

If you omit `nlsparams`, this function uses the default date language for your session.

The example in the slide converts a character string to a value of `TIMESTAMP`.

Note: You use the `TO_TIMESTAMP_TZ` function to convert a string of `CHAR`, `VARCHAR2`, `NCHAR`, or `NVARCHAR2` data type to a value of the `TIMESTAMP WITH TIME ZONE` data type. For more information about this function, see *Oracle Database SQL Language Reference* for Oracle Database 23c.

TO_YMINTERVAL

Display a date that is one year and two months after the hire date for the employees working in the department with the `DEPARTMENT_ID` 20.

```
SELECT hire_date,  
       hire_date + TO_YMINTERVAL('01-02') AS  
       HIRE DATE YMININTERVAL  
FROM   employees  
WHERE  department_id = 20;
```

	HIRE_DATE	HIRE_DATE_YMININTERVAL
1	17-FEB-12	17-APR-13
2	17-AUG-13	17-OCT-14

The `TO_YMINTERVAL` function converts a character string of `CHAR`, `VARCHAR2`, `NCHAR`, or `NVARCHAR2` data type to an `INTERVAL YEAR TO MONTH` data type. The `INTERVAL YEAR TO MONTH` data type stores a period of time using the `YEAR` and `MONTH` datetime fields. The format of `INTERVAL YEAR TO MONTH` is as follows:

`INTERVAL YEAR [(year_precision)] TO MONTH`

where `year_precision` is the number of digits in the `YEAR` datetime field. The default value of `year_precision` is 2.

The syntax of the `TO_YMINTERVAL` function is:

`TO_YMINTERVAL (char)`

where `char` is the character string to be converted.

The example in the slide calculates a date that is one year and two months after the hire date for the employees working in the department with the `DEPARTMENT_ID` 20 of the `EMPLOYEES` table.

TO_DSINTERVAL

Display a date that is 100 days and 10 hours after the hire date for all the employees.

```
SELECT last_name,  
       TO_CHAR(hire_date, 'mm-dd-yy:hh:mi:ss') hire_date,  
       TO_CHAR(hire_date +  
               TO_DSINTERVAL('100 10:00:00'),  
               'mm-dd-yy:hh:mi:ss') hiredate2  
FROM employees;
```

	LAST_NAME	HIRE_DATE	HIREDATE2
1	King	06-17-11:12:00:00	09-25-11:10:00:00
2	Kochhar	09-21-09:12:00:00	12-30-09:10:00:00
3	De Haan	01-13-09:12:00:00	04-23-09:10:00:00
4	Hunold	01-03-14:12:00:00	04-13-14:10:00:00
5	Ernst	05-21-15:12:00:00	08-29-15:10:00:00
6	Austin	06-25-13:12:00:00	10-03-13:10:00:00

...

TO_DSINTERVAL converts a character string of the CHAR, VARCHAR2, NCHAR, or NVARCHAR2 data type to an INTERVAL DAY TO SECOND data type.

In the example in the slide, the date 100 days and 10 hours after the hire date is obtained.

Daylight Saving Time (DST)

- Start of Daylight Saving:
 - Time jumps from 01:59:59 AM to 03:00:00 AM.
 - Values from 02:00:00 AM to 02:59:59 AM are not valid.
- End of Daylight Saving:
 - Time jumps back from 02:00:00 AM to 01:00:01 AM.
 - Values from 01:00:01 AM to 02:00:00 AM are ambiguous because they are visited twice.



Most Western nations advance the clock ahead one hour during the summer months. This period is called daylight saving time (DST). The DST lasts from the start of Daylight Saving to the end of Daylight Saving in most of the US, Mexico, and Canada. The nations of the European Union observe DST, but they call it the summertime period. Europe's summertime period begins a week earlier than its North American counterpart but ends at the same time.

The Oracle database automatically determines, for any given time zone region, whether the DST is in effect and returns local time values accordingly. The datetime value is sufficient for the Oracle database to determine whether daylight saving time is in effect for a given region in all cases except boundary cases.

A boundary case occurs during the period when the DST goes into or out of effect. For example, in the US/Eastern region, when DST goes into effect, the time changes from 01:59:59 AM to 03:00:00 AM. The one-hour interval between 02:00:00 AM and 02:59:59 AM does not exist. When daylight saving time goes out of effect, the time changes from 02:00:00 AM back to 01:00:01 AM, and the one-hour interval between 01:00:01 AM and 02:00:00 AM is repeated.

ERROR_ON_OVERLAP_TIME

The `ERROR_ON_OVERLAP_TIME` is a session parameter to notify the system to issue an error when it encounters a datetime that occurs in the overlapped period and no time zone abbreviation was specified to distinguish the period.

For example, the DST ends on October 31, at 02:00:01 AM. The overlapped periods are:

- 10/31/2016 01:00:01 AM to 10/31/2016 02:00:00 AM (EDT)
- 10/31/2016 01:00:01 AM to 10/31/2016 02:00:00 AM (EST)

If you input a datetime string that occurs in one of these two periods, you need to specify the time zone abbreviation (for example, EDT or EST) in the input string for the system to determine the period.

Without this time zone abbreviation, the system does the following:

If the `ERROR_ON_OVERLAP_TIME` parameter is `FALSE`, it assumes that the input time is standard time (for example, EST). Otherwise, an error is raised.

For Instructor Use Only.
This document should not be distributed.



SYSDATE and SYSTIMESTAMP Data Handling

For Instructor Use Only.

This document should not be distributed.

Lesson Agenda

- `CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP`
- `INTERVAL` data types
- Using the following functions:
 - `EXTRACT`
 - `TZ_OFFSET`
 - `FROM_TZ`
 - `TO_TIMESTAMP`
 - `TO_YMINTERVAL`
 - `TO_DSINTERVAL`
- `SYSDATE` and `SYSTIMESTAMP` data handling

This section discusses the following functions:

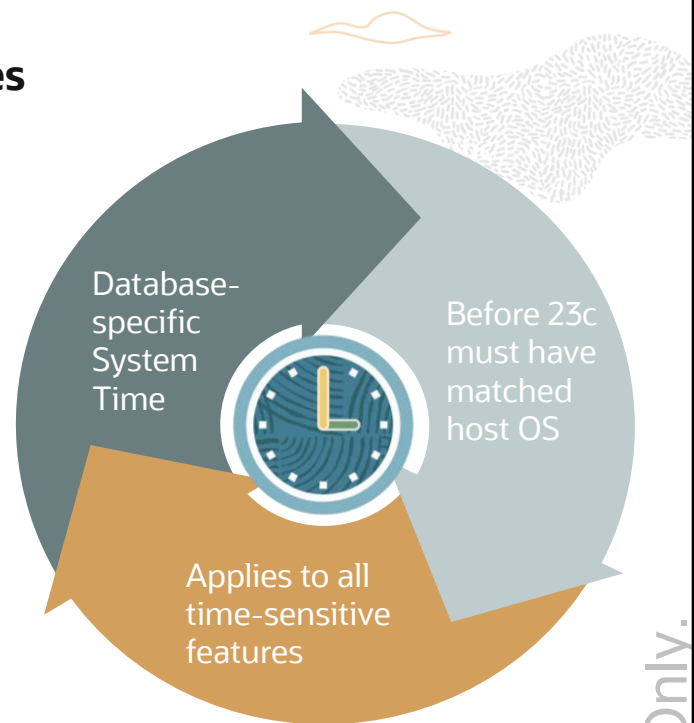
- `EXTRACT`
- `TZ_OFFSET`
- `FROM_TZ`
- `TO_TIMESTAMP`
- `TO_YMINTERVAL`
- `TO_DSINTERVAL`

Date and Time in Oracle Databases

System time can be set as local time zone of an individual database.

System time before DB 23C has always matched time zone of database host OS.

The “system time” applies to scheduler, flashback, materialized view refresh, RMAN, and other features.

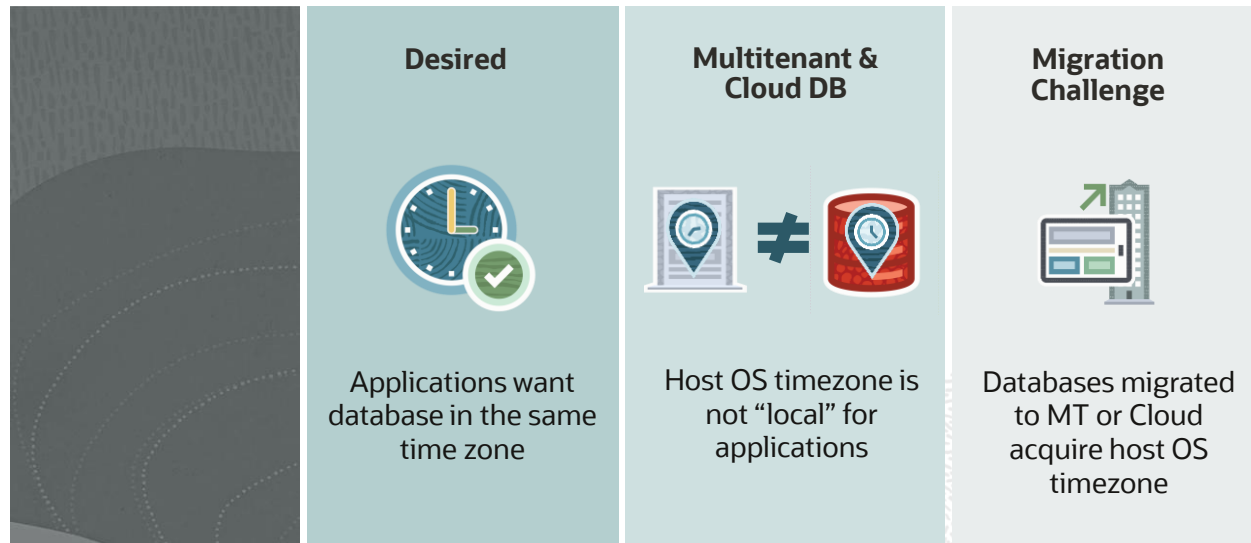


The “system time” of a database, visible as SYSDATE and SYSTIMESTAMP, can now be set as the local time zone of an individual database.

Prior to 23c, SYSDATE and SYSTIMESTAMP were always based upon the time zone of OS for the database host.

The “system time” is used not only for SYSDATE and SYSTIMESTAMP, but also for scheduler, flashback, materialize view refresh, RMAN, and other features.

Database Time Versus OS System Time



Many databases prefer to use a local time zone based upon where the database application is used and developed, rather than the local time zone where the database server is hosted.

- In the cloud and in multitenant environments, a database may often be hosted on a server in a different time zone from the customer's "local" time zone.

Databases migrated from on-premises to the cloud want to keep the same local time zone, even though the cloud database may be hosted in a different time zone.

Database-Specific Time in Oracle Database 23c



Any Cloud DB can set local time zone to customer's requirements.



Any Pluggable DB can set local time zone to customer's requirements.



Simplified migration to Multitenant or Cloud for applications.



More intuitive database development and monitoring

Any database using Oracle Database cloud services can now set a local time zone based upon the customer's requirements.

Any database that has been consolidated in a multitenant environment can now set a local time zone based upon the customer's requirements.

Simpler migrations to the cloud or to multitenant environments

- Because SYSDATE can now be set based on the customer time zone, no changes are necessary for applications that have hard-code dependencies on SYSDATE.

More intuitive database development and monitoring

- SYSDATE and other date-related settings consistently use local time zone.

Database Administrator Configures Database-Specific Time



Set the desired time zone for the database: **before 23c**

```
ALTER DATABASE SET TIME_ZONE='-05:00';
```

Enable SYSDATE/SYSTIMESTAMP to use database time zone: **23c and later**

```
ALTER SYSTEM SET CURRENT_TIME_AT_DBTIMEZONE = TRUE;
```

Set the desired time zone for the database (prior to 23c functionality):

```
ALTER DATABASE SET TIME_ZONE='-05:00';
```

Enable SYSDATE/SYSTIMESTAMP to use database time zone (new parameter):

```
ALTER SYSTEM SET CURRENT_TIME_AT_DBTIMEZONE = TRUE;
```

Summary



`CURRENT_DATE`, `CURRENT_TIMESTAMP`, and `LOCALTIMESTAMP`

Interval data types

Additional date functions

`SYSDATE` and `SYSTIMESTAMP` data handling

This lesson addressed some of the datetime functions available in the Oracle Database.

For Instructor Use Only.
This document should not be distributed.

Table Descriptions



For Instructor Use Only.

This document should not be distributed.

Schema Description

Overall Description

The Oracle Database sample schemas portray a sample company that operates worldwide to fill orders for different products. The company has three divisions:

- **Human Resources:** Tracks information about employees and facilities
- **Order Entry:** Tracks product inventories and sales through various channels
- **Sales History:** Tracks business statistics to facilitate business decisions

Each of these divisions is represented by a schema. In this course, you have access to the objects in all the schemas. However, the emphasis of the examples, demonstrations, and practices is on the `Human Resources (HR)` schema.

All scripts necessary to create the sample schemas reside in the `$ORACLE_HOME/demo/schema/` folder.

Human Resources (HR)

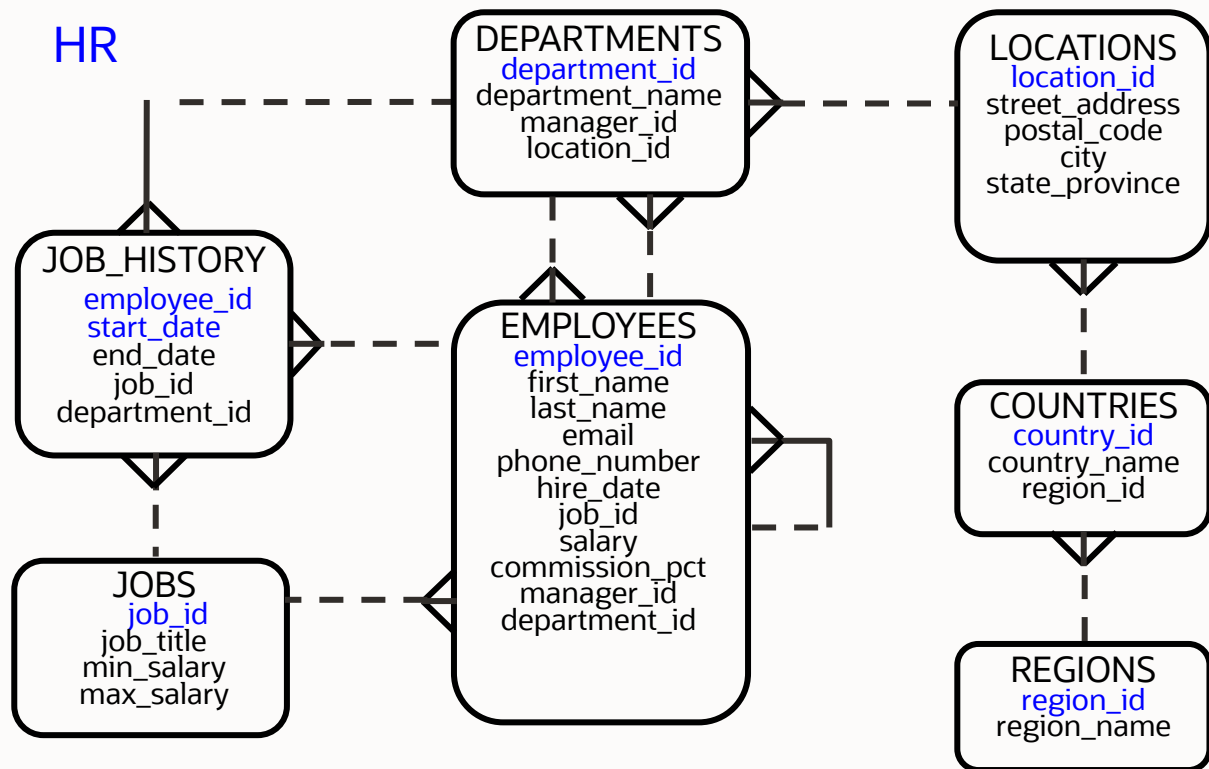
This is the schema that is used in this course. In the Human Resource (HR) records, each employee has an identification number, email address, job identification code, salary, and manager. Some employees earn commissions in addition to their salary.

The company also tracks information about the jobs within the organization. Each job has an identification code, job title, and a minimum and maximum salary range for the job. Some employees have been with the company for a long time and have held different positions within the company. When an employee resigns, the duration the employee was working for, the job identification number, and the department are recorded.

The sample company is regionally diverse, so it tracks the locations of its warehouses and departments. Each employee is assigned to a department, and each department is identified either by a unique department number or a short name. Each department is associated with one location, and each location has a full address that includes the street name, postal code, city, state or province, and the country code.

In places where the departments and warehouses are located, the company records details such as the country name, currency symbol, currency name, and the region where the country is located geographically.

HR Entity Relationship Diagram



For Instructor Use Only.
This document should not be distributed.

Human Resources (HR) Table Descriptions

- DESCRIBE countries

Name	Null	Type
COUNTRY_ID	NOT NULL	CHAR(2)
COUNTRY_NAME		VARCHAR2(40)
REGION_ID		NUMBER

- SELECT * FROM countries

	COUNTRY_ID	COUNTRY_NAME	REGION_ID
1	CA	Canada	2
2	DE	Germany	1
3	UK	United Kingdom	1
4	US	United States of America	2

- DESCRIBE departments

Name	Null	Type
DEPARTMENT_ID	NOT NULL	NUMBER(4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2(30)
MANAGER_ID		NUMBER(6)
LOCATION_ID		NUMBER(4)

- SELECT * FROM departments

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	(null)	1700

- DESCRIBE employees

Name	Null	Type
-----	-----	-----
EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)

- SELECT * FROM employees

	EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
1	100	Steven	King	SKING	515.123.4567	17-JUN-11	AD_PRES	24000	(null)	(null)	90
2	101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-09	AD_VP	17000	(null)	100	90
3	102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-09	AD_VP	17000	(null)	100	90
4	103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-14	AC_MGR	12008	(null)	102	60
5	104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-15	IT_PROG	6000	(null)	103	60
6	107	Diana	Lorentz	DLORENTZ	590.423.5567	07-FEB-15	IT_PROG	4200	(null)	103	60
7	124	Kevin	Mourgos	KMOURGOS	650.123.5234	16-NOV-15	ST_MAN	5800	(null)	100	50
8	141	Trenna	Rajs	TRAJS	650.121.8009	17-OCT-11	ST_CLERK	3500	(null)	124	50
9	142	Curtis	Davies	CDAVIES	650.121.2994	29-JAN-13	ST_CLERK	3100	(null)	124	50
10	143	Randall	Matos	RMATOS	650.121.2874	15-MAR-14	ST_CLERK	2600	(null)	124	50
11	144	Peter	Vargas	PVARGAS	650.121.2004	09-JUL-14	ST_CLERK	2500	(null)	124	50
12	149	Eleni	Zlotkey	EZLOTKEY	011.44.1344.429018	29-JAN-16	SA_MAN	10500	0.2	100	80
13	174	Ellen	Abel	EABEL	011.44.1644.429267	11-MAY-12	SA_REP	11000	0.3	149	80
14	176	Jonathon	Taylor	JTAYLOR	011.44.1644.429265	24-MAR-14	SA_REP	8600	0.2	149	80
15	178	Kimberely	Grant	KGRANT	011.44.1644.429263	24-MAY-15	SA_REP	7000	0.15	149	(null)
16	200	Jennifer	Whalen	JWHALEN	515.123.4444	17-SEP-11	AD_ASST	4400	(null)	101	10
17	201	Michael	Hartstein	MHARTSTE	515.123.5555	17-FEB-12	MK_MAN	13000	(null)	100	20
18	202	Pat	Fay	PFAY	603.123.6666	17-AUG-13	MK_REP	6000	(null)	201	20
19	205	Shelley	Higgins	SHIGGINS	515.123.8080	07-JUN-10	AC_MGR	12008	(null)	101	110
20	206	William	Gietz	WGIEZT	515.123.8181	07-JUN-10	AC_ACCOUNT	8300	(null)	205	80

- DESCRIBE job_history

Name	Null	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
START_DATE	NOT NULL	DATE
END_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
DEPARTMENT_ID		NUMBER(4)

- SELECT * FROM job_history

	EMPLOYEE_ID	START_DATE	END_DATE	JOB_ID	DEPARTMENT_ID
1	102	13-JAN-09	24-JUL-14	IT_PROG	60
2	101	21-SEP-09	27-OCT-16	AC_ACCOUNT	110
3	101	28-OCT-09	15-MAR-13	AC_MGR	110
4	201	17-FEB-12	19-DEC-15	MK_REP	20
5	114	24-MAR-14	31-DEC-15	ST_CLERK	50
6	122	01-JAN-15	31-DEC-15	ST_CLERK	50
7	200	17-SEP-95	17-JUN-09	AD_ASST	90
8	176	24-MAR-14	31-DEC-14	SA_REP	80
9	176	01-JAN-15	31-DEC-15	SA_MAN	80
10	200	01-JUL-10	31-DEC-14	AC_ACCOUNT	90

- DESCRIBE jobs

Name	Null	Type
-----	-----	-----
JOB_ID	NOT NULL	VARCHAR2(10)
JOB_TITLE	NOT NULL	VARCHAR2(35)
MIN_SALARY		NUMBER(6)
MAX_SALARY		NUMBER(6)

- SELECT * FROM jobs

	 JOB_ID	 JOB_TITLE	 MIN_SALARY	 MAX_SALARY
1	AD_PRES	President	20080	40000
2	AD_VP	Administration Vice President	15000	30000
3	AD_ASST	Administration Assistant	3000	6000
4	AC_MGR	Accounting Manager	8200	16000
5	AC_ACCOUNT	Public Accountant	4200	9000
6	SA_MAN	Sales Manager	10000	20080
7	SA_REP	Sales Representative	6000	12008
8	ST_MAN	Stock Manager	5500	8500
9	ST_CLERK	Stock Clerk	2008	5000
10	IT_PROG	Programmer	4000	10000
11	MK_MAN	Marketing Manager	9000	15000
12	MK_REP	Marketing Representative	4000	9000

- DESCRIBE locations

Name	Null	Type
LOCATION_ID	NOT NULL	NUMBER(4)
STREET_ADDRESS		VARCHAR2(40)
POSTAL_CODE		VARCHAR2(12)
CITY	NOT NULL	VARCHAR2(30)
STATE_PROVINCE		VARCHAR2(25)
COUNTRY_ID		CHAR(2)

- SELECT * FROM locations

	LOCATION_ID	STREET_ADDRESS	POSTAL_CODE	CITY	STATE_PROVINCE	COUNTRY_ID
1	1400	2014 Jabbawocky Rd	26192	Southlake	Texas	US
2	1500	2011 Interiors Blvd	99236	South San Francisco	California	US
3	1700	2004 Charade Rd	98199	Seattle	Washington	US
4	1800	460 Bloor St. W.	ON M5S 1X8	Toronto	Ontario	CA
5	2500	Magdalen Centre, The Oxford Science Park	OX9 9ZB	Oxford	Oxford	UK

- DESCRIBE regions

Name	Null	Type
-----	-----	-----
REGION_ID	NOT NULL	NUMBER
REGION_NAME		VARCHAR2(25)

- SELECT * FROM regions

	REGION_ID	REGION_NAME
1	1	Europe
2	2	Americas
3	3	Asia
4	4	Middle East and Africa

Using SQL Developer



For Instructor Use Only.

This document should not be distributed.

Objectives



List the key features of Oracle SQL Developer

Identify the menu items of Oracle SQL Developer

Create a database connection

Manage database objects

Use SQL Worksheet

Save and run SQL scripts

In this appendix, you are introduced to the graphical tool called SQL Developer. You learn how to use SQL Developer for your database development tasks. You learn how to use SQL Worksheet to execute SQL statements and SQL scripts.

Objectives



Create and save reports

Browse the data modeling options in SQL Developer

For Instructor Use Only.
This document should not be distributed.

What is Oracle SQL Developer?

- Oracle SQL Developer is a graphical tool that enhances productivity and simplifies database development tasks.
- You can connect to any target Oracle database schema by using standard Oracle Database authentication.



Oracle SQL Developer is a free graphical tool designed to improve your productivity and simplify the development of everyday database tasks. With just a few clicks, you can easily create and debug stored procedures, test SQL statements, and view optimizer plans.

SQL Developer, which is the visual tool for database development, simplifies the following tasks:

- Browsing and managing database objects
- Executing SQL statements and scripts
- Editing and debugging PL/SQL statements
- Creating reports

You can connect to any target Oracle Database schema by using standard Oracle Database authentication. When connected, you can perform operations on objects in the database.

SQL Developer is the interface to administer the Oracle Application Express Listener. The new interface enables you to specify global settings and multiple database settings with different database connections for the Application Express Listener. SQL Developer provides the option to drag objects by table or column name to the worksheet. It provides improved DB Diff comparison options, `GRANT` statement support in the SQL editor, and DB Doc reporting. Additionally, SQL Developer includes support for Oracle Database 23c features.

Specifications of SQL Developer

SQL Developer:

- Is shipped along with Oracle Database 23c Release 2
- Is developed in Java
- Supports Windows, Linux, and Mac OS X platforms
- Enables default connectivity using the JDBC Thin driver
- Connects to Oracle Database version 9.2.0.1 and later



SQL Developer is developed in Java, leveraging the Oracle JDeveloper integrated development environment (IDE). Therefore, it is a cross-platform tool. The tool runs on Windows, Linux, and Mac operating system (OS) X platforms.

The default connectivity to the database is through the Java Database Connectivity (JDBC) Thin driver, and therefore, no Oracle Home is required. SQL Developer does not require an installer, and you need to simply unzip the downloaded file. With SQL Developer, users can connect to Oracle Databases 9.2.0.1 and later and all Oracle database editions, including Express Edition.

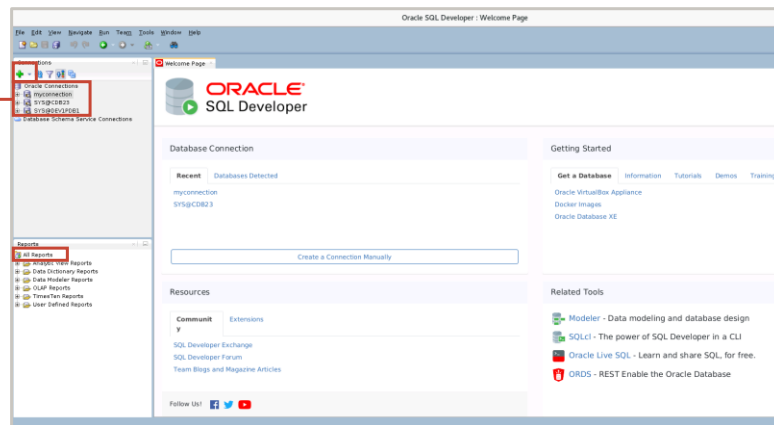
Note: For the latest version of SQL Developer, you will have to download and install SQL Developer. SQL Developer is freely downloadable from the following link:

<http://www.oracle.com/technetwork/developer-tools/sql-developer/downloads/index.html>

For instructions on how to install SQL Developer, see the website at:

<http://www.oracle.com/technetwork/developer-tools/sql-developer/overview/index.html>

SQL Developer 17.4.1 Interface



You must define a connection to start using SQL Developer for running SQL queries on a database schema.

The SQL Developer interface contains two main navigation tabs:

- **Connections:** By using this tab, you can browse database objects and users to which you have access.
- **Reports:** Identified by the Reports icon, this tab enables you to run predefined reports or create and add your own reports.

General Navigation and Use

SQL Developer uses the left side for navigation to find and select objects, and the right side to display information about selected objects. You can customize many aspects of the appearance and behavior of SQL Developer by setting preferences.

Note: You need to define at least one connection to be able to connect to a database schema and issue SQL queries or run procedures and functions.

Menus

The following menus contain standard entries, plus entries for features that are specific to SQL Developer:

- **View:** Contains options that affect what is displayed in the SQL Developer interface
- **Navigate:** Contains options for navigating to panes and for executing subprograms
- **Run:** Contains the Run File and Execution Profile options that are relevant when a function or procedure is selected, and also debugging options

- **Versioning:** Provides integrated support for the following versioning and source control systems: Concurrent Versions System (CVS) and Subversion
- **Tools:** Invokes SQL Developer tools such as SQL*Plus, Preferences, and SQL Worksheet. It also contains options related to migrating third-party databases to Oracle.

Note: The Run menu also contains options that are relevant when a function or procedure is selected for debugging.

For Instructor Use Only.
This document should not be distributed.

Creating a Database Connection

- You must have at least one database connection to use SQL Developer.
- You can create and test connections for multiple databases and multiple schemas.
- SQL Developer automatically imports any connections defined in the `tnsnames.ora`.
- You can export connections to an XML file.
- Each additional database connection created is listed in the Connections Navigator hierarchy.



A connection is a SQL Developer object that specifies the necessary information for connecting to a specific database as a specific user of that database. To use SQL Developer, you must have at least one database connection, which may be existing, created, or imported.

You can create and test connections for multiple databases and for multiple schemas.

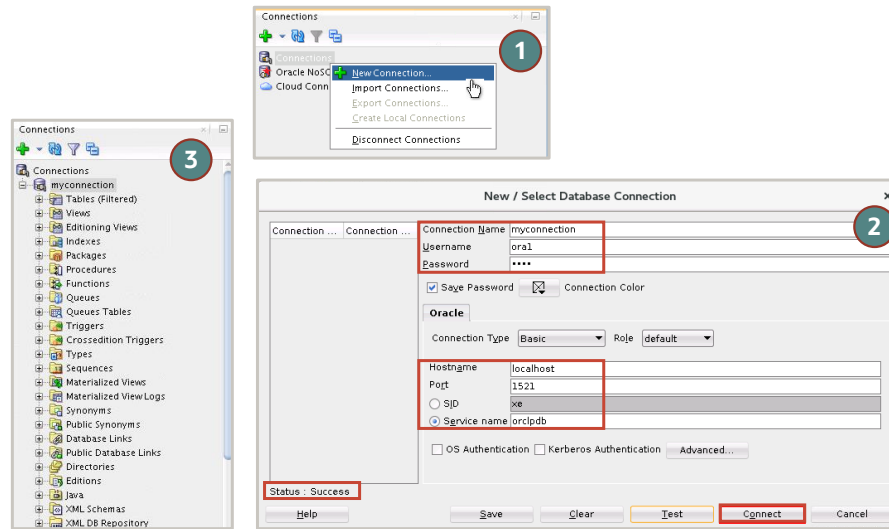
By default, the `tnsnames.ora` file is located in the `$ORACLE_HOME/network/admin` directory, but it can also be in the directory specified by the `TNS_ADMIN` environment variable or registry value. When you start SQL Developer and open the Database Connections dialog box, SQL Developer automatically imports any connections defined in the `tnsnames.ora` file on your system.

Note: On Windows, if the `tnsnames.ora` file exists, but its connections are not being used by SQL Developer, define `TNS_ADMIN` as a system environment variable.

You can export connections to an Extensible Markup Language (XML) file so that you can reuse it.

You can create additional connections as different users to the same database or to connect to the different databases.

Creating a Database Connection



To create a database connection, perform the following steps:

1. On the Connections tabbed page, right-click Connections and select New Connection.
2. In the New / Select Database Connection window, enter the connection name. Enter the username and password of the schema that you want to connect to.
 1. From the Role drop-down list, you can select either *default* or *SYSDBA*. (You choose *SYSDBA* for the *sys* user or any user with database administrator privileges.)
 2. You can select the connection type as:
 - **Basic:** In this type, enter host name and SID for the database that you want to connect to. Port is already set to 1521. You can also choose to enter the Service name directly if you use a remote database connection.
 - **TNS:** You can select any one of the database aliases imported from the `tnsnames.ora` file.
 - **LDAP:** You can look up database services in Oracle Internet Directory, which is a component of Oracle Identity Management.
 - **Advanced:** You can define a custom Java Database Connectivity (JDBC) URL to connect to the database.
 - **Local/Bequeath:** If the client and database exist on the same computer, a client connection can be passed directly to a dedicated server process without going through the listener.
3. Click Test to ensure that the connection has been set correctly.
4. Click Connect.

If you select the Save Password check box, the password is saved to an XML file. So, after you close the SQL Developer connection and open it again, you are not prompted for the password.

3. The connection gets added in the Connections Navigator. You can expand the connection to view the database objects and view object definitions (dependencies, details, statistics, and so on).

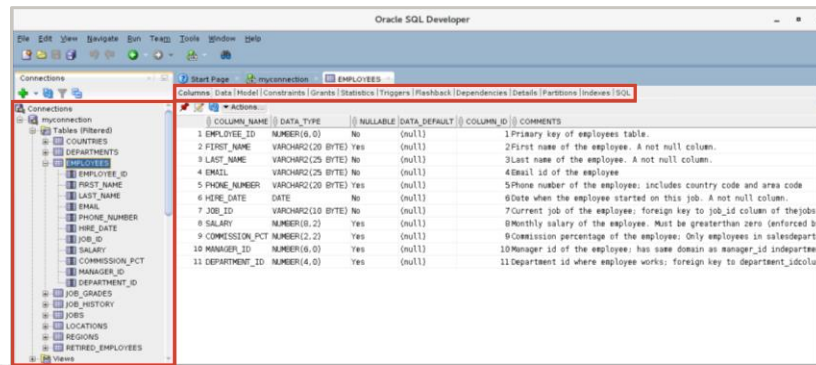
Note: From the same New/Select Database Connection window, you can define connections to non-Oracle data sources using the Access, MySQL, and SQL Server tabs. However, these connections are read-only connections that enable you to browse objects and data in that data source.

For Instructor Use Only.
This document should not be distributed.

Browsing Database Objects

Use the Connections Navigator to to:

- Browse through many objects in a database schema
- Review the definitions of objects at a glance



After you create a database connection, you can use the Connections Navigator to browse through many objects in a database schema, including Tables, Views, Indexes, Packages, Procedures, Triggers, and Types.

SQL Developer uses the left side for navigation to find and select objects and the right side to display information about the selected objects. You can customize many aspects of the appearance of SQL Developer by setting preferences.

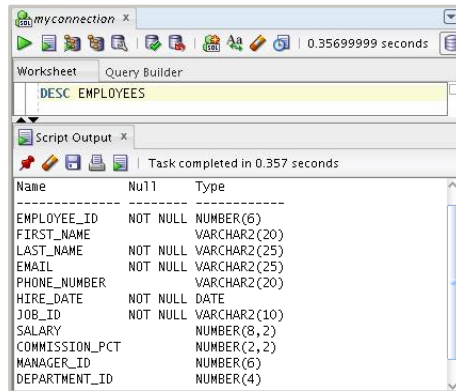
You can see the definition of the objects broken into tabs of information that is pulled out of the data dictionary. For example, if you select a table in the Navigator, details about columns, constraints, grants, statistics, triggers, and so on are displayed on an easy-to-read tabbed page.

If you want to see the definition of the `EMPLOYEES` table as shown in the slide, perform the following steps:

1. Expand the Connections node in the Connections Navigator.
2. Expand Tables.
3. Click `EMPLOYEES`. By default, the Columns tab is selected. It shows the column description of the table. Using the Data tab, you can view the table data and also enter new rows, update data, and commit these changes to the database.

Displaying the Table Structure

Use the `DESCRIBE` command to display the structure of a table:



The screenshot shows the SQL Developer interface. The 'Script Output' window displays the result of the `DESCRIBE EMPLOYEES` command. The output is a table with three columns: Name, Null, and Type. The table lists the columns of the EMPLOYEES table, including their names, whether they are nullable, and their data types.

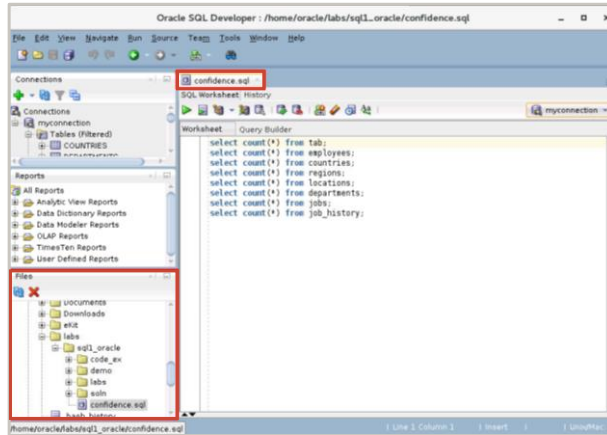
Name	Null	Type
EMPLOYEE_ID	NOT NULL	NUMBER(6)
FIRST_NAME		VARCHAR2(20)
LAST_NAME	NOT NULL	VARCHAR2(25)
EMAIL	NOT NULL	VARCHAR2(25)
PHONE_NUMBER		VARCHAR2(20)
HIRE_DATE	NOT NULL	DATE
JOB_ID	NOT NULL	VARCHAR2(10)
SALARY		NUMBER(8,2)
COMMISSION_PCT		NUMBER(2,2)
MANAGER_ID		NUMBER(6)
DEPARTMENT_ID		NUMBER(4)



In SQL Developer, you can also display the structure of a table by using the `DESCRIBE` command. The result of the command is a display of column names and data types, as well as an indication of whether a column must contain data.

Browsing Files

Use the File Navigator to explore the file system and open system files.



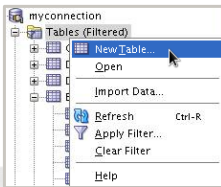
Browsing Database Objects

You can use the File Navigator to browse and open system files.

- To view the File Navigator, click the View tab and select Files, or select View > Files.
- To view the contents of a file, double-click a file name to display its contents in the SQL Worksheet area.

Creating a Schema Object

- SQL Developer supports the creation of any schema object by:
 - Executing a SQL statement in SQL Worksheet
 - Using the context menu
- Edit the objects by using an edit dialog box or one of the many context-sensitive menus.
- View the data definition language (DDL) for adjustments such as creating a new object or editing an existing schema object.

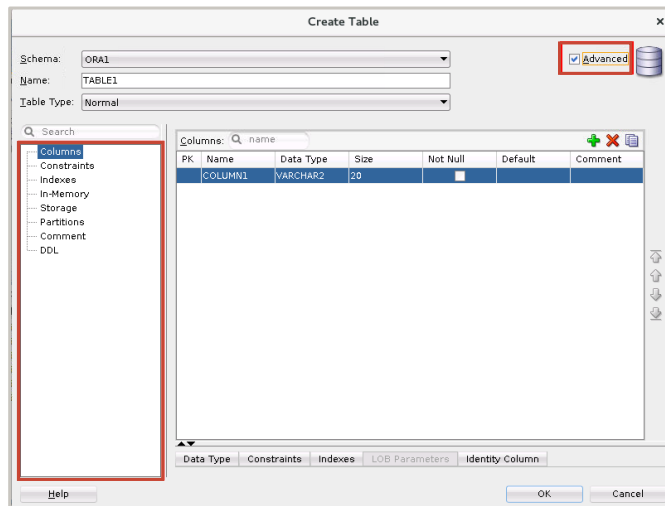


SQL Developer supports the creation of any schema object by executing a SQL statement in SQL Worksheet. Alternatively, you can create objects by using the context menus. When created, you can edit objects using an edit dialog box or one of the many context-sensitive menus.

As new objects are created or existing objects are edited, the DDL for those adjustments is available for review. An Export DDL option is available if you want to create the full DDL for one or more objects in the schema.

The slide shows how to create a table by using the context menu. To open a dialog box for creating a new table, right-click Tables and select New Table. The dialog boxes to create and edit database objects have multiple tabs, each reflecting a logical grouping of properties for that type of object.

Creating a New Table: Example



In the Create Table dialog box, if you do not select the Advanced check box, you can create a table quickly by specifying columns and some frequently used features.

If you select the Advanced check box, the Create Table dialog box changes to one with multiple options, in which you can specify an extended set of features while you create the table.

The example in the slide shows how to create the `DEPENDENTS` table by selecting the Advanced check box.

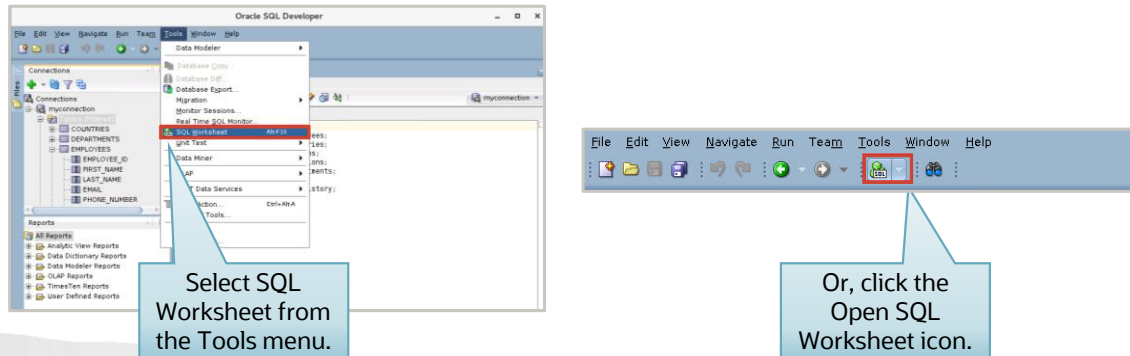
To create a new table, perform the following steps:

1. In the Connections Navigator, right-click Tables and select Create TABLE.
2. In the Create Table dialog box, select Advanced.
3. Specify the column information.
4. Click OK.

Although it is not required, you should also specify a primary key by using the Primary Key tab in the dialog box. Sometimes, you may want to edit the table that you have created; to do so, right-click the table in the Connections Navigator and select Edit.

Using SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



When you connect to a database, a SQL Worksheet window for that connection automatically opens. You can use SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements. SQL Worksheet supports SQL*Plus statements to a certain extent. SQL*Plus statements that are not supported by SQL Worksheet are ignored and not passed to the database.

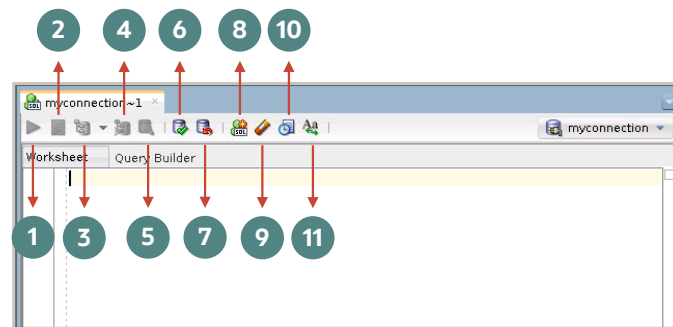
You can specify the actions that can be processed by the database connection associated with the worksheet, such as:

- Creating a table
- Inserting data
- Creating and editing a trigger
- Selecting data from a table
- Saving the selected data to a file

You can display a SQL Worksheet by using one of the following:

- Select Tools > SQL Worksheet.
- Click the Open SQL Worksheet icon.

Using SQL Worksheet

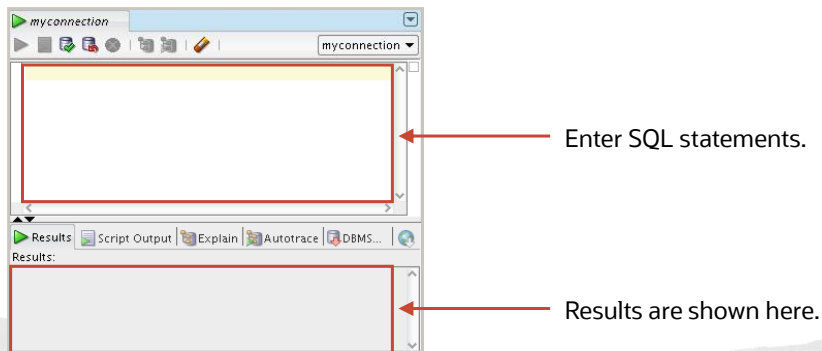


You may want to use the shortcut keys or icons to perform certain tasks such as executing a SQL statement, running a script, and viewing the history of the SQL statements that you have executed. You can use the SQL Worksheet toolbar that contains icons to perform the following tasks:

1. **Run Statement:** Executes the statement where the cursor is located in the Enter SQL Statement box. You can use bind variables in the SQL statements, but not substitution variables.
2. **Run Script:** Executes all the statements in the Enter SQL Statement box by using the Script Runner. You can use substitution variables in the SQL statements, but not bind variables.
3. **Explain Plan:** Generates the execution plan, which you can see by clicking the Explain tab
4. **Autotrace:** Generates trace information for the statement
5. **SQL Tuning Advisory:** Analyzes high-volume SQL statements and offers tuning recommendations
6. **Commit:** Writes any changes to the database and ends the transaction
7. **Rollback:** Discards any changes to the database, without writing them to the database, and ends the transaction
8. **Unshared SQL Worksheet:** Creates a separate unshared SQL Worksheet for a connection
9. **Clear:** Erases the statement or statements in the Enter SQL Statement box
10. **SQL History:** Displays a dialog box with information about the SQL statements that you have executed
11. **To Upper/Lower/InitCap:** Changes the selected text to uppercase, lowercase, or initcap, respectively

Using SQL Worksheet

- Use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements.
- Specify any actions that can be processed by the database connection associated with the worksheet.



When you connect to a database, a SQL Worksheet window for that connection automatically opens. You can use the SQL Worksheet to enter and execute SQL, PL/SQL, and SQL*Plus statements. All SQL and PL/SQL commands are supported as they are passed directly from the SQL Worksheet to the Oracle database. The SQL*Plus commands that are used in SQL Developer must be interpreted by the SQL Worksheet before being passed to the database.

The SQL Worksheet currently supports a number of SQL*Plus commands. Commands that are not supported by the SQL Worksheet are ignored and not sent to the Oracle database. Through the SQL Worksheet, you can execute the SQL statements and some of the SQL*Plus commands.

Executing SQL Statements

Use the Enter SQL Statement box to enter single or multiple SQL statements.

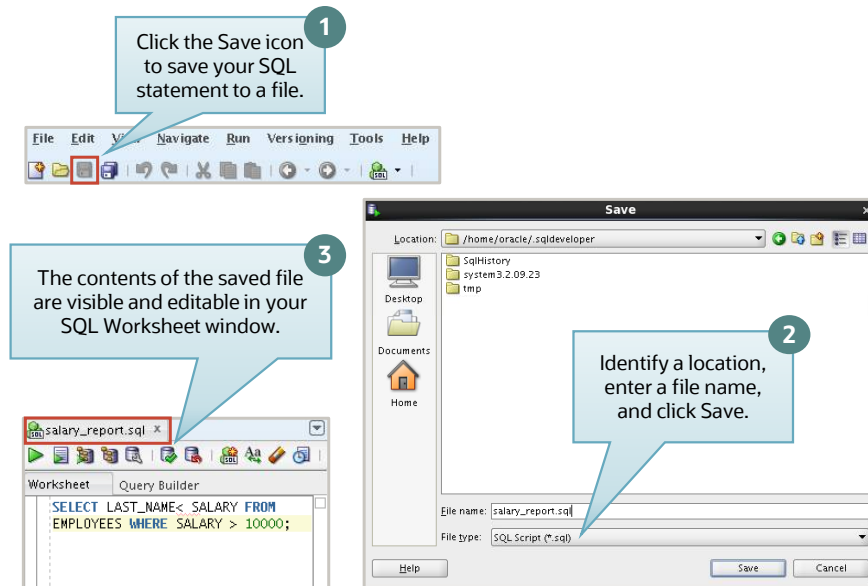
The screenshot shows the SQL Workshop interface with the 'myconnection' window. The 'Query Builder' tab is active, displaying the SQL statement: `SELECT employee_id, last_name from employees;`. A red box highlights the 'Run' button (a green play icon) and the 'Run Script' button (a green play icon with a document). Red arrows point from these buttons to callouts 'F9' and 'F5' respectively. Below the query editor, two windows are shown: 'Query Result' (labeled F9) and 'Script Output' (labeled F5). The 'Query Result' window shows a table with 5 rows of employee data. The 'Script Output' window shows the same data as a text output.

EMPLOYEE_ID	LAST_NAME
1	174 Abel
2	142 Davies
3	102 De Haan
4	104 Ernst
5	202 Fay

EMPLOYEE_ID	LAST_NAME
174	Abel
142	Davies
102	De Haan
104	Ernst
202	Fay
206	Gietz

The example in the slide shows the difference in output for the same query when the F9 key or Execute Statement is used versus the output when the F5 or Run Script is used.

Saving SQL Scripts



You can save your SQL statements from SQL Worksheet to a text file. To save the contents of the Enter SQL Statement box, perform the following steps:

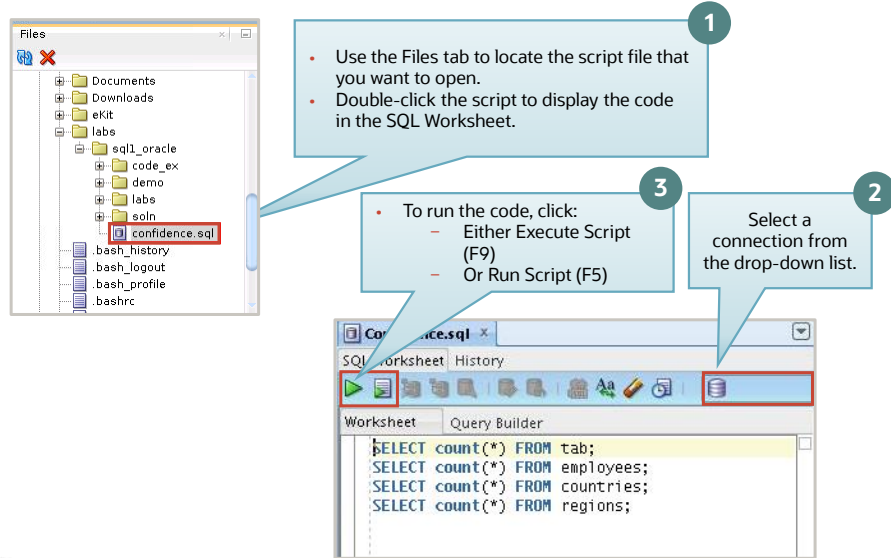
1. Click the Save icon or use the File > Save menu item.
2. In the Save dialog box, enter a file name and the location where you want the file saved.
3. Click Save.

After you save the contents to a file, the Enter SQL Statement window displays a tabbed page of your file contents. You can have multiple files open at the same time. Each file is displayed as a tabbed page.

Script Pathing

You can select a default path to look for scripts and to save scripts. Under Tools > Preferences > Database > Worksheet Parameters, enter a value in the “Select default path to look for scripts” field.

Executing Saved Script Files: Method 1



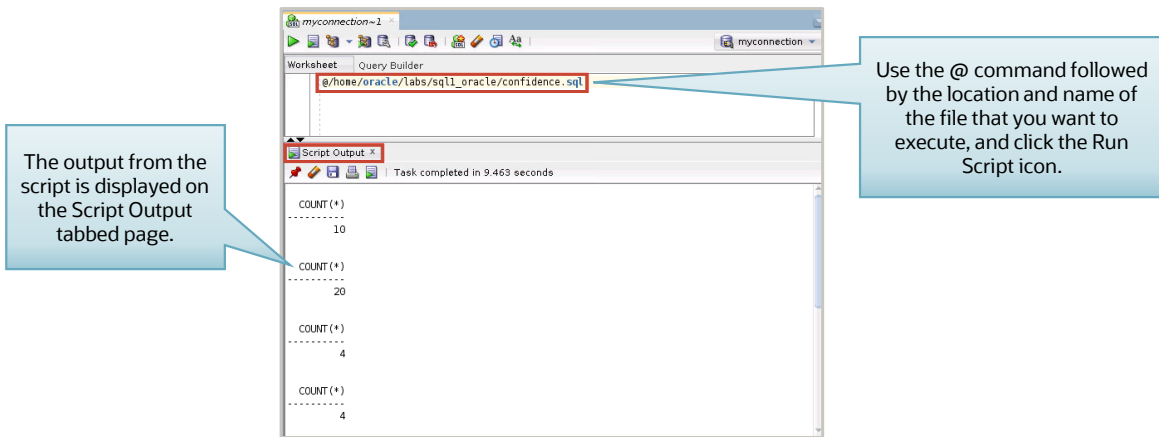
To open a script file and display the code in the SQL Worksheet area, perform the following steps:

1. In the files navigator, select (or navigate to) the script file that you want to open.
2. Double-click the file to open it. The code of the script file is displayed in the SQL Worksheet area.
3. Select a connection from the connection drop-down list.
4. To run the code, click the Run Script (F5) icon on the SQL Worksheet toolbar. If you have not selected a connection from the connection drop-down list, a connection dialog box will appear. Select the connection that you want to use for the script execution.

Alternatively, you can also do the following:

1. Select File > Open. The Open dialog box is displayed.
2. In the Open dialog box, select (or navigate to) the script file that you want to open.
3. Click Open. The code of the script file is displayed in the SQL Worksheet area.
4. Select a connection from the connection drop-down list.
5. To run the code, click the Run Script (F5) icon on the SQL Worksheet toolbar. If you have not selected a connection from the connection drop-down list, a connection dialog box will appear. Select the connection that you want to use for the script execution.

Executing Saved Script Files: Method 2



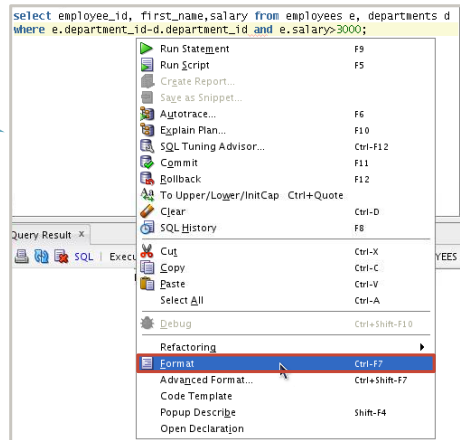
To run a saved SQL script, perform the following steps:

1. Use the @ command followed by the location and the name of the file that you want to run in the Enter SQL Statement window.
2. Click the Run Script icon.

The results from running the file are displayed on the Script Output tabbed page. You can also save the script output by clicking the Save icon on the Script Output tabbed page. The File Save dialog box appears and you can identify a name and location for your file.

Formatting the SQL Code

Before
formatting



After
formatting

```
SELECT employee_id,
       first_name,
       salary
FROM employees e,
     departments d
WHERE e.department_id=d.department_id
AND e.salary>3000;
```



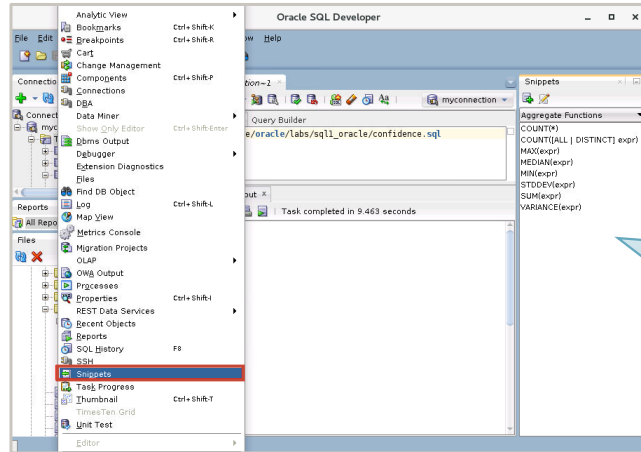
You may want to format the indentation, spacing, capitalization, and line separation of the SQL code. SQL Developer has a feature for formatting SQL code.

To format the SQL code, right-click in the statement area and select Format.

In the example in the slide, before formatting, the SQL code has the keywords not capitalized and the statement not properly indented. After formatting, the SQL code is beautified with the keywords capitalized and the statement properly indented.

Using Snippets

Snippets are code fragments that may be just syntax or examples.

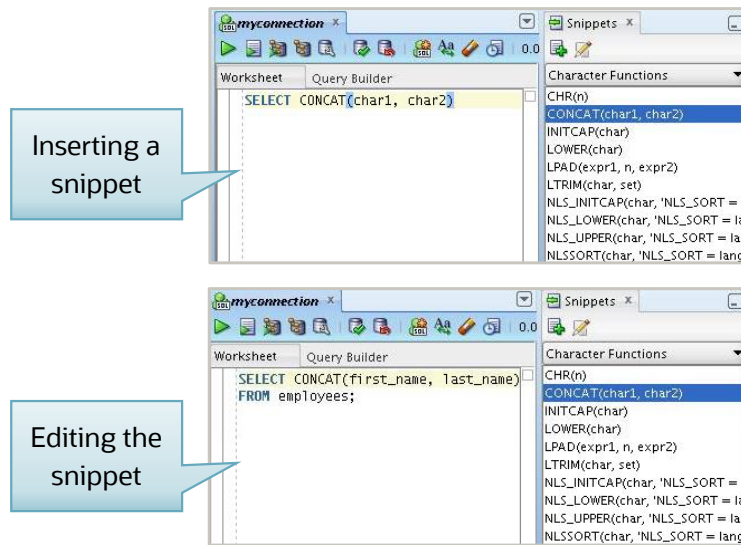


You may want to use certain code fragments when you use SQL Worksheet or create or edit a PL/SQL function or procedure. SQL Developer has a feature called Snippets. Snippets are code fragments, such as SQL functions, optimizer hints, and miscellaneous PL/SQL programming techniques. You can drag snippets to the Editor window.

To display Snippets, select View > Snippets.

The Snippets window is displayed on the right. You can use the drop-down list to select a group. A Snippets button is placed in the right window margin, so that you can display the Snippets window if it becomes hidden.

Using Snippets: Example



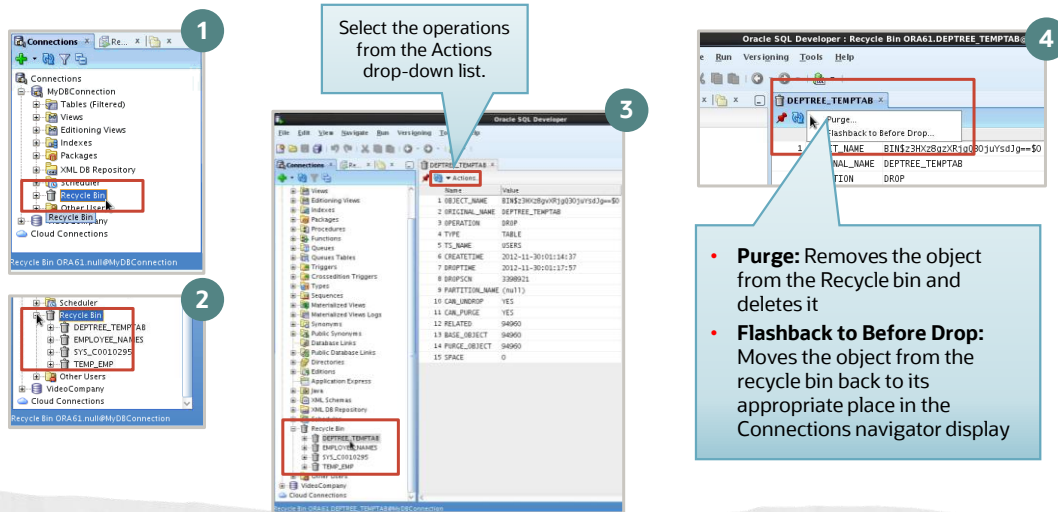
To insert a Snippet into your code in a SQL Worksheet or in a PL/SQL function or procedure, drag the snippet from the Snippets window to the desired place in your code. Then you can edit the syntax so that the SQL function is valid in the current context. To see a brief description of a SQL function in a tool tip, place the cursor over the function name.

The example in the slide shows that `CONCAT(char1, char2)` is dragged from the Character Functions group in the Snippets window. Then the `CONCAT` function syntax is edited, and the rest of the statement is added as in the following:

```
SELECT CONCAT(first_name, last_name)
FROM employees;
```


Using the Recycle Bin

The recycle bin holds objects that have been dropped.



The recycle bin is a data dictionary table containing information about dropped objects. Dropped tables and any associated objects such as indexes, constraints, nested tables, and the likes are not removed and still occupy space. They continue to count against user space quotas, until specifically purged from the recycle bin or the unlikely situation where they must be purged by the database because of tablespace space constraints.

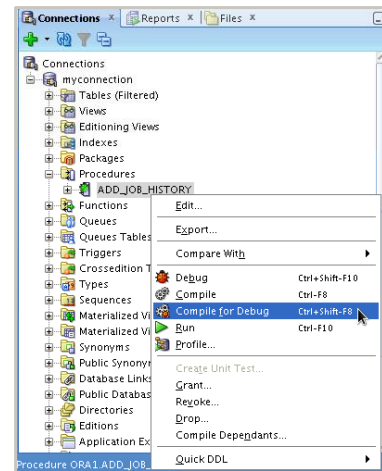
To use the recycle bin, perform the following steps:

1. In the Connections navigator, select (or navigate to) the recycle bin.
2. Expand Recycle Bin and click the object name. The object details are displayed in the SQL Worksheet area.
3. Click the Actions drop-down list and select the operation that you want to perform on the object.

Debugging Procedures and Functions

Use:

- SQL Developer to debug PL/SQL functions and procedures
- The “Compile for Debug” option to perform a PL/SQL compilation so that the procedure can be debugged
- The Debug menu options to set breakpoints and to perform Step Into and Step Over tasks



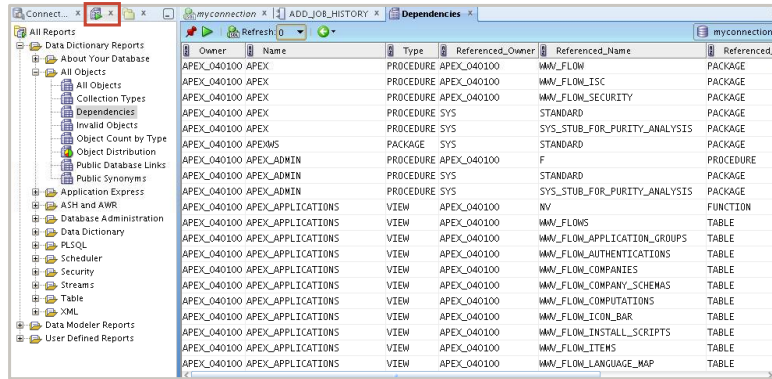
In SQL Developer, you can debug PL/SQL procedures and functions. Using the Debug menu options, you can perform the following debugging tasks:

- **Find Execution Point** goes to the next execution point.
- **Resume** continues execution.
- **Step Over** bypasses the next method and goes to the next statement after the method.
- **Step Into** goes to the first statement in the next method.
- **Step Out** leaves the current method and goes to the next statement.
- **Step to End of Method** goes to the last statement of the current method.
- **Pause** halts execution, but does not exit, thus allowing you to resume execution.
- **Terminate** halts and exits the execution. You cannot resume execution from this point; instead, to start running or debugging from the beginning of the function or procedure, click the Run or Debug icon on the Source tab toolbar.
- **Garbage Collection** removes invalid objects from the cache in favor of more frequently accessed and more valid objects.

These options are also available as icons on the Debugging tab of the output window.

Database Reporting

SQL Developer provides a number of predefined reports about the database and its objects.



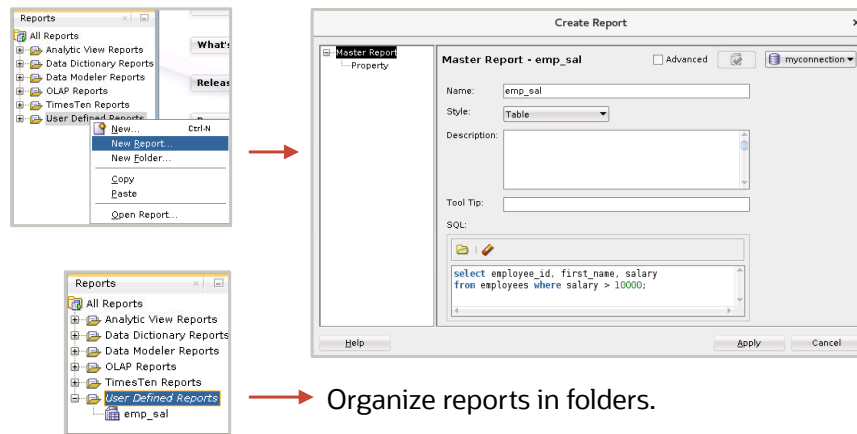
SQL Developer provides many reports about the database and its objects. These reports can be grouped into the following categories:

- About Your Database reports
- Database Administration reports
- Table reports
- PL/SQL reports
- Security reports
- XML reports
- Jobs reports
- Streams reports
- All Objects reports
- Data Dictionary reports
- User-Defined reports

To display reports, click the Reports tab on the left of the window. Individual reports are displayed in tabbed panes on the right of the window; for each report, you can select (using a drop-down list) the database connection for which to display the report. For reports about objects, the objects shown are only those visible to the database user associated with the selected database connection, and the rows are usually ordered by Owner. You can also create your own user-defined reports.

Creating a User-Defined Report

Create and save user-defined reports for repeated use.



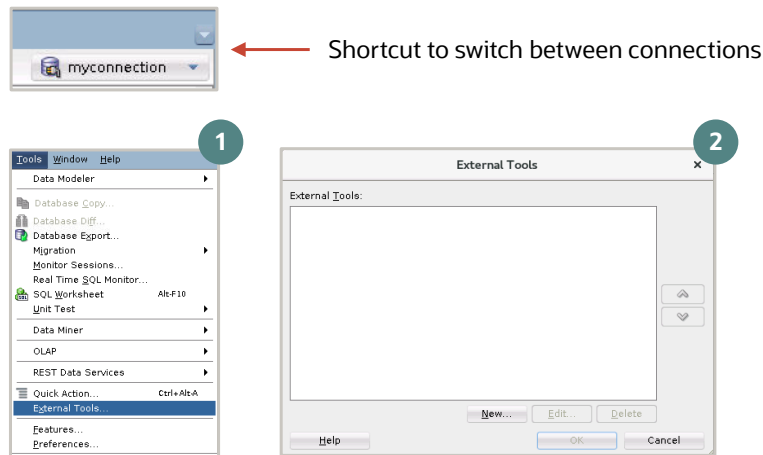
User-defined reports are created by SQL Developer users. To create a user-defined report, perform the following steps:

1. Right-click the User Defined Reports node under Reports and select Add Report.
2. In the Create Report dialog box, specify the report name and the SQL query to retrieve information for the report. Then click Apply.

In the example in the slide, the report name is specified as `emp_sal`. An optional description is provided indicating that the report contains details of employees with `salary >= 10000`. The complete SQL statement for retrieving the information to be displayed in the user-defined report is specified in the SQL box. You can also include an optional tool tip to be displayed when the cursor stays briefly over the report name in the Reports navigator display.

You can organize user-defined reports in folders, and you can create a hierarchy of folders and subfolders. To create a folder for user-defined reports, right-click the User Defined Reports node or any folder name under that node and select Add Folder. Information about user-defined reports, including any folders for these reports, is stored in a file named `UserReports.xml` in the directory for user-specific information.

External Tools



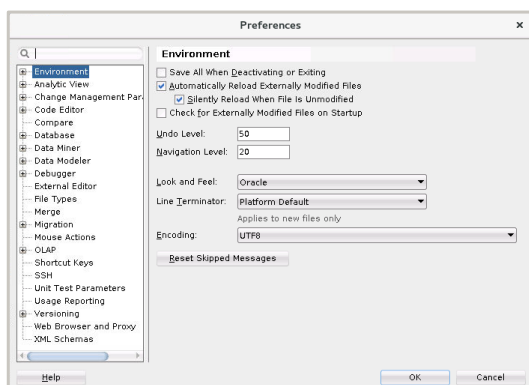
Shortcut icons are available for some of the frequently used tools such as Notepad, Microsoft Word, and Dreamweaver.

You can add external tools to the existing list or even delete shortcuts to the tools that you do not use frequently. To do so, perform the following steps:

1. From the Tools menu, select External Tools.
2. In the External Tools dialog box, select New to add new tools. Select Delete to remove any tool from the list.

Setting Preferences

- Customize the SQL Developer interface and environment.
- From the Tools menu, select Preferences.



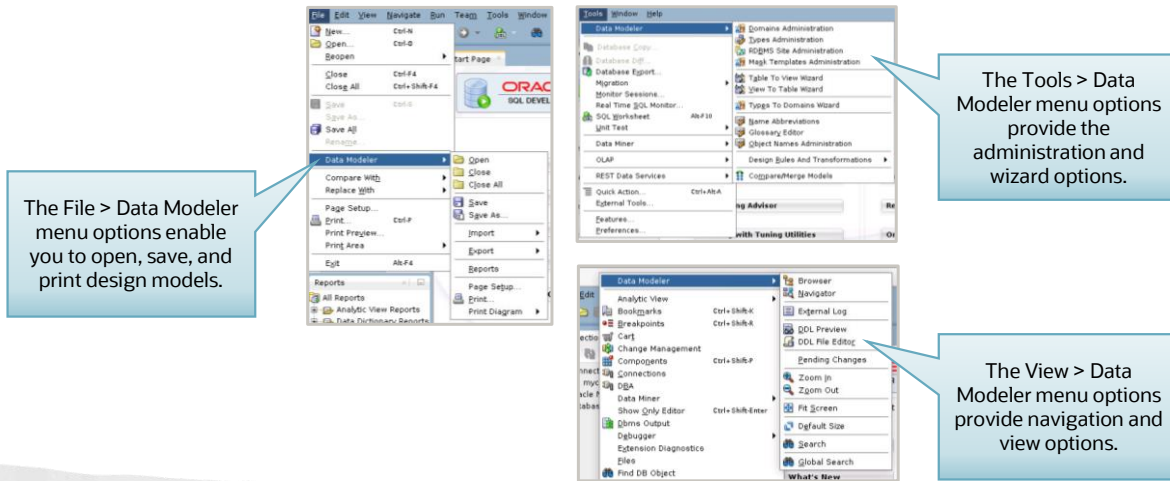
You can customize many aspects of the SQL Developer interface and environment by modifying SQL Developer preferences according to your needs. To modify SQL Developer preferences, select Tools and then Preferences.

The preferences are grouped into the following categories:

- Environment
- Change Management Parameter
- Code Editor
- Compare and Merge
- Database
- Data Miner
- Data Modeler
- Debugger
- Extensions
- External Editor
- File Types
- Migration
- Mouseover Popups
- Shortcut Keys
- Unit Test Parameters
- Versioning
- Web Browser and Proxy
- XML Schemas

Data Modeler in SQL Developer

SQL Developer includes an integrated version of SQL Developer Data Modeler.



Using the integrated version of the SQL Developer Data Modeler, you can:

- Create, open, import, and save a database design
- Create, modify, and delete Data Modeler objects

To display Data Modeler in a pane, click Tools and then Data Modeler. The Data Modeler menu under Tools includes additional commands, for example, that enable you to specify design rules and preferences.

Summary



Browse, create, and edit database objects

Execute SQL statements and scripts in SQL Worksheet

Create and save custom reports

Browse the data modeling options in SQL Developer

SQL Developer is a free graphical tool to simplify database development tasks. Using SQL Developer, you can browse, create, and edit database objects. You can use SQL Worksheet to run SQL statements and scripts. SQL Developer enables you to create and save your own special set of reports for repeated use.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Using SQL*Plus



For Instructor Use Only.

This document should not be distributed.

Objectives



Log in to SQL*Plus

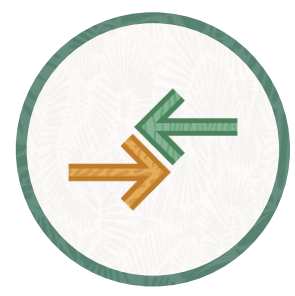
Edit SQL commands

Format the output by using SQL*Plus commands

Interact with script files

You might want to create `SELECT` statements that can be used repeatedly. This appendix covers the use of SQL*Plus commands to execute SQL statements. You learn how to format output using SQL*Plus commands, edit SQL commands, and save scripts in SQL*Plus.

For Instructor Use Only.
This document should not be distributed.



For Instructor Use Only.

This document should not be distributed.

art
s is

art
s is

- art
s is

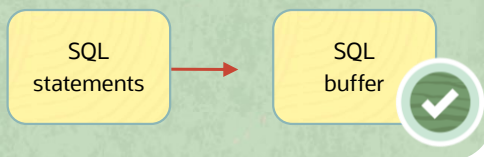
art
s is

- art
s is

SQL Statements Versus SQL*Plus Commands

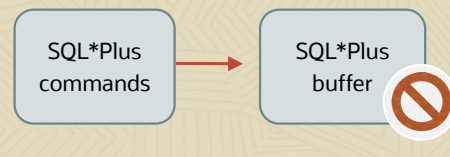
SQL:

- Is a language
- Is based on ANSI standard
- Cannot abbreviate keywords
- Manipulates data and table definitions in the database



SQL*Plus:

- Is an environment
- Is Oracle proprietary
- Can abbreviate keywords
- Commands do not allow manipulation of values in the database



The following table compares SQL and SQL*Plus:

SQL	SQL*Plus
Is a language for communicating with the Oracle server to access data	Recognizes SQL statements and sends them to the server
Is based on American National Standards Institute (ANSI)–standard SQL	Is the Oracle-proprietary interface for executing SQL statements
Manipulates data and table definitions in the database	Does not allow manipulation of values in the database
Is entered into the SQL buffer on one or more lines	Is entered one line at a time, not stored in the SQL buffer
Does not have a continuation character	Uses a dash (–) as a continuation character if the command is longer than one line
Cannot be abbreviated	Can be abbreviated
Uses a termination character to execute commands immediately	Does not require termination characters; executes commands immediately
Uses functions to perform some formatting	Uses commands to format data

SQL*Plus: Overview

- Log in to SQL*Plus.
- Describe the table structure.
- Edit your SQL statement.
- Execute SQL from SQL*Plus.
- Save SQL statements to files and append SQL statements to files.
- Execute saved files.
- Load commands from the file to buffer to edit.



SQL*Plus

SQL*Plus is an environment in which you can:

- Execute SQL statements to retrieve, modify, add, and remove data from the database
- Format, perform calculations on, store, and print query results in the form of reports
- Create script files to store SQL statements for repeated use in the future

SQL*Plus commands can be divided into the following main categories:

Category	Purpose
Environment	Affects the general behavior of SQL statements for the session
Format	Formats query results
File manipulation	Saves, loads, and runs script files
Execution	Sends SQL statements from the SQL buffer to the Oracle server
Edit	Modifies SQL statements in the buffer
Interaction	Creates and passes variables to SQL statements, prints variable values, and prints messages to the screen
Miscellaneous	Connects to the database, manipulates the SQL*Plus environment, and displays column definitions

Logging In to SQL*Plus

```
sqlplus [username[/password[@database]]]
```

How you invoke SQL*Plus depends on the type of operating system on which you are running Oracle Database.

To log in from a Linux environment, perform the following steps:

1. Right-click your Linux desktop and select terminal.
2. Enter the `sqlplus` command shown in the slide.
3. Enter the username, password, and database name.

In the syntax:

<i>username</i>	Your database username
<i>password</i>	Your database password (Your password is visible if you enter it here.)
<i>@database</i>	The database connect string

Note: To ensure the integrity of your password, do not enter it at the operating system prompt. Instead, enter only your username. Enter your password at the password prompt.

For Instructor Use Only.
This document should not be distributed.

Displaying the Table Structure

Use the SQL*Plus `DESCRIBE` command to display the structure of a table:

```
DESC[RIBE] tablename
```



In SQL*Plus, you can display the structure of a table by using the `DESCRIBE` command. The result of the command is a display of column names and data types, as well as an indication if a column must contain data.

In the syntax:

tablename The name of any existing table, view, or synonym that is accessible to the user

To describe the `DEPARTMENTS` table, use the following command:

```
SQL> DESCRIBE DEPARTMENTS
```

Name	Null	Type
-----	-----	-----
DEPARTMENT_ID	NOT NULL	NUMBER (4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2 (30)
MANAGER_ID		NUMBER (6)
LOCATION_ID		NUMBER (4)

Displaying the Table Structure

```
DESCRIBE departments
```

Name	Null	Type
DEPARTMENT_ID	NOT NULL	NUMBER (4)
DEPARTMENT_NAME	NOT NULL	VARCHAR2 (30)
MANAGER_ID		NUMBER (6)
LOCATION_ID		NUMBER (4)

The example in the slide displays information about the structure of the `DEPARTMENTS` table. In the result:

- **Null:** Specifies whether a column must contain data (`NOT NULL` indicates that a column must contain data.)
- **Type:** Displays the data type for a column

SQL*Plus Editing Commands

- A[PPEND] *text*
- C[HANGE] / *old* / *new*
- C[HANGE] / *text* /
- CL[EAR] BUFF[ER]
- DEL
- DEL *n*
- DEL *m n*



SQL*Plus commands are entered one line at a time and are not stored in the SQL buffer.

Command	Description
A[PPEND] <i>text</i>	Adds <i>text</i> to the end of the current line
C[HANGE] / <i>old</i> / <i>new</i>	Changes <i>old</i> text to <i>new</i> in the current line
C[HANGE] / <i>text</i> /	Deletes <i>text</i> from the current line
CL[EAR] BUFF[ER]	Deletes all lines from the SQL buffer
DEL	Deletes current line
DEL <i>n</i>	Deletes line <i>n</i>
DEL <i>m n</i>	Deletes lines <i>m</i> to <i>n</i> inclusive

Guidelines

- If you press Enter before completing a command, SQL*Plus prompts you with a line number.
- You terminate the SQL buffer either by entering one of the terminator characters (semicolon or slash) or by pressing Enter twice. The SQL prompt appears.

SQL*Plus Editing Commands

- I [NPUT]
- I [NPUT] *text*
- L [IST]
- L [IST] *n*
- L [IST] *m n*
- R [UN]
- *n*
- *n text*
- *0 text*



Command	Description
I [NPUT]	Inserts an indefinite number of lines
I [NPUT] <i>text</i>	Inserts a line consisting of <i>text</i>
L [IST]	Lists all lines in the SQL buffer
L [IST] <i>n</i>	Lists one line (specified by <i>n</i>)
L [IST] <i>m n</i>	Lists a range of lines (<i>m</i> to <i>n</i>) inclusive
R [UN]	Displays and runs the current SQL statement in the buffer
<i>n</i>	Specifies the line to make the current line
<i>n text</i>	Replaces line <i>n</i> with <i>text</i>
<i>0 text</i>	Inserts a line before line 1

Note: You can enter only one SQL*Plus command for each SQL prompt. SQL*Plus commands are not stored in the buffer. To continue a SQL*Plus command on the next line, end the first line with a hyphen (-).

Using LIST, n, and APPEND

```
LIST
1  SELECT last_name
2* FROM   employees
```

```
1
1* SELECT last_name
```

```
A , job_id
1* SELECT last_name, job_id
```

```
LIST
1  SELECT last_name, job_id
2* FROM   employees
```

- Use the `L[IST]` command to display the contents of the SQL buffer. The asterisk (*) beside line 2 in the buffer indicates that line 2 is the current line. Any edits that you made apply to the current line.
- Change the number of the current line by entering the number (n) of the line that you want to edit. The new current line is displayed.
- Use the `A[PPEND]` command to add text to the current line. The newly edited line is displayed. Verify the new contents of the buffer by using the `LIST` command.

Note: Many SQL*Plus commands, including `LIST` and `APPEND`, can be abbreviated to just their first letter. `LIST` can be abbreviated to `L`; `APPEND` can be abbreviated to `A`.

Using the CHANGE Command

```
LIST
1* SELECT * from employees
```

```
c/employees/departments
1* SELECT * from departments
```

```
LIST
1* SELECT * from departments
```

- Use `L[IST]` to display the contents of the buffer.
- Use the `C[HANGE]` command to alter the contents of the current line in the SQL buffer. In this case, replace the `employees` table with the `departments` table. The new current line is displayed.
- Use the `L[IST]` command to verify the new contents of the buffer.

SQL*Plus File Commands

- `SAVE filename`
- `GET filename`
- `START filename`
- `@ filename`
- `EDIT filename`
- `SPOOL filename`
- `EXIT`



SQL statements communicate with the Oracle server. SQL*Plus commands control the environment, format query results, and manage files. You can use the commands described in the following table:

Command	Description
<code>SAV[E] filename [.ext]</code> <code>[REPL[ACE]APP[END]]</code>	Saves the current contents of the SQL buffer to a file. Use <code>APPEND</code> to add to an existing file; use <code>REPLACE</code> to overwrite an existing file. The default extension is <code>.sql</code> .
<code>GET filename [.ext]</code>	Writes the contents of a previously saved file to the SQL buffer. The default extension for the file name is <code>.sql</code> .
<code>STA[RT] filename [.ext]</code>	Runs a previously saved command file
<code>@ filename</code>	Runs a previously saved command file (same as <code>START</code>)
<code>ED[IT]</code>	Invokes the editor and saves the buffer contents to a file named <code>afiedt.buf</code>
<code>ED[IT] [filename.ext]</code>	Invokes the editor to edit the contents of a saved file
<code>SPO[OL] [filename.ext] </code> <code>OFF OUT]</code>	Stores query results in a file. <code>OFF</code> closes the spool file. <code>OUT</code> closes the spool file and sends the file results to the printer.
<code>EXIT</code>	Quits SQL*Plus

Using the SAVE and START Commands

```
LIST
1  SELECT last_name, manager_id, department_id
2* FROM employees
```

```
SAVE my_query
Created file my_query
```

```
START my_query

LAST_NAME                MANAGER_ID DEPARTMENT_ID
-----
King                      100             90
Kochhar                   100             90
...
107 rows selected.
```

SAVE

Use the `SAVE` command to store the current contents of the buffer in a file. Thus, you can store frequently used scripts for use in the future.

START

Use the `START` command to run a script in SQL*Plus. You can also, alternatively, use the symbol `@` to run a script.

```
@my_query
```

SERVEROUTPUT Command

- Use the `SET SERVEROUT[PUT]` command to control whether to display the output of stored procedures or PL/SQL blocks in SQL*Plus.
- The `DBMS_OUTPUT` line length limit is increased from 255 bytes to 32767 bytes.
- The default size is now unlimited.
- Resources are not preallocated when `SERVEROUTPUT` is set.
- Because there is no performance penalty, use `UNLIMITED` unless you want to conserve physical memory.

```
SET SERVEROUT[PUT] {ON | OFF} [SIZE {n | UNL[IMITED]}]  
[FOR[MAT] {WRA[PPED] | WOR[D_WRAPPED] | TRU[NCATED]}]
```

Most of the PL/SQL programs perform input and output through SQL statements, to store data in database tables or query those tables. All other PL/SQL input/output is done through APIs that interact with other programs. For example, the `DBMS_OUTPUT` package has procedures, such as `PUT_LINE`. To see the result outside of PL/SQL requires another program, such as SQL*Plus, to read and display the data passed to `DBMS_OUTPUT`.

SQL*Plus does not display `DBMS_OUTPUT` data unless you first issue the SQL*Plus command `SET SERVEROUTPUT ON` as follows:

```
SET SERVEROUTPUT ON
```

Notes

- `SIZE` sets the number of bytes of the output that can be buffered within the Oracle Database server. The default is `UNLIMITED`. `n` cannot be less than 2000 or greater than 1,000,000.
- For additional information about `SERVEROUTPUT`, see *Oracle Database PL/SQL User's Guide and Reference 23c*.

Using the SQL*Plus SPOOL Command

```
SPOOL [file_name[.ext] [CRE[ATE] | REP[LACE] | APP[END]] | OFF | OUT]
```

Option	Description
file_name[.ext]	Spools output to the specified file name
CRE[ATE]	Creates a new file with the name specified
REP[LACE]	Replaces the contents of an existing file. If the file does not exist, REPLACE creates the file.
APP[END]	Adds the contents of the buffer to the end of the file that you specify
OFF	Stops spooling
OUT	Stops spooling and sends the file to your computer's standard (default) printer

The SPOOL command stores query results in a file or optionally sends the file to a printer. The SPOOL command has been enhanced. You can now append to, or replace an existing file, where previously you could use SPOOL only to create (and replace) a file. REPLACE is the default.

To spool the output generated by commands in a script without displaying the output on screen, use SET TERMOUT OFF. SET TERMOUT OFF does not affect the output from commands that run interactively.

You must use quotation marks around file names that contain white space. To create a valid HTML file using SPOOL APPEND commands, you must use PROMPT or a similar command to create the HTML page header and footer. The SPOOL APPEND command does not parse HTML tags. SET SQLPLUSCOMPAT[IBILITY] to 9.2 or earlier to disable the CREATE, APPEND, and SAVE parameters.

Using the AUTOTRACE Command

- It displays a report after the successful execution of SQL DML statements, such as SELECT, INSERT, UPDATE, or DELETE.
- The report can now include execution statistics and the query execution path.

```
SET AUTOT[RACE] {ON | OFF | TRACE[ONLY]} [EXP[LAIN]] [STAT[ISTICS]]
```

```
SET AUTOTRACE ON
-- The AUTOTRACE report includes both the optimizer
-- execution path and the SQL statement execution
-- statistics
```

EXPLAIN shows the query execution path by performing an EXPLAIN PLAN. STATISTICS displays SQL statement statistics. The formatting of your AUTOTRACE report may vary depending on the version of the server to which you are connected and the configuration of the server. The DBMS_XPLAN package provides an easy way to display the output of the EXPLAIN PLAN command in several predefined formats.

Notes

- For additional information about the package and subprograms, refer to *Oracle Database PL/SQL Packages and Types Reference 23c*.
- For additional information about the EXPLAIN PLAN, refer to *Oracle Database SQL Reference 23c*.
- For additional information about Execution Plans and the statistics, refer to *Oracle Database Performance Tuning Guide 23c*.

Summary



In this appendix, you should have learned how to use SQL*Plus as an environment to do the following:

Execute SQL statements

Edit SQL statements

Format the output

Interact with script files

SQL*Plus is an execution environment that you can use to send SQL commands to the database server and to edit and save SQL commands. You can execute commands from the SQL prompt or from a script file.

Commonly Used SQL Commands



For Instructor Use Only.

This document should not be distributed.

Objectives



Execute a basic `SELECT` statement

Create, alter, and drop a table using data definition language (DDL) statements

Insert, update, and delete rows from one or more tables using data manipulation language (DML) statements

Commit, roll back, and create savepoints by using transaction control statements

Perform join operations on one or more tables

This lesson explains how to obtain data from one or more tables by using the `SELECT` statement, to use DDL statements to alter the structure of data objects, to manipulate data in existing schema objects by using DML statements, to manage the changes made by DML statements, and to use joins to display data from multiple tables using SQL:1999 join syntax.

Note: Before the Oracle 9i release, the join syntax was proprietary. The SQL:1999-compliant join syntax does not offer any performance benefits over the Oracle-proprietary join syntax. Oracle Database 23c complies with ANSI/ISO SQL:2016 standards.

Basic SELECT Statement

- Use the `SELECT` statement to:
 - Identify the columns to be displayed
 - Retrieve data from one or more tables, object tables, views, object views, or materialized views
- A `SELECT` statement is also known as a query because it queries a database.
- Syntax:

```
SELECT {*|[DISTINCT] column|expression [alias],...}  
FROM   table;
```

In its simplest form, a `SELECT` statement must include the following:

- A `SELECT` clause, which specifies the columns to be displayed
- A `FROM` clause, which identifies the table containing the columns that are listed in the `SELECT` clause

In the syntax:

<code>SELECT</code>	Is a list of one or more columns
<code>*</code>	Selects all columns
<code>DISTINCT</code>	Suppresses duplicates
<code>column expression</code>	Selects the named column or the expression
<code>alias</code>	Gives different headings to the selected columns
<code>FROM table</code>	Specifies the table containing the columns

Note: Throughout this course, the words *keyword*, *clause*, and *statement* are used as follows:

- A *keyword* refers to an individual SQL element—for example, `SELECT` and `FROM` are keywords.
- A *clause* is a part of a SQL statement (for example, `SELECT employee_id, last_name`).
- A *statement* is a combination of two or more clauses (for example, `SELECT * FROM employees`).

SELECT Statement

- Select all columns:

```
SELECT *  
FROM job_history;
```

	EMPLOYEE_ID	START_DATE	END_DATE	JOB_ID	DEPARTMENT_ID
1	102	13-JAN-01	24-JUL-06	IT_PROG	60
2	101	21-SEP-97	27-OCT-01	AC_ACCOUNT	110
3	101	28-OCT-01	15-MAR-05	AC_MGR	110
4	201	17-FEB-04	19-DEC-07	MK_REP	20
5	114	24-MAR-06	31-DEC-07	ST_CLERK	50
6	122	01-JAN-07	31-DEC-07	ST_CLERK	50
7	200	17-SEP-95	17-JUN-01	AD_ASST	90
8	176	24-MAR-06	31-DEC-06	SA_REP	80
9	176	01-JAN-07	31-DEC-07	SA_MAN	80
10	200	01-JUL-02	31-DEC-06	AC_ACCOUNT	90

- Select specific columns:

```
SELECT manager_id, job_id  
FROM employees;
```

	MANAGER_ID	JOB_ID
1	(null)	AD_PRES
2	100	AD_VP
3	100	AD_VP
4	102	IT_PROG
5	103	IT_PROG
6	103	IT_PROG
7	100	ST_MAN
8	124	ST_CLERK
9	124	ST_CLERK
10	124	ST_CLERK

You can display all columns of data in a table by following the `SELECT` keyword with an asterisk (*) or by listing all the column names after the `SELECT` keyword. The first example in the slide displays all the rows from the `job_history` table. Specific columns of the table can be displayed by specifying the column names, separated by commas. The second example in the slide displays the `manager_id` and `job_id` columns from the `employees` table.

In the `SELECT` clause, specify the columns in the order in which you want them to appear in the output. For example, the following SQL statement displays the `location_id` column before displaying the `department_id` column:

```
SELECT location_id, department_id FROM departments;
```

Note: You can enter your SQL statement in a SQL Worksheet and click the Run Statement icon or press F9 to execute a statement in SQL Developer. The output displayed on the Results tabbed page appears as shown in the slide.

WHERE Clause

- Use the optional `WHERE` clause to:
 - Filter rows in a query
 - Produce a subset of rows

- Syntax:

```
SELECT * FROM    table
[WHERE    condition];
```

- Example:

```
SELECT location_id from departments
WHERE department_name = 'Marketing';
```

The `WHERE` clause specifies a condition to filter rows, producing a subset of the rows in the table. A condition specifies a combination of one or more expressions and logical (Boolean) operators. It returns a value of `TRUE`, `FALSE`, or `NULL`. The example in the slide retrieves the `location_id` of the marketing department.

The `WHERE` clause can also be used to update or delete data from the database.

Example:

```
UPDATE departments
SET  department_name = 'Administration'
WHERE department_id = 20;
and
DELETE from departments
WHERE department_id =20;
```


ORDER BY Clause

- Use the optional `ORDER BY` clause to specify the row order.

- Syntax:

```
SELECT * FROM    table
[WHERE    condition]
[ORDER BY {<column>|<position> } [ASC|DESC] [, ...] ];
```

- Example:

```
SELECT last_name, department_id, salary
FROM employees
ORDER BY department_id ASC, salary DESC;
```

The `ORDER BY` clause specifies the order in which the rows should be displayed. The rows can be sorted in ascending or descending fashion. By default, the rows are displayed in ascending order.

The example in the slide retrieves rows from the `employees` table ordered first by ascending order of `department_id` and then by descending order of `salary`.

GROUP BY Clause

- Use the optional `GROUP BY` clause to group columns that have matching values into subsets.
- Each group has no two rows having the same value for the grouping column or columns.
- Syntax:

```
SELECT <column1, column2, ... column_n>
FROM table
[WHERE condition]
[GROUP BY <column> [, ...] ]
[ORDER BY <column> [, ...] ] ;
```

- Example:

```
SELECT department_id, MIN(salary), MAX (salary)
FROM employees
GROUP BY department_id ;
```

The `GROUP BY` clause is used to group selected rows based on the value of `expr(s)` for each row. The clause groups rows but does not guarantee order of the result set. To order the groupings, use the `ORDER BY` clause.

Any `SELECT` list elements that are not included in aggregation functions must be included in the `GROUP BY` list of elements. This includes both columns and expressions. The database returns a single row of summary information for each group.

The example in the slide returns the minimum and maximum salaries for each department in the `employees` table.

Data Definition Language

- DDL statements are used to define, structurally change, and drop schema objects.
- The commonly used DDL statements are:
 - CREATE TABLE, ALTER TABLE, and DROP TABLE
 - GRANT, REVOKE
 - TRUNCATE



DDL statements enable you to alter the attributes of an object without altering the applications that access the object. You can also use DDL statements to alter the structure of objects while database users are performing work in the database. These statements are most frequently used to:

- Create, alter, and drop schema objects and other database structures, including the database itself and database users
- Delete all the data in schema objects without removing the structure of these objects
- Grant and revoke privileges and roles

Oracle Database implicitly commits the current transaction before and after every DDL statement.

CREATE TABLE Statement

- Use the CREATE TABLE statement to create a table in the database.

- Syntax:

```
CREATE TABLE tablename (  
  {column-definition | Table-level constraint}  
  [ , {column-definition | Table-level constraint} ] * )
```

- Example:

```
CREATE TABLE teach_dept (  
  department_id NUMBER(3) PRIMARY KEY,  
  department_name VARCHAR2(10));
```

Use the CREATE TABLE statement to create a table in the database. To create a table, you must have the CREATE TABLE privilege and a storage area in which to create objects.

The table owner and the database owner automatically gain the following privileges on the table after it is created:

- INSERT
- SELECT
- REFERENCES
- ALTER
- UPDATE

The table owner and the database owner can grant the preceding privileges to other users.

ALTER TABLE Statement

- Use the `ALTER TABLE` statement to modify the definition of an existing table in the database.

- Example 1:

```
ALTER TABLE teach_dept
ADD location_id NUMBER NOT NULL;
```

- Example 2:

```
ALTER TABLE teach_dept
MODIFY department_name VARCHAR2(30) NOT NULL;
```

The `ALTER TABLE` statement allows you to make changes to an existing table.

You can:

- Add a column to a table
- Add a constraint to a table
- Modify an existing column definition
- Drop a column from a table
- Drop an existing constraint from a table
- Increase the width of the `VARCHAR` and `CHAR` columns
- Change a table to have read-only status

Example 1 in the slide adds a new column called `location_id` to the `teach_dept` table.

Example 2 updates the existing `department_name` column from `VARCHAR2(10)` to `VARCHAR2(30)` and adds a `NOT NULL` constraint to it.

DROP TABLE Statement

- The **DROP TABLE** statement removes the table and all its data from the database.
- Example:

```
DROP TABLE teach_dept;
```

- **DROP TABLE** with the **PURGE** clause drops the table and releases the space that is associated with it.

```
DROP TABLE teach_dept PURGE;
```

- The **CASCADE CONSTRAINTS** clause drops all referential integrity constraints from the table.

```
DROP TABLE teach_dept CASCADE CONSTRAINTS;
```

The **DROP TABLE** statement enables you to remove a table and its contents from the database and pushes it to the recycle bin. Dropping a table invalidates dependent objects and removes object privileges on the table.

Use the **PURGE** clause along with the **DROP TABLE** statement to release back to the tablespace the space allocated for the table. You cannot roll back a **DROP TABLE** statement with the **PURGE** clause, nor can you recover the table if you have dropped it with the **PURGE** clause.

The **CASCADE CONSTRAINTS** clause enables you to drop the reference to the primary key and unique keys in the dropped table.

GRANT Statement

- The `GRANT` statement assigns the privilege to:
 - Insert or delete data
 - Create a foreign key reference to the named table or to a subset of columns from a table
 - Select data, a view, or a subset of columns from a table
 - Create a trigger on a table
 - Execute a specified function or procedure
- Example:

```
GRANT SELECT any table to PUBLIC;
```

You can use the `GRANT` statement to:

- Assign privileges to a specific user or role, or to all users, to perform actions on database objects
- Grant a role to a user, to `PUBLIC`, or to another role

Before you issue a `GRANT` statement, check that the `derby.database.sql` authorization property is set to `True`. This property enables SQL Authorization mode. You can grant privileges on an object if you are the owner of the database.

You can grant privileges to all users by using the `PUBLIC` keyword. When `PUBLIC` is specified, the privileges or roles affect all current and future users.

Privilege Types

Assign the following privileges by using the `GRANT` statement:

- `ALL PRIVILEGES`
- `DELETE`
- `INSERT`
- `REFERENCES`
- `SELECT`
- `UPDATE`



Oracle Database provides a variety of privilege types to grant privileges to a user or role:

- `ALL PRIVILEGES`: To grant all privileges to the user or role for the specified table
- `DELETE`: To grant permission to delete rows from the specified table
- `INSERT`: To grant permission to insert rows into the specified table
- `REFERENCES`: To grant permission to create a foreign key reference to the specified table
- `SELECT`: To grant permission to perform `SELECT` statements on a table or view
- `UPDATE`: To grant permission to use the `UPDATE` statement on the specified table

For Instructor Use Only.
This document should not be distributed.

REVOKE Statement

- Use the `REVOKE` statement to remove privileges from a user to perform actions on database objects.
- Revoke a *system privilege* from a user:

```
REVOKE DROP ANY TABLE  
FROM hr;
```

- Revoke a *role* from a user:

```
REVOKE dw_manager  
FROM sh;
```

The `REVOKE` statement removes privileges from a specific user (or users) or role to perform actions on database objects. It performs the following operations:

- Revokes a role from a user, from `PUBLIC`, or from another role
- Revokes privileges for an object if you are the owner of the object or the database owner

Note: To revoke a role or system privilege, you must have been granted the privilege with the `ADMIN OPTION`.

TRUNCATE TABLE Statement

- Use the `TRUNCATE TABLE` statement to remove all the rows from a table.
- Example:

```
TRUNCATE TABLE employees_demo;
```

- By default, Oracle Database performs the following tasks:
 - Deallocates space used by the removed rows
 - Sets the `NEXT` storage parameter to the size of the last extent removed from the segment by the truncation process

The `TRUNCATE TABLE` statement deletes all the rows from a specific table. Removing rows with the `TRUNCATE TABLE` statement can be more efficient than dropping and re-creating a table. Dropping and re-creating a table:

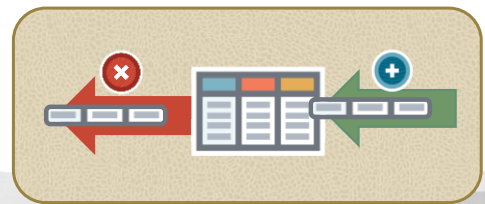
- Invalidates the dependent objects of the table
- Requires you to re-grant object privileges
- Requires you to re-create indexes, integrity constraints, and triggers
- Re-specify its storage parameters

The `TRUNCATE TABLE` statement spares you from these efforts.

Note: You cannot roll back a `TRUNCATE TABLE` statement.

Data Manipulation Language

- DML statements query or manipulate data in the existing schema objects.
- A DML statement is executed when:
 - New rows are added to a table by using the `INSERT` statement
 - Existing rows in a table are modified using the `UPDATE` statement
 - Existing rows are deleted from a table by using the `DELETE` statement
- A *transaction* consists of a collection of DML statements that form a logical unit of work.



Data Manipulation Language (DML) statements enable you to query or change the contents of an existing schema object. These statements are most frequently used to:

- Add new rows of data to a table or view by specifying a list of column values or using a subquery to select and manipulate existing data
- Change column values in the existing rows of a table or view
- Remove rows from tables or views

A collection of DML statements that forms a logical unit of work is called a transaction. Unlike DDL statements, DML statements do not implicitly commit the current transaction.

INSERT Statement

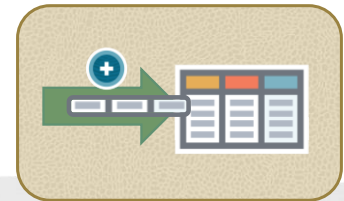
- Use the `INSERT` statement to add new rows to a table.
- Syntax:

```
INSERT INTO    table [(column [, column...])]  
VALUES        (value [, value...]);
```

- Example:

```
INSERT INTO    departments  
VALUES        (200, 'Development', 104, 1400);
```

1 rows inserted.



The `INSERT` statement adds rows to a table. Make sure to insert a new row containing values for each column and to list the values in the default order of the columns in the table. Optionally, you can also list the columns in the `INSERT` statement.

Example:

```
INSERT INTO job_history (employee_id, start_date, end_date, job_id)  
VALUES (120, '25-JUL-06', '12-FEB_08', 'AC_ACCOUNT');
```

The syntax discussed in the slide allows you to insert a single row at a time. The `VALUES` keyword assigns the values of expressions to the corresponding columns in the column list.

UPDATE Statement Syntax

- Use the `UPDATE` statement to modify the existing rows in a table.
- Update more than one row at a time (if required).

```
UPDATE      table
SET         column = value [, column = value, ...]
[WHERE      condition];
```

- Example:

```
UPDATE  copy_emp
SET
22 rows updated
```

- Specify `SET column_name= NULL` to update a column value to `NULL`.



The `UPDATE` statement modifies the existing values in a table. Confirm the update operation by querying the table to display the updated rows. You can modify a specific row or rows by specifying the `WHERE` clause.

Example:

```
UPDATE employees
SET      salary = 17500
WHERE    employee_id = 102;
```

In general, use the primary key column in the `WHERE` clause to identify the row to update. For example, to update a specific row in the `employees` table, use `employee_id` to identify the row instead of `employee_name`, because more than one employee may have the same name.

Note: Typically, the `condition` keyword is composed of column names, expressions, constants, subqueries, and comparison operators.

DELETE Statement

- Use the `DELETE` statement to delete the existing rows from a table.
- Syntax:

```
DELETE [FROM] table
[WHERE condition];
```

- Write the `DELETE` statement using the `WHERE` clause to delete specific rows from a table.

```
DELETE FROM departments
WHERE department_name = 'Finance';
1 rows deleted
```



The `DELETE` statement removes existing rows from a table. You must use the `WHERE` clause to delete a specific row or rows from a table based on the condition. The `condition` identifies the rows to be deleted. It may contain column names, expressions, constants, subqueries, and comparison operators.

The first example in the slide deletes the finance department from the `departments` table. You can confirm the delete operation by using the `SELECT` statement to query the table.

```
SELECT *
FROM departments
WHERE department_name = 'Finance';
```

If you omit the `WHERE` clause, all rows in the table are deleted. Example:

```
DELETE FROM copy_emp;
```

The preceding example deletes all the rows from the `copy_emp` table.

Transaction Control Statements

- Transaction control statements are used to manage the changes made by DML statements.
- The DML statements are grouped into transactions.
- Transaction control statements:
 - COMMIT
 - ROLLBACK
 - SAVEPOINT



A transaction is a sequence of SQL statements that Oracle Database treats as a single unit. Transaction control statements are used in a database to manage the changes made by DML statements and to group these statements into transactions.

Each transaction is assigned a unique `transaction_id`, and it groups SQL statements so that they are either all committed, which means they are applied to the database, or all rolled back, which means they are undone from the database.

COMMIT Statement

- Use the `COMMIT` statement to:
 - Permanently save the changes made to the database during the current transaction
 - Erase all savepoints in the transaction
 - Release transaction locks
- Example:

```
INSERT INTO    departments
VALUES        (201, 'Engineering', 106, 1400);
COMMIT;
```

```
1 rows inserted.
committed.
```

The `COMMIT` statement ends the current transaction by making all the pending data changes permanent. It releases all row and table locks and erases any savepoints that you may have marked since the last commit or rollback. The changes made using the `COMMIT` statement are visible to all users.

Oracle recommends that you explicitly end every transaction in your application programs with a `COMMIT` or `ROLLBACK` statement, including the last transaction, before disconnecting from Oracle Database. If you do not explicitly commit the transaction and the program terminates abnormally, the last uncommitted transaction is automatically rolled back.

Note: Oracle Database issues an implicit `COMMIT` before and after any data definition language (DDL) statement.

ROLLBACK Statement

- Use the `ROLLBACK` statement to undo changes made to the database during the current transaction.
- Use the `TO SAVEPOINT` clause to undo a part of the transaction after the savepoint.
- Example:

```
UPDATE      employees
SET         salary = 7000
WHERE      last_name = 'Ernst';
SAVEPOINT   Ernst_sal;

UPDATE      employees
SET         salary = 12000
WHERE      last_name = 'Mourgos';

ROLLBACK TO SAVEPOINT Ernst_sal;
```

The `ROLLBACK` statement undoes work done in the current transaction. To roll back the current transaction, no privileges are necessary.

Using `ROLLBACK` with the `TO SAVEPOINT` clause performs the following operations:

- Rolls back only the portion of the transaction after the savepoint
- Erases all savepoints created after that savepoint. The named savepoint is retained, so you can roll back to the same savepoint multiple times.

Using `ROLLBACK` without the `TO SAVEPOINT` clause performs the following operations:

- Ends the transaction
- Undoes all the changes in the current transaction
- Erases all savepoints in the transaction

SAVEPOINT Statement

- Use the `SAVEPOINT` statement to name and mark the current point in the processing of a transaction.
- Specify a name to each savepoint.
- Use distinct savepoint names within a transaction to avoid overriding.
- Syntax:

```
SAVEPOINT ;
```

The `SAVEPOINT` statement identifies a point in a transaction to which you can later roll back. You must specify a distinct name for each savepoint. If you create a second savepoint with the same identifier as an earlier savepoint, the earlier savepoint is erased.

After a savepoint has been created, you can continue processing, commit your work, roll back the entire transaction, or roll back to the savepoint.

A simple rollback or commit erases all savepoints. When you roll back to a savepoint, any savepoints marked after that savepoint are erased. The savepoint to which you have rolled back is retained.

When savepoint names are reused within a transaction, the Oracle Database moves (overrides) the savepoint from its old position to the current point in the transaction.

Joins

Use a join to query data from more than one table:

```
SELECT    table1.column, table2.column
FROM table1, table2
WHERE table1.column1 = table2.column2;
```

- Write the join condition in the `WHERE` clause.
- Prefix the column name with the table name when the same column name appears in more than one table.



When data from more than one table in the database is required, a *join* condition is used. Rows in one table can be joined to rows in another table according to common values that exist in the corresponding columns (usually primary and foreign key columns).

To display data from two or more related tables, write a simple join condition in the `WHERE` clause.

In the syntax:

<code>table1.column</code>	Denotes the table and column from which data is retrieved
<code>table1.column1 = table2.column2</code>	Is the condition that joins (or relates) the tables together

Guidelines

- When writing a `SELECT` statement that joins tables, precede the column name with the table name for clarity and to enhance database access.
- If the same column name appears in more than one table, the column name must be prefixed with the table name.
- To join n tables together, you need a minimum of $n-1$ join conditions. For example, to join four tables, a minimum of three joins is required. This rule may not apply if your table has a concatenated primary key, in which case more than one column is required to uniquely identify each row.

Types of Joins

- Natural join
- Equijoin
- Nonequijoin
- Outer join
- Self-join
- Cross join



To join tables, you can use Oracle's join syntax.

Note: The join syntax was proprietary in Oracle 9i and earlier releases. The SQL:1999-compliant join syntax does not offer any performance benefits over the Oracle-proprietary join syntax. Oracle Database 23c complies with ANSI/ISO SQL:2016 standards.

Qualifying Ambiguous Column Names

- Use table prefixes to:
 - Qualify column names that are in multiple tables
 - Improve performance
- Use table aliases instead of full table name prefixes.
- Table aliases give a table a shorter name.
- Use column aliases to distinguish columns that have identical names but reside in different tables.



When joining two or more tables, you need to qualify the names of the columns with the table name to avoid ambiguity. Without the table prefixes, the `DEPARTMENT_ID` column in the `SELECT` list could be from either the `DEPARTMENTS` table or the `EMPLOYEES` table. Therefore, it is necessary to add the table prefix to execute your query. If there are no common column names between the two tables, there is no need to qualify the columns. However, using a table prefix improves performance, because you tell the Oracle server exactly where to find the columns.

Qualifying column names with table names can be very time consuming, particularly if table names are lengthy. Therefore, you can use *table aliases*, instead of table names. Just as a column alias gives a column another name, a table alias gives a table another name. Table aliases help to keep SQL code smaller, thereby using less memory.

The table name is specified in full, followed by a space, and then the table alias. For example, the `EMPLOYEES` table can be given an alias of `e` and the `DEPARTMENTS` table an alias of `d`.

Guidelines

- Table aliases can be up to 30 characters in length, but shorter aliases are better than longer ones.
- If a table alias is used for a particular table name in the `FROM` clause, that table alias must be substituted for the table name throughout the `SELECT` statement.
- Table aliases should be meaningful.
- A table alias is valid only for the current `SELECT` statement.

Natural Join

- The `NATURAL JOIN` clause is based on all the columns in the two tables that have the same name.
- It selects rows from tables that have the same names and data values of columns.
- Example:

```
SELECT country_id, location_id, country_name, city
FROM countries NATURAL JOIN locations;
```

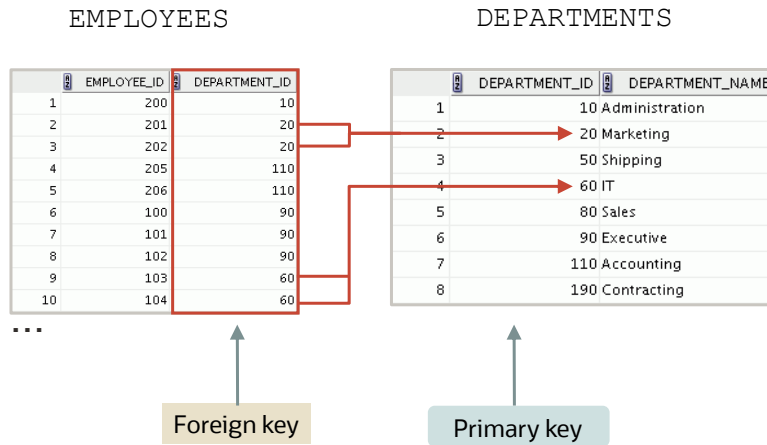
	COUNTRY_ID	LOCATION_ID	COUNTRY_NAME	CITY
1	US	1400	United States of America	Southlake
2	US	1500	United States of America	South San Francisco
3	US	1700	United States of America	Seattle
4	CA	1800	Canada	Toronto
5	UK	2500	United Kingdom	Oxford

You can join tables automatically based on the columns in the two tables that have matching data types and names. You do this by using the `NATURAL JOIN` keywords.

Note: The join can happen only on those columns that have the same names and data types in both tables. If the columns have the same name but different data types, the `NATURAL JOIN` syntax causes an error.

In the example in the slide, the `COUNTRIES` table is joined to the `LOCATIONS` table by the `COUNTRY_ID` column, which is the only column of the same name in both tables. If other common columns were present, the join would have used them all.

Equijoins



An **equijoin** is a join with a join condition containing an equality operator. An equijoin combines rows that have equivalent values for the specified columns. To determine an employee's department name, you compare the values in the `DEPARTMENT_ID` column in the `EMPLOYEES` table with the `DEPARTMENT_ID` values in the `DEPARTMENTS` table. The relationship between the `EMPLOYEES` and `DEPARTMENTS` tables is an *equijoin*; that is, values in the `DEPARTMENT_ID` column in both tables must be equal. Often, this type of join involves primary and foreign key complements.

Note: Equijoins are also called *simple joins*.

Retrieving Records with Equijoins

```
SELECT e.employee_id, e.last_name, e.department_id,  
       d.department_id, d.location_id  
FROM   employees e JOIN departments d  
ON     e.department_id = d.department_id;
```

EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID_1	LOCATION_ID
1	200 Whalen	10	10	1700
2	201 Hartstein	20	20	1800
3	202 Fay	20	20	1800
4	144 Vargas	50	50	1500
5	143 Matos	50	50	1500
6	142 Davies	50	50	1500
7	141 Rajs	50	50	1500
8	124 Mourgos	50	50	1500
9	103 Hunold	60	60	1400
10	104 Ernst	60	60	1400
11	107 Lorentz	60	60	1400

...

In the example in the slide:

- The **SELECT** clause specifies the column names to retrieve:
 - Employee last name, employee ID, and department ID, which are columns in the `EMPLOYEES` table
 - Department ID and location ID, which are columns in the `DEPARTMENTS` table
- The **FROM** clause specifies the two tables that the database must access:
 - `EMPLOYEES` table
 - `DEPARTMENTS` table
- The **WHERE** clause specifies how the tables are to be joined:

`e.department_id = d.department_id`

Because the `DEPARTMENT_ID` column is common to both tables, it must be prefixed with the table alias to avoid ambiguity. Other columns that are not present in both the tables need not be qualified by a table alias, but it is recommended for better performance.

Note: When you use the Execute Statement icon to run the query, SQL Developer suffixes a “_1” to differentiate between the two `DEPARTMENT_IDS`.

Add Additional Search Conditions to the ON clause Using AND or WHERE

```
SELECT d.department_id, d.department_name, l.city
FROM departments d JOIN locations l
ON d.location_id = l.location_id
AND d.department_id IN (20, 50);
```

	DEPARTMENT_ID	DEPARTMENT_NAME	CITY
1	20	Marketing	Toronto
2	50	Shipping	South San Francisco

```
SELECT d.department_id, d.department_name, l.city
FROM departments d JOIN locations l
ON d.location_id = l.location_id
WHERE d.department_id IN (20, 50);
```

In addition to the join, you may have criteria for your `WHERE` clause to restrict the rows in consideration for one or more tables in the join. The example in the slide performs a join on the `DEPARTMENTS` and `LOCATIONS` tables and, in addition, displays only those departments with ID equal to 20 or 50. To add additional conditions to the `ON` clause, you can add `AND` clauses. Alternatively, you can use a `WHERE` clause to apply additional conditions.

Both queries produce the same output.

	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Matos	50	Shipping

Retrieving Records with Nonequijoins

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e JOIN job_grades j
ON     e.salary
      BETWEEN j.lowest_sal AND j.highest_sal;
```

	LAST_NAME	SALARY	GRADE_LEVEL
1	Vargas	2500	A
2	Matos	2600	A
3	Davies	3100	B
4	Rajs	3500	B
5	Lorentz	4200	B
6	Whalen	4400	B
7	Fay	6000	C

...

The example in the slide creates a nonequijoin to evaluate an employee's salary grade. The salary must be between any pair of the low and high salary ranges.

It is important to note that all employees appear exactly once when this query is executed. No employee is repeated in the list. There are two reasons for this:

- None of the rows in the job grade table contain grades that overlap. That is, the salary value for an employee can lie only between the low salary and high salary values of one of the rows in the salary grade table.
- All of the employees' salaries lie within the limits that are provided by the job grade table. That is, no employee earns less than the lowest value contained in the `LOWEST_SAL` column or more than the highest value contained in the `HIGHEST_SAL` column.

Note: Other conditions (such as `<=` and `>=`) can be used, but `BETWEEN` is the simplest. Remember to specify the low value first and the high value last when using the `BETWEEN` condition. The Oracle server translates the `BETWEEN` condition to a pair of `AND` conditions. Therefore, using `BETWEEN` has no performance benefits, but should be used only for logical simplicity.

Table aliases have been specified in the example in the slide for performance reasons, not because of possible ambiguity.

Retrieving Records by Using the USING Clause

- You can use the `USING` clause to match only one column when more than one column matches.
- You cannot specify this clause with a `NATURAL` join.
- Do not qualify the column name with a table name or table alias.
- Example:

```
SELECT country_id, country_name, location_id, city
FROM   countries JOIN locations
      USING (country_id) ;
```

	COUNTRY_ID	COUNTRY_NAME	LOCATION_ID	CITY
1	US	United States of America	1400	Southlake
2	US	United States of America	1500	South San Francisco
3	US	United States of America	1700	Seattle
4	CA	Canada	1800	Toronto
5	UK	United Kingdom	2500	Oxford

In the example in the slide, the `COUNTRY_ID` columns in the `COUNTRIES` and `LOCATIONS` tables are joined, and thus the `LOCATION_ID` of the location where an employee works is shown.

Retrieving Records by Using the ON Clause

- The join condition for the natural join is basically an equijoin of all columns with the same name.
- Use the ON clause to specify arbitrary conditions or specify columns to join.
- The ON clause makes code easy to understand.

```
SELECT e.employee_id, e.last_name, j.department_id,  
FROM   employees e JOIN job_history j  
ON     (e.employee_id = j.employee_id);
```

	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID
1	101	Kochhar	110
2	101	Kochhar	110
3	102	De Haan	60
4	176	Taylor	80
5	176	Taylor	80
6	200	Whalen	90
7	200	Whalen	90
8	201	Hartstein	20

Use the ON clause to specify a join condition. With this, you can specify join conditions separate from any search or filter conditions in the WHERE clause.

In this example, the EMPLOYEE_ID columns in the EMPLOYEES and JOB_HISTORY tables are joined using the ON clause. Wherever an employee ID in the EMPLOYEES table equals an employee ID in the JOB_HISTORY table, the row is returned. The table alias is necessary to qualify the matching column names.

You can also use the ON clause to join columns that have different names. The parentheses around the joined columns, as in the example in the slide, (e.employee_id = j.employee_id), are optional. So, even ON e.employee_id = j.employee_id will work.

Note: When you use the Execute Statement icon to run the query, SQL Developer suffixes a '_1' to differentiate between the two employee_ids.

Left Outer Join

- A join between two tables that returns all matched rows, as well as the unmatched rows from the left table, is called a **LEFT OUTER JOIN**.
- Example:

```
SELECT c.country_id, c.country_name, l.location_id, l.city
FROM   countries c LEFT OUTER JOIN locations l
ON     (c.country_id = l.country_id) ;
```

	COUNTRY_ID	COUNTRY_NAME	LOCATION_ID	CITY
1	CA	Canada	1800	Toronto
2	DE	Germany	(null)	(null)
3	UK	United Kingdom	2500	Oxford
4	US	United States of America	1400	Southlake
5	US	United States of America	1500	South San Francisco
6	US	United States of America	1700	Seattle

This query retrieves all the rows in the **COUNTRIES** table, which is the left table, even if there is no match in the **LOCATIONS** table.

Right Outer Join

- A join between two tables that returns all matched rows, as well as the unmatched rows from the right table, is called a RIGHT OUTER JOIN.
- Example:

```
SELECT e.last_name, d.department_id, d.department_name
FROM   employees e RIGHT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```

	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Whalen	10	Administration
2	Hartstein	20	Marketing
3	Fay	20	Marketing
4	Davies	50	Shipping
...			
18	Higgins	110	Accounting
19	Gietz	110	Accounting
20	(null)	190	Contracting

This query retrieves all the rows in the DEPARTMENTS table, which is the table at the right, even if there is no match in the EMPLOYEES table.

Full Outer Join

- A join between two tables that returns all matched rows, as well as the unmatched rows from both tables, is called a `FULL OUTER JOIN`.
- Example:

```
SELECT e.last_name, d.department_id, d.manager_id,  
       d.department_name  
FROM   employees e FULL OUTER JOIN departments d  
ON     (e.manager_id = d.manager_id) ;
```

	LAST_NAME	DEPARTMENT_ID	MANAGER_ID	DEPARTMENT_NAME
1	King	(null)	(null)	(null)
...				
19	Higgins	(null)	(null)	(null)
20	Gietz	110		205 Accounting
21	(null)	190	(null)	Contracting
22	(null)	10		200 Administration

This query retrieves all the rows in the `EMPLOYEES` table, even if there is no match in the `DEPARTMENTS` table. It also retrieves all the rows in the `DEPARTMENTS` table, even if there is no match in the `EMPLOYEES` table.

Self-Join: Example

```
SELECT worker.last_name || ' works for '
       || manager.last_name
FROM   employees worker JOIN employees manager
       ON worker.manager_id = manager.employee_id
ORDER BY worker.last_name;
```

	WORKER.LAST_NAME 'WORKSFOR' MANAGER.LAST_NAME
1	Abel works for Zlotkey
2	Davies works for Mourgos
3	De Haan works for King
4	Ernst works for Hunold
5	Fay works for Hartstein
6	Gietz works for Higgins
7	Grant works for Zlotkey
8	Hartstein works for King
9	Higgins works for Kochhar

...

Sometimes you need to join a table to itself. To find the name of each employee's manager, you need to join the `EMPLOYEES` table to itself or perform a self-join. The example in the slide joins the `EMPLOYEES` table to itself. To simulate two tables in the `FROM` clause, there are two aliases, namely `worker` and `manager`, for the same table, `EMPLOYEES`.

In this example, the `WHERE` clause contains the join that means "where a worker's manager ID matches the employee ID for the manager."

Cross Join

- A `CROSS JOIN` is a `JOIN` operation that produces the Cartesian product of two tables.
- Example:

```
SELECT department_name, city  
FROM department CROSS JOIN location;
```

	DEPARTMENT_NAME	CITY
1	Administration	Oxford
2	Administration	Seattle
3	Administration	South San Francisco
4	Administration	Southlake
5	Administration	Toronto
6	Marketing	Oxford
7	Marketing	Seattle
8	Marketing	South San Francisco
9	Marketing	Southlake
10	Marketing	Toronto

...

The `CROSS JOIN` syntax specifies the cross product. It is also known as a Cartesian product. A cross join produces the cross product of two relations and is essentially the same as the comma-delimited Oracle Database notation.

You do not specify any `WHERE` condition between the two tables in the `CROSS JOIN`.

Summary



The `SELECT` statement to retrieve rows from one or more tables

DDL statements to alter the structure of objects

DML statements to manipulate data in the existing schema objects

Transaction control statements to manage the changes made by DML statements

Joins to display data from multiple tables

There are many commonly used commands and statements in SQL. It includes the DDL statements, DML statements, transaction control statements, and joins.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Generating Reports by Grouping Related Data



For Instructor Use Only.

This document should not be distributed.

Objectives



The `ROLLUP` operation to produce subtotal values

The `CUBE` operation to produce cross-tabulation values

The `GROUPING` function to identify the row values created by `ROLLUP` or `CUBE`

`GROUPING SETS` to produce a single result set

In this appendix, you learn how to:

- Group data to obtain subtotal values by using the `ROLLUP` operator
- Group data to obtain cross-tabulation values by using the `CUBE` operator
- Use the `GROUPING` function to identify the level of aggregation in the result set produced by a `ROLLUP` or `CUBE` operator
- Use `GROUPING SETS` to produce a single result set that is equivalent to a `UNION ALL`

Review of Group Functions

- Group functions operate on sets of rows to give one result per group.

```
SELECT      [column,] group_function(column) . . .  
FROM        table  
[WHERE      condition]  
[GROUP BY   group_by_expression]  
[ORDER BY   column];
```

- Example:

```
SELECT AVG(salary), STDDEV(salary),  
       COUNT(commission_pct), MAX(hire_date)  
FROM   employees  
WHERE  job_id LIKE 'SA%';
```

You can use the `GROUP BY` clause to divide the rows in a table into groups. You can then use group functions to return summary information for each group. Group functions can appear in select lists, and in `ORDER BY` and `HAVING` clauses. The Oracle server applies the group functions to each group of rows and returns a single result row for each group.

Types of group functions: Each of the group functions—`AVG`, `SUM`, `MAX`, `MIN`, `COUNT`, `STDDEV`, and `VARIANCE`—accepts one argument. The `AVG`, `SUM`, `STDDEV`, and `VARIANCE` functions operate only on numeric values. `MAX` and `MIN` can operate on numeric, character, or date data values. `COUNT` returns the number of non-NULL rows for the given expression. The example in the slide calculates the average salary, standard deviation on the salary, number of employees earning a commission, and the maximum hire date for those employees whose `JOB_ID` begins with SA.

Guidelines for Using Group Functions

- The data types for the arguments can be `CHAR`, `VARCHAR2`, `NUMBER`, or `DATE`.
- All group functions except `COUNT (*)` ignore null values. To substitute a value for null values, use the `NVL` function. `COUNT` returns either a number or zero.
- The Oracle server implicitly sorts the result set in ascending order of the grouping columns specified, when you use a `GROUP BY` clause. To override this default ordering, you can use `DESC` in an `ORDER BY` clause.

Review of the GROUP BY Clause

- Syntax:

```
SELECT      [column,] group_function(column). . .  
FROM        table  
[WHERE      condition]  
[GROUP BY   group_by_expression]  
[ORDER BY   column];
```

- Example:

```
SELECT  department_id, job_id, SUM(salary),  
        COUNT(employee_id)  
FROM    employees  
GROUP BY department_id, job_id ;
```

The example illustrated in the slide is evaluated by the Oracle server as follows:

- The `SELECT` clause specifies that the following columns be retrieved:
 - Department ID and job ID columns from the `EMPLOYEES` table
 - The sum of all the salaries and the number of employees in each group that you have specified in the `GROUP BY` clause
- The `GROUP BY` clause specifies how the rows should be grouped in the table. The total salary and the number of employees are calculated for each job ID within each department. The rows are grouped by department ID and then grouped by job within each department.

Review of the HAVING Clause

- Use the HAVING clause to specify which groups are to be displayed.
- You further restrict the groups on the basis of a limiting condition.

```
SELECT      [column,] group_function(column) ...  
FROM        table  
[WHERE      condition]  
[GROUP BY   group by expression]  
[HAVING     having_expression]  
[ORDER BY   column];
```

Groups are formed and group functions are calculated before the HAVING clause is applied to the groups. The HAVING clause can precede the GROUP BY clause, but it is recommended that you place the GROUP BY clause first, because the GROUP BY clause is more logical than the HAVING clause.

The Oracle server performs the following steps when you use the HAVING clause:

1. It groups rows.
2. It applies the group functions to the groups and displays the groups that match the criteria in the HAVING clause.

GROUP BY with ROLLUP and CUBE Operators

Use `ROLLUP` or `CUBE` with `GROUP BY` to produce superaggregate rows by cross-referencing columns.

- `ROLLUP` grouping produces a result set containing the regular grouped rows and the subtotal values.
- `CUBE` grouping produces a result set containing the rows from `ROLLUP` and cross-tabulation rows.



You specify `ROLLUP` and `CUBE` operators in the `GROUP BY` clause of a query. `ROLLUP` grouping produces a result set containing the regular grouped rows and subtotal rows. The `ROLLUP` operator also calculates a grand total. The `CUBE` operation in the `GROUP BY` clause groups the selected rows based on the values of all possible combinations of expressions in the specification and returns a single row of summary information for each group. You can use the `CUBE` operator to produce cross-tabulation rows.

Note: When working with `ROLLUP` and `CUBE`, make sure that the columns following the `GROUP BY` clause have meaningful, real-life relationships with each other; otherwise, the operators return irrelevant information.

ROLLUP Operator

- ROLLUP is an extension of the GROUP BY clause.
- Use the ROLLUP operation to produce cumulative aggregates, such as subtotals.

```
SELECT      [column, ]group_function(column). . .
FROM        table
[WHERE      condition]
[GROUP BY   ROLLUP group_by_expression]
[HAVING     having_expression];
[ORDER BY   column];
```

The ROLLUP operator delivers aggregates and superaggregates for expressions within a GROUP BY statement. The ROLLUP operator can be used by report writers to extract statistics and summary information from result sets. The cumulative aggregates can be used in reports, charts, and graphs.

The ROLLUP operator creates groupings by moving in one direction, from right to left, along the list of columns specified in the GROUP BY clause. It then applies the aggregate function to these groupings.

Notes

- To produce subtotals in n dimensions (that is, n columns in the GROUP BY clause) without a ROLLUP operator, $n+1$ SELECT statements must be linked with UNION ALL. This makes the query execution inefficient because each of the SELECT statements causes table access. The ROLLUP operator gathers its results with just one table access. The ROLLUP operator is useful when there are many columns involved in producing the subtotals.
- Subtotals and totals are produced with ROLLUP. CUBE produces totals as well, but effectively rolls up in each possible direction, thereby producing cross-tabular data.

ROLLUP Operator: Example

```
SELECT department_id, job_id, SUM(salary)
FROM employees
WHERE department_id < 60
GROUP BY ROLLUP(department_id, job_id);
```

	DEPARTMENT_ID	JOB_ID	SUM(SALARY)	
1	10	AD_ASST	4400	1
2	10	(null)	4400	
3	20	MK_MAN	13000	
4	20	MK_REP	6000	
5	20	(null)	19000	
6	30	PU_MAN	11000	
7	30	PU_CLERK	13900	
8	30	(null)	24900	
9	40	HR_REP	6500	
10	40	(null)	6500	2
11	50	ST_MAN	36400	
12	50	SH_CLERK	64300	
13	50	ST_CLERK	55700	
14	50	(null)	156400	
15	(null)	(null)	211200	3

In the example in the slide:

- Total salaries for every job ID within a department for those departments whose department ID is less than 60 are displayed by the `GROUP BY` clause
- The `ROLLUP` operator displays:
 - The total salary for each department whose department ID is less than 60
 - The total salary for all departments whose department ID is less than 60, irrespective of the job IDs

In this example, 1 indicates a group totaled by both `DEPARTMENT_ID` and `JOB_ID`, 2 indicates a group totaled only by `DEPARTMENT_ID`, and 3 indicates the grand total.

The `ROLLUP` operator creates subtotals that roll up from the most detailed level to a grand total, following the grouping list specified in the `GROUP BY` clause. First, it calculates the standard aggregate values for the groups specified in the `GROUP BY` clause (in the example, the sum of salaries grouped on each job within a department). Then it creates progressively higher-level subtotals, moving from right to left through the list of grouping columns. (In the example, the sum of salaries for each department is calculated, followed by the sum of salaries for all departments.)

- Given n expressions in the `ROLLUP` operator of the `GROUP BY` clause, the operation results in $n + 1$ (in this case, $2 + 1 = 3$) groupings.
- Rows based on the values of the first n expressions are called rows or regular rows, and the others are called superaggregate rows.

CUBE Operator

- CUBE is an extension of the GROUP BY clause.
- Use the CUBE operator to produce cross-tabulation values with a single SELECT statement.

```
SELECT      [column, ] group_function(column)...  
FROM        table  
[WHERE      condition]  
[GROUP BY   CUBE group_by_expression]  
[HAVING     having_expression]  
[ORDER BY   column];
```

The CUBE operator is an additional switch in the GROUP BY clause in a SELECT statement. The CUBE operator can be applied to all aggregate functions, including AVG, SUM, MAX, MIN, and COUNT. It is used to produce result sets that are typically used for cross-tabular reports. ROLLUP produces only a fraction of possible subtotal combinations, whereas CUBE produces subtotals for all possible combinations of groupings specified in the GROUP BY clause, and a grand total.

The CUBE operator is used with an aggregate function to generate additional rows in a result set. Columns included in the GROUP BY clause are cross-referenced to produce a superset of groups. The aggregate function specified in the select list is applied to these groups to produce summary values for the additional superaggregate rows. The number of extra groups in the result set is determined by the number of columns included in the GROUP BY clause.

In fact, every possible combination of the columns or expressions in the GROUP BY clause is used to produce superaggregates. If you have n columns or expressions in the GROUP BY clause, there will be 2^n possible superaggregate combinations. Mathematically, these combinations form an n -dimensional cube, which is how the operator got its name.

By using application or programming tools, these superaggregate values can then be fed into charts and graphs that convey results and relationships visually and effectively.

CUBE Operator: Example

```
SELECT  department_id, job_id, SUM(salary)
FROM    employees
WHERE   department_id < 60
GROUP BY CUBE (department_id, job_id) ;
```

	DEPARTMENT_ID	JOB_ID	SUM(SALARY)	
1	(null)	(null)	211200	1
2	(null)	HR_REP	6500	
3	(null)	MK_MAN	13000	
4	(null)	MK_REP	6000	
5	(null)	PU_MAN	11000	
6	(null)	ST_MAN	36400	2
7	(null)	AD_ASST	4400	
8	(null)	PU_CLERK	13900	
9	(null)	SH_CLERK	64300	
10	(null)	ST_CLERK	55700	
11	10	(null)	4400	
12	10	AD_ASST	4400	3
13	20	(null)	19000	
14	20	MK_MAN	13000	
15	20	MK_REP	6000	
16	30	(null)	24900	4

The output of the `SELECT` statement in the example can be interpreted as follows:

- The total salary for every job within a department (for departments with a department ID of lower than 60)
- The total salary for each department with a department ID of lower than 60
- The total salary for each job irrespective of the department
- The total salary for those departments with a department ID of lower than 60, irrespective of the job title

In this example, 1 indicates the grand total, 2 indicates the rows totaled by `JOB_ID` alone, 3 indicates some of the rows totaled by `DEPARTMENT_ID` and `JOB_ID`, and 4 indicates some of the rows totaled by `DEPARTMENT_ID` alone.

The `CUBE` operator has also performed the `ROLLUP` operation to display the subtotals for those departments whose department ID is less than 60 and the total salary for those departments whose department ID is less than 60, irrespective of the job titles. Further, the `CUBE` operator displays the total salary for every job irrespective of the department.

Note: Similar to the `ROLLUP` operator, producing subtotals in n dimensions (that is, n columns in the `GROUP BY` clause) without a `CUBE` operator requires that 2^n `SELECT` statements be linked with `UNION ALL`. Thus, a report with three dimensions requires $2^3 = 8$ `SELECT` statements to be linked with `UNION ALL`.

GROUPING Function

- Is used:
 - With either the CUBE or ROLLUP operator
 - To find the groups forming the subtotal in a row
 - To differentiate stored NULL values from NULL values created by ROLLUP or CUBE
- Returns 0 or 1

```
SELECT    [column,] group_function(column) .. ,  
          GROUPING(expr)  
FROM      table  
[WHERE    condition]  
[GROUP BY [ROLLUP][CUBE] group_by_expression]  
[HAVING   having_expression]  
[ORDER BY column];
```

- The GROUPING function can be used with either the CUBE or the ROLLUP operator to help you understand how a summary value has been obtained.
- It uses a single column as its argument. The *expr* in the GROUPING function must match one of the expressions in the GROUP BY clause. The function returns a value of 0 or 1.
- The values returned by the GROUPING function are useful to:
 - Determine the level of aggregation of a given subtotal (that is, the group or groups on which the subtotal is based)
 - Identify whether a NULL value in the expression column of a row of the result set indicates:
 - A NULL value from the base table (stored NULL value)
 - A NULL value created by ROLLUP or CUBE (as a result of a group function on that expression)
- A value of 0 returned by the GROUPING function based on an expression indicates one of the following:
 - The expression has been used to calculate the aggregate value.
 - The NULL value in the expression column is a stored NULL value.
- A value of 1 returned by the GROUPING function based on an expression indicates one of the following:
 - The expression has not been used to calculate the aggregate value.
 - The NULL value in the expression column is created by ROLLUP or CUBE as a result of grouping.

GROUPING Function: Example

```
SELECT  department_id DEPTID, job_id JOB,
        SUM(salary),
        GROUPING(department_id) GRP_DEPT,
        GROUPING(job_id) GRP_JOB
FROM    employees
WHERE   department_id < 50
GROUP BY ROLLUP(department_id, job_id);
```

	DEPTID	JOB	SUM(SALARY)	GRP_DEPT	GRP_JOB
1	10	AD_ASST	4400	0	0
2	10	(null)	4400	0	1
3	20	MK_MAN	13000	0	0
4	20	MK_REP	6000	0	0
5	20	(null)	19000	0	1
6	30	PU_MAN	11000	0	0
7	30	PU_CLERK	13900	0	0
8	30	(null)	24900	0	1
9	40	HR_REP	6500	0	0
10	40	(null)	6500	0	1
11	(null)	(null)	54800	1	1

In the example in the slide, consider the summary value 4400 in the first row (labeled 1). This summary value is the total salary for the job ID of AD_ASST within department 10. To calculate this summary value, both the DEPARTMENT_ID and JOB_ID columns have been taken into account. Thus, a value of 0 is returned for both the GROUPING(department_id) and GROUPING(job_id) expressions.

Consider the summary value 4400 in the second row (labeled 2). This value is the total salary for department 10 and has been calculated by taking into account the DEPARTMENT_ID column; thus, a value of 0 has been returned by GROUPING(department_id). Because the JOB_ID column has not been taken into account to calculate this value, a value of 1 has been returned for GROUPING(job_id). You can observe similar output in the fifth row.

In the last row, consider the summary value 54800 (labeled 3). This is the total salary for those departments whose department ID is less than 50 and all job titles. To calculate this summary value, neither the DEPARTMENT_ID nor the JOB_ID columns has been taken into account. Thus, a value of 1 is returned for both GROUPING(department_id) and GROUPING(job_id) expressions.

GROUPING SETS

- The `GROUPING SETS` syntax is used to define multiple groupings in the same query.
 - All groupings specified in the `GROUPING SETS` clause are computed and the results of individual groupings are combined with a `UNION ALL` operation.
- Grouping set efficiency:
 - Only one pass over the base table is required.
 - There is no need to write complex `UNION` statements.
 - The more elements `GROUPING SETS` has, the greater is the performance benefit.



`GROUPING SETS` is a further extension of the `GROUP BY` clause that you can use to specify multiple groupings of data. Doing so facilitates efficient aggregation, which in turn facilitates analysis of data across multiple dimensions.

A single `SELECT` statement can now be written by using `GROUPING SETS` to specify various groupings (which can also include `ROLLUP` or `CUBE` operators), rather than multiple `SELECT` statements combined by `UNION ALL` operators.

For example:

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY
GROUPING SETS
((department_id, job_id, manager_id),
 (department_id, manager_id),(job_id, manager_id));
```

This statement calculates aggregates over three groupings:

`(department_id, job_id, manager_id)`, `(department_id, manager_id)`, and `(job_id, manager_id)`

Without this feature, multiple queries combined together with `UNION ALL` are required to obtain the output of the preceding `SELECT` statement. A multiquery approach is inefficient because it requires multiple scans of the same data.

Compare the previous example with the following alternative:

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY CUBE(department_id, job_id, manager_id);
```

This statement computes all the 8 ($2 * 2 * 2$) groupings, although only the (department_id, job_id, manager_id), (department_id, manager_id), and (job_id, manager_id) groups are of interest to you.

Another alternative is the following statement:

```
SELECT department_id, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY department_id, job_id, manager_id
UNION ALL
SELECT department_id, NULL, manager_id, AVG(salary)
FROM employees
GROUP BY department_id, manager_id
UNION ALL
SELECT NULL, job_id, manager_id, AVG(salary)
FROM employees
GROUP BY job_id, manager_id;
```

This statement requires three scans of the base table, which makes it inefficient.

CUBE and ROLLUP can be thought of as grouping sets with very specific semantics and results. The following equivalencies show this fact:

CUBE(a, b, c) is equivalent to	GROUPING SETS ((a, b, c), (a, b), (a, c), (b, c), (a), (b), (c), ())
ROLLUP(a, b, c) is equivalent to	GROUPING SETS ((a, b, c), (a, b), (a), ())

GROUPING SETS: Example

```
SELECT  department_id, job_id,
        manager_id, AVG(salary)
FROM    employees
GROUP BY GROUPING SETS
        ((department_id, job_id), (job_id, manager_id));
```

	DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVG(SALARY)
1	(null)	SH_CLERK	122	3200
2	(null)	AC_MGR	101	12000
3	(null)	ST_MAN	100	7280
4	...	(null)	121	2675

1

	DEPARTMENT_ID	JOB_ID	MANAGER_ID	AVG(SALARY)
39	110	AC_MGR	(null)	12000
40	90	AD_PRES	(null)	24000
41	60	IT_PROG	(null)	5760
42	100	FI_MGR	(null)	12000

2

...

The query in the slide calculates aggregates over two groupings. The table is divided into the following groups:

- Department ID, Job ID
- Job ID, Manager ID

The average salaries for each of these groups are calculated. The result set displays the average salary for each of the two groups.

In the output, the group marked as 1 can be interpreted as the following:

- The average salary of all employees with the SH_CLERK job ID under manager 122 is \$ 3,200.
- The average salary of all employees with the AC_MGR job ID under manager 101 is \$ 12,000, and so on.

The group marked as 2 in the output is interpreted as the following:

- The average salary of all employees with the AC_MGR job ID in department 110 is \$ 12,000.
- The average salary of all employees with the AD_PRES job ID in department 90 is \$ 24,000, and so on.

The example in the slide can also be written as:

```
SELECT department_id, job_id, NULL as manager_id,  
       AVG(salary) as AVGSAL  
  
FROM employees  
  
GROUP BY department_id, job_id  
  
UNION ALL  
  
SELECT NULL, job_id, manager_id, avg(salary) as AVGSAL  
  
FROM employees  
  
GROUP BY job_id, manager_id;
```

In the absence of an optimizer that looks across query blocks to generate the execution plan, the preceding query will need two scans of the base table `EMPLOYEES`. This could be very inefficient. Therefore, the usage of the `GROUPING SETS` statement is recommended.

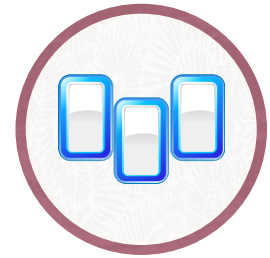
For Instructor Use Only.
This document should not be distributed.

Composite Columns

- A composite column is a collection of columns that are treated as a unit.

```
ROLLUP (a, (b, c), d)
```

- Use parentheses within the `GROUP BY` clause to group columns so that they are treated as a unit while computing `ROLLUP` or `CUBE` operations.
- When used with `ROLLUP` or `CUBE`, composite columns would require skipping aggregation across certain levels.



A composite column is a collection of columns that are treated as a unit during the computation of groupings. You specify the columns in parentheses as in the following statement: `ROLLUP (a, (b, c), d)`

Here, `(b, c)` forms a composite column and is treated as a unit. In general, composite columns are useful in `ROLLUP`, `CUBE`, and `GROUPING SETS`. For example, in `CUBE` or `ROLLUP`, composite columns would require skipping aggregation across certain levels.

That is, `GROUP BY ROLLUP(a, (b, c))` is equivalent to:

```
GROUP BY a, b, c UNION ALL
```

```
GROUP BY a UNION ALL
```

```
GROUP BY ()
```

Here, `(b, c)` is treated as a unit and `ROLLUP` is not applied across `(b, c)`. It is as though you have an alias—for example, `z` as an alias for `(b, c)`, and the `GROUP BY` expression reduces to: `GROUP BY ROLLUP(a, z)`.

Note: `GROUP BY ()` is typically a `SELECT` statement with `NULL` values for the columns `a` and `b` and only the aggregate function. It is generally used for generating grand totals.

```
SELECT NULL, NULL, aggregate_col
```

```
FROM <table_name>
```

```
GROUP BY ();
```

Compare this with the normal ROLLUP.

For example:

```
GROUP BY ROLLUP(a, b, c)
```

This would be equivalent to:

```
GROUP BY a, b, c UNION ALL
```

```
GROUP BY a, b UNION ALL
```

```
GROUP BY a UNION ALL
```

```
GROUP BY ()
```

Similarly:

```
GROUP BY CUBE((a, b), c)
```

This would be equivalent to:

```
GROUP BY a, b, c UNION ALL
```

```
GROUP BY a, b UNION ALL
```

```
GROUP BY c UNION ALL
```

```
GROUP BY ()
```

The following table shows the GROUPING SETS specification and the equivalent GROUP BY specification.

GROUPING SETS Statements	Equivalent GROUP BY Statements
GROUP BY GROUPING SETS(a, b, c)	GROUP BY a UNION ALL GROUP BY b UNION ALL GROUP BY c
GROUP BY GROUPING SETS(a, b, (b, c)) (The GROUPING SETS expression has a composite column.)	GROUP BY a UNION ALL GROUP BY b UNION ALL GROUP BY b, c
GROUP BY GROUPING SETS((a, b, c))	GROUP BY a, b, c
GROUP BY GROUPING SETS(a, (b), ())	GROUP BY a UNION ALL GROUP BY b UNION ALL GROUP BY ()
GROUP BY GROUPING SETS (a, ROLLUP(b, c)) (The GROUPING SETS expression has a composite column.)	GROUP BY a UNION ALL GROUP BY ROLLUP(b, c)

Composite Columns: Example

```
SELECT  department_id, job_id, manager_id,  
        SUM(salary)  
FROM    employees  
GROUP BY ROLLUP( department_id, (job_id, manager_id) );
```

	DEPARTMENT_ID	JOB_ID	MANAGER_ID	SUM(SALARY)
1	(null)	SA_REP	149	7000
2	(null)	(null)	(null)	7000
3	10	AD_ASST	101	4400
4	10	(null)	(null)	4400
5	20	MK_MAN	100	13000
6	20	MK_REP	201	6000
7	20	(null)	(null)	19000
...				
	100	FL_MGR	101	12000
	100	FL_ACCOUNT	108	39600
	100	(null)	(null)	51600
	110	AC_MGR	101	12000
	110	AC_ACCOUNT	205	8300
	110	(null)	(null)	20300
	(null)	(null)	(null)	691400

Consider the example:

```
SELECT department_id, job_id, manager_id, SUM(salary)  
  
FROM    employees  
  
GROUP BY ROLLUP( department_id, job_id, manager_id);
```

This query results in the Oracle server computing the following groupings:

- (job_id, manager_id)
- (department_id, job_id, manager_id)
- (department_id)
- Grand total

If you are interested only in specific groups, you cannot limit the calculation to those groupings without using composite columns. With composite columns, this is possible by treating `JOB_ID` and `MANAGER_ID` columns as a single unit while rolling up. Columns enclosed in parentheses are treated as a unit while computing `ROLLUP` and `CUBE`. This is illustrated in the example in the slide. By enclosing the `JOB_ID` and `MANAGER_ID` columns in parentheses, you indicate to the Oracle server to treat `JOB_ID` and `MANAGER_ID` as a single unit, that is, as a composite column.

The example in the slide computes the following groupings:

- department_id, job_id, manager_id)
- department_id)
- ()

The example in the slide displays the following:

- Total salary for every job and manager (labeled 1)
- Total salary for every department, job, and manager (labeled 2)
- Total salary for every department (labeled 3)
- Grand total (labeled 4)

The example in the slide can also be written as:

```
SELECT      department_id, job_id, manager_id, SUM(salary)

FROM        employees

GROUP BY    department_id, job_id, manager_id

UNION ALL   SELECT department_id, TO_CHAR(NULL), TO_NUMBER(NULL),
              SUM(salary)

FROM        employees

GROUP BY    department_id

UNION ALL   SELECT  TO_NUMBER(NULL),
                  TO_CHAR(NULL), TO_NUMBER(NULL), SUM(salary)

FROM        employees

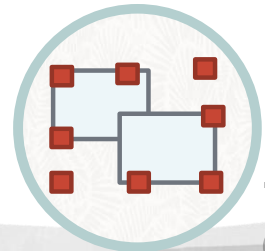
GROUP BY    ();
```

In the absence of an optimizer that looks across query blocks to generate the execution plan, the preceding query would need three scans of the base table, EMPLOYEES. This could be very inefficient. Therefore, the use of composite columns is recommended.

Concatenated Groupings

- Concatenated groupings offer a concise way to generate useful combinations of groupings.
- To specify concatenated grouping sets, you separate multiple grouping sets, `ROLLUP`, and `CUBE` operations with commas, so that the Oracle server combines them into a single `GROUP BY` clause.
- The result is a cross-product of groupings from each `GROUPING SET`.

```
GROUP BY GROUPING SETS(a, b), GROUPING SETS(c, d)
```



Concatenated groupings offer a concise way to generate useful combinations of groupings. The concatenated groupings are specified by listing multiple grouping sets, `CUBE`s, and `ROLLUP`s and separating them with commas. The following is an example of concatenated grouping sets:

```
GROUP BY GROUPING SETS(a, b), GROUPING SETS(c, d)
```

This SQL example defines the following groupings:

```
(a, c), (a, d), (b, c), (b, d)
```

Concatenation of grouping sets is very helpful for these reasons:

- **Ease of query development:** You need not manually enumerate all groupings.
- **Use by applications:** SQL generated by online analytical processing (OLAP) applications often involves concatenation of grouping sets, with each `GROUPING SET` defining groupings needed for a dimension.

Concatenated Groupings: Example

```
SELECT  department_id, job_id, manager_id,
        SUM(salary)
FROM    employees
GROUP BY department_id,
        ROLLUP (job_id),
        CUBE (manager_id);
```

	DEPARTMENT_ID	JOB_ID	MANAGER_ID	SUM(SALARY)
1	(null)	SA_REP	149	7000
2		10 AD_ASST	101	4400
3		20 MK_MAN	100	13000
4		20 MK_REP	201	6000
...				
1		90 AD_VP	100	34000
		90 AD PRES	(null)	24000
...				
		(null) SA_REP	(null)	7000
		10 AD_ASST	(null)	4400
...				
2		110 (null)	101	12000
		110 (null)	205	8300
		110 (null)	(null)	20300

The example in the slide results in the following groupings:

- (department_id,job_id,) (1)
- (department_id,manager_id) (2)
- (department_id) (3)

The total salary for each of these groups is calculated.

The following is another example of a concatenated grouping.

```
SELECT department_id, job_id, manager_id, SUM(salary) total
FROM employees
WHERE department_id<60
GROUP BY GROUPING SETS(department_id),
GROUPING SETS (job_id, manager_id);
```

Summary



ROLLUP operation to produce subtotal values

CUBE operation to produce cross-tabulation values

GROUPING function to identify the row values created by ROLLUP or CUBE

GROUPING SETS syntax to define multiple groupings in the same query

GROUP BY clause to combine expressions in various ways:

- Composite columns
- Concatenated grouping sets

- ROLLUP and CUBE are extensions of the GROUP BY clause.
- ROLLUP is used to display subtotal and grand total values.
- CUBE is used to display cross-tabulation values.
- The GROUPING function enables you to determine whether a row is an aggregate produced by a CUBE or ROLLUP operator.
- With the GROUPING SETS syntax, you can define multiple groupings in the same query. GROUP BY computes all the groupings specified and combines them with UNION ALL.
- Within the GROUP BY clause, you can combine expressions in various ways:
 - To specify composite columns, you group columns within parentheses so that the Oracle server treats them as a unit while computing ROLLUP or CUBE operations.
 - To specify concatenated grouping sets, you separate multiple grouping sets, ROLLUP, and CUBE operations with commas so that the Oracle server combines them into a single GROUP BY clause. The result is a cross-product of groupings from each grouping set.

For Instructor Use Only.
This document should not be distributed.

Hierarchical Retrieval



For Instructor Use Only.

This document should not be distributed.

Objectives



Interpret the concept of a hierarchical query

Create a tree-structured report

Format hierarchical data

Exclude branches from the tree structure

In this appendix, you learn how to use hierarchical queries to create tree-structured reports.

Sample Data from the EMPLOYEES Table

EMPLOYEE_ID	LAST_NAME	JOB_ID	MANAGER_ID
1	100 King	AD_PRES	(null)
2	101 Kochhar	AD_VP	100
3	102 De Haan	AD_VP	100
4	103 Hunold	IT_PROG	102
5	104 Ernst	IT_PROG	103
6	107 Lorentz	IT_PROG	103
...			
16	200 Whalen	AD_ASST	101
17	201 Hartstein	MK_MAN	100
18	202 Fay	MK_REP	201
19	205 Higgins	AC_MGR	101
20	206 Gietz	AC_ACCOUNT	205

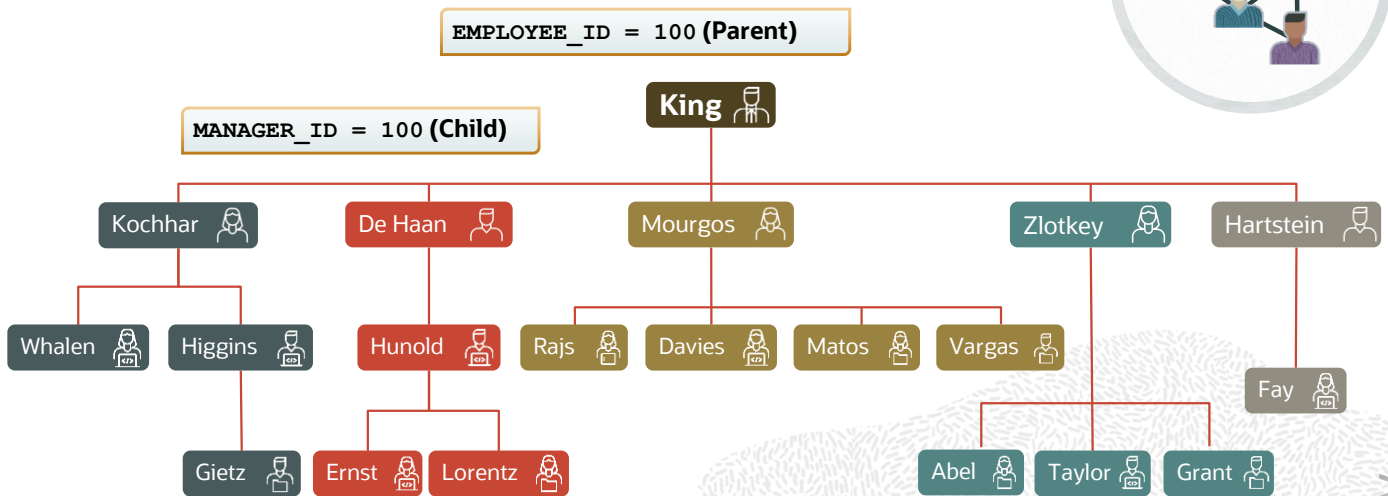
Using hierarchical queries, you can retrieve data based on a natural hierarchical relationship between the rows in a table. A relational database does not store records in a hierarchical way. However, where a hierarchical relationship exists between the rows of a single table, a process called *tree walking* enables the hierarchy to be constructed. A hierarchical query is a method of reporting, with the branches of a tree in a specific order.

Imagine a family tree with the eldest members of the family found close to the base or trunk of the tree and the youngest members representing branches of the tree. Branches can have their own branches, and so on.

A hierarchical query is possible when a relationship exists between rows in a table. For example, in the slide, you see that Kochhar, De Haan, and Hartstein report to `MANAGER_ID` 100, which is King's `EMPLOYEE_ID`.

Note: Hierarchical trees are used in various fields such as human genealogy (family trees), livestock (breeding purposes), corporate management (management hierarchies), manufacturing (product assembly), evolutionary research (species development), and scientific research.

Natural Tree Structure



The `EMPLOYEES` table has a tree structure representing the management reporting line. The hierarchy can be created by looking at the relationship between equivalent values in the `EMPLOYEE_ID` and `MANAGER_ID` columns. This relationship can be exploited by joining the table to itself. The `MANAGER_ID` column contains the employee number of the employee's manager.

The parent/child relationship of a tree structure enables you to control:

- The direction in which the hierarchy is walked
- The starting point inside the hierarchy

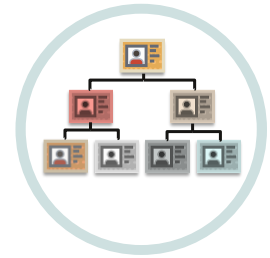
Note: The slide displays an inverted tree structure of the management hierarchy of the employees in the `EMPLOYEES` table.

Hierarchical Queries

```
SELECT [LEVEL], column, expr...  
FROM table  
[WHERE condition(s)]  
[START WITH condition(s)]  
[CONNECT BY PRIOR condition(s)] ;
```

condition:

```
expr comparison_operator expr
```



Keywords and Clauses

Hierarchical queries can be identified by the presence of the `CONNECT BY` and `START WITH` clauses.

In the syntax:

<code>SELECT</code>	Is the standard <code>SELECT</code> clause
<code>LEVEL</code>	For each row returned by a hierarchical query, returns 1 for a root row, 2 for a child of a root, and so on
<code>FROM table</code>	Specifies the table, view, or snapshot containing the columns. You can select from only one table.
<code>WHERE</code>	Restricts the rows returned by the query without affecting other rows of the hierarchy
<code>condition</code>	Is a comparison with expressions
<code>START WITH</code>	Specifies the root rows of the hierarchy (where to start). This clause is required for a true hierarchical query.
<code>CONNECT BY PRIOR</code>	Specifies the columns in which the relationship between parent and child <code>PRIOR</code> rows exist. This clause is required for a hierarchical query.

Walking the Tree

Starting Point

- Specifies the condition that must be met
- Accepts any valid condition

```
START WITH column1 = value
```

- Using the EMPLOYEES table, start with the employee whose last name is Kochhar.

```
...START WITH last_name = 'Kochhar'
```

The row or rows to be used as the root of the tree are determined by the `START WITH` clause. The `START WITH` clause can contain any valid condition.

Examples

Using the EMPLOYEES table, start with King, the president of the company.

```
... START WITH manager_id IS NULL
```

Using the EMPLOYEES table, start with employee Kochhar. A `START WITH` condition can contain a subquery.

```
... START WITH employee_id = (SELECT employee_id
                               FROM   employees
                               WHERE  last_name =A 'Kochhar')
```

If the `START WITH` clause is omitted, the tree walk is started with all the rows in the table as root rows.

Note: The `CONNECT BY` and `START WITH` clauses are not American National Standards Institute (ANSI) SQL standard.

Walking the Tree

```
CONNECT BY PRIOR column1 = column2
```

- Walk from the top down, using the EMPLOYEES table.

```
... CONNECT BY PRIOR employee_id = manager_id
```

Direction

Top down	→	Column1 = Parent Key Column2 = Child Key
Bottom up	→	Column1 = Child Key Column2 = Parent Key

The direction of the query is determined by the `CONNECT BY PRIOR` column placement. For top-down, the `PRIOR` operator refers to the parent row. For bottom-up, the `PRIOR` operator refers to the child row. To find the child rows of a parent row, the Oracle server evaluates the `PRIOR` expression for the parent row and the other expressions for each row in the table. Rows for which the condition is true are the child rows of the parent. The Oracle server always selects child rows by evaluating the `CONNECT BY` condition with respect to a current parent row.

Examples

Walk from the top down using the `EMPLOYEES` table. Define a hierarchical relationship in which the `EMPLOYEE_ID` value of the parent row is equal to the `MANAGER_ID` value of the child row:

```
... CONNECT BY PRIOR employee_id = manager_id
```

Walk from the bottom up using the `EMPLOYEES` table:

```
... CONNECT BY PRIOR manager_id = employee_id
```

The `PRIOR` operator does not necessarily need to be coded immediately following `CONNECT BY`. Thus, the following `CONNECT BY PRIOR` clause gives the same result as the one in the preceding example:

```
... CONNECT BY employee_id = PRIOR manager_id
```

Note: The `CONNECT BY` clause cannot contain a subquery.

Walking the Tree: From the Bottom Up

```
SELECT employee_id, last_name, job_id, manager_id
FROM   employees
START WITH employee_id = 101
CONNECT BY PRIOR manager_id = employee_id ;
```

	EMPLOYEE_ID	LAST_NAME	JOB_ID	MANAGER_ID
1	101	Kochhar	AD_VP	100
2	100	King	AD_PRES	(null)

The example in the slide displays a list of managers starting with the employee whose employee ID is 101.

Walking the Tree: From the Top Down

```
SELECT last_name||' reports to '||  
PRIOR last_name "Walk Top Down"  
FROM employees  
START WITH last_name = 'King'  
CONNECT BY PRIOR employee_id = manager_id ;
```

Walk Top Down
1 King reports to
2 King reports to
3 Kochhar reports to King
4 Greenberg reports to Kochhar
5 Faviet reports to Greenberg
...
105 Grant reports to Zlotkey
106 Johnson reports to Zlotkey
107 Hartstein reports to King
108 Fay reports to Hartstein

Walking from the top down, display the names of the employees and their manager. Use employee King as the starting point. Print only one column.

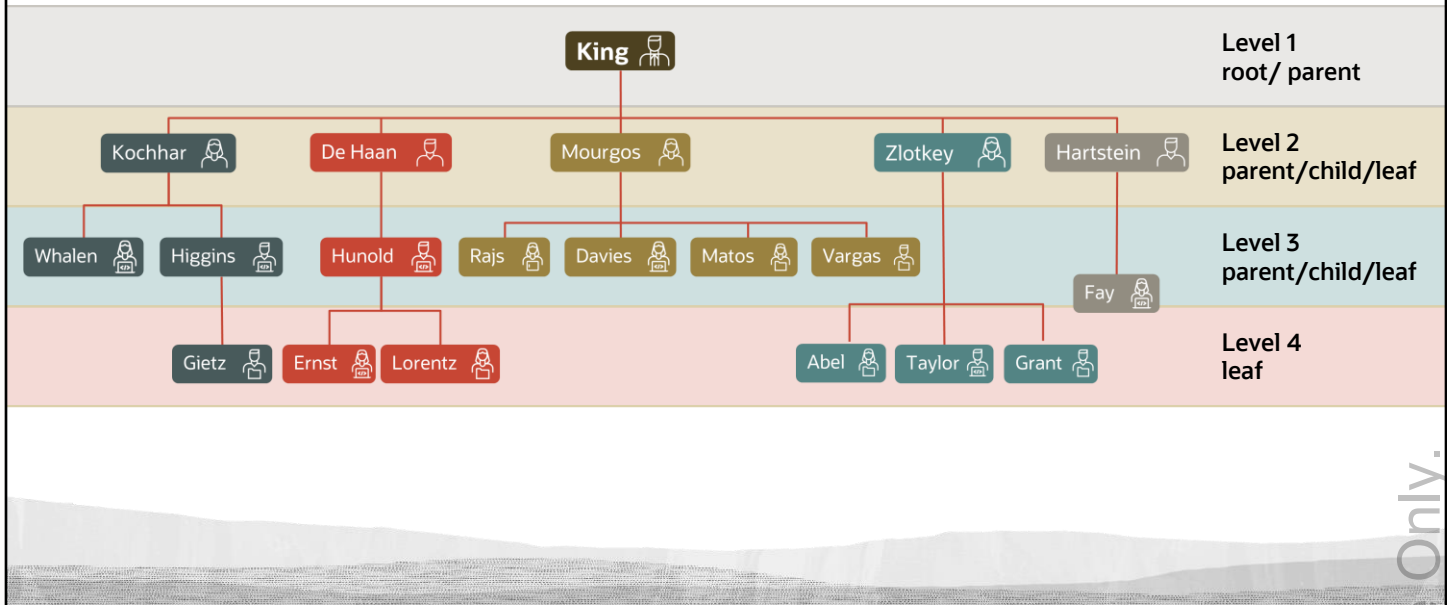
Example

In the following example, `EMPLOYEE_ID` values are evaluated for the parent row and `MANAGER_ID` and `SALARY` values are evaluated for the child rows. The `PRIOR` operator applies only to the `EMPLOYEE_ID` value.

```
... CONNECT BY PRIOR employee_id = manager_id  
AND salary > 15000;
```

To qualify as a child row, a row must have a `MANAGER_ID` value equal to the `EMPLOYEE_ID` value of the parent row and must have a `SALARY` value greater than \$15,000.

Ranking Rows with the LEVEL Pseudocolumn



You can explicitly show the rank or level of a row in the hierarchy by using the `LEVEL` pseudocolumn. This will make your report more readable. The forks where one or more branches split away from a larger branch are called nodes, and the very end of a branch is called a leaf or leaf node. The graphic in the slide shows the nodes of the inverted tree with their `LEVEL` values. For example, employee Higgins is a parent and a child, whereas employee Davies is a child and a leaf.

LEVEL Pseudocolumn

Value	Level for Top Down	Level for Bottom up
1	A root node	A root node
2	A child of a root node	The parent of a root node
3	A child of a child, and so on	A parent of a parent, and so on

In the slide, King is the root or parent (`LEVEL = 1`). Kochhar, De Haan, Mourgos, Zlotkey, Hartstein, Higgins, and Hunold are children and also parents (`LEVEL = 2`). Whalen, Rajs, Davies, Matos, Vargas, Gietz, Ernst, Lorentz, Abel, Taylor, Grant, and Fay are children and leaves (`LEVEL = 3` and `LEVEL = 4`).

Note: A *root node* is the highest node within an inverted tree. A *child node* is any nonroot node. A parent node is any node that has children. A leaf node is any node without children. The number of levels returned by a hierarchical query may be limited by available user memory.

Formatting Hierarchical Reports Using LEVEL and LPAD

Create a report displaying company management levels, beginning with the highest level and indenting each of the following levels.

```
COLUMN org_chart FORMAT A12
SELECT LPAD(last_name, LENGTH(last_name) + (LEVEL*2) - 2, '_')
       AS org_chart
FROM   employees
START WITH first_name='Steven' AND last_name='King'
CONNECT BY PRIOR employee_id=manager_id
```

The nodes in a tree are assigned level numbers from the root. Use the LPAD function in conjunction with the LEVEL pseudocolumn to display a hierarchical report as an indented tree.

In the example in the slide:

- `LPAD(char1, n [, char2])` returns *char1*, left-padded to length *n* with the sequence of characters in *char2*. The argument *n* is the total length of the return value as it is displayed on your terminal screen.
- `LPAD(last_name, LENGTH(last_name) + (LEVEL*2) - 2, '_')` defines the display format
- *char1* is the `LAST_NAME`, *n* the total length of the return value, is length of the `LAST_NAME` + $(LEVEL*2) - 2$, and *char2* is `'_'`

That is, this tells SQL to take the `LAST_NAME` and left-pad it with the `'_'` character until the length of the resultant string is equal to the value determined by $LENGTH(last_name) + (LEVEL*2) - 2$.

For King, `LEVEL` = 1. Therefore, $(2 * 1) - 2 = 2 - 2 = 0$. So King does not get padded with any `'_'` character and is displayed in column 1.

For Kochhar, `LEVEL = 2`. Therefore, $(2 * 2) - 2 = 4 - 2 = 2$. So Kochhar gets padded with 2 ' _ ' characters and is displayed indented.

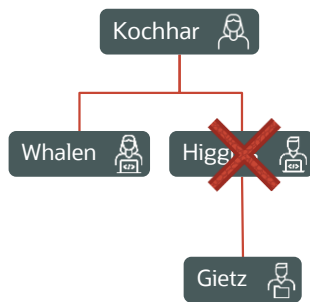
The rest of the records in the `EMPLOYEES` table are displayed similarly.

	ORG_CHART
1	King
2	_Kochhar
3	__Greenberg
4	____Faviet
5	_____Chen
6	_____Sciarra
7	_____Urman
8	_____Popp
9	____Whalen
10	____Mavris
11	____Baer
12	____Higgins
13	_____Gietz
14	____De Haan
15	____Hunold
16	_____Ernst
17	_____Austin

Pruning Branches

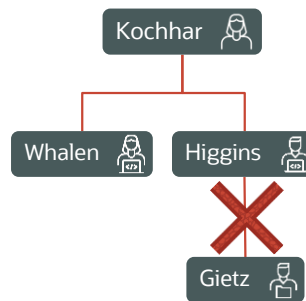
Use the `WHERE` clause
to eliminate a node.

```
WHERE last_name != 'Higgins'
```



Use the `CONNECT BY` clause
to eliminate a branch.

```
CONNECT BY PRIOR  
employee_id = manager_id  
AND last_name != 'Higgins'
```



You can use the `WHERE` and `CONNECT BY` clauses to prune the tree (that is, to control which nodes or rows are displayed). The predicate you use acts as a Boolean condition.

Examples

Starting at the root, walk from the top down and eliminate employee Higgins in the result, but process the child rows.

```
SELECT department_id, employee_id, last_name, job_id, salary  
FROM employees  
WHERE last_name != 'Higgins'  
START WITH manager_id IS NULL  
CONNECT BY PRIOR employee_id = manager_id;
```

Starting at the root, walk from the top down and eliminate employee Higgins and all child rows.

```
SELECT department_id, employee_id, last_name, job_id, salary  
FROM employees  
START WITH manager_id IS NULL  
CONNECT BY PRIOR employee_id = manager_id  
AND last_name != 'Higgins';
```


Summary



Use hierarchical queries to view a hierarchical relationship between rows in a table

Specify the direction and starting point of the query

Eliminate nodes or branches by pruning

You can use hierarchical queries to retrieve data based on a natural hierarchical relationship between rows in a table. The `LEVEL` pseudocolumn counts how far down a hierarchical tree you have traveled. You can specify the direction of the query using the `CONNECT BY PRIOR` clause. You can specify the starting point using the `START WITH` clause. You can use the `WHERE` and `CONNECT BY` clauses to prune the tree branches.

Writing Advanced Scripts



For Instructor Use Only.

This document should not be distributed.

Objectives



Describe the type of problems that are solved by using SQL to generate SQL

Create a basic SQL script

Capture the output in a file

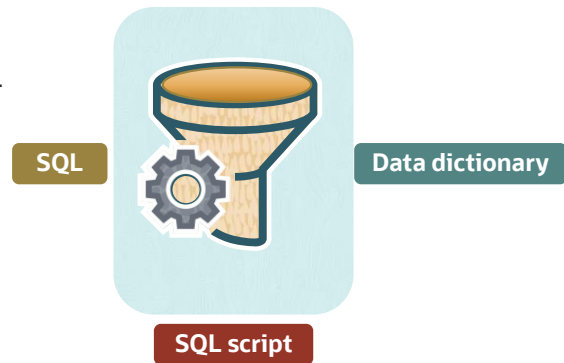
Dump the contents of a table to a file

Generate a dynamic predicate

In this appendix, you learn how to write a SQL script to generate a SQL script.

Using SQL to Generate SQL

- SQL can be used to generate scripts in SQL.
- The data dictionary is:
 - A collection of tables and views that contain database information
 - Created and maintained by the Oracle server



SQL can be a powerful tool to generate other SQL statements. In most cases, this involves writing a script file. You can use SQL-from-SQL to:

- Avoid repetitive coding
- Access information from the data dictionary
- Drop or re-create database objects
- Generate dynamic predicates that contain run-time parameters

The examples used in this appendix involve selecting information from the data dictionary. The data dictionary is a collection of tables and views that contain information about the database. This collection is created and maintained by the Oracle server. All data dictionary tables are owned by the `sys` user. Information stored in the data dictionary includes names of Oracle server users, privileges granted to users, database object names, table constraints, and audit information. There are four categories of data dictionary views. Each category has a distinct prefix that reflects its intended use.

Prefix	Description
USER_	Contains details of objects owned by the user
ALL_	Contains details of objects to which the user has been granted access rights, in addition to objects owned by the user
DBA_	Contains details of users with DBA privileges to access any object in the database
V\$_	Stores information about database server performance and locking; available only to the DBA

Creating a Basic Script

```
SELECT 'CREATE TABLE ' || table_name ||  
      '_test ' || 'AS SELECT * FROM '  
      || table_name || ' WHERE 1=2;'  
      AS "Create Table Script"  
FROM   user_tables;
```

1	Create Table Script
1	CREATE TABLE REGIONS_test AS SELECT * FROM REGIONS WHERE 1=2;
2	CREATE TABLE LOCATIONS_test AS SELECT * FROM LOCATIONS WHERE 1=2;
3	CREATE TABLE DEPARTMENTS_test AS SELECT * FROM DEPARTMENTS WHERE 1=2;
4	CREATE TABLE JOBS_test AS SELECT * FROM JOBS WHERE 1=2;
5	CREATE TABLE EMPLOYEES_test AS SELECT * FROM EMPLOYEES WHERE 1=2;
6	CREATE TABLE JOB_HISTORY_test AS SELECT * FROM JOB_HISTORY WHERE 1=2;

The example in the slide produces a report with `CREATE TABLE` statements from every table you own. Each `CREATE TABLE` statement produced in the report includes the syntax to create a table using the table name with a suffix of `_test` and having only the structure of the corresponding existing table. The old table name is obtained from the `TABLE_NAME` column of the data dictionary view `USER_TABLES`.

The next step is to enhance the report to automate the process.

Note: You can query the data dictionary tables to view various database objects that you own. The data dictionary views frequently used include:

- `USER_TABLES`: Displays description of the user's own tables
- `USER_OBJECTS`: Displays all the objects owned by the user
- `USER_TAB_PRIVS_MADE`: Displays all grants on objects owned by the user
- `USER_COL_PRIVS_MADE`: Displays all grants on columns of objects owned by the user

Controlling the Environment

```
SET ECHO OFF  
SET FEEDBACK OFF  
SET PAGESIZE 0
```

SQL statement

```
SET FEEDBACK ON  
SET PAGESIZE 24  
SET ECHO ON
```

Set system variables to appropriate values.

Set system variables back to the default value.

To execute the SQL statements that are generated, you must capture them in a file that can then be run. You must also plan to clean up the output that is generated and make sure that you suppress elements such as headings, feedback messages, top titles, and so on. In SQL Developer, you can save these statements to a script. To save the contents of the Enter SQL Statement box, click the Save icon or select Save from the File menu. Alternatively, you can right-click in the Enter SQL Statement box and select the Save File option from the drop-down menu.

Note: Some of the SQL*Plus statements are not supported by SQL Worksheet. For the complete list of SQL*Plus statements that are supported, and not supported by SQL Worksheet, refer to the topic titled *SQL*Plus Statements Supported and Not Supported in SQL Worksheet* in SQL Developer Online Help.

The Complete Picture

```
SET ECHO OFF
SET FEEDBACK OFF
SET PAGESIZE 0

SELECT 'DROP TABLE ' || object_name || ';'
FROM   user_objects
WHERE  object_type = 'TABLE'
/

SET FEEDBACK ON
SET PAGESIZE 24
SET ECHO ON
```

The output of the command in the slide is saved into a file called `dropem.sql` in SQL Developer. To save the output into a file in SQL Developer, you use the Save File option under the Script Output pane. The `dropem.sql` file contains the following data. This file can now be started from SQL Developer by locating the script file, loading it, and executing it.

	'DROPTABLE' OBJECT_NAME ';'
1	DROP TABLE REGIONS;
2	DROP TABLE COUNTRIES;
3	DROP TABLE LOCATIONS;
4	DROP TABLE DEPARTMENTS;
5	DROP TABLE JOBS;
6	DROP TABLE EMPLOYEES;
7	DROP TABLE JOB_HISTORY;
8	DROP TABLE JOB_GRADES;

Dumping the Contents of a Table to a File

```
SET HEADING OFF ECHO OFF FEEDBACK OFF
SET PAGESIZE 0

SELECT
  'INSERT INTO departments_test VALUES
  (' || department_id || ', ' || department_name ||
  ', ' || location_id || ');'
  AS "Insert Statements Script"
FROM   departments
/

SET PAGESIZE 24
SET HEADING ON ECHO ON FEEDBACK ON
```

Sometimes, it is useful to have the values for the rows of a table in a text file in the format of an `INSERT INTO VALUES` statement. This script can be run to populate the table in case the table has been dropped accidentally.

The example in the slide produces `INSERT` statements for the `DEPARTMENTS_TEST` table, captured in the `data.sql` file using the Save File option in SQL Developer.

The contents of the `data.sql` script file are as follows:

```
INSERT INTO departments_test VALUES
  (10, 'Administration', 1700);
INSERT INTO departments_test VALUES
  (20, 'Marketing', 1800);
INSERT INTO departments_test VALUES
  (50, 'Shipping', 1500);
INSERT INTO departments_test VALUES
  (60, 'IT', 1400);
...
```


Dumping the Contents of a Table to a File

Source	Result
<code>'''X'''</code>	<code>'X'</code>
<code>''''</code>	<code>'</code>
<code>'''' department_name ''''</code>	<code>'Administration'</code>
<code>''', '''</code>	<code>', '</code>
<code>''');'</code>	<code>');'</code>

You may have noticed the large number of single quotation marks in the previous slide. A set of four single quotation marks produces one single quotation mark in the final statement. Also remember that character and date values must be enclosed within quotation marks.

Within a string, to display one quotation mark, you need to prefix it with another single quotation mark. For example, in the fifth example in the slide, the surrounding quotation marks are for the entire string. The second quotation mark acts as a prefix to display the third quotation mark. Thus, the result is a single quotation mark followed by the parenthesis, followed by the semicolon.

Generating a Dynamic Predicate

```
COLUMN my_col NEW_VALUE dyn_where_clause

SELECT DECODE('&&deptno', null,
DECODE ('&&hiredate', null, ' ',
'WHERE hire_date=TO_DATE('' || '&&hiredate'', 'DD-MON-YYYY''))',
DECODE ('&&hiredate', null,
'WHERE department_id = ' || '&&deptno',
'WHERE department_id = ' || '&&deptno' ||
' AND hire_date = TO_DATE('' || '&&hiredate'', 'DD-MON-YYYY''))'
AS my_col FROM dual;
```

```
SELECT last_name FROM employees &dyn_where_clause;
```

The example in the slide generates a `SELECT` statement that retrieves data of all employees in a department who were hired on a specific day. The script generates the `WHERE` clause dynamically.

Note: After the user variable is in place, you must use the `UNDEFINE` command to delete it.

The first `SELECT` statement prompts you to enter the department number. If you do not enter any department number, the department number is treated as null by the `DECODE` function and the user is then prompted for the hire date. If you do not enter any hire date, the hire date is treated as null by the `DECODE` function and the dynamic `WHERE` clause that is generated is also a null, which causes the second `SELECT` statement to retrieve all the rows from the `EMPLOYEES` table.

Note: The `NEW_V[ALUE]` variable specifies a variable to hold a column value. You can reference the variable in `TTITLE` commands. Use `NEW_VALUE` to display column values or the date in the top title. You must include the column in a `BREAK` command with the `SKIP PAGE` action. The variable name cannot contain a pound sign (`#`). `NEW_VALUE` is useful for master/detail reports in which there is a new master record for each page.

Note: Here, the hire date must be entered in the DD-MON-YYYY format.

The SELECT statement in the slide can be interpreted as follows:

```
IF    (<<deptno>> is not entered) THEN
      IF    (<<hiredate>> is not entered) THEN
          return empty string
      ELSE
          return the string 'WHERE hire_date = TO_DATE('<<hiredate>>',
'DD-MON-YYYY')'
      ELSE
          IF    (<<hiredate>> is not entered) THEN
              return the string 'WHERE department_id = <<deptno>>
entered'
          ELSE
              return the string 'WHERE department_id = <<deptno>>
entered
AND hire_date = TO_DATE('
<<hiredate>>', 'DD-MON-YYYY')'
      END IF
```

The returned string becomes the value of the DYN_WHERE_CLAUSE variable, which will be used in the second SELECT statement.

Note: Use SQL*Plus for these examples.

When the first example in the slide is executed, the user is prompted for the values for DEPTNO and HIREDATE:

Enter the values of DEPTNO and HIREDATE: 10 and 17-SEP-2007

The following value for MY_COL is generated:

MY_COL
1 WHERE department_id = 10 AND hire_date = TO_DATE('17-SEP-2007','DD-MON-YYYY')

When the second example in the slide is executed, the following output is generated:

LAST_NAME
Whalen

For Instructor Use Only.
This document should not be distributed.

Summary



Create a basic SQL script

Capture the output in a file

Dump the contents of a table to a file

Generate a dynamic predicate

SQL can be used to generate SQL scripts. These scripts can be used to avoid repetitive coding, drop or re-create objects, get help from the data dictionary, and generate dynamic predicates that contain run-time parameters.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

Oracle Database Architectural Components



For Instructor Use Only.

This document should not be distributed.

Objectives



List the major database architectural components

Describe the background processes

Explain the memory structures

Correlate the logical and physical storage structures

This appendix provides an overview of the Oracle Database architecture. You learn about the physical and logical structures and the various components of Oracle Database and their functions.

Oracle Database Architecture: Overview

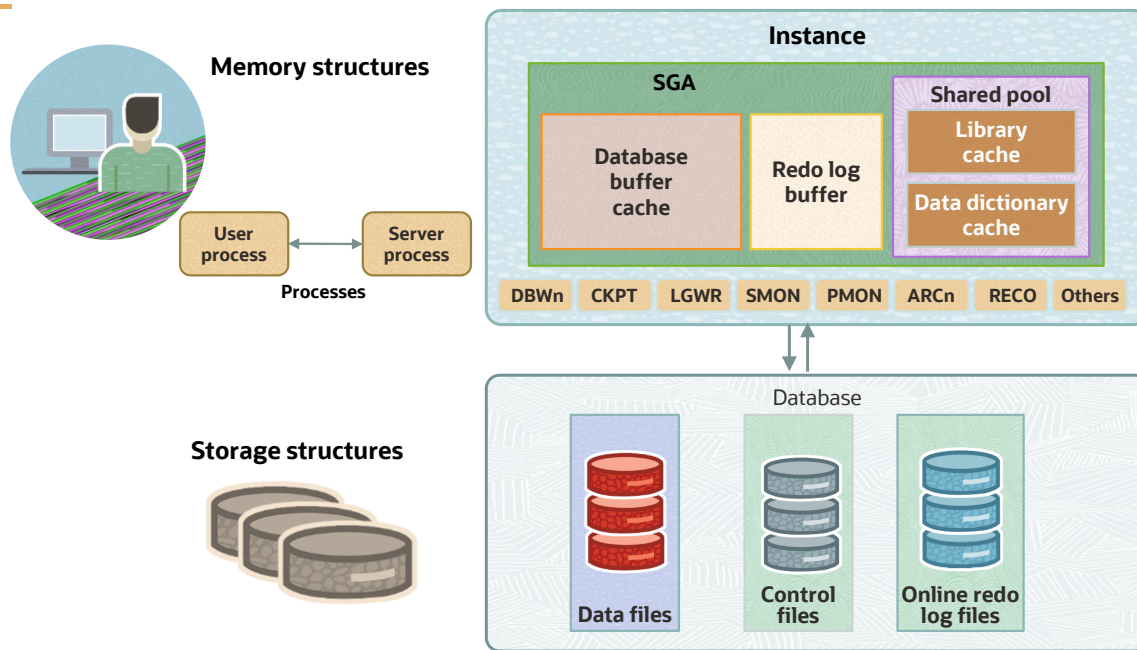
The Oracle Relational Database Management System (RDBMS) is a database management system that provides an open, comprehensive, integrated approach to information management.



A database is a collection of data treated as a unit. The purpose of a database is to store and retrieve related information.

An Oracle Database reliably manages a large amount of data in a multiuser environment so that many users can concurrently access the same data. This is accomplished while delivering high performance. At the same time, it prevents unauthorized access and provides efficient solutions for failure recovery.

Oracle Database Server Structures



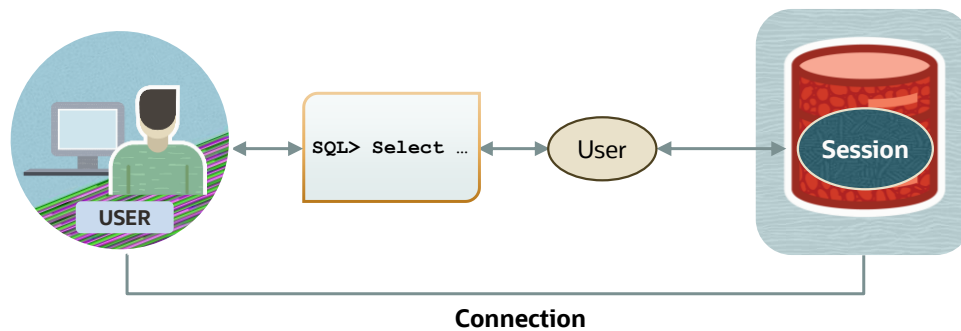
Oracle Database consists of two main components—the instance and the database.

- The instance consists of the System Global Area (SGA), which is a collection of memory structures, and the background processes that perform tasks within the database. Every time an instance is started, the SGA is allocated, and the background processes are started.
- The database consists of both physical structures and logical structures. Because the physical and logical structures are separate, the physical storage of data can be managed without affecting access to logical storage structures. The physical storage structures include:
 - The control files where the database configuration is stored
 - The redo log files that have information required for database recovery
 - The data files where all data is stored

An Oracle instance uses memory structures and processes to manage and access the database storage structures. All memory structures exist in the main memory of the computers that constitute the database server. Processes are jobs that work in the memory of these computers. A process is defined as a “thread of control” or a mechanism in an operating system that can run a series of steps.

Connecting to the Database

- **Connection:** A communication pathway between a user process and a database instance
- **Session:** A specific connection of a user to a database instance through a user process



To access information in the database, the user needs to connect to the database using a tool (such as SQL*Plus). After the user establishes a connection, a session is created for the user. Connection and session are closely related to the user process but are very different in meaning.

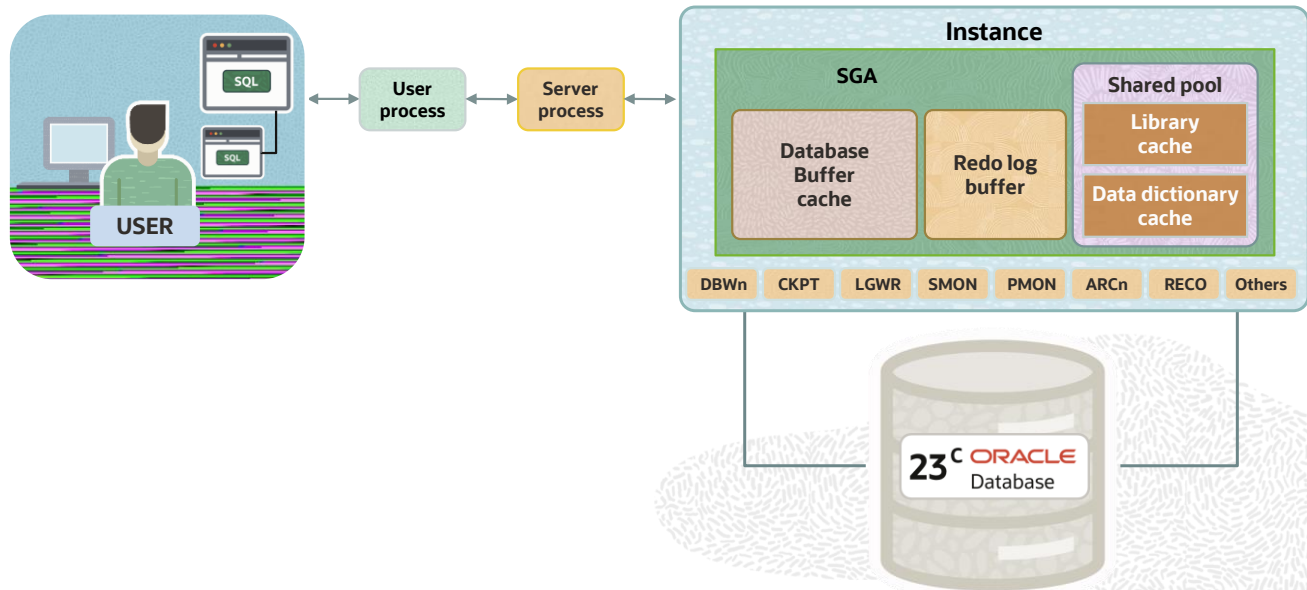
A connection is a communication pathway between a user process and an Oracle Database instance. A communication pathway is established by using available interprocess communication mechanisms or network software (when different computers run the database application and Oracle Database and communicate through a network).

A session represents the state of a current user login to the database instance. For example, when a user starts SQL*Plus, the user must provide a valid username and password, and then a session is established for that user. A session lasts from the time the user connects until the time the user disconnects or exits the database application.

In the case of a dedicated connection, the session is serviced by a permanent dedicated process. In the case of a shared connection, the session is serviced by an available server process selected from a pool, either by the middle tier or by Oracle shared server architecture.

Multiple sessions can be created and exist concurrently for a single Oracle Database user using the same username, but through different applications, or multiple invocations of the same application.

Interacting with an Oracle Database



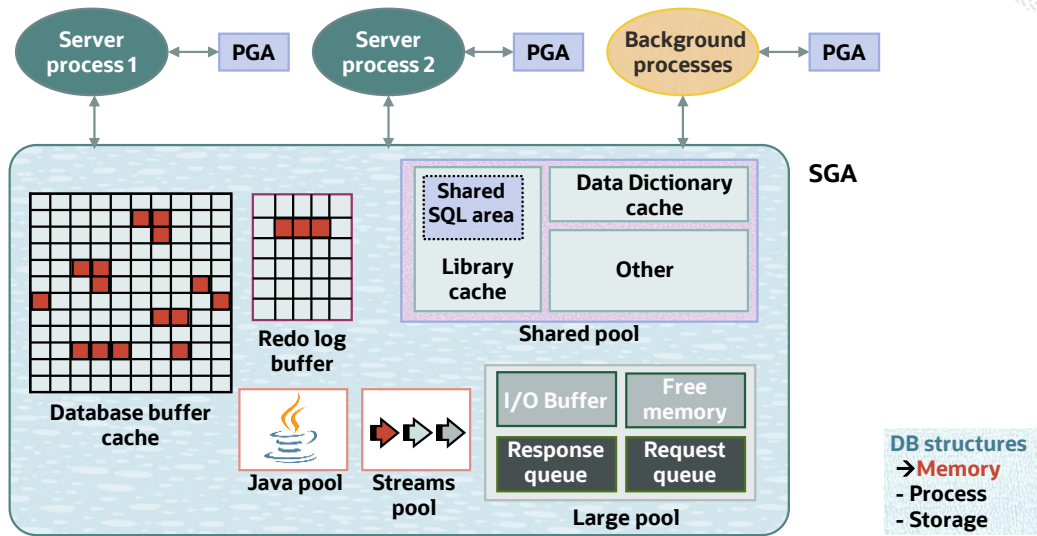
The following example describes Oracle Database operations at the most basic level. It illustrates an Oracle Database configuration where the user and the associated server process are on separate computers, connected through a network.

1. An instance has started on a node where Oracle Database is installed, often called the host or database server.
2. A user starts an application spawning a user process. The application attempts to establish a connection to the server. (The connection may be local, client server, or a three-tier connection from a middle tier.)
3. The server runs a listener that has the appropriate Oracle Net Services handler. The server detects the connection request from the application and creates a dedicated server process on behalf of the user process.
4. The user runs a DML-type SQL statement and commits the transaction. For example, the user changes the address of a customer in a table and commits the change.
5. The server process receives the statement and checks the shared pool (an SGA component) for any shared SQL area that contains a similar SQL statement. If a shared SQL area is found, the server process checks the user's access privileges to the requested data, and the existing shared SQL area is used to process the statement. If not, a new shared SQL area is allocated for the statement, so it can be parsed and processed.
6. The server process retrieves any necessary data values, either from the actual data file (in which the table is stored) or from those cached in the SGA.

7. The server process modifies data in the SGA. Because the transaction is committed, the log writer process (LGWR) immediately records the transaction in the redo log file. The database writer (DBWn) process writes modified blocks permanently to disk when doing so is efficient.
8. If the transaction is successful, the server process sends a message across the network to the application. If it is not successful, an error message is transmitted.
9. Throughout this entire procedure, the other background processes run, watching for conditions that require intervention. In addition, the database server manages other users' transactions and prevents contention between transactions that request the same data.

For Instructor Use Only.
This document should not be distributed.

Oracle Memory Architecture



Oracle Database creates and uses memory structures for various purposes. For example, memory stores program code being run, data shared among users, and private data areas for each connected user. Two basic memory structures are associated with an instance:

- The SGA is a group of shared memory structures, known as SGA components, that contain data and control information for one Oracle Database instance. The SGA is shared by all server and background processes. Examples of data stored in the SGA include cached data blocks and shared SQL areas.
- The Program Global Areas (PGA) are memory regions that contain data and control information for a server or background process. A PGA is nonshared memory created by Oracle Database when a server or background process is started. Access to the PGA is exclusive to the server process. Each server process and background process has its own PGA.

The SGA is the memory area that contains data and control information for the instance. The SGA includes the following data structures:

- **Database buffer cache:** Caches blocks of data retrieved from the database
- **Redo Log buffer:** Caches redo information (used for instance recovery) until it can be written to the physical redo log files stored on the disk
- **Shared pool:** Caches various constructs that can be shared among users
- **Large pool:** Is an optional area that provides large memory allocations for certain large processes, such as Oracle backup and recovery operations, and input/output (I/O) server processes
- **Java pool:** Is used for all session-specific Java code and data within the Java Virtual Machine (JVM)
- **Streams pool:** Is used by Oracle Streams to store information required by capture and apply

When you start the instance by using Enterprise Manager or SQL*Plus, the amount of memory allocated for the SGA is displayed.

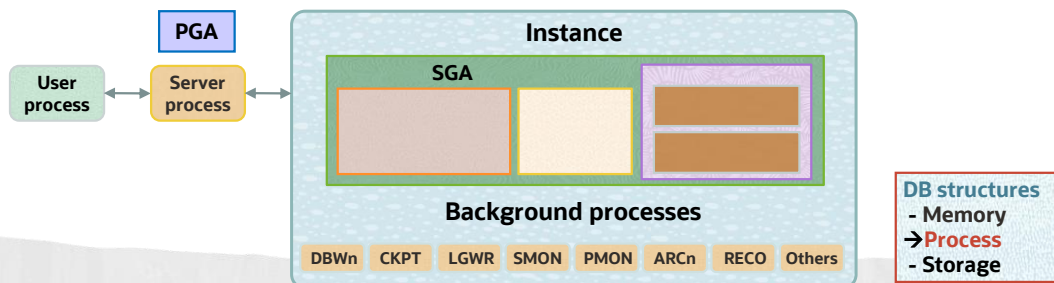
With the dynamic SGA infrastructure, the size of the database buffer cache, the shared pool, the large pool, the Java pool, and the Streams pool changes without shutting down the instance.

Oracle Database uses initialization parameters to create and configure memory structures. For example, the `SGA_TARGET` parameter specifies the total size of the SGA components. If you set `SGA_TARGET` to 0, Automatic Shared Memory Management is disabled.

For Instructor Use Only.
This document should not be distributed.

Process Architecture

- User process:
 - Is started when a database user or a batch process connects to Oracle Database
- Database processes:
 - Server process: Connects to the Oracle instance and is started when a user establishes a session
 - Background processes: Are started when an Oracle instance is started



The processes in an Oracle Database server can be categorized into two major groups:

- User processes that run the application or Oracle tool code
- Oracle Database processes that run the Oracle database server code. These include server processes and background processes.

When a user runs an application program or an Oracle tool such as SQL*Plus, Oracle Database creates a user process to run the user's application. Oracle Database also creates a server process to execute the commands issued by the user process. In addition, the Oracle server also has a set of background processes for an instance that interact with each other and with the operating system to manage the memory structures and asynchronously perform I/O to write data to disk and perform other required tasks.

The process structure varies for different Oracle Database configurations, depending on the operating system and the choice of Oracle Database options. The code for connected users can be configured as a dedicated server or a shared server.

- With a dedicated server, for each user, the database application is run by a different process (a user process) than the one that runs the Oracle server code (a dedicated server process).
- A shared server eliminates the need for a dedicated server process for each connection. A dispatcher directs multiple incoming network session requests to a pool of shared server processes. A shared server process serves any client request.

Server Processes

Oracle Database creates server processes to handle the requests of user processes connected to the instance. In some situations, when the application and Oracle Database operate on the same computer, it is possible to combine the user process and the corresponding server process into a single process to reduce system overhead. However, when the application and Oracle Database operate on different computers, a user process always communicates with Oracle Database through a separate server process.

Server processes created on behalf of each user's application can perform one or more of the following:

- Parse and run SQL statements issued through the application.
- Read necessary data blocks from data files on disk into the shared database buffers of the SGA, if the blocks are not already present in the SGA.
- Return results in such a way that the application can process the information.

Background Processes

To maximize performance and accommodate many users, a multiprocess Oracle Database system uses some additional Oracle Database processes called background processes. An Oracle Database instance can have many background processes.

The following background processes are required for a successful startup of the database instance:

- Database writer (DBWn)
- Log writer (LGWR)
- Checkpoint (CKPT)
- System monitor (SMON)
- Process monitor (PMON)

The following background processes are a few examples of optional background processes that can be started if required:

- Recoverer (RECO)
- Job queue
- Archiver (ARCn)
- Queue monitor (QMn)

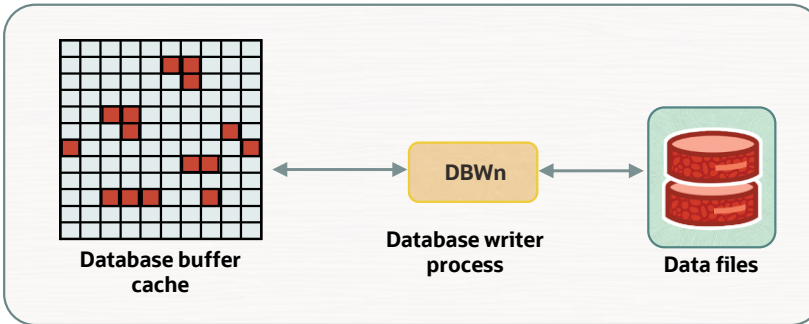
Other background processes may be found in more advanced configurations such as Real Application Clusters (RAC). See the `V$BGPROCESS` view for more information about the background processes. On many operating systems, background processes are created automatically when an instance is started.

For Instructor Use Only.
This document should not be distributed.

Database Writer Process

Writes modified (dirty) buffers in the database buffer cache to disk:

- Asynchronously while performing other processing
- Periodically to advance the checkpoint

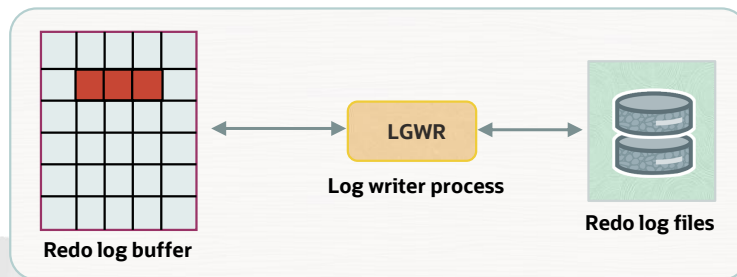


The database writer (DBWn) process writes the contents of buffers to data files. The DBWn processes are responsible for writing modified (dirty) buffers in the database buffer cache to disk. Although one database writer process (DBW0) is adequate for most systems, you can configure additional processes (DBW1 through DBW9 and DBW_a through DBW_j) to improve write performance if your system modifies data heavily. These additional DBWn processes are not useful on uniprocessor systems.

When a buffer in the database buffer cache is modified, it is marked “dirty” and is added to the LRUW list of dirty buffers that is kept in system change number (SCN) order, thereby matching the order of Redo corresponding to these changed buffers that is written to the Redo logs. When the number of available buffers in the buffer cache falls below an internal threshold such that server processes find it difficult to obtain available buffers, DBWn writes dirty buffers to the data files in the order that they were modified by following the order of the LRUW list.

Log Writer Process

- Writes the redo log buffer to a redo log file on disk
- The Log Writer (LGWR) writes:
 - When a process commits a transaction
 - When the redo log buffer is one-third full
 - Before a DBWn process writes modified buffers to disk



The log writer (LGWR) process is responsible for redo log buffer management by writing the redo log buffer entries to a redo log file on disk. LGWR writes all redo entries that have been copied into the buffer since the last time it wrote.

The redo log buffer is a circular buffer. When LGWR writes redo entries from the redo log buffer to a redo log file, server processes can then copy new entries over the entries in the redo log buffer that have been written to disk. The LGWR normally writes fast enough to ensure that space is always available in the buffer for new entries, even when access to the redo log is heavy.

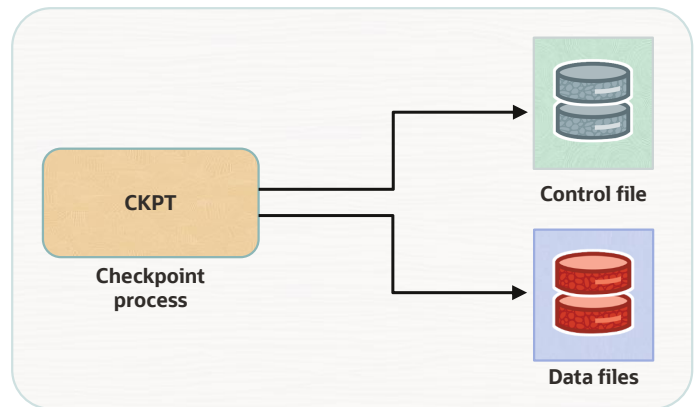
LGWR writes one contiguous portion of the buffer to disk. LGWR writes:

- When a user process commits a transaction
- When the redo log buffer is one-third full
- Before a DBWn process writes modified buffers to disk, if necessary

Checkpoint Process

Records checkpoint information in:

- The control file
- Each data file header



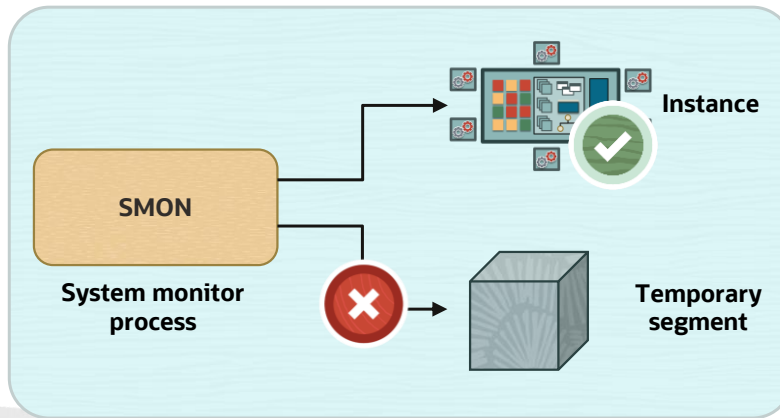
A checkpoint is a data structure that defines an SCN in the redo thread of a database. Checkpoints are recorded in the control file and each data file header, and are a crucial element of recovery.

When a checkpoint occurs, Oracle Database must update the headers of all data files to record the details of the checkpoint. This is done by the CKPT process. The CKPT process does not write blocks to disk; DBWR always performs that work. The SCNs recorded in the file headers guarantee that all the changes made to database blocks before that SCN have been written to disk.

The statistic DBWR checkpoints displayed by the `SYSTEM_STATISTICS` monitor in Oracle Enterprise Manager indicate the number of checkpoint requests completed.

System Monitor Process

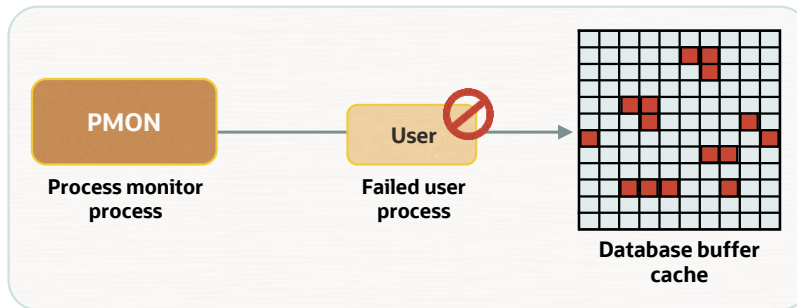
- Performs recovery at instance startup
- Cleans up unused temporary segments



The system monitor (SMON) process performs recovery, if necessary, at instance startup. It is also responsible for cleaning up temporary segments that are no longer in use. If any terminated transactions were skipped during instance recovery because of file-read or offline errors, SMON recovers them when the tablespace or the file is brought back online. SMON checks regularly to see whether it is needed. Other processes can call SMON if they detect a need for it.

Process Monitor Process

- Performs process recovery when a user process fails
- Monitors sessions for idle session timeout
- Dynamically registers database services with listeners

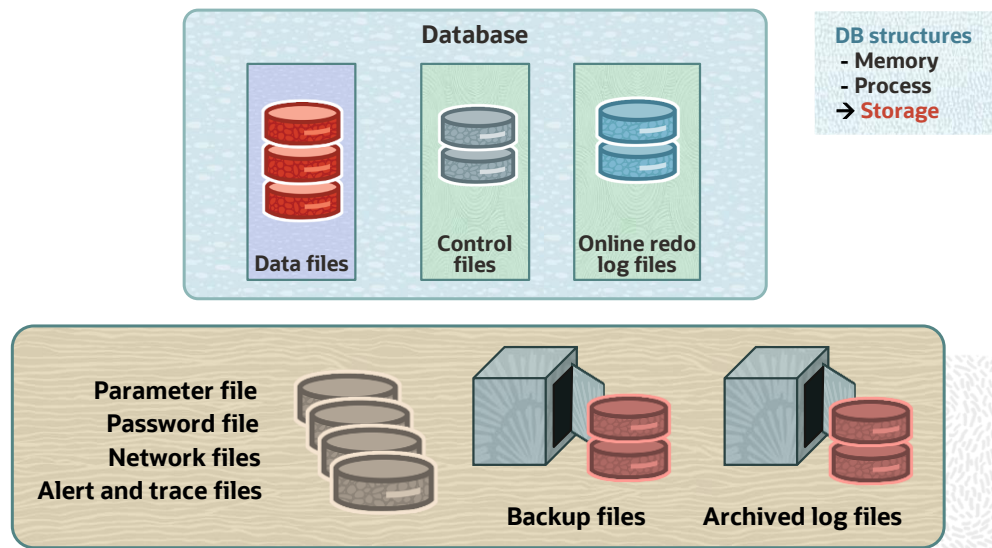


The process monitor (PMON) performs process recovery when a user process fails. PMON is responsible for cleaning up the database buffer cache and freeing resources that the user process was using. For example, it resets the status of the active transaction table, releases locks, and removes the process ID from the list of active processes.

PMON periodically checks the status of dispatcher and server processes and restarts any that have stopped running (but not any that Oracle Database has terminated intentionally). PMON also registers information about the instance and dispatcher processes with the network listener.

Like SMON, PMON checks regularly to see whether it is needed and can be called if another process detects the need for it.

Oracle Database Storage Architecture



The files that constitute an Oracle database are organized into the following:

- **Control files:** Contain data about the database itself (that is, physical database structure information). These files are critical to the database. Without them, you cannot open data files to access the data within the database.
- **Data files:** Contain the user or application data of the database, as well as metadata and the data dictionary.
- **Online redo log files:** Allow for instance recovery of the database. If the database server crashes and does not lose any data files, the instance can recover the database with the information in these files.

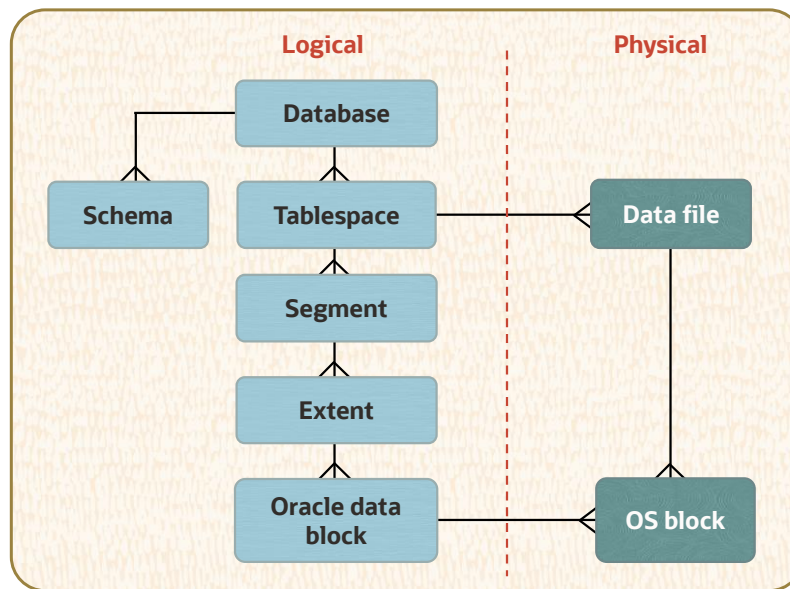
The following additional files are important to the successful running of the database:

- **Backup files:** Are used for database recovery. You typically restore a backup file when a media failure or user error has damaged or deleted the original file.
- **Archived log files:** Contain an ongoing history of the data changes (redo) that are generated by the instance. Using these files and a backup of the database, you can recover a lost data file. That is, archive logs enable the recovery of restored data files.
- **Parameter file:** Is used to define how the instance is configured when it starts up.
- **Password file:** Allows `sysdba/sysoper/sysasm` to connect remotely to the database and perform administrative tasks.
- **Network files:** Are used for starting the database listener and store information required for user connections.

- **Trace files:** Each server and background process can write to an associated trace file. When an internal error is detected by a process, the process dumps information about the error to its trace file. Some of the information written to a trace file is intended for the database administrator, whereas other information is for Oracle Support Services.
- **Alert log files:** These are special trace entries. The alert log of a database is a chronological log of messages and errors. Each instance has one alert log file. Oracle recommends that you review this alert log periodically.

For Instructor Use Only.
This document should not be distributed.

Logical and Physical Database Structures



An Oracle database has logical and physical storage structures.

Tablespaces

A database is divided into logical storage units called tablespaces, which group related logical structures together. For example, tablespaces commonly group all of an application's objects to simplify some administrative operations. You may have a tablespace for application data and an additional one for application indexes.

Databases, Tablespaces, and Data Files

The relationship among databases, tablespaces, and data files is illustrated in the slide. Each database is logically divided into one or more tablespaces. One or more data files are explicitly created for each tablespace to physically store the data of all logical structures in a tablespace. If it is a `TEMPORARY` tablespace, instead of a data file, the tablespace has a temporary file.

Schemas

A schema is a collection of database objects that are owned by a database user. Schema objects are the logical structures that directly refer to the database's data. Schema objects include such structures as tables, views, sequences, stored procedures, synonyms, indexes, clusters, and database links. In general, schema objects include everything that your application creates in the database.

Data Blocks

At the finest level of granularity, an Oracle database's data is stored in data blocks. One data block corresponds to a specific number of bytes of physical database space on the disk. A data block size is specified for each tablespace when it is created. A database uses and allocates free database space in Oracle data blocks.

Extents

The next level of logical database space is called an extent. An extent is a specific number of contiguous data blocks (obtained in a single allocation) that are used to store specific type of information.

Segments

The level of logical database storage above an extent is called a segment. A segment is a set of extents allocated for a certain logical structure. For example, the different types of segments include:

- **Data segments:** Each nonclustered, non-indexed-organized table has a data segment with the exception of external tables, global temporary tables, and partitioned tables, where each table has one or more segments. All of the table's data is stored in the extents of its data segment. For a partitioned table, each partition has a data segment. Each cluster has a data segment. The data of every table in the cluster is stored in the cluster's data segment.
- **Index segments:** Each index has an index segment that stores all of its data. For a partitioned index, each partition has an index segment.
- **Undo segments:** One `UNDO` tablespace is created per database instance that contains numerous undo segments to temporarily store *undo* information. The information in an undo segment is used to generate read-consistent database information and, during database recovery, to roll back uncommitted transactions for users.
- **Temporary segments:** Temporary segments are created by Oracle Database when a SQL statement needs a temporary work area to complete execution. When the statement finishes execution, the temporary segment's extents are returned to the instance for future use. Specify a default temporary tablespace for every user or a default temporary tablespace, which is used databasewide.

Oracle Database dynamically allocates space. When the existing extents of a segment are full, additional extents are added. Because extents are allocated as needed, the extents of a segment may or may not be contiguous on the disk.

Processing a SQL Statement

- Connect to an instance using:
 - The user process
 - The server process
- The Oracle server components that are used depend on the type of SQL statement.
- Some Oracle server components do not participate in SQL statement processing.

Not all the components of an Oracle instance are used to process SQL statements. The user and server processes are used to connect a user to an Oracle instance. These processes are not a part of the Oracle instance, but are required to process a SQL statement.

Some of the background processes, SGA structures, and database files are used to process SQL statements. Depending on the type of SQL statement, different components are used:

- Queries require additional processing to return rows to the user.
- DML statements require additional processing to log the changes made to the data.
- Commit processing ensures that the modified data in a transaction can be recovered.

Some required background processes do not directly participate in processing a SQL statement, but are used to improve performance and to recover the database. For example, the optional Archiver background process, `ARCn`, is used to ensure that a production database can be recovered.

Processing a Query

- **Parse:**
 - Searches for an identical statement
 - Checks the syntax, object names, and privileges
 - Locks the objects used during parse
 - Creates and stores the execution plan
- **Execute:**
 - Identifies the rows selected
- **Fetch:**
 - Returns the rows to the user process



Queries are different from other types of SQL statements because, if successful, they return data as results. Other statements simply return success or failure, whereas a query can return one row or thousands of rows.

There are three main stages in the processing of a query:

- Parse
- Execute
- Fetch

During the parse stage, the SQL statement is passed from the user process to the server process, and a parsed representation of the SQL statement is loaded into a shared SQL area.

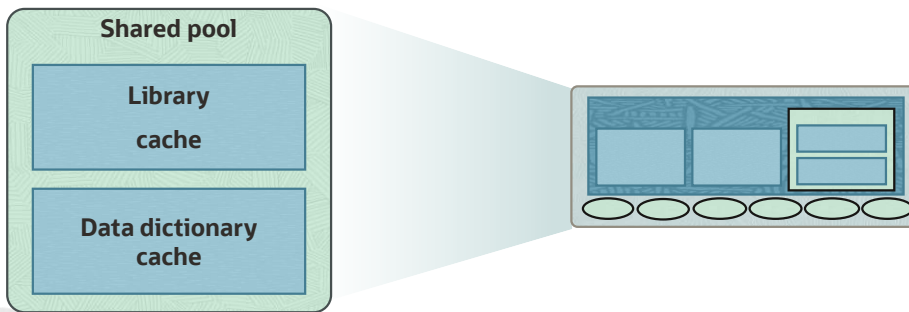
During parse, the server process performs the following functions:

- Searches for an existing copy of the SQL statement in the shared pool
- Validates the SQL statement by checking its syntax
- Performs data dictionary lookups to validate table and column definitions

The execute stage executes the statement using the best optimizer approach and the fetch stage retrieves the rows back to the user.

Shared Pool

- The library cache contains the SQL statement text, parsed code, and execution plan.
- The data dictionary cache contains table, column, and other object definitions and privileges.
- The shared pool is sized by `SHARED_POOL_SIZE`.



During the parse stage, the server process uses the area in the SGA known as the shared pool to compile the SQL statement. The shared pool has two primary components:

- Library cache
- Data dictionary cache

Library Cache

The library cache stores information about the most recently used SQL statements in a memory structure called a shared SQL area. The shared SQL area contains:

- The text of the SQL statement
- The parse tree, which is a compiled version of the statement
- The execution plan, with steps to be taken when executing the statement

The optimizer is the function in the Oracle server that determines the optimal execution plan.

If a SQL statement is reexecuted and a shared SQL area already contains the execution plan for the statement, then the server process does not need to parse the statement. The library cache improves the performance of applications that reuse SQL statements by reducing parse time and memory requirements. If the SQL statement is not reused, it is eventually aged out of the library cache.

Data Dictionary Cache

The data dictionary cache, also known as the dictionary cache or row cache, is a collection of the most recently used definitions in the database. It includes information about database files, tables, indexes, columns, users, privileges, and other database objects.

During the parse phase, the server process looks for the information in the dictionary cache to resolve the object names specified in the SQL statement and to validate the access privileges. If necessary, the server process initiates the loading of this information from the data files.

Sizing the Shared Pool

The size of the shared pool is specified by the `SHARED_POOL_SIZE` initialization parameter.

For Instructor Use Only.
This document should not be distributed.

Database Buffer Cache

- The database buffer cache stores the most recently used blocks.
- The size of a buffer is based on `DB_BLOCK_SIZE`.
- The number of buffers is defined by `DB_BLOCK_BUFFERS`.



When a query is processed, the server process looks in the database buffer cache for any blocks it needs. If the block is not found in the database buffer cache, the server process reads the block from the data file and places a copy in the buffer cache. Because subsequent requests for the same block may find the block in memory, the requests may not require physical reads. The Oracle server uses a least recently used algorithm to age out buffers that have not been accessed recently to make room for new blocks in the buffer cache.

Sizing the Database Buffer Cache

The size of each buffer in the buffer cache is equal to the size of an Oracle block, and it is specified by the `DB_BLOCK_SIZE` parameter. The number of buffers is equal to the value of the `DB_BLOCK_BUFFERS` parameter.

Program Global Area

A Program Global Area (PGA):

- Is not shared
- Is writable only by the server process
- Contains:
 - Sort area
 - Session information
 - Cursor state
 - Stack space



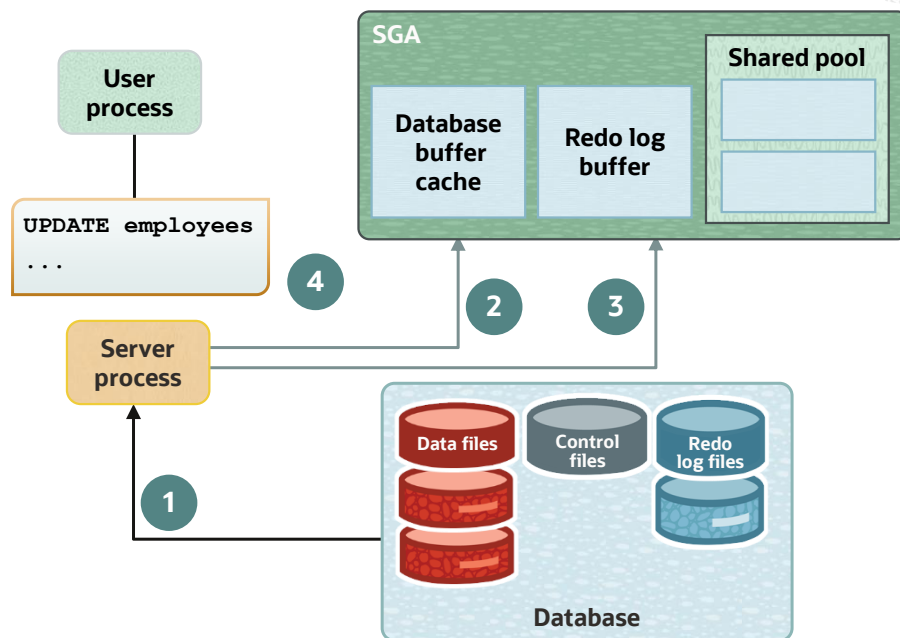
A Program Global Area (PGA) is a memory region that contains data and control information for a server process. It is a nonshared memory created by Oracle when a server process is started. Access to it is exclusive to that server process and is read and written only by the Oracle server code acting on behalf of it. The PGA memory allocated by each server process attached to an Oracle instance is referred to as the aggregated PGA memory allocated by the instance.

In a dedicated server configuration, the PGA of the server includes the following components:

- **Sort area:** Is used for any sorts that may be required to process the SQL statement
- **Session information:** Includes user privileges and performance statistics for the session
- **Cursor state:** Indicates the stage in the processing of the SQL statements that are currently used by the session
- **Stack space:** Contains other session variables

The PGA is allocated when a process is created and deallocated when the process is terminated.

Processing a DML Statement



A data manipulation language (DML) statement requires only two phases of processing:

- Parse is the same as the parse phase used for processing a query.
- Execute requires additional processing to make data changes.

DML Execute Phase

To execute a DML statement:

- If the data and rollback blocks are not already in the buffer cache, the server process reads them from the data files into the buffer cache.
- The server process places locks on the rows that are to be modified.
- In the redo log buffer, the server process records the changes to be made to the rollback and data blocks.
- The rollback block changes record the values of the data before it is modified. The rollback block is used to store the “before image” of the data, so that the DML statements can be rolled back if necessary.
- The data block changes record the new values of the data.

The server process records the “before image” to the rollback block and updates the data block. Both of these changes are done in the database buffer cache. Any changed blocks in the buffer cache are marked as dirty buffers (that is, buffers that are not the same as the corresponding blocks on the disk).

The processing of a `DELETE` or `INSERT` command uses similar steps. The “before image” for a `DELETE` contains the column values in the deleted row, and the “before image” of an `INSERT` contains the row location information.

Because the changes made to the blocks are only recorded in memory structures and are not written immediately to disk, a computer failure that causes the loss of the SGA can also lose these changes.

Redo Log Buffer

Redo Log Buffer:

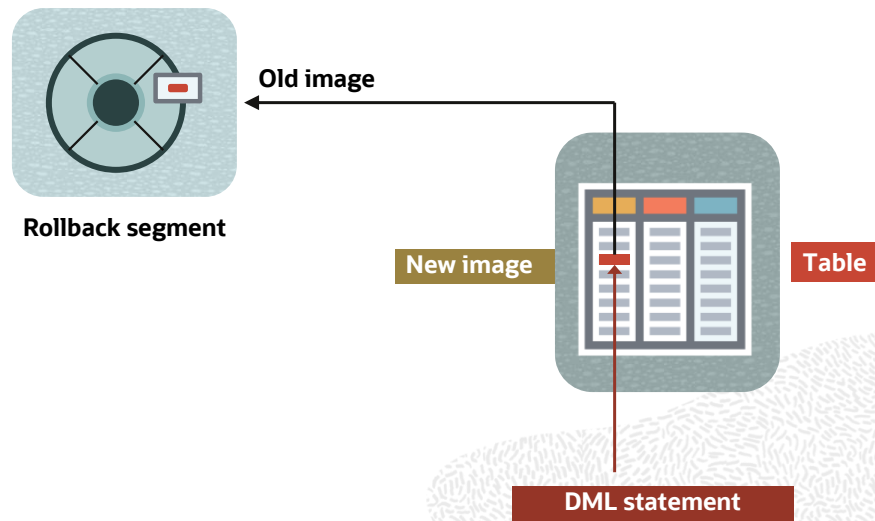
- Has its size defined by `LOG_BUFFER`
- Records changes made through the instance
- Is used sequentially
- Is a circular buffer



The server process records most of the changes made to data file blocks in the redo log buffer, which is a part of the SGA. The redo log buffer has the following characteristics:

- Its size in bytes is defined by the `LOG_BUFFER` parameter.
- It records the block that is changed, the location of the change, and the new value in a redo entry. A redo entry makes no distinction between the types of block that is changed; it only records which bytes are changed in the block.
- The redo log buffer is used sequentially, and changes made by one transaction may be interleaved with changes made by other transactions.
- It is a circular buffer that is reused after it is filled, but only after all the old redo entries are recorded in the redo log files.

Rollback Segment



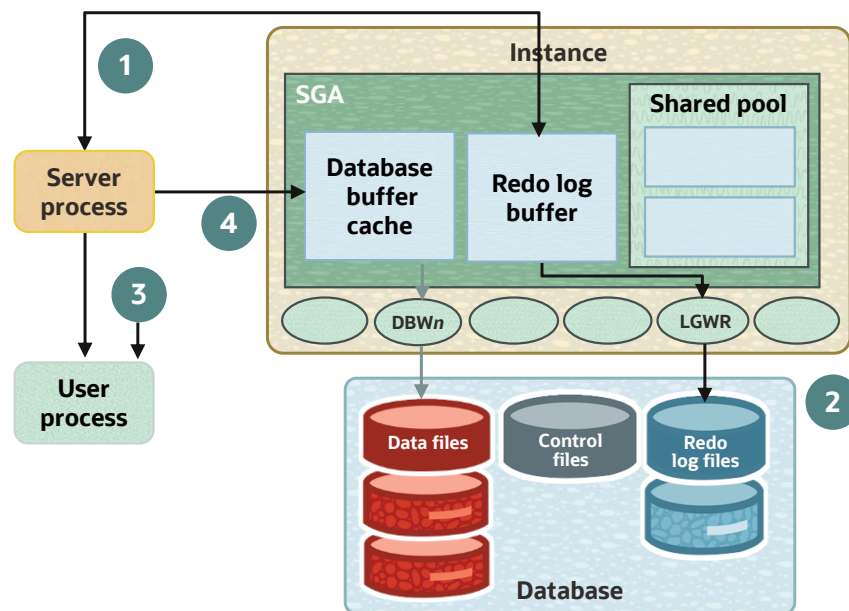
Before making a change, the server process saves the old data value in a rollback segment. This “before image” is used to:

- Undo the changes if the transaction is rolled back
- Provide read consistency by ensuring that other transactions do not see uncommitted changes made by the DML statement
- Recover the database to a consistent state in case of failures

Rollback segments, such as tables and indexes, exist in data files, and rollback blocks are brought into the database buffer cache as required. Rollback segments are created by the DBA.

Changes to rollback segments are recorded in the redo log buffer.

COMMIT Processing



The Oracle server uses a fast `COMMIT` mechanism that guarantees that the committed changes can be recovered in case of instance failure.

System Change Number

Whenever a transaction commits, the Oracle server assigns a commit SCN to the transaction. The SCN is monotonically incremented and is unique within the database. It is used by the Oracle server as an internal time stamp to synchronize data and to provide read consistency when data is retrieved from the data files. Using the SCN enables the Oracle server to perform consistency checks without depending on the date and time of the operating system.

Steps in Processing `COMMIT`s

When a `COMMIT` is issued, the following steps are performed:

1. The server process places a commit record, along with the SCN, in the redo log buffer.
2. LGWR performs a contiguous write of all the redo log buffer entries up to and including the commit record to the redo log files. After this point, the Oracle server can guarantee that the changes will not be lost even if there is an instance failure.
3. The user is informed that the `COMMIT` is complete.
4. The server process records information to indicate that the transaction is complete and that resource locks can be released.

Flushing of the dirty buffers to the data file is performed independently by DBW0 and can occur either before or after the commit.

Advantages of the Fast COMMIT

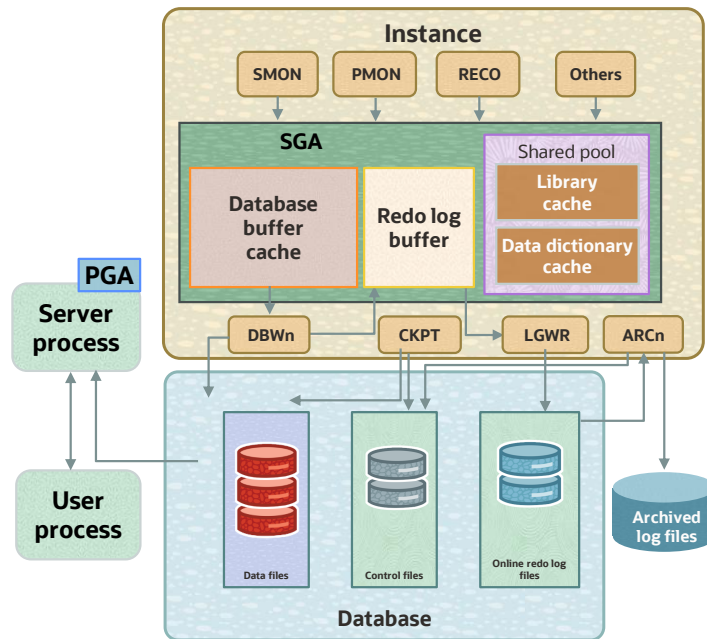
The fast COMMIT mechanism ensures data recovery by writing changes to the redo log buffer instead of the data files. It has the following advantages:

- Sequential writes to the log files are faster than writing to different blocks in the data file.
- Only the minimal information that is necessary to record changes is written to the log files; writing to the data files would require whole blocks of data to be written.
- If multiple transactions request to commit at the same time, the instance piggybacks redo log records into a single write.
- Unless the redo log buffer is particularly full, only one synchronous write is required per transaction. If piggybacking occurs, there can be less than one synchronous write per transaction.
- Because the redo log buffer may be flushed before the COMMIT, the size of the transaction does not affect the amount of time needed for an actual COMMIT operation.

Note: Rolling back a transaction does not trigger LGWR to write to disk. The Oracle server always rolls back uncommitted changes when recovering from failures. If there is a failure after a rollback, before the rollback entries are recorded on disk, the absence of a commit record is sufficient to ensure that the changes made by the transaction are rolled back.

For Instructor Use Only.
This document should not be distributed.

Summary of the Oracle Database Architecture



An Oracle database comprises an instance and its associated database:

- An instance comprises the SGA and the background processes
 - **SGA:** Database buffer cache, redo log buffer, shared pool, and so on
 - **Background processes:** SMON, PMON, DBWn, CKPT, LGWR, and so on
- A database comprises storage structures:
 - **Logical:** Tablespaces, schemas, segments, extents, and Oracle block
 - **Physical:** Data files, control files, redo log files

When a user accesses the Oracle database through an application, a server process communicates with the instance on behalf of the user process.

Summary



List the major database architectural components

Describe the background processes

Explain the memory structures

Correlate the logical and physical storage structures

For Instructor Use Only.

This document should not be distributed.

Regular Expression Support



For Instructor Use Only.

This document should not be distributed.

Objectives



List the benefits of using regular expressions

Use regular expressions to search for, match, and replace strings

In this appendix, you learn to use the regular expression support feature. Regular expression support is available in both SQL and PL/SQL.

For Instructor Use Only.
This document should not be distributed.

What Are Regular Expressions?

- You use regular expressions to search for (and manipulate) simple and complex patterns in string data by using standard syntax conventions.
- You use a set of SQL functions and conditions to search for and manipulate strings in SQL and PL/SQL.
- You specify a regular expression by using:
 - Metacharacters
 - Literals



Oracle Database provides support for regular expressions. The implementation complies with the Portable Operating System for UNIX (POSIX) standard, controlled by the Institute of Electrical and Electronics Engineers (IEEE), for ASCII data-matching semantics and syntax. Oracle's multilingual capabilities extend the matching capabilities of the operators beyond the POSIX standard. Regular expressions are a method of describing both simple and complex patterns for searching and manipulating.

String manipulation and searching contribute to a large percentage of the logic within a web-based application. Usage ranges from the simple, such as finding the words "San Francisco" in a specified text, to the complex task of extracting all URLs from the text and the more complex task of finding all words whose every second character is a vowel.

When coupled with native SQL, the use of regular expressions allows for very powerful search and manipulation operations on any data stored in an Oracle database. You can use this feature to easily solve problems that would otherwise involve complex programming.

You specify a regular expression by using:

- Metacharacters, which are operators that specify the search algorithms
- Literals, which are the characters for which you are searching

Benefits of Using Regular Expressions

Regular expressions enable you to implement complex match logic in the database with the following benefits:

- By centralizing match logic in Oracle Database, you avoid intensive string processing of SQL result sets by middle-tier applications.
- Using server-side regular expressions to enforce constraints, you eliminate the need to code data validation logic on the client.
- The built-in SQL and PL/SQL regular expression functions and conditions make string manipulations more powerful and easier than in previous releases of Oracle Database.



Regular expressions are a powerful text-processing component of programming languages such as PERL and Java. For example, a PERL script can process each HTML file in a directory, read its contents into a scalar variable as a single string, and then use regular expressions to search for URLs in the string. One reason for many developers writing in PERL is that it has a robust pattern-matching functionality. Oracle's support of regular expressions enables developers to implement complex match logic in the database. Regular expressions were introduced in Oracle Database 10g.

Using Regular Expressions Functions and Conditions in SQL and PL/SQL



Function or Condition Name	Description
REGEXP_LIKE	Is similar to the <code>LIKE</code> operator, but performs regular expression matching instead of simple pattern matching (condition)
REGEXP_REPLACE	Searches for a regular expression pattern and replaces it with a replacement string
REGEXP_INSTR	Searches a string for a regular expression pattern and returns the position where the match is found
REGEXP_SUBSTR	Searches for a regular expression pattern within a given string and extracts the matched substring
REGEXP_COUNT	Returns the number of times a pattern match is found in an input sting

Oracle Database provides a set of SQL functions that you use to search and manipulate strings by using regular expressions. You use these functions on a text literal, bind variable, or any column that holds character data, such as `CHAR`, `NCHAR`, `CLOB`, `NCLOB`, `NVARCHAR2`, and `VARCHAR2` (but not `LONG`). A regular expression must be enclosed within single quotation marks. This ensures that the entire expression is interpreted by the SQL function and can improve the readability of your code.

- **REGEXP_LIKE:** This condition searches a character column for a pattern. Use this condition in the `WHERE` clause of a query to return rows matching the regular expression that you specify.
- **REGEXP_REPLACE:** This function searches for a pattern in a character column and replaces each occurrence of that pattern with the pattern that you specify.
- **REGEXP_INSTR:** This function searches a string for a given occurrence of a regular expression pattern. You specify which occurrence you want to find and the start position to search from. This function returns an integer indicating the position in the string where the match is found.
- **REGEXP_SUBSTR:** This function returns the actual substring matching the regular expression pattern that you specify.
- **REGEXP_COUNT:** This function returns the number of times a pattern match is found in the input string.

What are metacharacters?

- Metacharacters are special characters that have a special meaning.
- You can use several predefined metacharacter symbols in the pattern matching.
- For example, the `^(f|ht)tps?:$` regular expression searches for the following from the beginning of the string:
 - The literals `f` or `ht`
 - The `t` literal
 - The `p` literal, optionally followed by the `s` literal
 - The colon `:` literal at the end of the string

Metacharacters are special characters that have a special meaning, such as a wildcard, a repeating character, a nonmatching character, or a range of characters.

The regular expression in the slide matches the `http:`, `https:`, `ftp:`, and `ftps:` strings.

Using Metacharacters with Regular Expressions

Syntax	Description
.	Matches any character in the supported character set, except NULL
+	Matches one or more occurrences
?	Matches zero or one occurrence
*	Matches zero or more occurrences of the preceding subexpression
{m}	Matches exactly m occurrences of the preceding expression
{m, }	Matches at least m occurrences of the preceding subexpression
{m, n}	Matches at least m, but not more than n, occurrences of the preceding subexpression
[...]	Matches any single character in the list within the brackets
	Matches one of the alternatives
(...)	Treats the enclosed expression within the parentheses as a unit. The subexpression can be a string of literals or a complex expression containing operators.

- **Any character**, “.”: `a.b` matches the strings `abb`, `acb`, and `adb`, but not `acc`.
- **One or more**, “+”: `a+` matches the strings `a`, `aa`, and `aaa`, but does not match `bbb`.
- **Zero or one**, “?”: `ab?c` matches the strings `abc` and `ac`, but does not match `abbc`.
- **Zero or more**, “*”: `ab*c` matches the strings `ac`, `abc`, and `abbc`, but does not match `abb`.
- **Exact count**, “{m}”: `a{3}` matches the strings `aaa`, but does not match `aa`.
- **At least count**, “{m,}”: `a{3,}` matches the strings `aaa` and `aaaa`, but not `aa`.
- **Between count**, “{m,n}”: `a{3,5}` matches the strings `aaa`, `aaaa`, and `aaaaa`, but not `aa`.
- **Matching character list**, “[...]”: `[abc]` matches the first character in the strings `all`, `bill`, and `coll`, but does not match any characters in `doll`.
- **Or**, “|”: `a|b` matches character `a` or character `b`.
- **Subexpression**, “(...)”: `(abc)?def` matches the optional string `abc`, followed by `def`. The expression matches `abcdefghi` and `def`, but does not match `ghi`. The subexpression can be a string of literals or a complex expression containing operators.

Using Metacharacters with Regular Expressions

Syntax	Description
<code>^</code>	Matches the beginning of a string
<code>\$</code>	Matches the end of a string
<code>\</code>	Treats the subsequent metacharacter in the expression as a literal
<code>\n</code>	Matches the nth (1–9) preceding subexpression of whatever is grouped within parentheses. The parentheses cause an expression to be remembered; a backreference refers to it.
<code>\d</code>	A digit character
<code>[:class:]</code>	Matches any character belonging to the specified POSIX character class
<code>[^ :class:]</code>	Matches any single character not in the list within the brackets

- **Beginning/end of line anchor, “`^`” and “`$`”:** `^def` matches `def` in the string `defghi`, but does not match `def` in `abcdef`. `def$` matches `def` in the string `abcdef`, but does not match `def` in the string `defghi`.
- **Escape character “`\`”:** `\+` searches for a `+`. It matches the plus character in the string `abc+def`, but does not match `Abcdef`.
- **Backreference, “`\n`”:** `(abc|def)xy\1` matches the strings `abcxyabc` and `defxydef`, but does not match `abcxydef` or `abcxy`. A backreference enables you to search for a repeated string without knowing the actual string ahead of time. For example, the expression `^(.*)\1$` matches a line consisting of two adjacent instances of the same string.
- **Digit character, “`\d`”:** The expression `^\[\d{3}\] \d{3}-\d{4}$` matches `[650] 555-1212`, but does not match `650-555-1212`.
- **Character class, “`[ :class:]`”:** `[ :upper:]` searches for one or more consecutive uppercase characters. This matches `DEF` in the string `abcDEFghi`, but does not match the string `abcdefghi`.
- **Nonmatching character list (or class), “`[^ . . .]`”:** `[^abc]` matches the character `d` in the string `abcdef`, but not `a`, `b`, or `c`.

Regular Expressions Functions and Conditions: Syntax

```
REGEXP_LIKE (source_char, pattern [,match_option]
```

```
REGEXP_INSTR (source_char, pattern [, position  
[, occurrence [, return_option  
[, match_option [, subexpr]]]])
```

```
REGEXP_SUBSTR (source_char, pattern [, position  
[, occurrence [, match_option  
[, subexpr]]]])
```

```
REGEXP_REPLACE(source_char, pattern [,replacestr  
[, position [, occurrence  
[, match_option]]]])
```

```
REGEXP_COUNT (source_char, pattern [, position  
[, occurrence [, match_option]]]])
```

The syntax for the regular expressions functions and conditions is as follows:

- **source_char**: A character expression that serves as the search value
- **pattern**: A regular expression, a text literal
- **occurrence**: A positive integer indicating which occurrence of pattern in **source_char** Oracle Server should search for. The default is 1.
- **position**: A positive integer indicating the character of **source_char**, where Oracle Server should begin the search. The default is 1.
- **return_option**:
 - 0: Returns the position of the first character of the occurrence (default)
 - 1: Returns the position of the character following the occurrence
- **Replacestr**: Character string replacing pattern
- **match_parameter**:
 - “c”: Uses case-sensitive matching (default)
 - “i”: Uses non-case-sensitive matching
 - “n”: Allows match-any-character operator
 - “m”: Treats source string as multiple lines
- **subexpr**: Fragment of pattern enclosed in parentheses. You learn more about subexpressions later in this appendix.

Performing a Basic Search by Using the REGEXP_LIKE Condition

```
REGEXP_LIKE(source_char, pattern [, match_parameter ])
```

```
SELECT first_name, last_name  
FROM employees  
WHERE REGEXP_LIKE (first_name, '^Ste(v|ph)en$');
```

	FIRST_NAME	LAST_NAME
1	Steven	King
2	Steven	Markle
3	Stephen	Stiles

REGEXP_LIKE is similar to the LIKE condition, except that REGEXP_LIKE performs regular-expression matching instead of the simple pattern matching performed by LIKE. This condition evaluates strings by using characters as defined by the input character set.

Example of REGEXP_LIKE

In this query, against the EMPLOYEES table, all employees with first names containing either Steven or Stephen are displayed. In the expression used '^Ste(v|ph)en\$':

- ^ indicates the beginning of the expression
- \$ indicates the end of the expression
- | indicates either/or

Replacing Patterns by Using the REGEXP_REPLACE Function

```
REGEXP_REPLACE(source_char, pattern [,replacestr  
[, position [, occurrence [, match_option]]])
```

```
SELECT last_name, REGEXP_REPLACE(phone_number, '\.', '-')  
AS phone  
FROM employees;
```

Original

	LAST_NAME	PHONE
1	OConnell	650.507.9833
2	Grant	650.507.9844
3	Whalen	515.123.4444
4	Hartstein	515.123.5555

Partial results

	LAST_NAME	PHONE
1	OConnell	650-507-9833
2	Grant	650-507-9844
3	Whalen	515-123-4444
4	Hartstein	515-123-5555

Using the REGEXP_REPLACE function, you reformat the phone number to replace the period (.) delimiter with a dash (-) delimiter. Here is an explanation of each of the elements used in the regular expression example:

- phone_number is the source column.
- '\.' is the search pattern.
 - Use single quotation marks (' ') to search for the literal character period ().
 - Use a backslash (\) to search for a character that is normally treated as a metacharacter.
- '-' is the replace string.

Finding Patterns by Using the REGEXP_INSTR Function

```
REGEXP_INSTR (source_char, pattern [, position [, occurrence [, return_option [, match_option]]]])
```

```
SELECT street_address,  
       REGEXP_INSTR(street_address,'[[:alpha:]]') AS  
         First_Alpha_Position  
FROM   locations;
```

	STREET_ADDRESS	FIRST_ALPHA_POSITION
1	1297 Via Cola di Rie	6
2	93091 Calle della Testa	7
3	2017 Shinjuku-ku	6
4	9450 Kamiya-cho	6

In this example, the `REGEXP_INSTR` function is used to search the street address to find the location of the first alphabetic character, regardless of whether it is in uppercase or lowercase. Note that `[<class>:]` implies a character class and matches any character from within that class; `[[:alpha:]]` matches with any alphabetic character. The partial results are displayed.

In the expression used in the query `'[[:alpha:]]'`:

- `[` starts the expression
- `[[:alpha:]]` indicates alphabetic character class
- `]` ends the expression

Note: The POSIX character class operator enables you to search for an expression within a character list that is a member of a specific POSIX character class. You can use this operator to search for specific formatting, such as uppercase characters, or you can search for special characters such as digits or punctuation characters. The full set of POSIX character classes is supported. Use the syntax `[[:class:]]`, where `class` is the name of the POSIX character class to search for. The following regular expression searches for one or more consecutive uppercase characters: `[[:upper:]]+`.

Extracting Substrings by Using the REGEXP_SUBSTR Function

```
REGEXP_SUBSTR (source_char, pattern [, position  
                [, occurrence [, match_option]]])
```

```
SELECT REGEXP_SUBSTR(street_address , ' [^ ]+ ') AS Road  
FROM locations;
```

	ROAD
1	Via
2	Calle
3	(null)
4	(null)
5	labberwocky

In this example, the road names are extracted from the `LOCATIONS` table. To do this, the contents in the `STREET_ADDRESS` column that are after the first space are returned by using the `REGEXP_SUBSTR` function. In the expression used in the query `' [^ ]+ '`:

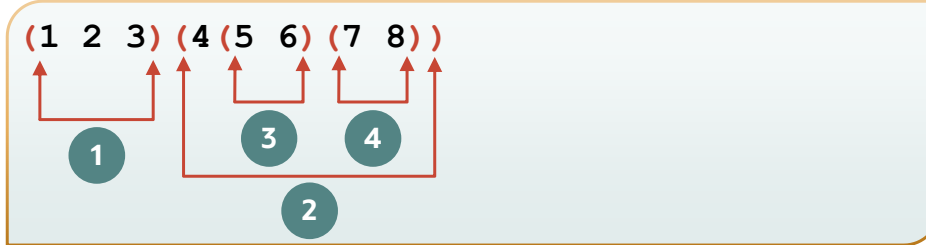
- `[` starts the expression
- `^` indicates NOT
- `\ '` indicates space
- `]` ends the expression
- `+` indicates 1 or more

Subexpressions

- Examine this expression:

```
(1 2 3) (4 (5 6) (7 8))
```

- The subexpressions are:



Oracle Database provides regular expression support parameters to access a subexpression. In the example in the slide, a string of digits is shown. The parentheses identify the subexpressions within the string of digits. Reading from left to right, and from outer parentheses to the inner parentheses, the subexpressions in the string of digits are:

- 1) 123
- 2) 45678
- 3) 56
- 4) 78

You can search for any of those subexpressions with the `REGEXP_INSTR` and `REGEXP_SUBSTR` functions.

Using Subexpressions with Regular Expression Support

```
SELECT
  REGEXP_INSTR
1  ('0123456789',      -- source char or search value
2  '(123) (4(56) (78))', -- regular expression patterns
3  1,                  -- position to start searching
4  1,                  -- occurrence
5  0,                  -- return option
6  'i',                -- match option (case insensitive)
7  1)                  -- subexpression on which to search
  "Position"
FROM dual;
```

Position	
1	2

REGEXP_INSTR and REGEXP_SUBSTR have an optional SUBEXPR parameter that lets you target a particular substring of the regular expression being evaluated.

In the example in the slide, you may want to search for the first subexpression pattern in your list of subexpressions. The example shown identifies several parameters for the REGEXP_INSTR function.

1. The string you are searching is identified.
2. The subexpressions are identified. The first subexpression is 123. The second subexpression is 45678, the third is 56, and the fourth is 78.
3. The third parameter identifies from which position to start searching.
4. The fourth parameter identifies the occurrence of the pattern you want to find. 1 means find the first occurrence.
5. The fifth parameter is the return option. This is the position of the first character of the occurrence. (If you specify 1, the position of the character following the occurrence is returned.)
6. The sixth parameter identifies whether your search should be case-sensitive or not.
7. The last parameter specifies which subexpression you want to find. In the example shown, you are searching for the first subexpression, which is 123.

Why access the nth subexpression?

- A more realistic use: DNA sequencing
- You may need to find a specific subpattern that identifies a protein needed for immunity in mouse DNA.

SELECT

```
REGEXP_INSTR('ccacotttccctccactcctcacgtttctcacctgtaaagcgctccctccctcatccccatgc  
ccccttaccctgcagggtagagtaggctagaaccagagagctccaagctccatctgtggagaggtgccatcctt  
gggctgcagagagaggagaatttgcccaaagctgcctgcagagcttcaccacccttagtctcacaagccttga  
gttcatagcatttcttgagttttcacctgccagcaggacactgcagcaccctaaagggttcccaggagtaggg  
ttgccctcaagaggctcttgggtctgatggccacatcctggaattgtttcaagttgatgggtcacagccctgagg  
catgtaggggctggggatgcgctctgctctgctctcctctcctgaaccctgaaccctctgggtacccagagc  
acttagagccag',
```

```
      '(gtc(tcac)(aaag))',  
      1, 1, 0, 'i',  
      1) "Position"
```

FROM dual;

1	Position
1	195

In life sciences, you may need to extract the offsets of subexpression matches from a DNA sequence for further processing. For example, you may need to find a specific protein sequence, such as the begin offset for the DNA sequence preceded by `gtc` and followed by `tcac`, followed by `aaag`. To accomplish this goal, you can use the `REGEXP_INSTR` function, which returns the position where a match is found.

In the example in the slide, the position of the first subexpression (`gtc`) is returned. `gtc` appears starting in position 195 of the DNA string.

If you modify the example in the slide to search for the second subexpression (`tcac`), the query results in the following output. `tcac` appears starting in position 198 of the DNA string.

1	Position
1	198

If you modify the example in the slide to search for the third subexpression (`aaag`), the query results in the following output. `aaag` appears starting in position 202 of the DNA string.

1	Position
1	202

REGEXP_SUBSTR: Example

```
SELECT
  REGEXP_SUBSTR
  ① ('acgctgcactgca', -- source char or search value
  ② 'acg(.*)gca',    -- regular expression pattern
  ③ 1,               -- position to start searching
  ④ 1,               -- occurrence
  ⑤ 'i',             -- match option (case insensitive)
  ⑥ 1)              -- sub-expression
  "Value"
FROM dual;
```

	Value
1	ctgcact

In the example in the slide:

1. acgctgcactgca is the source to be searched.
2. acg(.*)gca is the pattern to be searched. Find acg followed by gca with potential characters between the acg and the gca.
3. Start searching at the first character of the source.
4. Search for the first occurrence of the pattern.
5. Use non-case-sensitive matching on the source.
6. Use a nonnegative integer value that identifies the *n*th subexpression to be targeted. This is the subexpression parameter. In this example, 1 indicates the first subexpression. You can use a value from 0–9. A zero means that no subexpression is targeted. The default value for this parameter is 0.

Using the REGEXP_COUNT Function

```
REGEXP_COUNT (source_char, pattern [, position
              [, occurrence [, match_option]])
```

```
SELECT REGEXP_COUNT (
  'ccacctttccctccactcctcacgtttctcacctgtaaagcgctccctccctcatcccatgcccccttaccctgcag
  ggtagagtaggctagaaaccagagagctccaagctccatctgtggagaggtgccatccttgggctgcagagagaggagaat
  ttgccccaaagctgctgcagagcttcaccacccttagtctcacaagccttgagttcatagcatttcttgagttttcacc
  ctgcccagcaggacactgcagcaccctgaagggttcccaggagtaggggtgcctcaagaggctcttggtctgatggcca
  catcctggaattgttttcaagttgatggtcacagccctgaggcatgtagggcgctggggatgcgctctgctctgctctcct
  ctctgaaccctgaaccctctggctacccagagcacttagagccag',
  'gtc') AS Count
FROM dual;
```

	COUNT
1	4

The REGEXP_COUNT function evaluates strings by using characters as defined by the input character set. It returns an integer indicating the number of occurrences of the pattern. If no match is found, the function returns 0.

In the example in the slide, the number of occurrences for a DNA substring is determined by using the REGEXP_COUNT function.

The following example shows that the number of times the pattern 123 occurs in the string 123123123123 is three times. The search starts from the second position of the string.

```
SELECT REGEXP_COUNT
  ('123123123123', -- source char or search value
  '123',          -- regular expression pattern
  2,              -- position where the search should start
  'i')            -- match option (case insensitive)
  AS Count
FROM dual;
```

	COUNT
1	3

Regular Expressions and Check Constraints: Examples

```
ALTER TABLE emp8
ADD CONSTRAINT email_addr
CHECK (REGEXP_LIKE(email, '@')) NOVALIDATE;
```

```
INSERT INTO emp8 VALUES
(500, 'Christian', 'Patel', 'ChrisP2creme.com',
1234567890, '12-Jan-2004', 'HR_REP', 2000, null, 102, 40);
```

```
Error starting at line 1 in command:
INSERT INTO emp8 VALUES
(500, 'Christian', 'Patel',
'ChrisP2creme.com', 1234567890,
'12-Jan-2004', 'HR_REP', 2000, null, 102, 40)
Error report:
SQL Error: ORA-02290: check constraint (TEACH.B.EMAIL_ADDR) violated
02290. 00000 - "check constraint (%s.%s) violated"
*Cause: The values being inserted do not satisfy the named check
*Action: do not insert values that violate the constraint.
```

Regular expressions can also be used in CHECK constraints. In this example, a CHECK constraint is added on the EMAIL column of the EMPLOYEES table. This ensures that only strings containing an “@” symbol are accepted. The constraint is tested. The CHECK constraint is violated because the email address does not contain the required symbol. The NOVALIDATE clause ensures that existing data is not checked.

For the example in the slide, the emp8 table is created by using the following code:

```
CREATE TABLE emp8 AS SELECT * FROM employees;
```

Note: The slide example is executed by using the Execute Statement option in SQL Developer. The output format differs if you use the Run Script option.



Quiz

With the use of regular expressions in SQL and PL/SQL, you can:

- Avoid intensive string processing of SQL result sets by middle-tier applications
- Avoid data validation logic on the client
- Enforce constraints on the server

Answer: a, b, c

For Instructor Use Only.

This document should not be distributed.

Summary



In this appendix, you should have learned how to use regular expressions to search for, match, and replace strings.

In this appendix, you have learned to use the regular expression support features. Regular expression support is available in both SQL and PL/SQL.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.

MySQL Syntax



For Instructor Use Only.

This document should not be distributed.

Objectives



Compare MySQL SQL Syntax with Oracle SQL Syntax

In this appendix, you compare the MySQL SQL Syntax to the Oracle SQL Syntax.

Icons Used in This Course



**Indicates the output from
Oracle Database**



**Indicates the output from
MySQL**

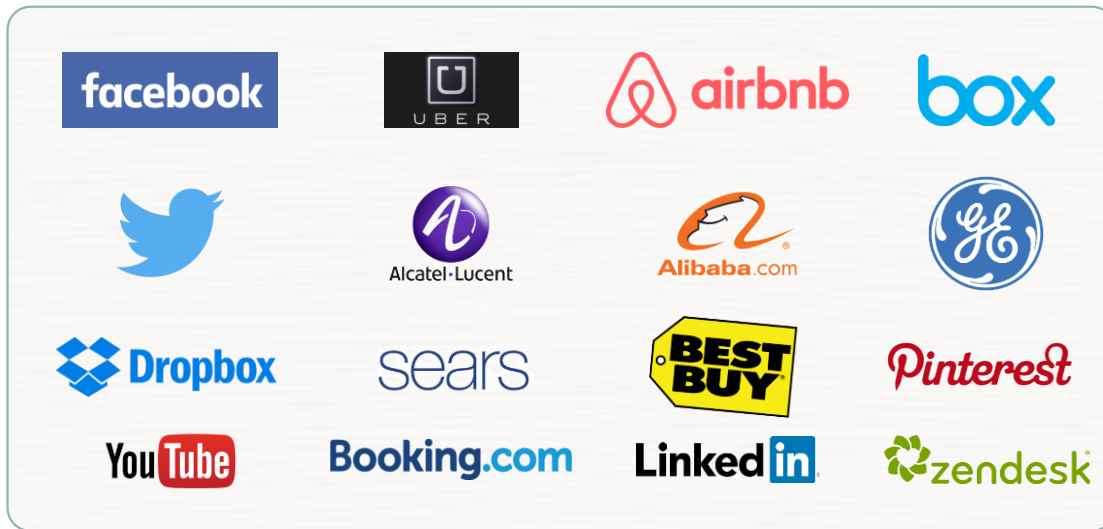
The above icons have been used throughout the course to indicate the output results from Oracle Database and MySQL.

The respective icons have also been used in the title of the slides to indicate exclusive Oracle Database– or MySQL-only content.

Look out for these icons throughout the course!

MySQL: A Modern Database for the Digital Age

Digital Disruptors and Large Enterprises Rely on MySQL to Innovate



The slide lists several digital disruptors and companies that have redefined customer expectations. They rely on MySQL, and so do large enterprises that drive digital transformation initiatives.

They choose MySQL because it is a modern database designed for web-based applications, which allows them to innovate quickly. But those are not the only reasons. As the world's most popular open-source database with over 20 years of existence, MySQL is a proven, battle-tested, and mature technology.

High Scalability with MySQL



- MySQL Cluster powers a mobile network in Asia that has a count of over 800 million subscribers.
- A government, also in Asia, has processed over one billion IDs for its citizens.



MySQL-Supported Operating Systems

MySQL:

- Provides control and flexibility for users
- Supports multiple commodity platforms, including:
 - Windows (x86, x86_64)
 - Linux (x86, x86_64)
 - Oracle Solaris (SPARC_64, x86_64, x86)
 - Mac OS X (x86, x86_64)
- Can be compiled to run on other platforms

For a full list of supported operating systems, see the website at:
<http://www.mysql.com/support/supportedplatforms/database.html>.

For Instructor Use Only.
This document should not be distributed.

MySQL Enterprise Edition



Advanced Features

- Scalability
 - MySQL Enterprise Thread Pool
 - Group Replication
 - InnoDB Cluster
- High Availability
 - MySQL Enterprise HA
- Security
 - Network Access Control
 - MySQL Enterprise Authentication
 - MySQL Enterprise Audit
 - MySQL Enterprise Encryption/Transparent Data Encryption (TDE)
 - MySQL Enterprise Firewall



Management Tools

- Monitoring
 - MySQL Enterprise Monitor
- Backup
 - MySQL Enterprise Backup
- Development
 - MySQL Connectors
- Administration
 - MySQL Workbench
 - MySQL Utilities
- Migration



Support

- Technical and Consultative Support
 - Oracle Premier Support for MySQL
- Oracle Certifications
 - Oracle Certified MySQL Database Administrator
 - Oracle Certified MySQL Developer

Scalability

- MySQL Enterprise Thread Pool allows you to scale the performance of your application in the face of increasing user, query, and data loads.
- The MySQL Group Replication feature is a multi-master, update-anywhere replication plug-in for MySQL.
- InnoDB Cluster is an integrated, native, full stack high availability solution for MySQL.

MySQL Enterprise High Availability

MySQL Enterprise High Availability enables you to meet the availability requirements of even the most demanding, mission-critical applications. MySQL Group Replication provides native high availability with built-in group membership management, data consistency guarantees, conflict detection and handling, node failure detection, and database failover-related operations, all without the need for manual intervention or custom tooling.

MySQL Security

- **Network Access Control:** Restrict connections to your MySQL instances.
- **MySQL Enterprise Authentication:** Authenticate MySQL users against your existing directory services and security rules.
- **MySQL Enterprise Audit:** Generate a complete audit trail to track MySQL access and usage.
- **MySQL Enterprise Encryption:** Protect the sensitive data that is stored in MySQL databases, in backups, or during transfer.
- **MySQL Transparent Data Encryption (TDE):** Protect data at rest and securely manage your encryption keys.
- **MySQL Enterprise Firewall:** Ensure real-time protection against database-specific attacks.

Why MySQL Enterprise Edition?

In addition to all the features you love...



For Instructor Use Only.

This document should not be distributed.

Oracle Premier Support for MySQL



- Largest MySQL engineering and support organization
- Backed by MySQL developers
- World-class support in 29 languages
- Hot fixes and maintenance releases
- 24 x 7 x 365
- Unlimited incidents
- Consultative support
- Global scale and reach

Get immediate help for any MySQL issue, plus expert advice.

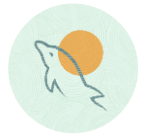


"The MySQL support service has been essential in helping us with troubleshooting and providing recommendations for the production cluster. Thanks."

– Carlos Morales (playfulplay.com)

For Instructor Use Only:
This document should not be distributed.

MySQL and Oracle Integration



MySQL Integrates into the Oracle Environment

- Oracle Linux
- Oracle Solaris
- Oracle OpenStack
- Oracle Fusion Middleware
- Oracle Enterprise Manager
- Oracle Secure Backup
- Oracle Clusterware
- Oracle Audit Vault and Database Firewall



In addition to Oracle Enterprise Manager, MySQL is now integrated with nearly all relevant Oracle products.

Oracle wants to make it very easy for existing Oracle customers to integrate and manage MySQL in their current environment.

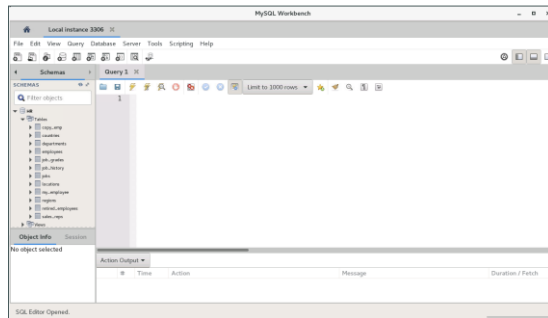
For Instructor Use Only.
This document should not be distributed.

Development Environments for SQL in MySQL

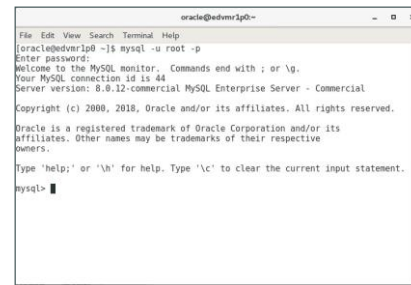


There are two MySQL development environments available for this course:

- For this course, the primary tool is MySQL Workbench.
- `mysql`, the MySQL command-line tool, can be used instead.



MySQL
Workbench



mysql Command-
Line Tool

There are various development environments for writing SQL statements. MySQL Workbench and the `mysql` command-line tool are commonly used for MySQL database development.

MySQL Workbench

This is a graphical interface for developing SQL. It provides features to view the database components and update values by navigating a graphical user interface in addition to entering SQL statements. In this course, MySQL Workbench is used to create and execute example SQL statements. You will use MySQL Workbench to perform the hands-on exercises.

mysql – the MySQL Command-Line Tool

If you like working with a command-line interface, you can use `mysql`, which is the command-line interpreter for MySQL Database. It processes SQL statements and also MySQL-specific commands. It can also be used to run all SQL commands covered in this course.

MySQL Websites

- <http://www.mysql.com>:
 - Product information
 - Services (Training, Certification, Consulting, and Support)
 - White papers, webinars, and other resources
 - MySQL Enterprise Edition downloads (trial versions)
- <http://dev.mysql.com>:
 - Developer Zone (forums, articles, Planet MySQL, and more)
 - Documentation
 - Downloads
- <https://github.com/mysql>:
 - Source code for MySQL Server and other MySQL products



Download MySQL Community Edition general availability (GA) and development releases from the <http://dev.mysql.com> website. This site also hosts the online documentation, which is an extremely valuable resource for both beginners and expert users of MySQL, and includes several example databases. Download trial versions of the MySQL Enterprise Edition software, view detailed product information, and find out more about Oracle MySQL services at <http://www.mysql.com>.

For Instructor Use Only.
This document should not be distributed.

MySQL Community Resources

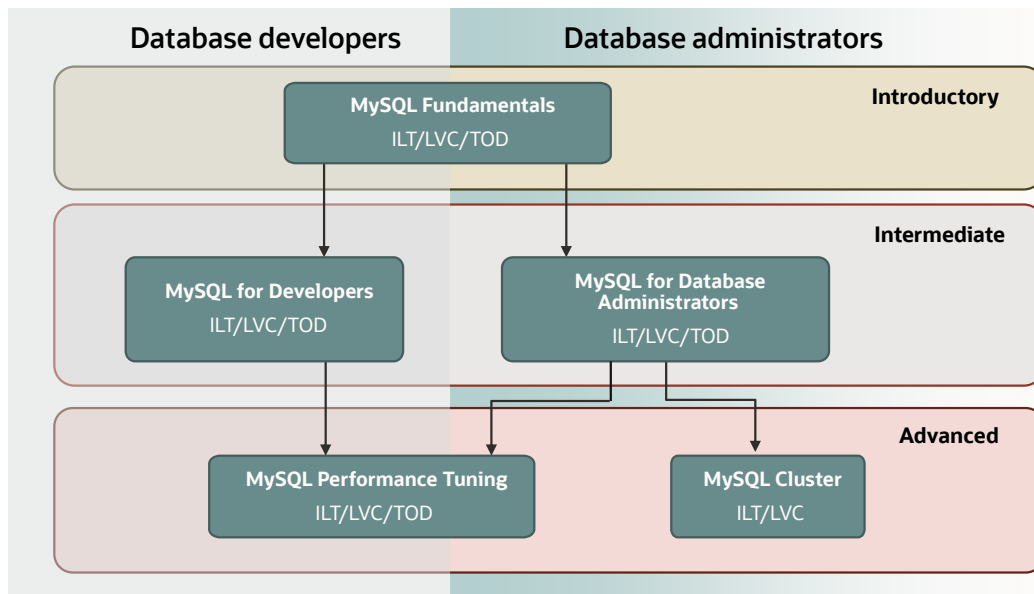
- Mailing lists
- Forums
- Developer articles
- MySQL Newsletter (published monthly)
- Planet MySQL blogs
- Social media channels
 - Facebook, Twitter, and Google+
- Physical and virtual events, including:
 - Developer days
 - MySQL Tech Tours
 - Webinars
- Bug tracking
- GitHub repositories



MySQL has a very large and active community. Engage with your peers, report bugs, share information, and discover what is happening in the world of MySQL by using the following resources:

- Mailing lists (<http://lists.mysql.com>)
- Forums (<http://forums.mysql.com>)
- Developer articles (<http://forums.mysql.com>)
- MySQL Newsletter (<http://www.mysql.com/news-and-events/newsletter/>)
- Planet MySQL blogs (<http://planet.mysql.com>)
- Social media channels:
 - Facebook: <http://facebook.com/mysql>
 - Twitter: https://twitter.com/mysql_community and <http://twitter.com/MySQL>
- Physical and virtual events (<http://www.mysql.com/news-and-events>)
- Bug tracking (<http://bugs.mysql.com>)
- GitHub repositories

Oracle University: MySQL Training

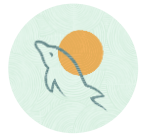


Training Formats

- **Instructor-Led Training (ILT):** Delivered in a classroom with the instructor and students present at the same location and time
- **Live Virtual Class (LVC):** Delivered by using video and audio through a web-based delivery system (WebEx) in which geographically distributed instructor and students participate, interact, and collaborate in a virtual class environment
- **Training On Demand (TOD):** On-demand training that takes traditional classroom training (complete with all classroom content, including lectures, white boarding, and practice videos) and makes it available in a video-based, online format so that you can start your customized training at your convenience

For full details about MySQL training options, go to <http://education.oracle.com/mysql>.

MySQL Certification



- The Oracle Certification Program validates your expertise in the following areas:
 - Oracle Certified Professional: MySQL 5.6 Database Administrator
 - Oracle Certified Professional: MySQL 5.6 Developer
- Take the examination at a local Pearson VUE test center:
 - At a time suitable to you
 - In over 175 countries

For full details of MySQL and other Oracle Certification options, visit:
<http://education.oracle.com/certification>.

For Instructor Use Only.
This document should not be distributed.

Selecting All Columns

Oracle SQL Developer:

```
SELECT *  
FROM departments;
```



	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	(null)	1700

MySQL Workbench:



#	department_id	department_name	manager_id	location_id
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	NULL	1700
*	NULL	NULL	NULL	NULL

You can display all columns of data in a table by following the `SELECT` keyword with an asterisk (*). In the example in the slide, the `departments` table contains four columns: `department_id`, `department_name`, `manager_id`, and `location_id`. The table contains eight rows, one for each department.

You can also display all columns in the table by listing them after the `SELECT` keyword. For example, the following SQL statement (like the example in the slide) displays all columns and all rows of the `departments` table:

```
SELECT department_id, department_name, manager_id, location_id  
FROM departments;
```

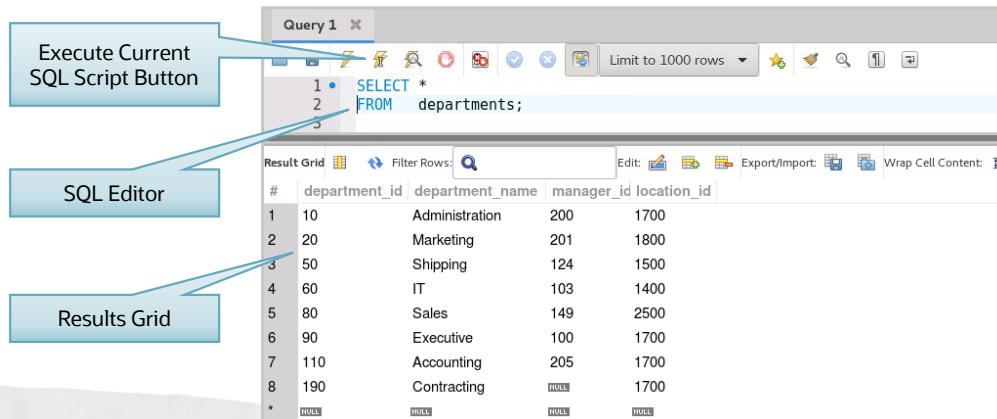
The examples in the slide show the output from executing the displayed statement in both Oracle SQL Developer and MySQL Workbench. Before considering other `SELECT` statements, the next few topics explain how to execute statements and what the output looks like in SQL Developer and MySQL Workbench.

Note: In this course, for examples in slides when both Oracle and MySQL use the same syntax, the outputs of both Oracle SQL Developer and MySQL Workbench appear. Your output should appear like the output of the database you are using. For topics where the databases use a different syntax or have other significant differences, separate slides are provided. You can skip over slides that do not apply to the database you are using.

Executing SQL Statements in MySQL Workbench



Enter statements in SQL Editor. To execute a single statement, place the cursor anywhere in the statement and click the **Execute Current SQL Script** button or press **Ctrl + Enter**. The results display in the Results Grid.



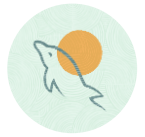
You can enter multiple statements in SQL Editor. You can select one statement or multiple statements in SQL Editor and click the **Execute SQL Script** button (to the left of the Execute Current SQL Script button) or press **Ctrl + Shift + Enter** to execute the selected statement or statements or execute all statements in SQL Editor if nothing is selected.

By default, the headings of the results in the Result Grid are the column names, displayed left-aligned above the results. Run the following SQL statement and observe the column headings in the output:

```
SELECT last_name, hire_date, salary
FROM employees;
```

You can override the column heading display with an alias. Column aliases are covered later in this lesson.

Executing SQL Statements in `mysql` Command-Line Client



Enter statements in the `mysql` command-line client. Press **Enter** to continue a statement to another line. Terminate a statement with semicolon (;) and press **Enter** to execute the statement. Results appear in a text table.

```
mysql> SELECT *
-> FROM departments;
+-----+-----+-----+-----+
| department_id | department_name | manager_id | location_id |
+-----+-----+-----+-----+
| 10 | Administration | 200 | 1700 |
| 20 | Marketing | 201 | 1800 |
| 50 | Shipping | 124 | 1500 |
| 60 | IT | 103 | 1400 |
| 80 | Sales | 149 | 2500 |
| 90 | Executive | 100 | 1700 |
| 110 | Accounting | 205 | 1700 |
| 190 | Contracting | NULL | 1700 |
+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

The default prompt for `mysql` command-line client is `mysql>`. When you press Enter before terminating the statement with a semicolon, the prompt becomes `->`. The MySQL examples in this course use MySQL Workbench, but you can use the `mysql` command-line client if you prefer a command-line tool.

By default, the headings of the results are the column names, displayed left-aligned. Run the following SQL statement and observe the column headings in the output:

```
SELECT last_name, hire_date, salary
FROM employees;
```

You can override the column heading display with an alias. Column aliases are covered later in this lesson.

Selecting Specific Columns

Oracle SQL Developer:

```
SELECT department_id, location_id
FROM departments;
```



	DEPARTMENT_ID	LOCATION_ID
1	10	1700
2	20	1800
3	50	1500
4	60	1400
5	80	2500
6	90	1700
7	110	1700
8	190	1700

MySQL Workbench:



#	department_id	location_id
1	60	1400
2	50	1500
3	10	1700
4	90	1700
5	110	1700
6	190	1700
7	20	1800
8	80	2500
*		

You can use the `SELECT` statement to display specific columns of the table by specifying the column names, separated by commas. The example in the slide displays all the department numbers and location numbers from the departments table. The results from Oracle SQL Developer and MySQL Workbench are the same. The order of the results might differ unless you specify an order. You learn to specify an order for the results in the lesson titled “Restricting and Sorting Data.”

In the `SELECT` clause, specify the columns that you want, in the order in which you want them to appear in the output. For example, to display the location before department number (from left to right), you use the following statement:

```
SELECT location_id, department_id
FROM departments;
```




Selecting Constant Expressions in MySQL

MySQL accepts the `FROM DUAL` clause but ignores it. The following statements are equivalent:

```
SELECT SYSDATE();
```

#	SYSDATE()
1	2018-08-17 18:24:44

```
SELECT SYSDATE()  
FROM DUAL;
```

#	SYSDATE()
1	2018-08-17 18:24:44

Selecting from the `dual` table is useful for computing a constant expression with the `SELECT` statement. It returns the value only once. Alternatively, you can select a constant, pseudocolumn, or expression from any table, but the value will be returned as many times as there are rows in the table.

For example, if you want to compute the expression $12 * 4 / 5 + 5 * 8$, use either of the following equivalent statements:

```
SELECT 12*4/5+5*8  
FROM dual;
```

```
SELECT 12*4/5+5*8;
```

MySQL does not create a `dual` table. In MySQL, the statement `SELECT * from dual;` will generate an error message because the `dual` table does not exist.

For Instructor Use Only.
This document should not be distributed.

Using Arithmetic Operators

```
SELECT last_name, salary, salary + 300
FROM employees;
```



	LAST_NAME	SALARY	SALARY+300
1	King	24000	24300
2	Kochhar	17000	17300
3	De Haan	17000	17300
4	Hunold	9000	9300
5	Ernst	6000	6300
6	Lorentz	4200	4500
7	Mourgos	5800	6100
8	Rajs	3500	3800
9	Davies	3100	3400
10	Matos	2600	2900

...



#	last_name	salary	salary + 300
1	King	24000.00	24300.00
2	Kochhar	17000.00	17300.00
3	De Haan	17000.00	17300.00
4	Hunold	9000.00	9300.00
5	Ernst	6000.00	6300.00
6	Lorentz	4200.00	4500.00
7	Mourgos	5800.00	6100.00
8	Rajs	3500.00	3800.00
9	Davies	3100.00	3400.00
10	Matos	2600.00	2900.00

...



The example in the slide uses the addition operator to calculate a salary increase of \$300 for all employees. The slide also displays a `SALARY+300` column in the output.

Note that the resultant calculated column, `SALARY+300`, is not a new column in the `EMPLOYEES` table; it is for display only. By default, the name of a new column comes from the calculation that generated it—in this case, `salary+300`.

Remember that the Oracle server ignores blank spaces before and after the arithmetic operator.

Rules of Precedence

- Multiplication and division occur before addition and subtraction.
- Operators of the same priority are evaluated from left to right.
- Parentheses are used to override the default precedence or to clarify the statement.

Operator Precedence

```
SELECT last_name, salary, 12*salary+100
FROM employees;
```



	LAST_NAME	SALARY	12*SALARY+100
1	King	24000	288100
2	Kochhar	17000	204100
3	De Haan	17000	204100
4	Hunold	9000	108100
...			



#	last_name	salary	12*salary+100
1	King	24000.00	288100.00
2	Kochhar	17000.00	204100.00
3	De Haan	17000.00	204100.00
4	Hunold	9000.00	108100.00
...			

```
SELECT last_name, salary, 12*(salary+100)
FROM employees;
```



	LAST_NAME	SALARY	12*(SALARY+100)
1	King	24000	289200
2	Kochhar	17000	205200
3	De Haan	17000	205200
4	Hunold	9000	109200
...			



#	last_name	salary	12*(salary+100)
1	King	24000.00	289200.00
2	Kochhar	17000.00	205200.00
3	De Haan	17000.00	205200.00
4	Hunold	9000.00	109200.00
...			

The first example in the slide displays the last name, salary, and annual compensation of employees. It calculates the annual compensation by multiplying the monthly salary with 12, plus a one-time bonus of \$100. Note that multiplication is performed before addition.

Note: Use parentheses to reinforce the standard order of precedence and to improve clarity. For example, the expression in the slide can be written as $(12 * \text{salary}) + 100$ with no change in the result.

Using Parentheses

You can override the rules of precedence by using parentheses to specify the desired order in which the operators are to be executed.

The second example in the slide displays the last name, salary, and annual compensation of employees. It calculates the annual compensation as follows: adding a monthly bonus of \$100 to the monthly salary and then multiplying that subtotal with 12. Because of the parentheses, addition takes priority over multiplication.

Defining a Null Value

- Null is a value that is unavailable, unassigned, unknown, or inapplicable.
- Null is not the same as zero or a blank space.

```
SELECT last_name, job_id, salary, commission_pct  
FROM employees;
```

#	LAST_NAME	JOB_ID	SALARY	COMMISSION_PCT
1	King	AD_PRES	24000	(null)
2	Kochhar	AD_VP	17000	(null)
3	De Haan	AD_VP	17000	(null)

...

12	Zlotkey	SA_MAN	10500	0.2
13	Abel	SA_REP	11000	0.3
14	Taylor	SA_REP	8600	0.2
15	Grant	SA_REP	7000	0.15

...

18	Fay	MK_REP	6000	(null)
19	Higgins	AC_MGR	12008	(null)
20	Gietz	AC_ACCOUNT	8300	(null)

#	last_name	job_id	salary	commission_pct
1	King	AD_PRES	24000.00	NULL
2	Kochhar	AD_VP	17000.00	NULL
3	De Haan	AD_VP	17000.00	NULL

...

12	Zlotkey	SA_MAN	10500.00	0.20
13	Abel	SA_REP	11000.00	0.30
14	Taylor	SA_REP	8600.00	0.20
15	Grant	SA_REP	7000.00	0.15

...

18	Fay	MK_REP	6000.00	NULL
19	Higgins	AC_MGR	12008.00	NULL
20	Gietz	AC_ACC...	8300.00	NULL

If a row lacks a data value for a particular column, that value is said to be `NULL` or to contain a null.

You can select columns with `NULL` value in a `SELECT` query and `NULL` values can be part of an arithmetic expression. Any arithmetic expression using `NULL` values results into `NULL`.

Columns of any data type can contain nulls. However, some constraints (`NOT NULL` and `PRIMARY KEY`) prevent nulls from being used in the column.

In the example in the slide, notice that only a sales manager or sales representative can earn a commission in the `COMMISSION_PCT` column of the `EMPLOYEES` table. Other employees are not entitled to earn commissions. A null represents that fact.

You can see that by default, SQL Developer uses the literal, `(null)`, to identify null values, and MySQL Workbench displays `NULL` as a graphic in white on gray. The `mysql` command-line client displays the word `NULL`.

Null Values in Arithmetic Expressions

Arithmetic expressions containing a null value evaluate to null.

```
SELECT last_name, 12*salary*commission_pct  
FROM employees;
```



	LAST_NAME	12*SALARY*COMMISSION_PCT
1	King	(null)
2	Kochhar	(null)
3	De Haan	(null)

...

12	Zlotkey	25200
13	Abel	39600
14	Taylor	20640
15	Grant	12600

...

17	Hartstein	(null)
18	Fay	(null)
19	Higgins	(null)
20	Gietz	(null)



#	last_name	12*salary*commission_pct
1	King	NULL
2	Kochhar	NULL
3	De Haan	NULL

...

12	Zlotkey	25200.0000
13	Abel	39600.0000
14	Taylor	20640.0000
15	Grant	12600.0000

...

17	Hartstein	NULL
18	Fay	NULL
19	Higgins	NULL
20	Gietz	NULL

If any column value in an arithmetic expression is null, the result is null. For example, if you attempt to perform division by zero, you get an error. However, if you divide a number by null, the result is a null or unknown.

In the example in the slide, employee King does not get any commission. Because the `commission_pct` column in the arithmetic expression is null, the result is null.

Using Column Aliases

```
SELECT last_name AS name, commission_pct comm
FROM employees;
```



	NAME	COMM
1	King	(null)
2	Kochhar	(null)
3	De Haan	(null)
4	Hunold	(null)

...



#	name	comm
1	King	NULL
2	Kochhar	NULL
3	De Haan	NULL
4	Hunold	NULL

...

```
SELECT last_name "Name", salary*12 "Annual Salary"
FROM employees;
```



	Name	Annual Salary
1	King	288000
2	Kochhar	204000
3	De Haan	204000
4	Hunold	108000

...



#	Name	Annual Salary
1	King	288000.00
2	Kochhar	204000.00
3	De Haan	204000.00
4	Hunold	108000.00

...

The first example displays the names and the commission percentages of all the employees. Note that the optional `AS` keyword has been used before the column alias 'name'. The result of the query is the same whether the `AS` keyword is used or not. Also, note that the SQL statement has the column aliases, `name` and `comm`, in lowercase, whereas the result of the query in Oracle displays the column headings in uppercase. As mentioned in the preceding slide, column headings appear in uppercase by default.

The second example displays the last names and annual salaries of all the employees. Because `Annual Salary` contains a space, it has been enclosed in double quotation marks. Note that the column heading in the output is exactly the same as the column alias.

Note: In MySQL the second example does not require the double quotation marks for 'Name'. It would be equivalent as the following:

```
SELECT last_name Name, salary*12 "Annual Salary"
FROM employees;
```

Concatenation Function in MySQL: CONCAT ()



The `CONCAT ()` function:

- Links columns or character strings to other columns
- Is a function that concatenates the values provided to it
- Creates a resultant column that is a character expression

```
SELECT CONCAT(last_name, job_id) AS "Employees"
FROM employees;
```



#	Employees
1	KingAD_PRES
2	KochharAD_VP
3	De HaanAD_VP
4	HunoldIT_PROG
5	ErnstIT_PROG
6	LorentzIT_PROG
7	MourgosST_MAN
8	RajsST_CLERK

You can link columns to other columns, arithmetic expressions, or constant values to create a character expression by using `CONCAT ()`. Columns names, separated by commas, are combined to make a single output column.

In the example, `Last_name` and `Job_id` are concatenated, and given the alias `Employees`. Note that the last name of the employee and the job code are combined to make a single output column, with no space between them.

The `AS` keyword before the alias name makes the `SELECT` clause easier to read.

Null Values with the Concatenation Operator

In MySQL, if you use the `CONCAT ()` function to concatenate a null value with a character string, the result is `NULL`.

Using Literal Character Strings in MySQL

```
SELECT CONCAT(last_name, ' is a ', job_id)
       AS 'Employee Details'
FROM   employees;
```



#	Employee Details
1	King is a AD_PRES
2	Kochhar is a AD_VP
3	De Haan is a AD_VP
4	Hunold is a IT_PROG
5	Ernst is a IT_PROG
6	Lorentz is a IT_PROG
7	Mourgos is a ST_MAN
8	Rajs is a ST_CLERK
9	Davies is a ST_CLERK
10	Matos is a ST_CLERK
...	

The example in the slide displays the last names and job codes of all employees. The column has the heading Employee Details. Note the spaces between the single quotation marks in the `SELECT` statement. The spaces improve the readability of the output.

In the following example, the last name and salary for each employee are concatenated with a literal, to give the returned rows more meaning:

```
SELECT CONCAT(last_name, ': 1 Month salary = ', salary) AS Monthly
FROM   employees;
```


Including a Single Quotation Mark in a String with an Escape Sequence in MySQL



To indicate a quotation mark is to be included in a string, use the \ ' escape sequence.

```
SELECT CONCAT(department_name,  
              ' Department\'s Manager Id: ', manager_id)  
      AS "Department and Manager"  
FROM departments;
```

#	Department and Manager
1	Administration Department's Manager Id: 200
2	Marketing Department's Manager Id: 201
3	Shipping Department's Manager Id: 124
4	IT Department's Manager Id: 103
5	Sales Department's Manager Id: 149
6	Executive Department's Manager Id: 100
7	Accounting Department's Manager Id: 205
8	NULL

Many SQL statements use character literals in expressions or conditions. If the literal itself contains a single quotation mark, you can use the (\) escape sequence to indicate that the quotation mark is part of the string.

In the example shown, the string contains a single quotation mark, which is normally interpreted as a delimiter of a character string. By using the \ ' escape sequence, however, the single quotation mark is included as part of the string. MySQL supports a number of other escape sequences.

For Instructor Use Only.
This document should not be distributed.

Duplicate Rows

The default display of queries is all rows, including duplicate rows.

1
`SELECT department_id
FROM employees;`



#	DEPARTMENT_ID
1	90
2	90
3	90
4	60
5	60
6	60
7	50
8	50
...	...



#	department_id
1	90
2	10
3	20
4	20
5	50
6	50
7	50
8	50
9	50
...	...

2
`SELECT DISTINCT department_id
FROM employees;`



#	DEPARTMENT_ID
1	(null)
2	90
3	20
4	110
5	50
6	80
7	60
8	10



#	department_id
1	90
2	10
3	20
4	50
5	60
6	80
7	90
8	110

Unless you indicate otherwise, SQL displays the results of a query without eliminating the duplicate rows. The first example in the slide displays all the department numbers from the `EMPLOYEES` table. Note that the department numbers are repeated.

To eliminate duplicate rows in the result, include the `DISTINCT` keyword in the `SELECT` clause immediately after the `SELECT` keyword. In the second example in the slide, the `EMPLOYEES` table actually contains 20 rows, but there are only seven unique department numbers in the table.

You can specify multiple columns after the `DISTINCT` qualifier. The `DISTINCT` qualifier affects all the selected columns, and the result is every distinct combination of the columns.

```
SELECT DISTINCT department_id, job_id  
FROM employees;
```

Displaying Table Structure by Using the DESCRIBE Command

Syntax:

```
DESCRIBE tablename
```

Example:

```
DESCRIBE employees
```



```
DESCRIBE Employees
Name          Null    Type
-----
EMPLOYEE_ID   NOT NULL NUMBER(6)
FIRST_NAME
LAST_NAME     NOT NULL VARCHAR2(25)
EMAIL         NOT NULL VARCHAR2(25)
PHONE_NUMBER  VARCHAR2(20)
HIRE_DATE     NOT NULL DATE
JOB_ID        NOT NULL VARCHAR2(10)
SALARY        NUMBER(8,2)
COMMISSION_PCT NUMBER(2,2)
MANAGER_ID    NUMBER(6)
DEPARTMENT_ID NUMBER(4)
```



#	Field	Type	Null	Key	Default	Extra
1	employee_id	int(11)	NO	PRI		
2	first_name	varchar(20)	YES			
3	last_name	varchar(25)	NO	MUL		
4	email	varchar(25)	NO	UNI		
5	phone_number	varchar(20)	YES			
6	hire_date	date	NO			
7	job_id	varchar(10)	NO	MUL		
8	salary	decimal(8,2)	YES			
9	commission_pct	decimal(2,2)	YES			
10	manager_id	int(11)	YES	MUL		
11	department_id	int(11)	YES	MUL		

In the syntax, *tablename* is the name of any existing table, view, or synonym that is accessible to the user.

The example in the slide displays information about the structure of the `employees` table using the `DESCRIBE` command.

In the resulting display, *Null* indicates that the values for this column may be unknown. `NOT NULL` in SQL Developer or `NO` in MySQL Workbench indicates that a column must contain data. *Type* displays the data type for a column.

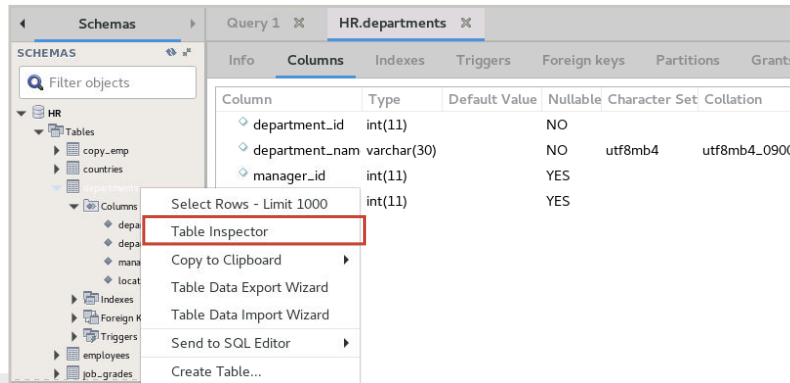
Data types are explained in the lesson titled “Introduction to Data Definition Language.”

Data Type	Description
NUMBER (<i>p</i> , <i>s</i>)	Number value having a maximum number of digits <i>p</i> , with <i>s</i> digits to the right of the decimal point
VARCHAR2 (<i>s</i>)	Variable-length character value of maximum size <i>s</i>
DATE	Date and time value between January 1, 4712 B.C. and December 31, A.D. 9999

Displaying Table Structure by Using MySQL Workbench



- Use the `DESCRIBE` command to display the structure of a table.
- Alternatively, right-click the table in the Navigator and select **Table Inspector** from the menu. Select the **Columns** tab.



You can display the structure of a table in MySQL Workbench by using the `DESCRIBE` command. The command displays the column names and the data types, and it shows you whether a column *must* contain data (that is, whether the Nullable column indicates `NO`).

Using the SQL Developer GUI interface, you can right-click the table in the Navigator and select **Table Inspector** from the menu. Then click the **Columns** tab.

Note: `DESCRIBE` is abbreviated as `DESC`.

Limiting Rows by Using a Selection

```
SELECT employee_id, last_name, job_id, department_id
FROM employees;
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90
4	103	Hunold	IT_PROG	60
5	104	Ernst	IT_PROG	60
6	107	Lorentz	IT_PROG	60

...



#	employee_id	last_name	job_id	department_id
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90
4	103	Hunold	IT_PROG	60
5	104	Ernst	IT_PROG	60
6	107	Lorentz	IT_PROG	60

...

What if you want to retrieve all employees in department 90, but not other departments?



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90



#	employee_id	last_name	job_id	department_id
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90

In the example in the slide, assume that you want to display all the employees in department 90. The rows with a value of 90 in the `DEPARTMENT_ID` column are the only ones that are returned. This method of restriction is the basis of the `WHERE` clause in SQL.

Using the WHERE Clause

```
SELECT employee_id, last_name, job_id, department_id
FROM   employees
WHERE  department_id = 90 ;
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90



#	employee_id	last_name	job_id	department_id
1	100	King	AD_PRES	90
2	101	Kochhar	AD_VP	90
3	102	De Haan	AD_VP	90
*	NULL	NULL	NULL	NULL

In the example, the `SELECT` statement retrieves the employee ID, last name, job ID, and department number of all employees who are in department 90.

Note: You cannot use column alias in the `WHERE` clause.

Character Strings and Dates

- Character strings and date values are enclosed within single quotation marks (' ').
- Character values are case-sensitive and date values are format-sensitive.
- The default display format for date is DD-MON-RR in Oracle databases and YYYY-MM-DD in MySQL.

```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  last_name = 'Whalen' ;
```



	LAST_NAME	JOB_ID	DEPARTMENT_ID
1	Whalen	AD_ASST	10



#	last_name	job_id	department_id
1	Whalen	AD_ASST	10

```
SELECT last_name
FROM   employees
WHERE  hire_date = '17-OCT-11' ;
```



	LAST_NAME
1	Rajs

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date = '2011-10-17' ;
```



#	last_name	hire_date
1	Rajs	2011-10-17

You must enclose character strings and dates in the `WHERE` clause within single quotation marks (' '). Number constants, however, need not be enclosed within single quotation marks.

Remember that all character searches are case-sensitive. In the following example, observe that no rows are returned because the `EMPLOYEES` table stores all the last names in mixed case:

```
SELECT last_name, job_id, department_id
FROM   employees
WHERE  last_name = 'WHALEN' ;
```

Oracle and MySQL databases store dates in an internal numeric format, representing the century, year, month, day, hours, minutes, and seconds. The default date display is in the `DD-MON-RR` format in Oracle and `YYYY-MM-DD` in MySQL.

Note: For details about the `RR` format and about changing the default date format, see the lesson titled “Using Single-Row Functions to Customize Output.” Also, you learn about the use of single-row functions such as `UPPER` and `LOWER` to override case-sensitivity in the same lesson.

Using Comparison Operators

Let us look at some examples:

```
SELECT last_name, salary
FROM employees
WHERE salary <= 3000 ;
```



#	LAST_NAME	SALARY
1	Matos	2600
2	Vargas	2500



#	last_name	salary
1	Matos	2600.00
2	Vargas	2500.00

```
SELECT *
FROM employees
WHERE last_name = 'Abel' ;
```

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID	
1	174	Ellen	Abel	EABEL	011.44.1644.429267	11-MAY-12	SA_REP	11000	0.3	149	8



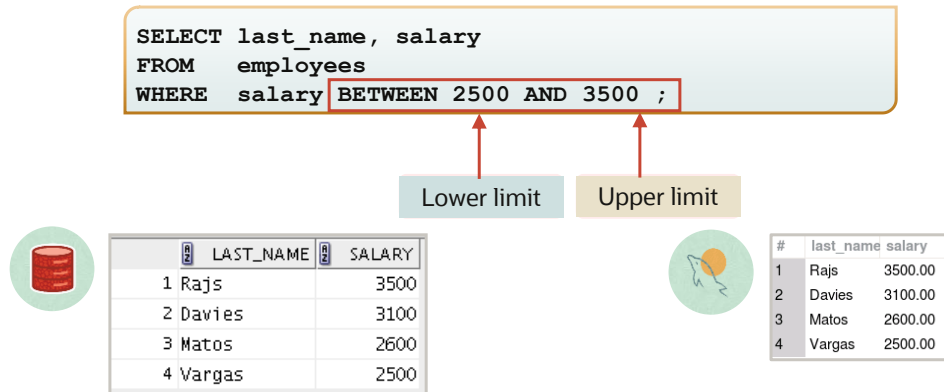
#	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
1	174	Ellen	Abel	EABEL	011.44.1644....	2012-05-11	SA_REP	11000.00	0.30	149	80
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

In the first example in the slide, the `SELECT` statement retrieves the last name and salary from the `employees` table for any employee whose salary is less than or equal to \$3,000. Observe that there is an explicit value supplied to the `WHERE` clause. The explicit value of 3000 is compared to the salary value in the `salary` column of the `employees` table.

In the second example, the `SELECT` statement retrieves all rows where the last name is `Abel`. Because `*` is used in the `SELECT` statement, all fields from the `employees` table appear in the result set.

Range Conditions Using the BETWEEN Operator

You can use the BETWEEN operator to display rows based on a range of values:



You can display rows based on a range of values using the BETWEEN operator. The range that you specify contains a lower limit and an upper limit.

The SELECT statement in the slide returns rows from the employees table for any employee whose salary is between \$2,500 and \$3,500.

Values that are specified with the BETWEEN operator are inclusive. Remember, you must specify the lower limit first.

You can also use the BETWEEN operator on character values:

```
SELECT last_name FROM employees
WHERE last_name BETWEEN 'King' AND 'Whalen';
```

Using the IN Operator

Use the IN operator to test for values in a list:

```
SELECT employee_id, last_name, salary, manager_id
FROM   employees
WHERE  manager_id IN (100, 101, 201)
```



	EMPLOYEE_ID	LAST_NAME	SALARY	MANAGER_ID
1	101	Kochhar	17000	100
2	102	De Haan	17000	100
3	124	Mourgos	5800	100
4	149	Zlotkey	10500	100
5	201	Hartstein	13000	100
6	200	Whalen	4400	101
7	205	Higgins	12008	101
8	202	Fay	6000	201



#	employee_id	last_name	salary	manager_id
1	101	Kochhar	17000.00	100
2	102	De Haan	17000.00	100
3	124	Mourgos	5800.00	100
4	149	Zlotkey	10500.00	100
5	201	Hartstein	13000.00	100
6	200	Whalen	4400.00	101
7	205	Higgins	12008.00	101
8	202	Fay	6000.00	201
*	NULL	NULL	NULL	NULL

You can use the IN operator to test for values among a specified set of values. The condition defined using the IN operator is also known as the *membership condition*.

The example in the slide displays employee IDs, last names, salaries, and manager's employee IDs for all the employees whose manager's employee ID is 100, 101, or 201.

Note: The set of values can be specified in any random order—for example, (201,100,101).

The IN operator can be used with any data type. The following example returns rows from the employees table, for any employee whose last name is included with the IN operator:

```
SELECT employee_id, manager_id, department_id
FROM   employees
WHERE  last_name IN ('Hartstein', 'Vargas');
```

If characters or dates are used in a list, they must be enclosed within single quotation marks (' ').

Pattern Matching Using the LIKE Operator

- You can use the LIKE operator to perform wildcard searches of valid string patterns.
- The search conditions can contain either literal characters or numbers:
 - % denotes zero or more characters.
 - _ denotes one character.

```
SELECT first_name
FROM employees
WHERE first_name LIKE 'S%';
```



#	FIRST_NAME
1	Shelley
2	Steven



#	first_name
1	Shelley
2	Steven

You may not always know the exact value to search for. You can select rows that match a character pattern by using the LIKE operator. The character pattern-matching operation is referred to as a *wildcard* search. Two symbols can be used to construct the search string.

Symbol	Description
%	Represents any sequence of zero or more characters
_	Represents any single character

The SELECT statement in the slide returns the first name from the employees table for any employee whose first name begins with the letter “S.” Note the uppercase “S.” Consequently, names beginning with a lowercase “s” are not returned.

The LIKE operator can be used as a shortcut for some BETWEEN comparisons. In Oracle, the following example displays the last names and hire dates of all employees who joined between January 2015 and December 2015:

```
SELECT last_name, hire_date
FROM employees
WHERE hire_date LIKE '%15'
```

In MySQL, the following example displays the last names and hire dates of all employees who joined between January 2015 and December 2015:

```
SELECT last_name, hire_date
FROM employees
WHERE hire_date LIKE '2015%';
```

Combining Wildcard Symbols

- You can combine the two wildcard symbols (% , _) with literal characters for pattern matching:

```
SELECT last_name
FROM employees
WHERE last_name LIKE '_o%';
```



	LAST_NAME
1	Kochhar
2	Lorentz
3	Mourgos



#	last_name
1	Kochhar
2	Lorentz
3	Mourgos

- You can use the ESCAPE identifier to search for the actual % and _ symbols.

The % and _ symbols can be used in any combination with literal characters. The example in the slide displays the names of all employees whose last names have the letter “o” as the second character.

When you need to have an exact match for the actual % and _ characters, use the ESCAPE identifier. For example, in Oracle, to find the last name and job ID of all the employees whose job ID contains 'SA_', use the following statement:

```
SELECT last_name, job_id
FROM employees
WHERE job_id LIKE 'SA\_%' ESCAPE '\';
```

To do the same thing in MySQL, use the following statement:

```
SELECT last_name, job_id
FROM employees
WHERE job_id LIKE 'SA\_%';
```

Using NULL Conditions

You can use the `IS NULL` operator to test for NULL values in a column.

```
SELECT last_name, manager_id
FROM   employees
WHERE  manager_id IS NULL ;
```



#	LAST_NAME	#	MANAGER_ID
1	King		(null)



#	last_name	manager_id
1	King	NULL

There are two types of NULL conditions:

- `IS NULL` tests for NULL values
- `IS NOT NULL` tests for values that are not NULL

A null value means that the value is unavailable, unassigned, unknown, or inapplicable. Therefore, you cannot test with `=`, because a null cannot be equal or unequal to any value. The example in the slide retrieves the `last_name` and `manager_id` of all employees who do not have a manager.

Here is another example: To display the last name, job ID, and commission for all employees who are *not* entitled to receive a commission, use the following SQL statement:

```
SELECT last_name, job_id, commission_pct
FROM   employees
WHERE  commission_pct IS NULL;
```

Using the AND Operator

AND requires both the component conditions to be true:

```
SELECT employee_id, last_name, job_id, salary
FROM   employees
WHERE  salary >= 10000
AND    job_id LIKE '%MAN%';
```



#	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	149	Zlotkey	SA_MAN	10500
2	201	Hartstein	MK_MAN	13000



#	employee_id	last_name	job_id	salary
1	149	Zlotkey	SA_MAN	10500.00
2	201	Hartstein	MK_MAN	13000.00
*	NULL	NULL	NULL	NULL

In the example, both the component conditions must be true for any record to be selected. Therefore, only those employees who have a job title that contains the string 'MAN' *and* earn \$10,000 or more are selected.

All character searches are case-sensitive, that is, no rows are returned if 'MAN' is not uppercase. Further, character strings must be enclosed within quotation marks.

AND Truth Table

The following table shows the results of combining two expressions with AND:

AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

Using the OR Operator

OR requires either component condition to be true:

```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary >= 10000
OR job_id LIKE '%MAN%';
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	100	King	AD_PRES	24000
2	101	Kochhar	AD_VP	17000
3	102	De Haan	AD_VP	17000
4	124	Mourgos	ST_MAN	5800
5	149	Zlotkey	SA_MAN	10500
6	174	Abel	SA_REP	11000
7	201	Hartstein	MK_MAN	13000
8	205	Higgins	AC_MGR	12008



#	employee_id	last_name	job_id	salary
1	100	King	AD_PRES	24000.00
2	101	Kochhar	AD_VP	17000.00
3	102	De Haan	AD_VP	17000.00
4	124	Mourgos	ST_MAN	5800.00
5	149	Zlotkey	SA_MAN	10500.00
6	174	Abel	SA_REP	11000.00
7	201	Hartstein	MK_MAN	13000.00
8	205	Higgins	AC_MGR	12008.00
*	NULL	NULL	NULL	NULL

In the example, either component condition can be true for any record to be selected. Therefore, any employee who has a job ID that contains the string 'MAN' or earns \$10,000 or both is selected.

OR Truth Table

The following table shows the results of combining two expressions with OR:

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

Using the NOT Operator

NOT is used to negate a condition:

```
SELECT last_name, job_id
FROM employees
WHERE job_id
      NOT IN ('IT_PROG', 'ST_CLERK',
'SA_REP');
```



#	LAST_NAME	JOB_ID
1	De Haan	AD_VP
2	Fay	MK_REP
3	Gietz	AC_ACCOUNT
4	Hartstein	MK_MAN
5	Higgins	AC_MGR
6	King	AD_PRES
7	Kochhar	AD_VP
8	Mourgos	ST_MAN
9	Whalen	AD_ASST
10	Zlotkey	SA_MAN



#	last_name	job_id
1	King	AD_PRES
2	Kochhar	AD_VP
3	De Haan	AD_VP
4	Mourgos	ST_MAN
5	Zlotkey	SA_MAN
6	Whalen	AD_ASST
7	Hartstein	MK_MAN
8	Fay	MK_REP
9	Higgins	AC_MGR
10	Gietz	AC_ACCOUNT

The example in the slide displays the last name and job ID of all employees whose job ID *is not* IT_PROG, ST_CLERK, or SA_REP.

NOT Truth Table

The following table shows the result of applying the NOT operator to a condition:

NOT	TRUE	FALSE	NULL
	FALSE	TRUE	TRUE/FALSE

Rules of Precedence

1

```
SELECT last_name, department_id, salary
FROM employees
WHERE department_id = 60
OR department_id = 80
AND salary > 10000;
```



	LAST_NAME	DEPARTMENT_ID	SALARY
1	Hunold	60	9000
2	Ernst	60	6000
3	Lorentz	60	4200
4	Zlotkey	80	10500
5	Abel	80	11000



#	last_name	department_id	salary
1	Hunold	60	9000.00
2	Ernst	60	6000.00
3	Lorentz	60	4200.00
4	Zlotkey	80	10500.00
5	Abel	80	11000.00

2

```
SELECT last_name, department_id, salary
FROM employees
WHERE (department_id = 60
OR department_id = 80)
AND salary > 10000;
```



	LAST_NAME	DEPARTMENT_ID	SALARY
1	Zlotkey	80	10500
2	Abel	80	11000



#	last_name	department_id	salary
1	Zlotkey	80	10500.00
2	Abel	80	11000.00

1. Precedence of the AND Operator: Example

- In this example, there are two conditions:
 - o The first condition is that the department ID is 80 and the salary is greater than \$10,000.
 - o The second condition is that the department ID is 60.
- Therefore, the SELECT statement reads as follows:
- "Select the row if an employee's department ID is 80 and earns more than \$10,000, or if the employee's department ID is 60."

2. Using Parentheses: Example

- In this example, there are two conditions:
 - o The first condition is that the department ID is 80 or 60.
 - o The second condition is that the salary is greater than \$10,000.
- Therefore, the SELECT statement reads as follows:
- "Select the row if an employee's department ID is 80 or 60, and if the employee earns more than \$10,000."

Using the ORDER BY Clause

You can sort the retrieved rows with the ORDER BY clause:

- ASC: Ascending order, default
- DESC: Descending order

```
SELECT last_name, job_id, department_id, hire_date
FROM employees
ORDER BY hire_date ;
```



	LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
1	De Haan	AD_VP	90	13-JAN-09
2	Kochhar	AD_VP	90	21-SEP-09
3	Higgins	AC_MGR	110	07-JUN-10
4	Gietz	AC_ACCOUNT	110	07-JUN-10
5	King	AD_PRES	90	17-JUN-11
6	Whalen	AD_ASST	10	17-SEP-11
7	Rajs	ST_CLERK	50	17-OCT-11

...



#	last_name	job_id	department_id	hire_date
1	De Haan	AD_VP	90	2009-01-13
2	Kochhar	AD_VP	90	2009-09-21
3	Gietz	AC_ACCOUNT	110	2010-06-07
4	Higgins	AC_MGR	110	2010-06-07
5	King	AD_PRES	90	2011-06-17
6	Whalen	AD_ASST	10	2011-09-17
7	Rajs	ST_CLERK	50	2011-10-17

...

The order of rows that are returned in a query result is undefined. The ORDER BY clause can be used to sort the rows. You can specify an expression, an alias, or a column position as the sort condition. You can specify multiple expressions in the ORDER BY clause. Rows are first sorted based on their values for the first expression. Rows with the same value for the first expression are then sorted based on their values for the second expression, and so on.

Syntax

```
SELECT          expr
FROM            table
[WHERE          condition(s)]
[ORDER BY      {column, expr, numeric_position} [ASC|DESC]];
```

In the syntax:

ORDER BY	Specifies the order in which the retrieved rows are displayed
ASC	Orders the rows in ascending order (This is the default order.)
DESC	Orders the rows in descending order

If the ORDER BY clause is not used, the sort order is undefined, and the Oracle server may not fetch rows in the same order for the same query twice. Use the ORDER BY clause to display the rows in a specific order.

Using SQL Row Limiting Clause in a Query in MySQL



You specify the row limiting clause in the SQL `SELECT` statement by placing it after the `ORDER BY` clause.

Syntax:

```
SELECT ...  
  FROM ...  
  [ WHERE ... ]  
  [ ORDER BY ... ]  
  [LIMIT {[offset,] row_count | row_count OFFSET offset}]
```

You specify the row limiting clause in the SQL `SELECT` statement by placing it after the `ORDER BY` clause. Note that an `ORDER BY` clause is required if you want consistent output. Without an `ORDER BY` clause, the first rows are not predictable. MySQL uses either of the following syntaxes in the row limiting clause.

- Use the `LIMIT` keyword followed by a nonnegative numeric integer for the maximum number of rows, followed by the `OFFSET` keyword, followed by the offset of the first row to return. The offset of the initial row is 0. For example:
 - `LIMIT 7 OFFSET 5` returns 7 rows starting at row 6, that is rows 6 - 12.
- Use the `LIMIT` keyword, followed by either one or two nonnegative numeric arguments separated by a comma. With two arguments, the first argument specifies the offset of the first row to return, and the second specifies the maximum number of rows to return. The offset of the initial row is 0. For example:
 - `LIMIT 5, 7` returns 7 rows starting at row 6, that is, rows 6 - 12. Note that this is the same as `LIMIT 7 OFFSET 5`.
 - `LIMIT 0, 7` returns the first 7 rows, that is, rows 1 - 7
 - `LIMIT 7` returns the first 7 rows, that is, rows 1 - 7

SQL Row Limiting Clause: Example in MySQL

```
SELECT employee_id, first_name
FROM employees
ORDER BY employee_id
LIMIT 7;
```

```
SELECT employee_id, first_name
FROM employees
ORDER BY employee_id
LIMIT 7 OFFSET 5;
```



#	employee_id	first_name
1	100	Steven
2	101	Neena
3	102	Lex
4	103	Alexander
5	104	Bruce
6	107	Diana
7	124	Kevin
*	NULL	NULL



#	employee_id	first_name
1	107	Diana
2	124	Kevin
3	141	Trenna
4	142	Curtis
5	143	Randall
6	144	Peter
7	149	Eleni
*	NULL	NULL

The first example in the slide returns the first seven employees after sorting the rows in ascending order of the `employee_id`.

The second example in the slide returns seven employees, starting with the sixth employee after sorting the rows in ascending order of the `employee_id`. For the second example, `LIMIT 5, 7` would be equivalent to `LIMIT 7 OFFSET 5`.

Note: If `employee_id` is assigned sequentially by the date when the employee joined the organization, these examples give us the top 7 employees and then employees 6-12, all in terms of seniority.

Using the SET Statement in MySQL



- Use the SET statement to create a user-defined variable and assign a value to it.
- User-defined variables are entered as `@var_name`.

```
SET @employee_num = 200;
```

```
SELECT employee_id, last_name, salary, department_id  
FROM employees  
WHERE employee_id = @employee_num ;
```



#	employee_id	last_name	salary	department_id
1	200	Whalen	4400.00	10
•	NULL	NULL	NULL	NULL

The example shown in the slide creates a user-defined variable for an employee number by using the SET command. At run time, the variable value is automatically substituted in the SELECT statement, which displays the employee number, name, salary, and department number for that employee.

The `employee_num` variable is specific to the session and remains assigned until the user assigns a different value or exits the session.

Case-Conversion Functions

- You can use the `LOWER` and `UPPER` functions to convert the case of character strings.
- For example:

```
SELECT last_name, UPPER(last_name), job_id, LOWER(job_id)
FROM   employees
WHERE  department_id = 90;
```



	LAST_NAME	UPPER(LAST_NAME)	JOB_ID	LOWER(JOB_ID)
1	King	KING	AD_PRES	ad_pres
2	Kochhar	KOCHHAR	AD_VP	ad_vp
3	De Haan	DE HAAN	AD_VP	ad_vp



#	last_name	UPPER(last_name)	job_id	LOWER(job_id)
1	King	KING	AD_PRES	ad_pres
2	Kochhar	KOCHHAR	AD_VP	ad_vp
3	De Haan	DE HAAN	AD_VP	ad_vp

`LOWER`, `UPPER`, and `INITCAP` are the three case-conversion functions.

- `LOWER`: Converts mixed-case or uppercase character strings to lowercase
- `UPPER`: Converts mixed-case or lowercase character strings to uppercase
- `INITCAP`: (Oracle) Converts the first letter of each word to uppercase and the remaining letters to lowercase

The slide shows an example of using `LOWER` and `UPPER` functions.

Case-Insensitive Queries in MySQL



By default, MySQL uses a case-insensitive character set and collation. Sorting and string comparisons consider upper and lower case of the same character to be equivalent in value. To display the employee number, name, and department number for employee Higgins:

```
SELECT employee_id, last_name, department_id
FROM employees
WHERE last_name = 'higgins';
```



#	employee_id	last_name	department_id
1	205	Higgins	110
*	NULL	NULL	NULL

The example in the slide displays the employee number, name, and department number of employee Higgins.

The `WHERE` clause of the SQL statement specifies the employee name as `higgins`. Even though all the data in the `employees` table is stored in proper case, the name `'higgins'` finds a match in the table and displays the results in the proper case. Any other combination, like `'hiGGins'`, `'HIGGINS'`, or `'Higgins'` would also match.

The lesson titled "Restricting and Sorting Data" explained that, by default, MySQL uses a case-insensitive character set and collation, so that in sorting, upper and lowercase letters sort as equivalent values. Also, in comparisons in a `WHERE` clause, upper and lowercase letters are considered equivalent and are found as matches.

Using Character-Manipulation Functions

1

```
SELECT last_name, CONCAT('Job category is ', job_id)
AS Job FROM employees
WHERE SUBSTR(job_id, 4) = 'REP';
```



	LAST_NAME	JOB
1	Abel	Job category is SA_REP
2	Fay	Job category is MK_REP
3	Grant	Job category is SA_REP
4	Taylor	Job category is SA_REP



#	last_name	Job
1	Abel	Job category is SA_REP
2	Taylor	Job category is SA_REP
3	Grant	Job category is SA_REP
4	Fay	Job category is MK_REP

2

```
SELECT employee_id, CONCAT(first_name, last_name) NAME,
LENGTH(last_name), INSTR(last_name, 'a') "Contains 'a'?"
FROM employees
WHERE SUBSTR(last_name, -1, 1) = 'n';
```



	EMPLOYEE_ID	NAME	LENGTH(LAST_NAME)	Contains 'a'?
1	102	LexDe Haan	7	5
2	200	JenniferWhalen	6	3
3	201	MichaelHartstein	9	2



#	employee_id	NAME	LENGTH(last_name)	Contains 'a'?
1	102	LexDe Haan	7	5
2	201	MichaelHartstein	9	2
3	200	JenniferWhalen	6	3

The first example in the slide displays employee last names and job IDs for all employees who have the string, REP, contained in the job ID, starting at the fourth position of the job ID.

The second SQL statement in the slide displays data such as employee ID, concatenated first name and last name, length of the last name, and the position of the first occurrence of the letter 'a' in the last name for those employees whose last names end with the letter "n."

Nesting Functions: Example

```
SELECT last_name,  
       UPPER(CONCAT(SUBSTR(LAST_NAME, 1, 8), ' US'))  
FROM   employees  
WHERE  department_id = 60;
```



#	LAST_NAME	#	UPPER(CONCAT(SUBSTR(LAST_NAME,1,8),'_US'))
1	Hunold		HUNOLD_US
2	Ernst		ERNST_US
3	Lorentz		LORENTZ_US



#	last_name	UPPER(CONCAT(SUBSTR(LAST_I
1	Hunold	HUNOLD_US
2	Ernst	ERNST_US
3	Lorentz	LORENTZ_US

The example in the slide displays the last names of employees in department 60. The evaluation of the SQL statement involves three steps:

1. The inner function retrieves the first eight characters of the last name.

Result1 = SUBSTR(LAST_NAME, 1, 8)

2. The outer function concatenates the result with _US.

Result2 = CONCAT(Result1, '_US')

3. The outermost function converts the results to uppercase.

Result3 = UPPER(Result2)

Result3 is displayed. The entire expression becomes the column heading because no column alias was given.

Numeric Functions

- **ROUND:** Rounds value to a specified decimal
- **TRUNC (Oracle) or TRUNCATE (MySQL):** Truncates value to a specified decimal
- **CEIL:** Returns the smallest whole number greater than or equal to a specified number
- **FLOOR:** Returns the largest whole number equal to or less than a specified number
- **MOD:** Returns remainder of division

Function	Result
ROUND (45.926, 2)	45.93
TRUNC (45.926, 2)	45.92
TRUNCATE (45.926, 2)	45.92
CEIL (2.83)	3
FLOOR (2.83)	2
MOD (1600, 300)	100

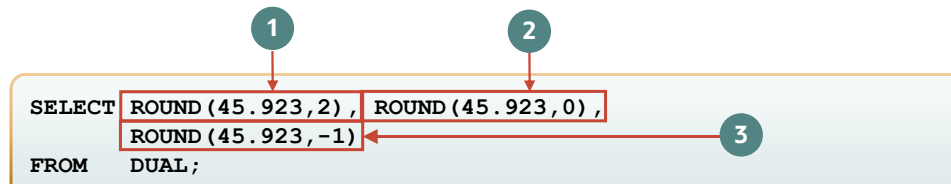
Numeric functions accept numeric input and return numeric values. This section describes some of the numeric functions.

Function	Purpose
ROUND (<i>column expression</i> , <i>n</i>)	Rounds the column, expression, or value to <i>n</i> decimal places or, if <i>n</i> is omitted, no decimal places (If <i>n</i> is negative, numbers to the left of decimal point are rounded.)
TRUNC (<i>column expression</i> , <i>n</i>) TRUNCATE (<i>column expression</i> , <i>n</i>)	Truncates the column, expression, or value to <i>n</i> decimal places or, if <i>n</i> is omitted, <i>n</i> defaults to zero
MOD (<i>m</i> , <i>n</i>)	Returns the remainder of <i>m</i> divided by <i>n</i>

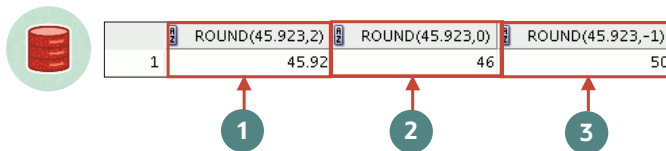
Note: This list contains only some of the available numeric functions. More information is available in the Oracle and MySQL documentation.

Using the ROUND Function

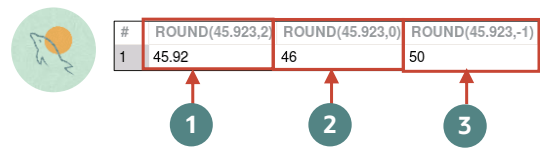
```
SELECT ROUND(45.923,2), ROUND(45.923,0),  
       ROUND(45.923,-1)  
FROM DUAL;
```



	1	2	3
1	ROUND(45.923,2) 45.92	ROUND(45.923,0) 46	ROUND(45.923,-1) 50



#	1	2	3
1	ROUND(45.923,2) 45.92	ROUND(45.923,0) 46	ROUND(45.923,-1) 50



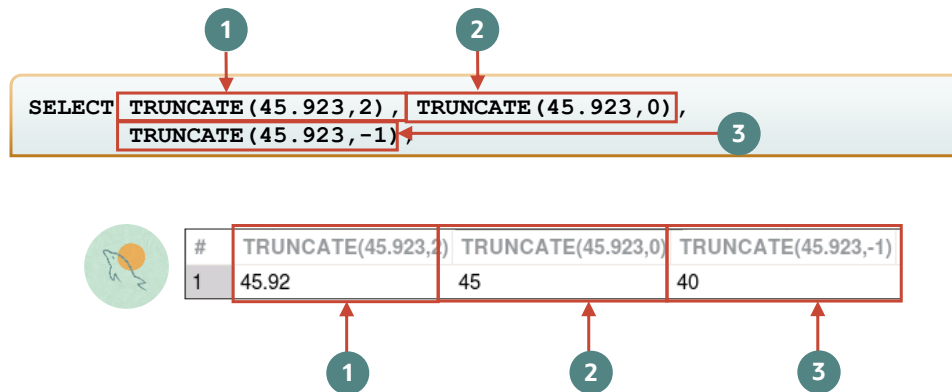
The `ROUND` function rounds the column, expression, or value to n decimal places. If the second argument is 0 or is missing, the value is rounded to zero decimal places. If the second argument is 2, the value is rounded to two decimal places. Conversely, if the second argument is -2, the value is rounded to two decimal places to the left (rounded to the nearest unit of 100).

Recall `DUAL` Table in Oracle

In Oracle, the `DUAL` table is owned by the user `SYS` and can be accessed by all users. It contains one column, `DUMMY`, and one row with the value `X`. The `DUAL` table is useful when you want to return a value only once (for example, the value of a constant, pseudocolumn, or expression that is not derived from a table with user data). The `DUAL` table is generally used for completeness of the `SELECT` clause syntax, because in Oracle, both `SELECT` and `FROM` clauses are mandatory, and several calculations do not need to select from the actual tables.

MySQL does not create a `DUAL` table and does not require its use but accepts the `FROM DUAL` clause for compatibility.

Using the TRUNCATE Function in MySQL



The `TRUNCATE` function truncates the column, expression, or value to n decimal places.

The `TRUNCATE` function works with arguments similar to those of the `ROUND` function, except that **both** arguments are required.

If the second argument is 0, the value is truncated to zero decimal places.

If the second argument is a positive number, the value is truncated to that number of decimal places to the right of the decimal point (tenths, hundredths, thousandths etc.)

Conversely, if the second argument is a negative number, the value is truncated to that number of decimal places to the left of the decimal point (tens, hundreds, thousands, etc.)

Note that the `FROM DUAL` clause is not required for MySQL.

Using the MOD Function

Display the employee records where `employee_id` is an even number:

```
SELECT employee_id AS Even_Numbers, last_name
FROM employees
WHERE MOD(employee_id,2) = 0;
```

	EVEN_NUMBERS	LAST_NAME
1	174	Abel
2	142	Davies
3	102	De Haan
4	104	Ernst
5	202	Fay
6	206	Gietz
7	178	Grant
8	100	King
9	124	Mourgos
10	176	Taylor
11	144	Vargas
12	200	Whalen



The `MOD` function finds the remainder of the first argument divided by the second argument. The example in the slide displays employee records where the `employee_id` is an even number.

Note: The MOD function is often used to determine whether a value is odd or even.



Working with Dates in MySQL Databases

The MySQL default date entry and display format is 'YYYY-MM-DD'. To display employees hired before February 1, 2013, enter the following query:

```
SELECT last_name, hire_date
FROM   employees
WHERE  hire_date < '2013-02-01';
```

#	last_name	hire_date
1	King	2011-06-17
2	Kochhar	2009-09-21
3	De Haan	2009-01-13
4	Rajs	2011-10-17
5	Davies	2013-01-29
6	Abel	2012-05-11
7	Whalen	2011-09-17
8	Hartstein	2012-02-17
9	Higgins	2010-06-07
10	Gietz	2010-06-07

The supported range of dates is '100-01-01' through '9999-12-31'. The MM portion of the date must be between 1 and 12 and the DD portion between 1 and 31. MySQL accepts dates in several formats, including other delimiters or no delimiters. For example, any of the following would be accepted for February 1, 2013:

- '2013-02-01'
- '2013/02/01'
- '20130201'

For Instructor Use Only.
This document should not be distributed.

Displaying the Current Date in MySQL



- The `CURDATE()` function returns the current date.
 - `CURRENT_DATE()` and `CURRENT_DATE` are synonyms for `CURDATE()`.
- The `NOW()` function returns the current date and time.
 - `CURRENT_TIMESTAMP()` and `CURRENT_TIMESTAMP` are synonyms for `NOW()`.
- The `SYSDATE()` function returns the current date and time.

```
SELECT CURDATE(), NOW(), SYSDATE();
```



#	CURDATE()	NOW()	SYSDATE()
1	2018-08-22	2018-08-22 15:33:14	2018-08-22 15:33:14

You can use the `CURDATE()` function in any statement that requires a date and `NOW()` in any statement that requires a date and time combination.

Date-Manipulation Functions in MySQL

Function	Result
<code>DATE_ADD(date, INTERVAL expr unit)</code>	Date after an interval is added to a date
<code>DATE_SUB(date, INTERVAL expr unit)</code>	Date after an interval is subtracted from a date
<code>DATEDIFF(date, date)</code>	Difference in days between two dates
<code>LAST_DAY(date)</code>	Last day of the month
<code>MONTH(date)</code>	The month number of the date
<code>YEAR(date)</code>	The year of the date

Date functions operate on MySQL dates. The functions listed in the slide are just a small portion of the functions available to operate on MySQL dates.

Using Date Functions in MySQL



The following example uses MySQL date functions in the output as well as in the WHERE clause:

```
SELECT employee_id, hire_date,  
       DATE_ADD(hire_date, INTERVAL 6 MONTH) AS Review,  
       DATEDIFF(CURDATE(), hire_date) AS Tenure  
FROM employees  
WHERE hire_date > DATE_SUB(CURDATE(), INTERVAL 4 YEAR);
```



#	employee_id	hire_date	Review	Tenure
1	104	2015-05-21	2015-11-21	1189
2	107	2015-02-07	2015-08-07	1292
3	124	2015-11-16	2016-05-16	1010
4	149	2016-01-29	2016-07-29	936
5	178	2015-05-24	2015-11-24	1186

The query displays the following:

- Employee ID
- Hire date
- The date of the employee's first six-month review with a column heading of Review. This is calculated by adding an interval of 6 months to the hire date.
- The number of days since the employee was hired with a column heading of Tenure. This is calculated as the difference between the hire date and the current date.

The query selects those employees who were hired less than 4 years ago. It compares the hire date to see if it is later (greater) than the date that was four years before today, which is calculated by subtracting an interval of 4 years from the current date.

Extracting the Month or Year Portion of Dates in MySQL



Use the `MONTH()` or `YEAR()` function to extract those portions of a date. For example, the following query displays the employee number, hire date, and starting month for employees that started in 2010.

```
SELECT employee_id, hire_date,  
       MONTH(hire_date)  
FROM employees  
WHERE YEAR(hire_date) = '2010';
```



#	employee_id	hire_date	MONTH(hire_date)
1	205	2010-06-07	6
2	206	2010-06-07	6

There are several functions that enable you to work with dates in MySQL.

Implicit Data Type Conversion of Strings to Numbers

In expressions, the database can automatically convert strings to numbers. In this example, two strings are concatenated and then implicitly converted to a number for comparison with the numeric department ID in the `WHERE` clause.

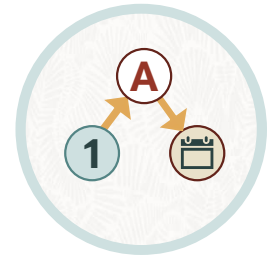
```
SELECT first_name, last_name, department_id
FROM employees WHERE department_id < CONCAT('9', '0');
```



	FIRST_NAME	LAST_NAME	DEPARTMENT_ID
1	Ellen	Abel	80
2	Curtis	Davies	50
3	Bruce	Ernst	60
4	Pat	Fay	20
5	Michael	Hartstein	20
6	Alexander	Hunold	60
7	Diana	Lorentz	60
8	Randall	Matos	50
9	Kevin	Mourgos	50
10	Trenna	Rajs	50
11	Jonathon	Taylor	80
12	Peter	Vargas	50
13	Jennifer	Whalen	10
14	Eleni	Zlotkey	80



#	first_name	last_name	department_id
1	Alexander	Hunold	60
2	Bruce	Ernst	60
3	Diana	Lorentz	60
4	Kevin	Mourgos	50
5	Trenna	Rajs	50
6	Curtis	Davies	50
7	Randall	Matos	50
8	Peter	Vargas	50
9	Eleni	Zlotkey	80
10	Ellen	Abel	80
11	Jonathon	Taylor	80
12	Jennifer	Whalen	10
13	Michael	Hartstein	20
14	Pat	Fay	20



The server can automatically perform data type conversion in an expression from `VARCHAR2` (Oracle), `VARCHAR` (MySQL), or `CHAR` to a numeric or date data type.

For example, the expression `hire_date > '01-JAN-90'` (Oracle) or `hire_date > '1990-01-01'` (MySQL) results in the implicit conversion from the string to a date. Therefore, a character expression or value can be implicitly converted to a number or date data type in an expression.

Note: `CHAR` to numeric data type conversions succeed only if the character string provided represents a valid number.

Implicit Data Type Conversion of Numbers to Strings

In expressions, the database can automatically convert numbers to strings. In this example, the salary column is converted to a string to determine if it contains a character.

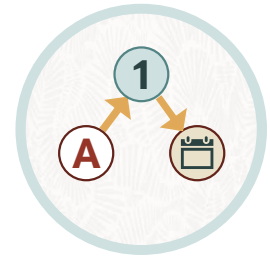
```
SELECT first_name, last_name, salary
FROM employees
WHERE INSTR(salary, '5') > 0;
```



	FIRST_NAME	LAST_NAME	SALARY
1	Kevin	Mourgos	5800
2	Trenna	Rajs	3500
3	Peter	Vargas	2500
4	Eleni	Zlotkey	10500



#	first_name	last_name	salary
1	Kevin	Mourgos	5800.00
2	Trenna	Rajs	3500.00
3	Peter	Vargas	2500.00
4	Eleni	Zlotkey	10500.00



The Oracle server can automatically perform data type conversion in an expression from NUMBER or DATE to VARCHAR2 or CHAR.

In general, the Oracle server uses the rule for expressions when a data type conversion is needed. For example, the expression `job_id = 2` results in the implicit conversion of the number 2 to the string "2" because `job_id` is a `VARCHAR(2)` column.

Using the CAST () Function in Oracle



CAST lets you convert one data type to another.

```
CAST(input_value as destination_type)
```

Examples:

```
SELECT first_name, last_name, department_id
FROM employees
WHERE department_id < CAST(CONCAT('9', '0') AS
DECIMAL(2,0));
```

1

```
SELECT first_name, last_name, salary
FROM employees
WHERE INSTR(CAST(salary AS VARCHAR2(30)), '5')
> 0;
```

2

CAST lets you convert built-in data types of one type into another built-in data type. You can convert a string to a number or a date, a number to a string, and so on.

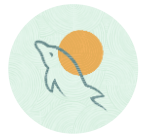
In the first example, two strings are concatenated and then explicitly converted to a `DECIMAL` numeric data type using the `CAST()` function to compare to the numeric department ID. The query displays the first name, last name, department ID of those employees whose department ID is less than 90.

In the second example, the salary column is explicitly converted to a string by using the `CAST()` function to determine if it contains a character. The query display the first name, last name, and salary of employees whose salary contains the digit '5'.

For more information about the `CAST()` function, refer to the *Oracle Database 18c SQL Language Reference* guide.

For Instructor Use Only.
This document should not be distributed.

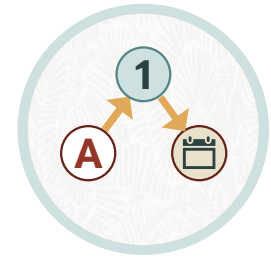
Explicit Data Type Conversion of Strings to Numbers in MySQL



You can use the `CAST()` function to explicitly convert strings to numbers. In this example, two strings are concatenated and then explicitly converted to a `DECIMAL` numeric data type to compare to the numeric department ID.

```
SELECT first_name, last_name, department_id FROM employees
WHERE department_id < CAST(CONCAT('9', '0') AS DECIMAL(2,0));
```

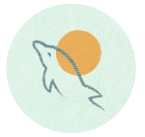
#	first_name	last_name	department_id
1	Alexander	Hunold	60
2	Bruce	Ernst	60
3	Diana	Lorentz	60
4	Kevin	Mourgos	50
5	Trenna	Rajs	50
6	Curtis	Davies	50
7	Randall	Matos	50
8	Peter	Vargas	50
9	Eleni	Zlotkey	80
10	Ellen	Abel	80
11	Jonathon	Taylor	80
12	Jennifer	Whalen	10
13	Michael	Hartstein	20
14	Pat	Fay	20



The `CAST()` function explicitly converts from a character to a decimal number value. (`INTEGER` data type is not accepted.)

For Instructor Use Only.
This document should not be distributed.

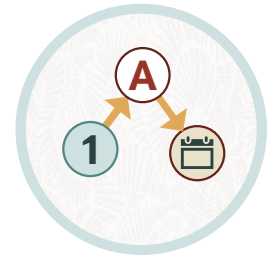
Explicit Data Type Conversion of Numbers to Strings in MySQL



You can use the `CAST()` function to explicitly convert strings to numbers. In this example, the salary column is explicitly converted to a string to determine if it contains a character.

```
SELECT first_name, last_name, salary
FROM employees
WHERE INSTR(CAST(salary AS CHAR), '5');
```

#	first_name	last_name	salary
1	Kevin	Mourgos	5800.00
2	Trenna	Rajs	3500.00
3	Peter	Vargas	2500.00
4	Eleni	Zlotkey	10500.00



For Instructor Use Only.
This document should not be distributed.

General Functions

The following functions pertain to using nulls and can be used with any data type:

`NVL (expr1, expr2)`

`NVL2 (expr1, expr2, expr3)`

`NULLIF (expr1, expr2)`

`COALESCE (expr1, expr2,
...,
exprn)`

`IFNULL (expr1, expr2)`
In MySQL

These functions work with any data type and pertain to the use of null values in the expression list.

Function	Description
NVL	Converts a null value to an actual value
NVL2	If <code>expr1</code> is not null, <code>NVL2</code> returns <code>expr2</code> . If <code>expr1</code> is null, <code>NVL2</code> returns <code>expr3</code> . The argument <code>expr1</code> can have any data type.
IFNULL	In MySQL, <code>IFNULL</code> substitutes a <code>NULL</code> value with a value that can be used in an expression. If <code>expr1</code> is <code>NULL</code> , returns <code>expr2</code> , otherwise returns <code>expr1</code> .
NULLIF	Compares two expressions and returns null if they are equal; returns the first expression if they are not equal
COALESCE	Returns the first non-null expression in the expression list

Note: For more information about the hundreds of functions available, see the “Functions” section in *Oracle Database SQL Language Reference* for 23c database.

NVL Function (Oracle) and IFNULL () Function (MySQL)

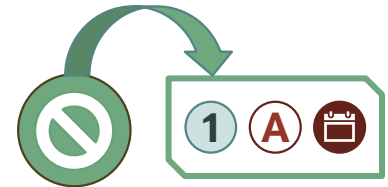
Converts a null value to an actual value:

- Data types that can be used are date, character, and number.
- Data types for both expressions must match.
- Examples:

Oracle	MySQL
<code>NVL(commission_pct,0)</code>	<code>IFNULL(commission_pct,0)</code>
<code>NVL(hire_date,'01-JAN-97')</code>	<code>IFNULL(hire_date, '1997-01-01')</code>
<code>NVL(job_id,'No Job Yet')</code>	<code>IFNULL(job_id,'No Job Yet')</code>

NVL (expr1, expr2)

IFNULL(expr1, expr2)



To convert a null value to an actual value, use the `NVL` function or `IFNULL` function.

Syntax

`NVL (expr1, expr2)`

`IFNULL (expr1, expr2)`

In the syntax:

- `expr1` is the source value or expression that may contain a null
- `expr2` is the target value for converting the null

You can use the `NVL` function with any data type, but the return value is always the same as the data type of `expr1`.

NVL Conversions for Various Data Types

Data Type	Conversion Example
NUMBER	<code>NVL (number_column, 9)</code>
DATE	<code>NVL (date_column, '01-JAN-95')</code>
CHAR or VARCHAR2	<code>NVL (character_column, 'Unavailable')</code>

Using the IFNULL Function in MySQL

```
SELECT last_name, salary, IFNULL(commission_pct, 0),  
       (salary*12) + (salary*12*IFNULL(commission_pct, 0)) AN_SAL  
FROM employees;
```

#	last_name	salary	IFNULL(commission_pct, 0)	AN_SAL
1	King	24000.00	0.00	288000.0000
2	Kochhar	17000.00	0.00	204000.0000
3	De Haan	17000.00	0.00	204000.0000
...				
12	Zlotkey	10500.00	0.20	151200.0000
13	Abel	11000.00	0.30	171600.0000
14	Taylor	8600.00	0.20	123840.0000
...				
18	Fay	6000.00	0.00	72000.0000
19	Higgins	12008.00	0.00	144096.0000
20	Gietz	8300.00	0.00	99600.0000

1

2

To calculate the annual compensation of all employees, you need to multiply the monthly salary by 12 and then add the commission percentage to the result:

```
SELECT last_name, salary, commission_pct,  
       (salary*12) + (salary*12*commission_pct) AN_SAL  
FROM employees;
```

Notice that the annual compensation is calculated for only those employees who earn a commission. If any column value in an expression is null, the result is null. To calculate values for all employees, you must convert the null value to a number before applying the arithmetic operator. In the example in the slide, the NVL function is used to convert null values to zero.

Using the NULLIF Function

NULLIF (expr1, expr2)

```
SELECT first_name, LENGTH(first_name) AS expr1, 1  
       last_name, LENGTH(last_name) AS expr2, 2  
       NULLIF(LENGTH(first_name), LENGTH(last_name)) AS Result 3  
FROM employees;
```



	FIRST_NAME	expr1	LAST_NAME	expr2	RESULT
1	Ellen	5	Abel	4	5
2	Curtis	6	Davies	6	(null)
3	Lex	3	De Haan	7	3
4	Bruce	5	Ernst	5	(null)
5	Pat	3	Fay	3	(null)
6	William	7	Gietz	5	7
7	Kimberely	9	Grant	5	9
8	Michael	7	Hartstein	9	7
9	Shelley	7	Higgins	7	(null)

...

1

2

3



	first_name	expr1	last_name	expr2	Result
▶	Ellen	5	Abel	4	5
	Curtis	6	Davies	6	NULL
	Lex	3	De Haan	7	3
	Bruce	5	Ernst	5	NULL
	Pat	3	Fay	3	NULL
	William	7	Gietz	5	7
	Kimberely	9	Grant	5	9
	Michael	7	Hartstein	9	7
	Shelley	7	Higgins	7	NULL

...

1

2

3

The NULLIF function compares two expressions.

Syntax

NULLIF (expr1, expr2)

In the syntax:

- NULLIF compares *expr1* and *expr2*. If they are equal, the function returns null. If they are not, the function returns *expr1*. However, you cannot specify the literal NULL for *expr1*.

In the example shown in the slide, the length of the first name in the EMPLOYEES table is compared with the length of the last name in the EMPLOYEES table. When the lengths of the names are equal, a null value is displayed. When the lengths of the names are not equal, the length of the first name is displayed.

Using the COALESCE Function

```
SELECT last_name, salary, commission_pct,
       COALESCE((salary+(commission_pct*salary)), salary+2000)
AS New_Salary
FROM employees;
```



	LAST_NAME	SALARY	COMMISSION_PCT	NEW_SALARY
1	King	24000	(null)	26000
2	Kochhar	17000	(null)	19000
3	De Haan	17000	(null)	19000
4	Hunold	9000	(null)	11000
5	Ernst	6000	(null)	8000
6	Lorentz	4200	(null)	6200
7	Mourgos	5800	(null)	7800
8	Rajs	3500	(null)	5500
9	Davies	3100	(null)	5100
10	Matos	2600	(null)	4600
11	Vargas	2500	(null)	4500
12	Zlotkey	10500	0.2	12600
13	Abel	11000	0.3	14300
14	Taylor	8600	0.2	10320
15	Grant	7000	0.15	8050
16	Whalen	4400	(null)	6400
17	Hartstein	13000	(null)	15000
18	Fay	6000	(null)	8000
19	Higgins	12008	(null)	14008
20	Gietz	8300	(null)	10300



#	last_name	salary	commission_pct	New_Salary
1	King	24000.00		26000.0000
2	Kochhar	17000.00		19000.0000
3	De Haan	17000.00		19000.0000
4	Hunold	9000.00		11000.0000
5	Ernst	6000.00		8000.0000
6	Lorentz	4200.00		6200.0000
7	Mourgos	5800.00		7800.0000
8	Rajs	3500.00		5500.0000
9	Davies	3100.00		5100.0000
10	Matos	2600.00		4600.0000
11	Vargas	2500.00		4500.0000
12	Zlotkey	10500.00	0.20	12600.0000
13	Abel	11000.00	0.30	14300.0000
14	Taylor	8600.00	0.20	10320.0000
15	Grant	7000.00	0.15	8050.0000
16	Whalen	4400.00		6400.0000
17	Hartstein	13000.00		15000.0000
18	Fay	6000.00		8000.0000
19	Higgins	12008.00		14008.0000
20	Gietz	8300.00		10300.0000

In the example shown in the slide, for the employees who do not get any commission, your organization wants to give a salary increment of \$2,000 and, for employees who get commission, the query should compute the new salary that is equal to the existing salary added to the commission amount.

Note: Examine the output. For employees who do not get any commission, the New Salary column shows the salary incremented by \$2,000, and for employees who get commission, the New Salary column shows the computed commission amount added to the salary.

Using the CASE Expression

```
SELECT last_name, job_id, salary,
       CASE job_id WHEN 'IT_PROG' THEN 1.10*salary
                   WHEN 'ST_CLERK' THEN 1.15*salary
                   WHEN 'SA_REP' THEN 1.20*salary
                   ELSE salary END AS REVISED_SALARY
FROM   employees;
```



#	LAST_NAME	JOB_ID	SALARY	REVISED_SALARY
1	King	AD_PRES	24000	24000
...				
4	Hunold	IT_PROG	9000	9900
5	Ernst	IT_PROG	6000	6600
6	Lorentz	IT_PROG	4200	4620
7	Mourgos	ST_MAN	5800	5800
8	Rajs	ST_CLERK	3500	4025
9	Davies	ST_CLERK	3100	3565
10	Matos	ST_CLERK	2600	2990
11	Vargas	ST_CLERK	2500	2875
...				
13	Abel	SA_REP	11000	13200
14	Taylor	SA_REP	8600	10320
15	Grant	SA_REP	7000	8400



#	last_name	job_id	salary	REVISED_SALARY
1	King	AD_PRES	24000.00	24000.00
...				
4	Hunold	IT_PROG	9000.00	9900.0000
5	Ernst	IT_PROG	6000.00	6600.0000
6	Lorentz	IT_PROG	4200.00	4620.0000
7	Mourgos	ST_MAN	5800.00	5800.00
8	Rajs	ST_CLERK	3500.00	4025.0000
9	Davies	ST_CLERK	3100.00	3565.0000
10	Matos	ST_CLERK	2600.00	2990.0000
11	Vargas	ST_CLERK	2500.00	2875.0000
...				
13	Abel	SA_REP	11000.00	13200.0000
14	Taylor	SA_REP	8600.00	10320.0000
15	Grant	SA_REP	7000.00	8400.0000

In the SQL statement in the slide, the value of `JOB_ID` is decoded.

If `JOB_ID` is `IT_PROG`, the salary increase is 10%.

If `JOB_ID` is `ST_CLERK`, the salary increase is 15%.

If `JOB_ID` is `SA_REP`, the salary increase is 20%.

For all other job roles, there is no increase in salary.

The same statement can be written with the `DECODE` function.

Using the AVG and SUM Functions

You can use the AVG and SUM functions for numeric data.

```
SELECT AVG(salary), MAX(salary),  
       MIN(salary), SUM(salary)  
FROM   employees  
WHERE  job_id LIKE '%REP%';
```



	AVG(SALARY)	MAX(SALARY)	MIN(SALARY)	SUM(SALARY)
1	8150	11000	6000	32600



#	AVG(salary)	MAX(salary)	MIN(salary)	SUM(salary)
1	8150.000000	11000.00	6000.00	32600.00

You can use the AVG, SUM, MIN, and MAX functions against the columns that can store numeric data. The example in the slide displays the average, highest, lowest, and sum of monthly salaries for all sales representatives.

Using the MIN and MAX Functions

You can use MIN and MAX for numeric, character, and date data types.

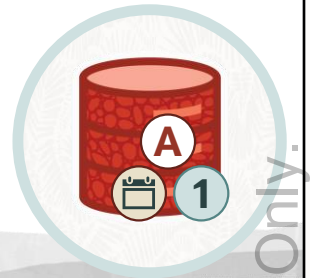
```
SELECT MIN(last_name), MAX(last_name),  
       MIN(hire_date), MAX(hire_date)  
FROM   employees;
```



	MIN(LAST_NAME)	MAX(LAST_NAME)	MIN(HIRE_DATE)	MAX(HIRE_DATE)
1	Abel	Zlotkey	13-JAN-09	29-JAN-16



#	MIN(last_name)	MAX(last_name)	MIN(hire_date)	MAX(hire_date)
1	Abel	Zlotkey	2009-01-13	2016-01-29



You can use the MAX and MIN functions for numeric, character, and date data types. The example in the slide displays the employee last name that is first and last in an alphabetic list of all employees and the date hired for the most senior and most junior employees.

Note: The AVG, SUM, VARIANCE, and STDDEV functions can be used only with numeric data types. MAX and MIN cannot be used with LOB or LONG data types.

Using the COUNT Function

- COUNT(*) returns the number of rows in a table:

```
SELECT COUNT(*)  
FROM employees  
WHERE department_id = 50;
```



COUNT(*)	
1	5



#	count(*)
1	5

- COUNT(expr) returns the number of rows with non-null values for expr:

```
SELECT COUNT(commission_pct)  
FROM employees  
WHERE department_id = 50;
```



COUNT(COMMISSION_PCT)	
1	0



#	COUNT(commission_pc
1	0

You can use the COUNT function in the following three formats:

- COUNT (*)
- COUNT (expr)
- COUNT (DISTINCT expr)

COUNT (*) returns the number of rows in a table that satisfy the criteria of the SELECT statement, including duplicate rows and rows containing null values in any of the columns. If a WHERE clause is included in the SELECT statement, COUNT (*) returns the number of rows that satisfy the condition in the WHERE clause.

In contrast, COUNT (expr) returns the number of non-null values that are in the column identified by expr.

COUNT (DISTINCT expr) returns the number of unique, non-null values that are in the column identified by expr.

Examples

1. The first example in the slide displays the number of employees in department 50.
2. The second example in the slide displays the number of employees in department 50 who can earn a commission.

Using the DISTINCT Keyword

- `COUNT(DISTINCT expr)` returns the number of distinct non-null values of *expr*.
- To display the number of distinct department values in the `EMPLOYEES` table:

```
SELECT COUNT(DISTINCT department_id)
FROM employees;
```



	COUNT(DISTINCTDEPARTMENT_ID)
1	7



#	COUNT(DISTINCT department_id)
1	7

Use the `DISTINCT` keyword to suppress the counting of any duplicate values in a column.

The example in the slide displays the number of distinct department values that are in the `EMPLOYEES` table.

Group Functions and Null Values in MySQL



- Group functions ignore null values in the column:

```
SELECT AVG(commission_pct)
FROM employees;
```

1



#	avg(commission_pc
1	0.212500

- The `IFNULL` function forces group functions to include null values:

```
SELECT AVG(IFNULL(commission_pct, 0))
FROM employees;
```

2



#	AVG(IFNULL(commission_pct, 0
1	0.042500

All group functions ignore null values in the column.

However, the `IFNULL` function forces group functions to include null values.

Examples

- The average is calculated based on *only* those rows in the table in which a valid value is stored in the `COMMISSION_PCT` column. The average is calculated as the total commission that is paid to all employees divided by the number of employees receiving a commission (four).
- The average is calculated based on *all* rows in the table, regardless of whether null values are stored in the `COMMISSION_PCT` column. The average is calculated as the total commission that is paid to all employees divided by the total number of employees in the company (20).

Using the GROUP BY Clause on Multiple Columns

```
SELECT department_id, job_id, SUM(salary)
FROM employees
WHERE department_id > 40
GROUP BY department_id, job_id
ORDER BY department_id;
```



#	DEPARTMENT_ID	JOB_ID	SUM(SALARY)
1	50	ST_CLERK	11700
2	50	ST_MAN	5800
3	60	IT_PROG	19200
4	80	SA_MAN	10500
5	80	SA_REP	19600
6	90	AD_PRES	24000
7	90	AD_VP	34000
8	110	AC_ACCOUNT	8300
9	110	AC_MGR	12008



#	department_id	job_id	SUM(salary)
1	50	ST_CLERK	11700.00
2	50	ST_MAN	5800.00
3	60	IT_PROG	19200.00
4	80	SA_MAN	10500.00
5	80	SA_REP	19600.00
6	90	AD_PRES	24000.00
7	90	AD_VP	34000.00
8	110	AC_ACC...	8300.00
9	110	AC_MGR	12008.00



You can return summary results for groups and subgroups by listing multiple GROUP BY columns. The GROUP BY clause groups rows but does not guarantee the order of the result set. To order the groupings, use the ORDER BY clause.

In the example in the slide, the SELECT statement that contains a GROUP BY clause is evaluated as follows:

- The SELECT clause specifies the columns to be retrieved:
 - DEPARTMENT_ID in the EMPLOYEES table
 - JOB_ID in the EMPLOYEES table
 - The sum of all salaries in the group that you specified in the GROUP BY clause
- The FROM clause specifies the tables that the database must access: the EMPLOYEES table.
- The WHERE clause reduces the result set to those rows where DEPARTMENT_ID is greater than 40.
- The GROUP BY clause specifies how you must group the resulting rows:
 - First, the rows are grouped by DEPARTMENT_ID.
 - Second, the rows are grouped by JOB ID in the DEPARTMENT_ID groups.
- The ORDER BY clause sorts the results by DEPARTMENT_ID.

Note: The SUM function is applied to the salary column for all job IDs in the result set in each DEPARTMENT_ID group. Also, note that the SA_REP row is not returned. DEPARTMENT_ID for this row is NULL and, therefore, does not meet the WHERE condition.

Illegal Queries Using Group Functions

Any column or expression in the `SELECT` list that is not an aggregate function must be in the `GROUP BY` clause:

```
SELECT department_id, COUNT(last_name)
FROM employees;
```

1

A `GROUP BY` clause must be added to count the last names for each `department_id`.

ORA-00937: not a single-group group function
00937. 00000 - "not a single-group group function"



Error Code: 1140. In aggregated query without `GROUP BY`, expression #1 of `SELECT` list contains nonaggregated column 'hr.employees.department_id'; this is incompatible with `sql_mode=only_full_group_by`

```
SELECT department_id, job_id, COUNT(last_name)
FROM employees
GROUP BY department_id;
```

2

Either add `job_id` in the `GROUP BY` clause or remove the `job_id` column from the `SELECT` list.

ORA-00979: not a `GROUP BY` expression
00979. 00000 - "not a `GROUP BY` expression"



Error Code: 1055. Expression #2 of `SELECT` list is not in `GROUP BY` clause and contains nonaggregated column 'hr.employees.job_id' which is not functionally dependent on columns in `GROUP BY` clause; this is incompatible with `sql_mode=only_full_group_by`

1. Whenever you use a mixture of individual items (`DEPARTMENT_ID`) and group functions (`COUNT`) in the same `SELECT` statement, you must include a `GROUP BY` clause that specifies the individual items (in this case, `DEPARTMENT_ID`). If the `GROUP BY` clause is missing, the error message "not a single-group group function" appears and an asterisk (*) points to the offending column. You can correct the error in the first example in the slide by adding the `GROUP BY` clause:
 - `SELECT department_id, COUNT(last_name)`
 - `FROM employees`
 - `GROUP BY department_id;`
2. Any column or expression in the `SELECT` list that is not an aggregate function must be in the `GROUP BY` clause. In the second example in the slide, `JOB_ID` is neither in the `GROUP BY` clause nor is it being used by a group function, so there is a "not a `GROUP BY` expression" error. You can correct the error in the second slide example by adding `JOB_ID` in the `GROUP BY` clause.
 - `SELECT department_id, job_id, COUNT(last_name)`
 - `FROM employees`
 - `GROUP BY department_id, job_id;`

Illegal Queries Using Group Functions in a WHERE Clause

- You use the `HAVING` clause to restrict groups using a group function.
- You cannot use group functions in the `WHERE` clause.

```
SELECT department_id, AVG(salary)
FROM employees
WHERE AVG(salary) > 8000
GROUP BY department_id;
```



ORA-00934: group function is not allowed here
00934. 00000 - "group function is not allowed here"
*Cause:
*Action:
Error at Line: 3 Column: 9



Error Code: 1111. Invalid use of group function

Cannot use a group function in a WHERE clause

You cannot use a group function in a `WHERE` clause. The `SELECT` statement in the example in the slide results in an error because it uses the `AVG()` function in the `WHERE` clause to restrict the display of the average salaries of those departments that have an average salary greater than \$8,000.

However, you can correct the error in the example by putting the group function in the `HAVING` clause to restrict groups:

- `SELECT department_id, AVG(salary)`
- `FROM employees`
- `GROUP BY department_id`
- `HAVING AVG(salary) > 8000;`

Using the HAVING Clause

```
SELECT department_id, MAX(salary)
FROM employees
GROUP BY department_id
HAVING MAX(salary) > 10000 ;
```



#	DEPARTMENT_ID	MAX(SALARY)
1	90	24000
2	20	13000
3	110	12008
4	80	11000



#	department_id	MAX(salary)
1	20	13000.00
2	80	11000.00
3	90	24000.00
4	110	12008.00

The example in the slide displays the department numbers and maximum salaries for departments with a maximum salary greater than \$10,000.

You can use the `GROUP BY` clause without using a group function in the `SELECT` list. If you restrict rows based on the result of a group function, you must have a `GROUP BY` clause as well as the `HAVING` clause.

The following example displays the department numbers and average salaries for departments with a maximum salary greater than \$10,000:

- `SELECT department_id, AVG(salary)`
- `FROM employees`
- `GROUP BY department_id`
- `HAVING max(salary) > 10000;`

Using the HAVING Clause

```
SELECT job_id, SUM(salary) PAYROLL
FROM employees
WHERE job_id NOT LIKE '%REP%'
GROUP BY job_id
HAVING SUM(salary) > 13000
ORDER BY SUM(salary);
```



	JOB_ID	PAYROLL
1	IT_PROG	19200
2	AD_PRES	24000
3	AD_VP	34000



#	job_id	PAYROLL
1	IT_PROG	19200.00
2	AD_PRES	24000.00
3	AD_VP	34000.00

The example in the slide displays the `JOB_ID` and total monthly salary for each job that has a total payroll exceeding \$13,000. The example excludes sales representatives and sorts the list by the total monthly salary.

Retrieving Records with Natural Joins

```
SELECT employee_id, first_name, job_id, job_title
from employees NATURAL JOIN jobs;
```



	EMPLOYEE_ID	FIRST_NAME	JOB_ID	JOB_TITLE
1	206	William	AC_ACCOUNT	Public Accountant
2	205	Shelley	AC_MGR	Accounting Manager
3	200	Jennifer	AD_ASST	Administration Assistant
4	100	Steven	AD_PRES	President
5	102	Lex	AD_VP	Administration Vice President
6	101	Neena	AD_VP	Administration Vice President
7	103	Alexander	IT_PROG	Programmer
8	104	Bruce	IT_PROG	Programmer
9	107	Diana	IT_PROG	Programmer
10	201	Michael	MK_MAN	Marketing Manager
11	202	Pat	MK_REP	Marketing Representative
12	149	Eleni	SA_MAN	Sales Manager
13	174	Ellen	SA_REP	Sales Representative
14	178	Kimberely	SA_REP	Sales Representative
15	176	Jonathon	SA_REP	Sales Representative
16	143	Randall	ST_CLERK	Stock Clerk
17	142	Curtis	ST_CLERK	Stock Clerk
18	141	Trenna	ST_CLERK	Stock Clerk
19	144	Peter	ST_CLERK	Stock Clerk
20	124	Kevin	ST_MAN	Stock Manager



#	employee_id	first_name	job_id	job_title
1	206	William	AC_ACC...	Public Accountant
2	205	Shelley	AC_MGR	Accounting Manager
3	200	Jennifer	AD_ASST	Administration Assistant
4	100	Steven	AD_PRES	President
5	101	Neena	AD_VP	Administration Vice President
6	102	Lex	AD_VP	Administration Vice President
7	103	Alexander	IT_PROG	Programmer
8	104	Bruce	IT_PROG	Programmer
9	107	Diana	IT_PROG	Programmer
10	201	Michael	MK_MAN	Marketing Manager
11	202	Pat	MK_REP	Marketing Representative
12	149	Eleni	SA_MAN	Sales Manager
13	174	Ellen	SA_REP	Sales Representative
14	176	Jonathon	SA_REP	Sales Representative
15	178	Kimberely	SA_REP	Sales Representative
16	141	Trenna	ST_CLERK	Stock Clerk
17	142	Curtis	ST_CLERK	Stock Clerk
18	143	Randall	ST_CLERK	Stock Clerk
19	144	Peter	ST_CLERK	Stock Clerk
20	124	Kevin	ST_MAN	Stock Manager

In the example in the slide, observe that the JOBS table is joined to the EMPLOYEES table by the JOB_ID column, which is the only column of the same name in both tables. If other common columns were present, the join would have used them all.

Natural Joins with a WHERE Clause

Additional restrictions on a natural join are implemented by using a WHERE clause. The following example limits the rows of output to those with a DEPARTMENT_ID equal to 20 or 50:

```
SELECT  department_id, department_name,
        location_id, city
FROM    departments
NATURAL JOIN locations
WHERE   department_id IN (20, 50);
```

Retrieving Records with the USING Clause

```
SELECT employee_id, last_name,  
       location_id, department_id  
FROM   employees JOIN departments  
       USING (department_id);
```



	EMPLOYEE_ID	LAST_NAME	LOCATION_ID	DEPARTMENT_ID
1	200	Whalen	1700	10
2	201	Hartstein	1800	20
3	202	Fay	1800	20
4	144	Vargas	1500	50
5	143	Matos	1500	50
6	142	Davies	1500	50
7	141	Rajs	1500	50
8	124	Mourgos	1500	50

...

18	206	Gietz	1700	110
19	205	Higgins	1700	110



#	employee_id	last_name	location_id	department_id
1	103	Hunold	1400	60
2	104	Ernst	1400	60
3	107	Lorentz	1400	60
4	124	Mourgos	1500	50
5	141	Rajs	1500	50
6	142	Davies	1500	50
7	143	Matos	1500	50
8	144	Vargas	1500	50

...

18	174	Abel	2500	80
19	176	Taylor	2500	80

In the example in the slide, the `DEPARTMENT_ID` columns in the `EMPLOYEES` and `DEPARTMENTS` tables are joined, and thus `LOCATION_ID` of the department where an employee works is shown.

Note that the `EMPLOYEE_ID` and `LAST_NAME` columns belong to the `EMPLOYEES` table, whereas `LOCATION_ID` belongs to the `DEPARTMENTS` table and the `DEPARTMENT_ID` column belongs to both these tables.

Retrieving Records with the ON Clause

```
SELECT e.employee_id, e.last_name, e.department_id,  
       d.department_id, d.location_id  
FROM   employees e JOIN departments d  
ON     (e.department_id = d.department_id);
```



#	EMPLOYEE_ID	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_ID_1	LOCATION_ID
1	200	Whalen	10	10	1700
2	201	Hartstein	20	20	1800
3	202	Fay	20	20	1800
4	124	Mourgos	50	50	1500
5	144	Vargas	50	50	1500
6	143	Matos	50	50	1500
7	142	Davies	50	50	1500
8	141	Rajs	50	50	1500
9	107	Lorentz	60	60	1400
10	104	Ernst	60	60	1400
11	103	Hunold	60	60	1400

...



#	employee_id	last_name	department_id	department_id_1	location_id
1	103	Hunold	60	60	1400
2	104	Ernst	60	60	1400
3	107	Lorentz	60	60	1400
4	124	Mourgos	50	50	1500
5	141	Rajs	50	50	1500
6	142	Davies	50	50	1500
7	143	Matos	50	50	1500
8	144	Vargas	50	50	1500
9	200	Whalen	10	10	1700
10	100	King	90	90	1700
11	101	Kochhar	90	90	1700

...

In this example, the `DEPARTMENT_ID` columns in the `EMPLOYEES` and `DEPARTMENTS` table are joined using the `ON` clause. Wherever a department ID in the `EMPLOYEES` table equals a department ID in the `DEPARTMENTS` table, the row is returned. The table alias is necessary to qualify the matching column names.

You can also use the `ON` clause to join columns that have different names. The parentheses around the joined columns, as in the example in the slide `(e.department_id = d.department_id)`, is optional. So, even `ON e.department_id = d.department_id` will work.

Note: When you use the Execute Statement icon to run the query, SQL Developer suffixes a `'_1'` to differentiate between the two `DEPARTMENT_IDS`.

Creating Three-Way Joins

```
SELECT employee_id, city, department_name
FROM employees e
JOIN departments d
ON d.department_id = e.department_id
JOIN locations l
ON d.location_id = l.location_id;
```



	EMPLOYEE_ID	CITY	DEPARTMENT_NAME
1	100	Seattle	Executive
2	101	Seattle	Executive
3	102	Seattle	Executive
4	103	Southlake	IT
5	104	Southlake	IT
6	107	Southlake	IT
7	124	South San Francisco	Shipping
8	141	South San Francisco	Shipping
9	142	South San Francisco	Shipping



#	employee_id	city	department_name
1	149	Oxford	Sales
2	174	Oxford	Sales
3	176	Oxford	Sales
4	200	Seattle	Administration
5	100	Seattle	Executive
6	101	Seattle	Executive
7	102	Seattle	Executive
8	205	Seattle	Accounting
9	206	Seattle	Accounting

A three-way join is a join of three tables. Here, the first join to be performed is `employees JOIN departments`. The first join condition can reference columns in `employees` and `departments` but cannot reference columns in `locations`. The second join condition can reference columns from all three tables.

Note: The code example in the slide can also be accomplished with the `USING` clause:

```
SELECT e.employee_id, l.city, d.department_name
FROM employees e
JOIN departments d
USING (department_id)
JOIN locations l
USING (location_id);
```

Self-Joins Using the ON Clause

```
SELECT worker.last_name emp, manager.last_name mgr
FROM   employees worker JOIN employees manager
ON     (worker.manager_id = manager.employee_id);
```



	EMP	MGR
1	Hunold	De Haan
2	Fay	Hartstein
3	Gietz	Higgins
4	Lorentz	Hunold
5	Ernst	Hunold
6	Zlotkey	King
7	Mourgos	King
8	Kochhar	King

...



#	emp	mgr
1	Kochhar	King
2	De Haan	King
3	Hunold	De Haan
4	Ernst	Hunold
5	Lorentz	Hunold
6	Mourgos	King
7	Rajs	Mourgos
8	Davies	Mourgos



The ON clause can also be used to join columns that have different names, within the same table or in a different table.

The example shown is a self-join of the EMPLOYEES table, based on the EMPLOYEE_ID and MANAGER_ID columns.

Retrieving Records with Nonequijoins

```
SELECT e.last_name, e.salary, j.grade_level
FROM   employees e JOIN job_grades j
ON     e.salary
      BETWEEN j.lowest_sal AND j.highest_sal;
```



#	LAST_NAME	SALARY	GRADE_LEVEL
1	Vargas	2500	A
2	Matos	2600	A
3	Davies	3100	B
4	Rajs	3500	B
5	Lorentz	4200	B
6	Whalen	4400	B
7	Mourgos	5800	B
8	Ernst	6000	C
9	Fay	6000	C
10	Grant	7000	C

...



#	last_name	salary	grade_level
1	King	24000.00	E
2	Kochhar	17000.00	E
3	De Haan	17000.00	E
4	Hunold	9000.00	C
5	Ernst	6000.00	C
6	Lorentz	4200.00	B
7	Mourgos	5800.00	B
8	Rajs	3500.00	B
9	Davies	3100.00	B
10	Matos	2600.00	A

The example in the slide creates a nonequijoin to evaluate an employee's salary grade. The salary must be *between* any pair of the low and high salary ranges.

It is important to note that all employees appear exactly once when this query is executed. No employee is repeated in the list. There are two reasons for this:

- None of the rows in the `JOB_GRADES` table contain grades that overlap. That is, the salary value for an employee can lie only between the low-salary and high-salary values of one of the rows in the salary grade table.
- All of the employees' salaries lie within the limits provided by the `JOB_GRADES` table. That is, no employee earns less than the lowest value contained in the `LOWEST_SAL` column or more than the highest value contained in the `HIGHEST_SAL` column.

Note: Other conditions (such as `<=` and `>=`) can be used, but `BETWEEN` is the simplest. Remember to specify the low value first and the high value last when using the `BETWEEN` condition. The Oracle server translates the `BETWEEN` condition to a pair of `AND` conditions. Therefore, using `BETWEEN` has no performance benefits, but should be used only for logical simplicity.

Table aliases have been specified in the example in the slide for performance reasons, not because of possible ambiguity.

LEFT OUTER JOIN

```
SELECT e.last_name, e.department_id, d.department_name
FROM   employees e LEFT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```



	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Whalen	10	Administration
2	Fay	20	Marketing
3	Hartstein	20	Marketing
4	Vargas	50	Shipping
5	Matos	50	Shipping

...		
16	Kochhar	90 Executive
17	King	90 Executive
18	Gietz	110 Accounting
19	Higgins	110 Accounting
20	Grant	(null) (null)



#	last_name	department_id	department_name
1	King	90	Executive
2	Kochhar	90	Executive
3	De Haan	90	Executive
4	Hunold	60	IT
5	Ernst	60	IT

...			
14	Taylor	80	Sales
15	Grant	NULL	NULL
16	Whalen	10	Administration
17	Hartstein	20	Marketing
18	Fay	20	Marketing
19	Higgins	110	Accounting
20	Gietz	110	Accounting

This query retrieves all the rows in the `EMPLOYEES` table, which is the left table, even if there is no match in the `DEPARTMENTS` table.

Basically, a `LEFT OUTER JOIN` will return all the rows that matches between the two tables on the given condition and also returns the unmatched rows of the left table, that is, the table mentioned first in the SQL statement.

RIGHT OUTER JOIN

```
SELECT e.last_name, d.department_id, d.department_name
FROM   employees e RIGHT OUTER JOIN departments d
ON     (e.department_id = d.department_id) ;
```



	LAST_NAME	DEPARTMENT_ID	DEPARTMENT_NAME
1	Whalen	10	Administration
2	Hartstein	20	Marketing
3	Fay	20	Marketing
4	Davies	50	Shipping
5	Vargas	50	Shipping
6	Rajs	50	Shipping
7	Mourgos	50	Shipping
8	Matos	50	Shipping
...			
18	Higgins	110	Accounting
19	Gietz	110	Accounting
20	(null)	190	Contracting



#	last_name	department_id	department_name
1	Whalen	10	Administration
2	Hartstein	20	Marketing
3	Fay	20	Marketing
4	Mourgos	50	Shipping
5	Rajs	50	Shipping
6	Davies	50	Shipping
7	Matos	50	Shipping
8	Vargas	50	Shipping
...			
18	Higgins	110	Accounting
19	Gietz	110	Accounting
20	NULL	190	Contracting

This query retrieves all the rows in the `departments` table, which is the table at the right, even if there is no match in the `employees` table.

Basically, a `RIGHT OUTER JOIN` will return all the rows that matches between the two tables on the given condition and also returns the unmatched rows of the right table, that is, the table mentioned second in the SQL statement.

Creating Cross Joins

- A `CROSS JOIN` is a `JOIN` operation that produces a Cartesian product of two tables.
- To create a Cartesian product, specify `CROSS JOIN` in your `SELECT` statement.

```
SELECT last_name, department_name  
FROM employees  
CROSS JOIN departments ;
```



#	LAST_NAME	DEPARTMENT_NAME
1	Abel	Administration
2	Davies	Administration
3	De Haan	Administration
4	Ernst	Administration
5	Fay	Administration
...		
158	Vargas	Contracting
159	Whalen	Contracting
160	Zlotkey	Contracting



#	last_name	department_name
1	Abel	Administration
2	Abel	Marketing
3	Abel	Shipping
4	Abel	IT
5	Abel	Sales
...		
156	Zlotkey	IT
157	Zlotkey	Sales
158	Zlotkey	Executive
159	Zlotkey	Accounting
160	Zlotkey	Contracting

The example in the slide produces a Cartesian product of `EMPLOYEES` and `DEPARTMENTS` tables.

It is a good practice to explicitly state `CROSS JOIN` in your `SELECT` statement when you intend to create a Cartesian product. Therefore, it is very clear that you intend for this to happen and it is not the result of missing joins.

Executing Single-Row Subqueries

```
SELECT last_name, job_id, salary
FROM employees
WHERE job_id = (SELECT job_id
                FROM employees
                WHERE last_name = 'Taylor')
AND salary > (SELECT salary
              FROM employees
              WHERE last_name = 'Taylor');
```



	LAST_NAME	JOB_ID	SALARY
1	Abel	SA_REP	11000



#	last_name	job_id	salary
1	Abel	SA_REP	11000.00

A `SELECT` statement can be considered as a query block. The example in the slide displays employees who do the same job as “Taylor” but earn more salary than him.

The example consists of three query blocks: the outer query and two inner queries. The inner query blocks are executed first, producing the query results `SA_REP` and `8600`, respectively. The outer query block is then processed and uses the values that were returned by the inner queries to complete its search conditions.

Both inner queries return single values (`SA_REP` and `8600`, respectively), so this SQL statement is called a single-row subquery.

Note: The outer and inner queries can get data from different tables.

Using Group Functions in a Subquery

```
SELECT last_name, job_id, salary
FROM employees
WHERE salary = (SELECT MIN(salary)
                FROM employees);
```



	LAST_NAME	JOB_ID	SALARY
1	Vargas	ST_CLERK	2500



#	last_name	job_id	salary
1	Vargas	ST_CLERK	2500.00

You can display data from a main query by using a group function in a subquery to return a single row. The subquery is in parentheses and is placed after the comparison condition.

The example in the slide displays the employee last name, job ID, and salary of all employees whose salary is equal to the minimum salary. The `MIN` group function returns a single value (2500) to the outer query.

HAVING Clause with Subqueries

The database server:

- Executes the subqueries first
- Returns the result into the HAVING clause of the main query

```
SELECT department_id, MIN(salary)
FROM employees
GROUP BY department_id
HAVING MIN(salary) > (SELECT MIN(salary)
                     FROM employees
                     WHERE department_id = 50);
```

2500



#	DEPARTMENT_ID	MIN(SALARY)
1	{null}	7000
2	90	17000
3	20	6000
4	110	8300
5	80	8600
6	60	4200
7	10	4400



#	department_id	MIN(salary)
1	NULL	7000.00
2	10	4400.00
3	20	6000.00
4	60	4200.00
5	80	8600.00
6	90	17000.00
7	110	8300.00

You can use subqueries not only in the WHERE clause but also in the HAVING clause. The server executes the subquery, and the results are returned into the HAVING clause of the main query.

The SQL statement in the slide displays all the departments that have a minimum salary greater than the minimum salary of department 30.

Another Example:

Find the job with the lowest average salary.

```
SELECT job_id, AVG(salary)
FROM employees
GROUP BY job_id
HAVING AVG(salary) = (SELECT MIN(AVG(salary))
                     FROM employees
                     GROUP BY job_id);
```

What is wrong with this statement?

```
SELECT employee_id, last_name
FROM employees
WHERE salary =
    (SELECT MIN(salary)
     FROM employees
     GROUP BY department_id);
```



ORA-01427: single-row subquery returns more than one row
01427. 00000 - "single-row subquery returns more than one row"
*Cause:
*Action:

Single-row operator with multiple-row subquery



Error Code: 1242. Subquery returns more than 1 row

A common error with subqueries occurs when more than one row is returned for a single-row subquery.

In the SQL statement in the slide, the subquery contains a `GROUP BY` clause, which implies that the subquery will return multiple rows, one for each group that it finds. In this case, the results of the subquery are 4400, 6000, 2500, 4200, 7000, 17000, and 8300.

The outer query takes those results and uses them in its `WHERE` clause. The `WHERE` clause contains an equal (`=`) operator, a single-row comparison operator that expects only one value. The `=` operator cannot accept more than one value from the subquery and, therefore, generates the error.

To correct this error, change the `=` operator to `IN`.

No Rows Returned by the Inner Query

```
SELECT last_name, job_id
FROM employees
WHERE job_id =
    (SELECT job_id
     FROM jobs
     WHERE job_title = 'Architect');
```



Query Result x	
All Rows Fetched: 0 in 0.003 seconds	
LAST_N...	JOB_ID

The subquery returns no rows because there is no job with the title "Architect."



#	last_name	job_id
0 row(s) returned		

Another common problem occurs with subqueries when no rows are returned by the inner query.

In the SQL statement in the slide, the subquery contains a `WHERE` clause. Presumably, the intention is to find the employee who works as an 'Architect'. The statement is correct, but selects no rows when executed because there is no job titled 'Architect'. Therefore, the subquery returns no rows.

The outer query takes the results of the subquery (null) and uses these results in its `WHERE` clause. The outer query finds no employee with a job ID equal to NULL and so returns no rows. Even if a job existed with a value of null, the row is not returned because comparison of two null values yields a null; therefore, the `WHERE` condition is not true.

Using the ANY Operator in Multiple-Row Subqueries

```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary < ANY (SELECT salary
                     FROM employees
                     WHERE job_id = 'IT_PROG')
AND job_id <> 'IT_PROG';
```



	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
	1	144 Vargas	ST_CLERK	2500
		143 Matos	ST_CLERK	2600
		142 Davies	ST_CLERK	3100
		141 Rajs	ST_CLERK	3500
	5	200 Whalen	AD_ASST	4400
...				
	9	206 Gietz	AC_ACCOUNT	8300
	10	176 Taylor	SA_REP	8600



#	employee_id	last_name	job_id	salary
1	124	Mourgos	ST_MAN	5800.00
2	141		ST_CLERK	3500.00
3	142		ST_CLERK	3100.00
4	143		ST_CLERK	2600.00
5	144	Vargas	ST_CLERK	2500.00
6	176	Taylor	SA_REP	8600.00
7	178	Grant	SA_REP	7000.00
8	200	Whalen	AD_ASST	4400.00
9	202	Fay	MK_REP	6000.00
10	206	Gietz	AC_ACC...	8300.00
*				



The ANY operator (and its synonym, the SOME operator) compares a value with *each* value returned by a subquery. The example in the slide displays employees who are not IT programmers and whose salary is less than that of any IT programmer. The maximum salary that a programmer earns is \$9,000.

- <ANY means less than the maximum.
- >ANY means more than the minimum.
- =ANY is equivalent to IN.

Using the ALL Operator in Multiple-Row Subqueries

```
SELECT employee_id, last_name, job_id, salary
FROM employees
WHERE salary < ALL
      (SELECT salary
       FROM employees
       WHERE job_id = 'IT_PROG')
AND job_id <> 'IT_PROG';
```

9000, 6000, 4200



	EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
1	141	Rajs	ST_CLERK	3500
2	142	Davies	ST_CLERK	3100
3	143	Matos	ST_CLERK	2600
4	144	Vargas	ST_CLERK	2500



#	employee_id	last_name	job_id	salary
1	141	Rajs	ST_CLERK	3500.00
2	142	Davies	ST_CLERK	3100.00
3	143	Matos	ST_CLERK	2600.00
4	144	Vargas	ST_CLERK	2500.00
*	NULL	NULL	NULL	NULL



The ALL operator compares a value with *every* value returned by a subquery. The example in the slide displays employees who are not IT programmers and whose salary is less than that of all IT programmers.

- >ALL means more than the maximum.
- <ALL means less than the minimum.
- The NOT operator can be used with IN, ANY, and ALL operators.

Multiple-Column Subquery: Example

Display all the employees with the lowest salary in each department.

```
SELECT first_name, department_id, salary
FROM employees
WHERE (salary, department_id) IN
      (SELECT min(salary), department_id
       FROM employees
       GROUP BY department_id)
ORDER BY department_id;
```



#	FIRST_NAME	DEPARTMENT_ID	SALARY
1	Jennifer	10	4400
2	Pat	20	6000
3	Peter	50	2500
4	Diana	60	4200
5	Jonathon	80	8600
6	Neena	90	17000
7	Lex	90	17000
8	William	110	8300



#	first_name	department_id	salary
1	Jennifer	10	4400.00
2	Pat	20	6000.00
3	Peter	50	2500.00
4	Diana	60	4200.00
5	Jonathon	80	8600.00
6	Neena	90	17000.00
7	Lex	90	17000.00
8	William	110	8300.00

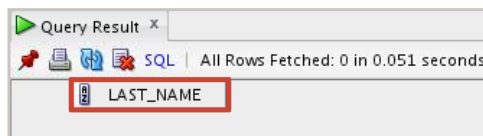
The example in the slide shows a multiple-column subquery in which the subquery returns more than one column.

The inner query is executed first, and it returns the lowest `salary` and `department_id` for each department. The main query block is then processed and uses the values that were returned by the inner query to complete its search condition.

Note: The employee with the first name Kimberly is not returned in the result because the `department_id` for the employee is `NULL`.

Null Values in a Subquery

```
SELECT emp.last_name
FROM   employees emp
WHERE  emp.employee_id NOT IN
      (SELECT mgr.manager_id
       FROM   employees mgr);
```



#	last_name
---	-----------

0 row(s) returned

The subquery returns no rows because one of the values returned by a subquery is null.

The SQL statement in the slide attempts to display all the employees who do not have any subordinates. Logically, this SQL statement should have returned 12 rows. However, the SQL statement does not return any rows. One of the values returned by the inner query is a null value and, therefore, the entire query returns no rows.

The reason is that all conditions that compare a null value result in a null. So whenever null values are likely to be part of the results set of a subquery, do not use the `NOT IN` operator. The `NOT IN` operator is equivalent to `<> ALL`.

Notice that the null value as part of the results set of a subquery is not a problem if you use the `IN` operator. The `IN` operator is equivalent to `=ANY`. For example, to display the employees who have subordinates, use the following SQL statement:

```
SELECT emp.last_name
FROM   employees emp
WHERE  emp.employee_id IN
      (SELECT mgr.manager_id
       FROM   employees mgr);
```

Alternatively, a `WHERE` clause can be included in the subquery to display all employees who do not have any subordinates:

```
SELECT last_name FROM employees
WHERE  employee_id NOT IN
      (SELECT manager_id
       FROM   employees
       WHERE  manager_id IS NOT NULL);
```

Using the UNION Operator

Display the job details of all the current and retired employees.
Display each job only once.

```
SELECT job_id
FROM employees
UNION
SELECT job_id
FROM retired_employees;
```



JOB_ID
1 AC_ACCOUNT
2 AC_MGR
3 AD_ASST
4 AD_PRES
5 AD_VP
6 FI_ACCOUNT
7 FI_MGR
8 IT_PROG
9 MK_MAN
10 MK_REP
11 PU_CLERK
12 PU_MAN
13 SA_MAN
14 SA_REP
15 ST_CLERK
16 ST_MAN



#	job_id
1	AC_ACCOUNT
2	AC_MGR
3	AD_ASST
4	AD_PRES
5	AD_VP
6	IT_PROG
7	MK_MAN
8	MK_REP
9	SA_MAN
10	SA_REP
11	ST_CLERK
12	ST_MAN
13	FI_MGR
14	FI_ACCOUNT
15	PU_MAN
16	PU_CLERK

The UNION operator eliminates any duplicate records. If records that occur in both the EMPLOYEES and the RETIRED_EMPLOYEES tables are identical, the records are displayed only once.

Using the UNION ALL Operator

Display the jobs and departments of all current and previous employees.

```
SELECT job_id, department_id
FROM employees
UNION ALL
SELECT job_id, department_id
FROM retired_employees
ORDER BY job_id;
```

#	JOB_ID	DEPARTMENT_ID
1	AC_ACCOUNT	110
2	AC_MGR	110
3	AD_ASST	10
4	AD_PRES	90
5	AD_PRES	90
6	AD_VP	90
7	AD_VP	80
8	AD_VP	90
9	AD_VP	90



28	SA_REP	80
29	SA_REP	80
30	SA_REP	(null)
31	ST_CLERK	50
32	ST_CLERK	50
33	ST_CLERK	50
34	ST_CLERK	50
35	ST_MAN	50

...

#	job_id	department_id
1	AC_ACCOUNT	110
2	AC_MGR	110
3	AD_ASST	10
4	AD_PRES	90
5	AD_PRES	90
6	AD_VP	90
7	AD_VP	90
8	AD_VP	80
9	AD_VP	90



28	SA_REP	80
29	SA_REP	80
30	SA_REP	80
31	ST_CLERK	50
32	ST_CLERK	50
33	ST_CLERK	50
34	ST_CLERK	50
35	ST_MAN	50

...

In the example in the slide, 35 rows are selected. The combination of the two tables totals to 35 rows. The UNION ALL operator does not eliminate duplicate rows. UNION returns all distinct rows selected by either query. UNION ALL returns all rows selected by either query, including all duplicates.

Consider the query in the slide, now written with the UNION clause:

```
SELECT job_id, department_id
FROM employees
UNION
SELECT job_id, department_id
FROM retired_employees
ORDER BY job_id;
```

The preceding query returns 19 rows. This is because it eliminates all the duplicate rows.

Matching SELECT Statements in MySQL



You must match the data type (using the `CAST` function) when columns do not exist in one or the other table.

```
SELECT location_id, department_name 'Department',  
       CAST(NULL AS CHAR) 'Warehouse location'  
FROM departments  
UNION  
SELECT location_id, CAST(NULL AS CHAR),  
       state_province  
FROM locations;
```



#	location_id	Department	Warehouse location
1	1700	Administration	NULL
2	1800	Marketing	NULL
3	1500	Shipping	NULL
4	1400	IT	NULL
5	2500	Sales	NULL
6	1700	Executive	NULL
7	1700	Accounting	NULL
8	1700	Contracting	NULL
9	1500	NULL	California
10	1800	NULL	Ontario
11	2500	NULL	Oxford
12	1400	NULL	Texas
13	1700	NULL	Washington

Because the expressions in the `SELECT` lists of the queries must match in number, you can use the dummy columns and the data type conversion functions to comply with this rule.

To match the column list explicitly, you can insert `NULL` columns at the missing positions so as to match the count and data type of selected columns in each `SELECT` statement.

In the slide, the name `Warehouse location` is given as the dummy column heading. The `CAST` function is used in the first query to match the `VARCHAR` data type of the `state_province` column that is retrieved by the second query. Similarly, the `CAST` function in the second query is used to match the `VARCHAR` data type of the `department_name` column that is retrieved by the first query.

For Instructor Use Only.

This document should not be distributed.

Matching the SELECT Statement: Example in MySQL



Using the UNION operator, display the employee name, job ID, and hire date of all employees.

```
SELECT first_name, job_id, hire_date 'Hire Date'
FROM employees
UNION
SELECT first_name, job_id, CAST(NULL AS DATE)
FROM retired_employees;
```

#	first_name	job_id	Hire Date
1	Steven	AD_PRES	2011-06-17
2	Neena	AD_VP	2009-09-21
3	Lex	AD_VP	2009-01-13
4	Alexander	IT_PROG	2014-01-03
5	Bruce	IT_PROG	2015-05-21
6	Diana	IT_PROG	2015-02-07
7	Kevin	ST_MAN	2015-11-16
8	Trenna	ST_CLERK	2011-10-17
9	Curtis	ST_CLERK	2013-01-29

...

28	Rahul	FI_MGR	NULL
29	Dany	FI_ACCOUNT	NULL
30	James	FI_ACCOUNT	NULL
31	Shana	FI_ACCOUNT	NULL
32	Peter	FI_ACCOUNT	NULL
33	Lui	FI_ACCOUNT	NULL
34	Del	PU_MAN	NULL
35	Alex	PU_CLERK	NULL

The employees and retired_employees tables have several columns in common (for example, employee_id, job_id, and department_id). However, what if you want the query to display first_name, job_id, and hire_date using the UNION operator, knowing that hire_date exists only in the employees table?

The code example in the slide matches the first_name and job_id columns in the employees and retired_employees tables. NULL is added to the retired_employees SELECT statement to match the hire_date column in the employees SELECT statement.

In the results shown in the slide, each row in the output that corresponds to a record from the retired_employees table contains a NULL in the hire_date column.

For Instructor Use Only.

This document should not be distributed.

Using the ORDER BY Clause with UNION in MySQL



- To use an ORDER BY clause to sort the entire UNION result, place the ORDER BY clause only once at the end of the compound query.
- The ORDER BY clause uses the columns of the first SELECT query.
- If a column to be sorted is aliased, the ORDER BY clause must use the alias rather than the column name.



You can use the ORDER BY clause to sort the entire result set in a compound query. Place the ORDER BY clause at the end of the query. In the ORDER BY clause, you can refer to a column alias or column name from the first SELECT statement. If the sort column is aliased, you must use the alias rather than the column name. You can use the column position in the ORDER BY clause, but this use is deprecated.

Note: You can use an ORDER BY clause in each SELECT statement, but it has no effect on the sort order of the result set, but rather is used by the LIMIT clause. To apply an ORDER BY clause to an individual SELECT statement, enclose the statement in parentheses, with the ORDER BY and LIMIT clauses inside the parentheses. For example, the following query selects and combines the first 10 employees from each table and sorts the results by Job ID.

```
(SELECT employee_id, job_id
FROM   employees ORDER BY employee_id LIMIT 10)
UNION
(SELECT employee_id, job_id
FROM   retired_employees ORDER BY employee_id LIMIT 10)
ORDER BY job_id;
```


Using the ORDER BY Clause with UNION: Example in MySQL



Display the employee ID and job ID of all current and retired employees, sorted by job ID.

```
SELECT employee_id, job_id
FROM employees
UNION
SELECT employee_id, job_id
FROM retired_employees
ORDER BY job_id;
```



#	employee_id	job_id
1	206	AC_ACCOUNT
2	205	AC_MGR
3	200	AD_ASST
4	100	AD_PRES
5	301	AD_PRES
6	101	AD_VP
7	102	AD_VP
8	303	AD_VP
9	302	AD_VP

...

28	178	SA_REP
29	176	SA_REP
30	174	SA_REP
31	144	ST_CLERK
32	143	ST_CLERK
33	142	ST_CLERK
34	141	ST_CLERK
35	124	ST_MAN

In the example shown in the slide, the output is sorted in the ascending order of `job_id`.

For Instructor Use Only.

This document should not be distributed.

Data Manipulation Language

- A DML statement is executed when you:
 - Add new rows to a table
 - Modify existing rows in a table
 - Remove existing rows from a table
- A transaction consists of a collection of DML statements that form a logical unit of work.

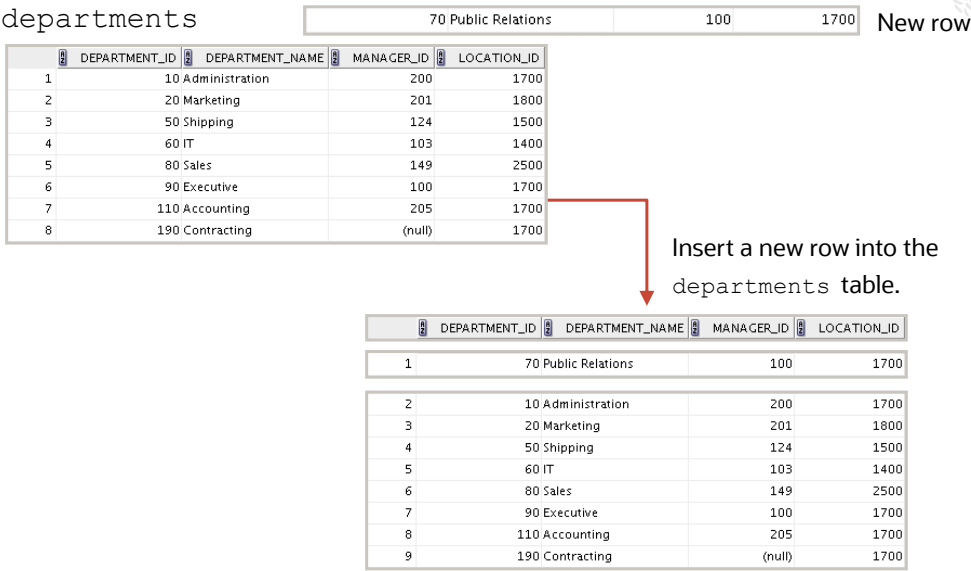


Data manipulation language (DML) is a core part of SQL. When you want to add, update, or delete data in the database, you execute a DML statement. A collection of DML statements that form a logical unit of work is called a *transaction*.

Consider a banking database. When a bank customer transfers money from a savings account to a checking account, the transaction might consist of three separate operations: decreasing the savings account, increasing the checking account, and recording the transaction in the transaction journal. The database server must guarantee that all the three SQL statements are performed to maintain the accounts in proper balance. When something prevents one of the statements in the transaction from executing, the other statements of the transaction must be undone.

Note: Most of the DML statements in this lesson assume that no constraints on the table are violated.

Adding a New Row to a Table



The graphic in the slide illustrates the addition of a new department record to the departments table.

INSERT Statement Syntax

- Add new rows to a table by using the `INSERT` statement:

```
INSERT INTO table [(column [, column...])]
VALUES
    (value [, value...]),
    (value [, value...]),
    ...
    (value [, value...]);
```

- Each comma-separated list of values, enclosed in parentheses, inserts a new row.
- Enclose character and date data in quotation marks.



You can add new rows to a table by issuing the `INSERT` statement.

In the syntax:

- **table:** The name of the table. The table name is required.
- **column:** The names of the columns in the table to populate. The column names are optional. As described below, the values listed depend on whether and how column names are provided.
- **value:** The corresponding value for each column. The list of values for each row are enclosed within parentheses and separated by commas. If the column names are provided, the list of values must be in the same order as the list of corresponding column names. If the column names are not provided, each list of values must include all columns in the order provided by the `DESCRIBE` statement.

Inserting New Rows: Listing Column Names

If you include the list of column names after the table name:

- You can list the columns in any order
- The values must be listed in the same order as the columns they populate
- The list of values has to include the same number of values as the number of columns

```
INSERT INTO departments(department_id,  
                        department_name, manager_id, location_id)  
VALUES (70, 'Public Relations', 100, 1700),  
       (150, 'Shareholder Services', 100, 1700);
```

2 row(s) affected

Records: 2 Duplicates: 0 Warnings: 0

The list of column names in parentheses after the table name in the `INSERT` statement is optional. If you include the list of column names, it does not have to be in the same order as the default order of the columns in the table. The list of column names can be in any order, but the values listed must be in the same order as the list of columns and has to have the same number of values as the list of columns.

The example in this slide inserts two rows. You could also create two separate `INSERT` statements, one for each row.

Inserting New Rows: Omitting Column Names

If you omit the list of column names after the table name:

- You must include a value for every column in the table
- The values must be listed in the same order as the default order of the columns, as displayed by a `DESCRIBE table_name;` statement

```
INSERT INTO departments
VALUES (120, 'Treasury', 100, 1700),
      (130, 'Corporate Tax', 100, 1700),
      (140, 'Control and Credit', 100, 1700);
```

3 row(s) affected

Records: 3 Duplicates: 0 Warnings: 0

The default order of columns is the order that columns are displayed as output from a `SELECT * FROM table_name;` query. You can find the default order by issuing a `DESCRIBE table_name;` statement.

Inserting Rows with Null Values

- **Implicit method:** If you omit a column from the column list, the value is set to the default value for the column or `NULL` if the column does not have a default value.

```
INSERT INTO departments (department_id,  
                        department_name)  
VALUES      (30, 'Purchasing');
```

1 row(s) affected

- **Explicit method:** You can specify the `NULL` keyword in the `VALUES` list.

```
INSERT INTO departments  
VALUES      (100, 'Finance', NULL, NULL);
```

1 row(s) affected

If you include a column list, but omit one or more of the columns, the value is set to the default value for the column or `NULL`. You learn about setting default values in the lesson titled "Introduction to Data Definition Language in MySQL." Be sure that you can use null values in the targeted column by verifying the `NULL` status with the `DESCRIBE` command.

Inserting Special Values in MySQL

The `CURDATE()` function records the current date and time in MySQL.

```
INSERT INTO employees (employee_id,
                        first_name, last_name,
                        email, phone_number,
                        hire_date, job_id, salary,
                        commission_pct, manager_id,
                        department_id)
VALUES (113,
        'Louis', 'Popp',
        'LPOPP', '515.124.4567',
        CURDATE(), 'AC_ACCOUNT', 6900,
        NULL, 205, 110);
```

1 row(s) affected

You can use functions to enter special values in your table.

The example in the slide records information for employee Popp in the `employees` table. It supplies the current date and time in the `hire_date` column. It uses the `CURDATE()` function that returns the current date.

Confirming Additions to the Table

```
SELECT employee_id, last_name, job_id, hire_date, commission_pct
FROM   employees
WHERE  employee_id = 113;
```


Inserting Specific Date and Time Values in MySQL

- Add a new employee.

```
INSERT INTO employees
VALUES (115,
       'Alexander', 'Khoo',
       'AKHOO', '515.127.4562',
       '2011-05-18',
       'SA_REP', 3100, 0.2, 100, 60);
```

1 row(s) affected

- Verify your addition.

```
SELECT * FROM employees WHERE employee_id = 115;
```

#	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
1	115	Alexander	Khoo	AKHOO	515.127.4562	2011-05-18	SA_REP	3100.00	0.20	100	60
*		NAME	NAME	NAME	NAME	NAME	NAME	NAME	NAME	NAME	NAME

You can enter dates by using the default date format, just as you do in `WHERE` conditions or other references to dates.

Inserting and Reformatting Specific Date and Time Values in MySQL

- Add a new employee.

```
INSERT INTO employees
VALUES
(114,
'Den', 'Raphealy',
'DRAPHEAL', '515.127.4561',
STR_TO_DATE('FEB 3, 2016', '%M %d, %Y'),
'SA_REP', 11000, 0.2, 100, 70);
```

1 row(s) affected

- Verify your addition.

```
SELECT * FROM employees WHERE employee_id = 114;
```

#	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
1	114	Den	Raphealy	DRAPHEAL	515.127.4561	2016-02-03	SA_REP	11000.00	0.20	100	70
*											

In MySQL, if you want to enter the date in a format other than the default format, you must use the `STR_TO_DATE` function.

The example in the slide records information for employee Raphealy in the `employees` table. It sets the `hire_date` column to be February 3, 2016, but stores and displays it in the default date format.

Copying Rows from Another Table

- Write your `INSERT` statement with a subquery:

```
INSERT INTO copy_emp  
SELECT * FROM employees;
```

23 row(s) affected
Records: 23 Duplicates: 0 Warnings: 0

```
INSERT INTO sales_reps(id, name, salary, commission_pct)  
SELECT employee_id, last_name, salary, commission_pct  
FROM employees WHERE job_id LIKE '%REP%';
```

6 row(s) affected
Records: 6 Duplicates: 0 Warnings: 0

- Do not use the `VALUES` clause.
- Match the number of columns in the `INSERT` clause to those in the subquery.
- It inserts all the rows returned by the subquery into the table.

You can use the `INSERT` statement to add rows to a table where the values are derived from existing tables. In the example in the slide, for the `INSERT INTO` statement to work, you must have already created the `sales_reps` table by using the `CREATE TABLE` statement. `CREATE TABLE` is discussed in the lesson titled “Introduction to Data Definition Language.”

In place of the `VALUES` clause, you use a subquery.

Syntax

```
INSERT INTO table [ column (, column) ] subquery;
```

In the syntax:

<i>table</i>	Is the name of the table
<i>column</i>	Is the name of the column in the table to populate
<i>subquery</i>	Is the subquery that returns rows to the table

The number of columns and their data types in the column list of the `INSERT` clause must match the number of values and their data types in the subquery. Zero or more rows are added depending on the number of rows returned by the subquery.

Changing Data in a Table

employees

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	MANAGER_ID	COMMISSION_PCT	DEPARTMENT_ID
100	Steven	King	24000	(null)	(null)	90
101	Neena	Kochhar	17000	100	(null)	90
102	Lex	De Haan	17000	100	(null)	90
103	Alexander	Hunold	9000	102	(null)	60
104	Bruce	Ernst	6000	103	(null)	60
107	Diana	Lorentz	4200	103	(null)	60
124	Kevin	Mourgos	5800	100	(null)	50

Update rows in the employees table:

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	SALARY	MANAGER_ID	COMMISSION_PCT	DEPARTMENT_ID
100	Steven	King	24000	(null)	(null)	90
101	Neena	Kochhar	17000	100	(null)	90
102	Lex	De Haan	17000	100	(null)	90
103	Alexander	Hunold	9000	102	(null)	80
104	Bruce	Ernst	6000	103	(null)	80
107	Diana	Lorentz	4200	103	(null)	80
124	Kevin	Mourgos	5800	100	(null)	50

The slide illustrates changing the department number for employees in department 60 to department 80.

UPDATE Statement Syntax

- Modify existing values in a table with the `UPDATE` statement:

```
UPDATE      table
SET         column = value [, column = value, ...]
[WHERE      condition];
```

- Update more than one row at a time (if required).



You can modify the existing values in a table by using the `UPDATE` statement.

In the syntax:

<i>table</i>	Is the name of the table
<i>column</i>	Is the name of the column in the table to populate
<i>value</i>	Is the corresponding value or subquery for the column
<i>Condition</i>	Identifies the rows to be updated and is composed of column names, expressions, constants, subqueries, and comparison operators

Confirm the update operation by querying the table to display the updated rows.

For more information, see the section on “`UPDATE`” in *Oracle Database SQL Language Reference* for 23c database.

Note: In general, use the primary key column in the `WHERE` clause to identify a single row for update. Using other columns can unexpectedly cause several rows to be updated. For example, identifying a single row in the `employees` table by last name may return more than one employee having the same last name.

Updating Rows in a Table

- Values for a specific row or rows are modified if you specify the `WHERE` clause:

```
UPDATE employees
SET   department_id = 50
WHERE employee_id = 113;
```

```
1 row(s) affected
Rows matched: 1 Changed: 1 Warnings: 0
```

- Values for all the rows in the table are modified if you omit the `WHERE` clause:

```
UPDATE   copy_emp
SET      department_id = 110;
```

```
20 row(s) affected
Rows matched: 23 Changed: 20 Warnings: 0
```

- Specify `SET column_name= NULL` to update a column value to `NULL`.

The `UPDATE` statement modifies the values of a specific row or rows if the `WHERE` clause is specified. The example in the slide shows the transfer of employee 113 (Popp) to department 50.

If you omit the `WHERE` clause, values for all the rows in the table are modified. Examine the updated rows in the `COPY_EMP` table.

```
SELECT last_name, department_id
FROM   copy_emp;
```

For example, an employee who was an `SA_REP` has now changed his job to an `IT_PROG`. Therefore, his `JOB_ID` needs to be updated and the commission field needs to be set to `NULL`.

```
UPDATE employees
SET job_id = 'IT_PROG', commission_pct = NULL
WHERE employee_id = 114;
```

Note: The `COPY_EMP` table has the same data as the `EMPLOYEES` table.

Updating Rows Based on Another Table

Use the subqueries in the `UPDATE` statements to update row values in a table based on values from another table:

```
UPDATE copy_emp
SET    department_id = (SELECT department_id
                        FROM employees
                        WHERE employee_id = 100)
WHERE  job_id         = (SELECT job_id
                        FROM employees
                        WHERE employee_id = 200);
```

1 row(s) affected
Rows matched: 1 Changed: 1 Warnings: 0

You can use the subqueries in the `UPDATE` statements to update values in a table. The example in the slide updates the `copy_emp` table based on the values from the `employees` table. It changes the department number to employee 100's current department number (90) for all the employees whose job ID is the same as employee 200's job ID. In MySQL, you cannot update a table and select from the same table in a subquery.

Removing a Row from a Table

departments

	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700
8	190	Contracting	(null)	1700

Delete a row from the departments table:

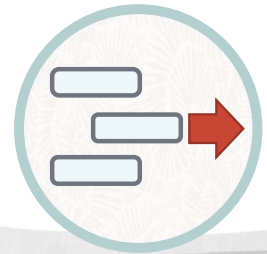
	DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID	LOCATION_ID
1	10	Administration	200	1700
2	20	Marketing	201	1800
3	50	Shipping	124	1500
4	60	IT	103	1400
5	80	Sales	149	2500
6	90	Executive	100	1700
7	110	Accounting	205	1700

The contracting department has been removed from the departments table (assuming no constraints on the departments table are violated), as shown in the graphic in the slide.

DELETE Statement

You can remove existing rows from a table by using the `DELETE` statement:

```
DELETE [FROM]    table
[WHERE          condition];
```



In the syntax:

<i>table</i>	Is the name of the table
<i>condition</i>	Identifies the rows to be deleted, and is composed of column names, expressions, constants, subqueries, and comparison operators

Note: If no rows are deleted, the message “0 rows deleted” is returned (on the Script Output tab in SQL Developer).

For more information, see the section on “`DELETE`” in *Oracle Database SQL Language Reference* for 23c database.

Deleting Rows from a Table

- Specific rows are deleted if you specify the `WHERE` clause:

```
DELETE FROM departments
WHERE department_name = 'Finance';
```

1 row(s) affected

- All rows in the table are deleted if you omit the `WHERE` clause:

```
DELETE FROM copy_emp;
```

23 row(s) affected

You can delete specific rows by specifying the `WHERE` clause in the `DELETE` statement. The first example in the slide deletes the accounting department from the `DEPARTMENTS` table. You can confirm the delete operation by trying to display the deleted rows by using the `SELECT` statement. The query returns 0 rows.

```
SELECT  *
FROM    departments
WHERE   department_name = 'Finance';
```

However, if you omit the `WHERE` clause, all rows in the table are deleted. The second example in the slide deletes all rows from the `COPY_EMP` table, because no `WHERE` clause was specified.

Example

Remove rows identified in the `WHERE` clause.

```
DELETE FROM employees WHERE employee_id = 115;
```

```
DELETE FROM departments WHERE department_id IN (130, 140);
```

Deleting Rows Based on Another Table

Use the subqueries in the `DELETE` statements to remove rows from a table based on values from another table:

```
DELETE FROM employees
WHERE department_id IN
    (SELECT department_id
     FROM departments
     WHERE department_name
       LIKE '%Public%');
```

1 row(s) affected

You can use the subqueries to delete rows from a table based on values from another table. The example in the slide deletes all the employees in a department, where the department name contains the string `Public`.

The subquery searches the `departments` table to find the department number based on the department name containing the string `Public`. The subquery then feeds the department number to the main query, which deletes rows of data from the `employees` table.

TRUNCATE Statement

- Removes all rows from a table, leaving the table empty and the table structure intact
- Is a data definition language (DDL) statement rather than a DML statement; cannot be undone
- Syntax:

```
TRUNCATE TABLE table_name;
```

- Example:

```
TRUNCATE TABLE copy_emp;
```

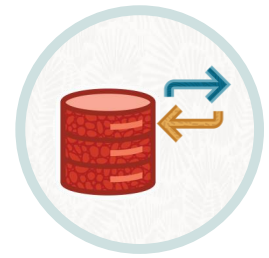
You can use the `TRUNCATE` statement to quickly remove all rows from a table or cluster efficiently. Removing rows with the `TRUNCATE` statement is faster than removing them with the `DELETE` statement for the following reasons:

- The `TRUNCATE` statement is a data definition language (DDL) statement and generates no rollback information. Rollback information is covered later in this lesson.

If the table is the parent of a referential integrity constraint, you cannot truncate the table. You need to disable the constraint before issuing the `TRUNCATE` statement. You will learn more about DDL statements and disabling constraints in the lesson titled “Introduction to Data Definition Language.”

Multiple-Statement Transactions

- A collection of statements that are treated as a single unit:
 - Groups multiple statements
 - Is useful when multiple clients can access data from the same table at the same time
 - Is important when statements affect the same or related data
- All or none of the steps succeed:
 - Execute if all steps are successful.
 - Cancel if any steps cause errors or do not complete.

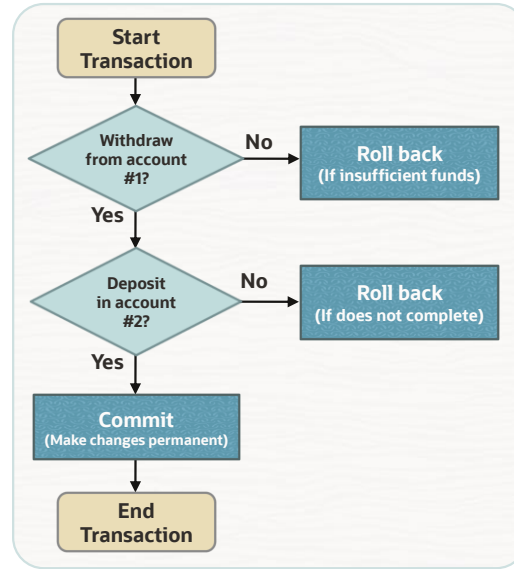


A transaction lets you execute one or more SQL statements as a single unit of work, so that either all or none of the statements succeed. This happens independently of work being done by any other transactions. If all the statements succeed, you commit the transaction to record the effect permanently in the database. If an error occurs during the transaction, you roll it back to cancel it. Any statements executed up to the point of canceling within the transaction are undone, leaving the database in the state it was in before the transaction.

Note: MySQL supports transactions only for tables that use a transactional storage engine (such as InnoDB). These statements have no noticeable effect on tables managed by non-transactional storage engines.

Transaction Diagram

Example of a banking transaction:



The diagram in the slide shows an attempted transfer of \$1,000 from your savings account to your checking account. You would not be happy if the money were successfully withdrawn from your savings account, but never reached your checking account.

To protect against this kind of error, the program that handles your transfer request begins a transaction and then issues the SQL statements needed to move the money from your savings to your checking account. It only commits the transaction if everything succeeds. If a problem occurs, the program instructs the server to undo all the changes made since the transaction began.

AUTOCOMMIT and Transaction Control Statements

- By default, MySQL runs with the `AUTOCOMMIT` mode enabled:
 - As soon as you execute a statement that modifies a table, MySQL stores the update on disk.
 - Each statement is treated like a transaction.
- Use the following statements to control a multiple-statement transaction:
 - `START TRANSACTION` (or `BEGIN`)
 - `COMMIT`
 - `ROLLBACK`

When you issue a `START TRANSACTION` statement, `AUTOCOMMIT` mode is disabled. Results of statements after the `START TRANSACTION` statement that modify a table are not written to the disk. The transaction remains open until you close it with a `COMMIT`, a `ROLLBACK`, or any statement that implicitly closes a transaction.

You can disable `AUTOCOMMIT`. After disabling `AUTOCOMMIT` mode, changes to transaction-safe tables (such as InnoDB) are not made permanent immediately. You must use a `COMMIT` statement to store your changes to disk or `ROLLBACK` statement to ignore the changes. The `AUTOCOMMIT` mode then reverts to its previous state.

To disable `AUTOCOMMIT`, enter the following statement:

```
SET AUTOCOMMIT=0;
```

Use the following statements to control a multiple-statement transaction:

`START TRANSACTION` (or `BEGIN`): Explicitly begins a new transaction and disables the `AUTOCOMMIT` mode

`COMMIT`: Makes the changes from the current transaction permanent

`ROLLBACK`: Cancels the changes from the current transaction

Committing Data in a Transaction

Make the changes to the data and commit the transaction:

```
START TRANSACTION;

INSERT INTO retired_employees (employee_id, first_name, last_name, email, retired_date,
job_id, salary, manager_id, department_id)
SELECT employee_id, first_name, last_name, email, CURDATE(), job_id, salary,
manager_id, department_id
FROM employees
WHERE employee_id = 113;

DELETE from employees where employee_id = 113;

COMMIT;
```

In the example in the slide, an employee is being moved from the `employees` table to the `retired_employees` table. Making this a transaction ensures that the employee can be added to the `retired_employees` table before being deleted from the `employees` table. If either of these actions fails, the transaction is rolled back. Because both statements execute correctly, the changes are saved by issuing the `COMMIT` statement. In this example, the columns to be inserted must be listed because not all of the columns from the `employees` table are used in the `retired_employees` table. Also, the `retired_employees` table has a `retired_date` column, which is set to the current date in this example, but could be set to a specified date if desired.

Rolling Back Changes

Make changes to data and then roll back the changes:

```
START TRANSACTION;

INSERT INTO departments
VALUES (200, 'Operations', NULL, 1700);

UPDATE employees
SET department_id = 200
WHERE department_id =
(SELECT department_id FROM departments WHERE department_name =
'IT');

ROLLBACK;
```

In the example in the slide, employees from the IT department are being transferred to the newly created Operations department (200). Creating a transaction ensures that the new Operations department is successfully added to the `departments` table before employees from the IT department have their department ID set to the new department. In this example, the `ROLLBACK` statement returns the `departments` table and the `employees` table back to the status they were in before the transaction began. That is, the department that was inserted is removed, and the employees from IT still have their previous department ID.

Rolling Back Changes to a Marker

- Create a marker in the current transaction by using the `SAVEPOINT` statement.
- Roll back to that marker by using the `ROLLBACK TO SAVEPOINT` statement.

```
START TRANSACTION;

INSERT INTO copy_emp
VALUES (113, 'Luis', 'Popp', 'LPOPP', '515.124.4567',
       STR_TO_DATE('07-12-2015', '%d-%m-%Y'), 'FI_ACCOUNT', 6900, NULL,
       205, 110);

SAVEPOINT first_insert;

INSERT INTO copy_emp
VALUES (175, 'Alyssa', 'Hutton', 'AHUTTON', '011.44.1644.429266',
       STR_TO_DATE('19-03-2013', '%d-%m-%Y'), 'SA_REP', 8800, .25, 149,
       80);

ROLLBACK TO first_insert;

COMMIT;
```

You can create a marker in the current transaction by using the `SAVEPOINT` statement, which divides the transaction into smaller sections. You can then discard pending changes back to that marker by using the `ROLLBACK TO SAVEPOINT` statement. The effect of the example in this slide is that the first `INSERT` statement is committed, but the second insert statement is rolled back. That is, employee 113 is inserted, but employee 175 is not.

Note that if you create a second savepoint with the same name as an earlier savepoint, the earlier savepoint is deleted.

The following transaction would commit the first and third `INSERT` statements (employees 177 and 179) but not the second one (employee 178):

```
START TRANSACTION;

INSERT INTO copy_emp
VALUES (177, 'Jack', 'Livingston', 'JLIVINGS', '011.44.1644.429264',
       STR_TO_DATE('23-04-2014', '%d-%m-%Y'), 'SA_REP', 8400, .20, 149, 80);

SAVEPOINT first_insert;

INSERT INTO copy_emp
VALUES (178, 'Kimberely', 'Grant', 'KGRANT', '011.44.1644.429263',
       STR_TO_DATE('24-05-2015', '%d-%m-%Y'), 'SA_REP', 7000, .15, 149, NULL);

ROLLBACK TO first_insert;

INSERT INTO copy_emp
VALUES (179, 'Charles', 'Johnson', 'CJOHNSON', '011.44.1644.429262',
       STR_TO_DATE('04-01-2016', '%d-%m-%Y'), 'SA_REP', 6200, .10, 149, 80);

COMMIT;
```

Consistent Reads

In busy database systems, some transactions are reading data while other transactions are changing that same data.

- A read operation does not place any locks on the tables it is reading. Other transactions are not blocked from modifying the data.
- A read operation reads a snapshot of the data. Other transactions can then modify the data.
- The timing of the snapshot depends on the isolation level.
- The default isolation level for the MySQL InnoDB storage engine is `REPEATABLE READ`.

MySQL supports other isolation levels. Those are covered in the MySQL Performance Tuning course.

The default isolation level for the MySQL InnoDB storage engine is `REPEATABLE READ`. With that isolation level, a snapshot is taken with the first read statement of a transaction, and all other reads within that transaction use the same snapshot.

FOR UPDATE Clause in a SELECT Statement

- The rows in the EMPLOYEES table where job_id is SA_REP are locked.

```
SELECT employee_id, salary, commission_pct, job_id
FROM employees
WHERE job_id = 'SA_REP'
ORDER BY employee_id
FOR UPDATE;
```

- This works only after START TRANSACTION or with AUTOCOMMIT set to 0.
- Lock is released only when you issue a ROLLBACK or a COMMIT.
- If the SELECT statement attempts to lock a row that is locked by another user, the database waits until the row is available, and then returns the results of the SELECT statement.
- In MySQL, the FOR UPDATE clause must be the last clause in the SELECT statement.

When you issue a SELECT statement against the database to query some records, no locks are placed on the selected rows. In general, this is required because the number of records locked at any given time is (by default) kept to the absolute minimum: only those records that have been changed but not yet committed are locked. Even then, others will be able to read those records as they appeared before the change (the “before image” of the data). There are times, however, when you may want to lock a set of records even before you change them in your program. MySQL offers the FOR UPDATE clause of the SELECT statement to perform this locking. This works only within a transaction (after a START TRANSACTION statement) or if AUTOCOMMIT is set to 0 (off) rather than the default value of 1 (ON).

When you issue a SELECT . . . FOR UPDATE statement in a transaction, the database automatically obtains exclusive row-level locks on all the rows identified by the SELECT statement, thereby holding the records “for your changes only.” No one else will be able to change any of these records until you perform a ROLLBACK or a COMMIT.

You can append the optional keyword NOWAIT to the FOR UPDATE clause to tell the server not to wait if the table has been locked by another user. In this case, the query returns an error, and control is returned immediately so that you can perform other work, or you can wait for a period of time before trying again. Without the NOWAIT clause, your process blocks until the table is available, when the locks are released by the other user through the other user's issue of a COMMIT or a ROLLBACK statement.

FOR UPDATE Clause: Examples

- You can use the `FOR UPDATE` clause in a `SELECT` statement against multiple tables.

```
SELECT e.employee_id, e.last_name, e.job_id
FROM employees e JOIN departments d
USING (department_id)
WHERE d.department_name = 'Shipping'
ORDER BY e.employee_id
FOR UPDATE;
```

- Rows from both the `EMPLOYEES` and `DEPARTMENTS` tables are locked.
- Use `FOR UPDATE OF table_name` to qualify the table that you intend to change; then only the rows from that specific table are locked.

In the example in the slide, the statement locks rows in the `employees` table with `department_id` of 50 (Shipping) and locks rows in the `departments` table with departments named Shipping.

In MySQL, you can use the `FOR UPDATE OF table_name` to qualify the table that you intend to change. When `OF table_name` is omitted, the query locks all tables referenced by the `FROM` clause. If an alias is specified as the table name in the `SELECT` statement, the `FOR UPDATE OF` clause must use the alias.

The following statements lock only those rows in the `employees` table with department ID of 50 (Shipping). No rows are locked in the `departments` table:

```
START TRANSACTION;

SELECT e.employee_id, e.last_name, e.job_id
FROM employees e JOIN departments d
USING (department_id)
WHERE d.department_name = 'Shipping'
ORDER BY e.employee_id
FOR UPDATE OF e;

COMMIT;
```

You can manually lock and unlock tables with the `LOCK TABLES` and `UNLOCK TABLES` statements. These statements are covered in the MySQL Performance Tuning course.

Creating a Database: Syntax

- General syntax for creating a database:

```
CREATE DATABASE [IF NOT EXISTS] database_name;
```

Or

```
CREATE SCHEMA [IF NOT EXISTS] database_name;
```

- The `IF NOT EXISTS` option prevents getting an error if the database already exists.

The terms database and schema are synonymous in MySQL. In this course, you have been working with one database or schema, the HR database, with tables for employees and jobs. You might also have an inventory database with tables for items, their locations, manufacturers, costs, and so on.

The statement to create the HR database is the following:

```
CREATE DATABASE IF NOT EXISTS HR;
```

MySQL Naming Conventions

- Database names, table names, and column names:
 - Can contain A–Z, a–z, 0–9, _, and \$
 - Cannot have more than 64 characters
 - Cannot contain reserved words or special characters, such as /, \, ., # or spaces unless you enclose the name with backticks
- For example, the following causes an error:

```
CREATE DATABASE my database;
```

- Correct the error by enclosing the name in backticks:

```
CREATE DATABASE `my database`;
```



Databases and tables in MySQL are created by using directories and files in the host file system. Because of this, if the host operating system is case-sensitive in its treatment of directory and file names, so is MySQL. Windows file names are not case-sensitive, so a server running on Windows does not treat database and table names as case-sensitive. MySQL servers running on UNIX usually treat database and table names as case-sensitive because UNIX file names are case-sensitive. Case sensitivity of database and table names depends on the host operating system.

Table names can be reserved words or contain special characters if the name is quoted in backticks (` `).

CREATE TABLE Statement

General syntax for creating a table:

```
CREATE TABLE [database.]table  
  (column_name datatype [, ...]);
```

- Example:

```
CREATE TABLE employees1  
  (employee_id    INTEGER,  
   first_name     VARCHAR(20),  
   last_name      VARCHAR(25),  
   email          VARCHAR(25),  
   phone_number   VARCHAR(20),  
   hire_date      DATE,  
   job_id         VARCHAR(10),  
   salary         DECIMAL(8,2),  
   commission_pct DECIMAL(2,2),  
   manager_id     INTEGER,  
   department_id  INTEGER);
```



You create tables to store data by executing the SQL `CREATE TABLE` statement. This statement is one of the DDL statements that are used to create, modify, or remove database structures. These statements record information in the data dictionary. The data dictionary is an important set of read-only tables that provide database information.

To create a table, a user must have the `CREATE TABLE` privilege. The example in this slide creates a table similar to the employees table. It creates a table with columns and their data types specified. This lesson describes numerous other options that can be used when creating tables. The table in this slide uses the data types in the employee table. MySQL supports many other data types.

Data Types: Overview

- **Numeric data types:** Integer, decimal, and floating point data
- **Date and time data types:** Calendar dates, with or without time data and time intervals
- **String data types:** Character strings and BLOBs
- **Spatial data types:** Data representing geographic features
- **JavaScript Object Notation (JSON):** Structured text data stored in JSON documents

MySQL supports a wide variety of data types. In this course, you have worked with examples of numeric, date, and character data. This lesson describes some of the other commonly used MySQL data types. Spatial data and JSON documents are beyond the scope of this course. Data types are covered in more depth in the MySQL Essentials course.

Numeric Data Types

- Integer:
 - Positive or negative whole numbers
 - Syntax: `INT[(M)] [UNSIGNED] [ZEROFILL]`
- Fixed Point:
 - Used for exact-value numbers: integer, fractional, or both
 - Syntax: `DECIMAL[(M[, D])] [UNSIGNED] [ZEROFILL]`
- Floating Point:
 - Used for approximate-value numbers: integer, fractional, or both
 - Syntax: `FLOAT[(M, D)] [UNSIGNED] [ZEROFILL]`

In the syntax, *M* represents an optional value for the maximum number of digits available, and, for `DECIMAL` and `FLOAT`, *D* represents the number of digits following the decimal point. `INTEGER` is a synonym for `INT`. For `INT`, if *M* is omitted, a default number of digits is provided.

`NUMERIC` and `DEC` are synonyms for `DECIMAL`. For `DECIMAL` data type, if *D* is omitted, the default is 0 decimal places, and if *M* is omitted, the default is 10 significant digits. The decimal point and (for negative numbers) the minus (–) sign are not counted in *M*.

Floating point numbers are used to represent very large or very small numbers (close to zero) using a small amount of space. For example, `6.022169E+23` represents 602216900000000000000000, while `6.022169E-23` represents .00000000000000000000006022169 as a value. If *M* and *D* are omitted, `FLOAT` values are stored to the limits permitted by the hardware.

MySQL supports a number of other integer, fixed point, and floating point data types for efficiency of storage or calculations. Other data types are covered in the course on MySQL Essentials.

Date and Time Data Types

- **DATE:**
 - YYYY-MM-DD as in 2018-01-04
- **TIME:**
 - HH:MM:SS as in 12:59:02
- **DATETIME:**
 - YYYY-MM-DD HH:MM:SS as in 2018-01-04 12:59:02

Here, YYYY, MM, DD, HH, MM, and SS stand for year, month, day of month, hour, minute, and second, respectively.

You have used the **DATE** data type previously in this course.

The **TIME** data type can express time of day or elapsed time.

The **DATETIME** data type is a combination of date and time.

The **TIME** and **DATETIME** data types can include fractional seconds up to 6 decimal places.

Other date and time data types and the use of fractional seconds are covered in the course on MySQL Essentials.

For Instructor Use Only.
This document should not be distributed.

String Data Types

- Fixed length character strings: `CHAR (M)`
 - Stored as fixed length *M*, right-padded with spaces
- Variable length character strings: `VARCHAR (M)`
 - Stored **up to** length *M*, **not** right-padded with spaces
- Extremely long character strings: `TEXT`
 - Variable length character strings, stored as a separately allocated object
- **Binary large objects**: `BLOB`
 - Variable length binary data, stored as a separately allocated object

The `CHAR` data type is efficient for storage when the character string column always has the same number of characters, such as for a 2-character state or region abbreviation or a 3-character country abbreviation. The `VARCHAR` data type is common for character strings that are variable in length, such as names of people or places. For `CHAR`, length can be up to 255, whereas for `VARCHAR`, length can be up to 65,535, but cannot cause the size of the row, including all columns, to exceed 65,535 bytes. For extremely long character strings, the `TEXT` type can hold a variable amount of data. A `BLOB` (binary large object) is binary data stored as a single value in a database.

The course on MySQL Essentials provides more coverage of the `TEXT` and `BLOB` data types, along with `BINARY`, `VARBINARY`, `ENUM`, and `SET` data types.

Indexes, Keys, and Constraints

- **Index:** A column or columns of table data designated to be used as pointers to table rows to speed up data retrieval
- **Key:** Normally a synonym for index in MySQL
- **Constraint:** A restriction placed on one or more column values of a table to enforce integrity rules

The following MySQL indexes (keys) include designated combinations of constraints:

- Primary Keys
- Unique Keys
- Foreign Keys

A constraint is a condition that data must meet when being added to or removed from a table. For example, a unique constraint requires all values in an index to be distinct. If you try to add a new row with a value in the index that matches a value in an existing row, an error occurs. Primary keys, unique keys, and foreign keys are indexes that have specific combinations of constraints. The sections in this lesson titled "Primary Keys," "Unique Key Constraints," and "Foreign Key Constraints" describe the constraints specific to these indexes and how you can include these types of indexes in a table.

Table Indexes

- An index is a collection of pointers to records in a table.
- It helps to:

- Locate rows quickly
- Avoid full table scans
- Improve query performance

- Unnecessary indexes are wasteful:

- Take up space in the database
- Updated whenever data is updated

Name_idx		Students		
Name		ID	Name	Grade
Cohen	→	101	Jones	C
Gibb	→	102	Smith	C-
Jones	→	103	Gibb	B+
Mars	→	104	Walsh	A
Smith	→	105	Cohen	A-
Walsh	→	106	Mars	D

By default, when MySQL attempts to locate a record, it scans the entire table until a match is found (full table scan). This can really slow down large queries.

An index helps MySQL find specific column values quickly. The index entries act as pointers to the rows that match these criteria so they can be accessed without a full table scan.

You can assign indexes when you create a table, or at any time afterward. You can create indexes on single columns or multiple columns (composite indexes). For example, you can use the phone number for an individual in a phone book as a single column index, or you can use the last name and first name to create a composite index.

You can index any table column, but do not overdo it. Indexes take up space. Too many indexes can slow performance because indexes are updated whenever you add or delete rows or modify data in indexed columns. Avoid indexing fields that change regularly. Selecting columns for indexing is covered in the MySQL Database Administrators course.

Note: A B-tree is a type of data structure used by DBMSs to store indexes. The data is kept sorted at all times, which makes for fast lookup of exact matches or comparisons of values.

Primary Keys

A primary key has the following constraints:

- Uniquely identifies a single row in the table
- Null values not allowed
- Only one primary key allowed per table

```
CREATE TABLE employees2
  (employee_id    INTEGER,
   first_name     VARCHAR(20),
   last_name      VARCHAR(25),
   email          VARCHAR(25),
   phone_number   VARCHAR(20),
   hire_date      DATE,
   job_id         VARCHAR(10),
   salary         DECIMAL(8,2),
   commission_pct DECIMAL(2,2),
   manager_id     INTEGER,
   department_id  INTEGER,
   PRIMARY KEY (employee_id));
```

The primary key is the identifying key for the table. Each row in the table must have a unique value for the primary key and it cannot be NULL. The primary key can be a composite key, which is covered elsewhere in this lesson. The InnoDB engine stores tables sorted by the primary key.

The example in this slide uses the `PRIMARY KEY` clause at the end of the statement. Another option is to use the `PRIMARY KEY` attribute in the column definition for `ID`, as shown in this example:

```
CREATE TABLE employees2
  (employee_id    INTEGER PRIMARY KEY,
   first_name     VARCHAR(20),
   last_name      VARCHAR(25),
   email          VARCHAR(25),
   phone_number   VARCHAR(20),
   hire_date      DATE,
   job_id         VARCHAR(10),
   salary         DECIMAL(8,2),
   commission_pct DECIMAL(2,2),
   manager_id     INTEGER,
   department_id  INTEGER);
```

Unique Key Constraints

Unique keys have the following constraints:

- The value must be unique for that column within the table.
- Null values are allowed for unique keys.
- Multiple unique keys are allowed per table.

```
CREATE TABLE employees3
  (employee_id    INTEGER,
   first_name     VARCHAR(20),
   last_name      VARCHAR(25),
   email          VARCHAR(25),
   phone_number   VARCHAR(20),
   hire_date      DATE,
   job_id         VARCHAR(10),
   salary         DECIMAL(8,2),
   commission_pct DECIMAL(2,2),
   manager_id     INTEGER,
   department_id  INTEGER,
   PRIMARY KEY (employee_id),
   UNIQUE KEY (email));
```

The unique key constraint requires that all values in the column are different. Primary keys are unique keys that have the additional constraint that null values are not allowed and only one primary key is allowed per table. You can specify unique keys that do not have the additional constraints of primary keys. In the example in this slide, each employee's email must be unique. The example in this slide uses `UNIQUE KEY` clauses at the end of the statement. Another option is to use `UNIQUE KEY` attributes in the column definitions, as shown in this example:

```
CREATE TABLE employees3
  (employee_id    INTEGER PRIMARY KEY,
   first_name     VARCHAR(20),
   last_name      VARCHAR(25),
   email          VARCHAR(25) UNIQUE KEY,
   phone_number   VARCHAR(20),
   hire_date      DATE,
   job_id         VARCHAR(10),
   salary         DECIMAL(8,2),
   commission_pct DECIMAL(2,2),
   manager_id     INTEGER,
   department_id  INTEGER);
```


Foreign Key Constraints

Foreign keys maintain referential integrity by enforcing a link between the data in two tables:

- The foreign key in the child table references the primary key in the parent table.
- The foreign key constraint prevents actions that would destroy the links between the child and parent tables.

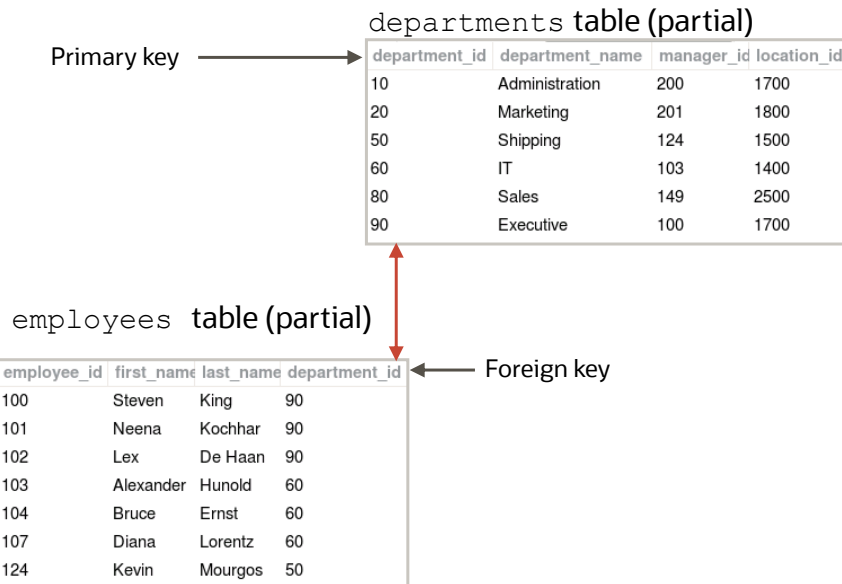
When you add a record to a table with a foreign key constraint, MySQL checks whether the value being added for the foreign key column has a matching value in the column referenced in the parent table and rejects the operation if no match is found. When you delete or update a record in a parent table, MySQL checks whether the value in the primary key column has any matching values in the foreign key column of the child table. Depending on how the foreign key is defined, the operation might be rejected, or the row might be deleted, updated, or set to a specified value.

For example:

- When inserting or updating a row in a child table, the value for the foreign key column being inserted or updated must already exist as the primary key for a row in the parent table. Otherwise, the insert or update is rejected.
- When deleting or updating a row in the parent table, the foreign key constraint determines what happens to any rows in the child table with a foreign key matching the column being deleted or updated.

The actions are described in the section titled "FOREIGN KEY Constraint: Referential Actions" in this lesson.

Foreign Key Constraint: Example Tables



The foreign key (or referential integrity) constraint designates a column or a combination of columns as a foreign key and establishes a relationship with a primary key or a unique key in the same table or a different table.

In the example in the slide, `department_id` has been defined as the foreign key in the `employees` table (child table); it references the `department_id` column of the `departments` table (the parent table). If you try to insert a row into the `employees` table or change `department_id` of a row in the `employees` table, the value provided for `department_id` must already be a primary key in the `departments` table. If you try to delete a row from the `departments` table or try to change the value for `department_id` in the `departments` table, it will first check whether there is a row in the `employees` table with that foreign key and take the action defined for the foreign key, described later in this lesson.

Guidelines

- A foreign key value must match an existing value in the parent table or be `NULL`.
- Foreign keys are based on data values and are purely logical, rather than physical, pointers.
- The primary key and foreign key might have the same column name (as in this example), but that is not required and often not appropriate.

FOREIGN KEY Constraint: Example Statement

```
CREATE TABLE employees4
  (employee_id    INTEGER,
   first_name     VARCHAR(20),
   last_name      VARCHAR(25),
   email          VARCHAR(25),
   phone_number   VARCHAR(20),
   hire_date      DATE,
   job_id         VARCHAR(10),
   salary         DECIMAL(8,2),
   commission_pct DECIMAL(2,2),
   manager_id     INTEGER,
   department_id  INTEGER,
   PRIMARY KEY (employee_id),
   UNIQUE KEY (email),
   CONSTRAINT emp4_dept_fk
     FOREIGN KEY (department_id)
       REFERENCES departments (department_id));
```

The example in the slide defines a FOREIGN KEY constraint on the department_id column of the employees table. This makes department_id in the employees4 child table a foreign key that must match a value for the primary key department_id in the departments table. The name of the constraint is emp4_dept_fk. The name must be unique for all tables in the database. If you do not provide a name for the foreign key constraint, MySQL will generate a name. The CONSTRAINT keyword is optional. If CONSTRAINT is omitted, the foreign key name must be omitted.

FOREIGN KEY Constraint: Referential Actions

- To control what happens to rows in a child table when a `DELETE` or `UPDATE` operation affects a key value in a parent table that has matching rows in the child table, you can include one or both of these optional subclauses at the end of the `FOREIGN KEY` clause:

```
[ON DELETE referential_action]
[ON UPDATE referential_action]
```

- The *referential_action* can be one of the following:
 - RESTRICT or NO ACTION
 - CASCADE
 - SET NULL

If `ON DELETE` or `ON UPDATE` is not included at the end of a `FOREIGN KEY` clause, the default is to reject a deletion or update on the parent table. This is the same as specifying `RESTRICT` or `NO ACTION`. Specifying `ON DELETE CASCADE` in the example with the `department_id` foreign key would mean that, if you delete a department from the `departments` table, all employees with that same `department_id` would be deleted. Specifying `ON UPDATE CASCADE` in the example would mean that, if you change the `department_id` in the `department` table, all employees with that `department_id` would be assigned the newly updated `department_id`. If you specify `ON DELETE SET NULL` or `ON UPDATE SET NULL`, be sure the column in the child table is not defined as `NOT NULL`.

The *referential_action* can be one of the following:

- RESTRICT or NO ACTION:** Rejects the `DELETE` or `UPDATE` operation on the parent table. It is the default action if `ON DELETE` or `ON UPDATE` is omitted.
- CASCADE:** Deletes or updates the row in the parent table and automatically deletes or updates the matching rows in the child table.
- SET NULL:** Deletes or updates the row in the parent table and sets the foreign key column or columns in the child table to `NULL`.

Secondary Indexes

A secondary index does not have any constraints and is used to speed up access to data. For example:

```
CREATE TABLE employees5
(employee_id    INTEGER,
 first_name     VARCHAR(20),
 last_name      VARCHAR(25),
 email          VARCHAR(25),
 phone_number   VARCHAR(20),
 hire_date      DATE,
 job_id         VARCHAR(10),
 salary         DECIMAL(8,2),
 commission_pct DECIMAL(2,2),
 manager_id     INTEGER,
 department_id  INTEGER,
 PRIMARY KEY (employee_id),
 UNIQUE KEY (email),
 INDEX emp5_name_ix (last_name),
 CONSTRAINT emp5_dept_fk
 FOREIGN KEY (department_id)
 REFERENCES departments (department_id));
```

A secondary index (or key) does not have to be unique or have any referential constraints. The index provides faster access to data. For example, if you know that many searches will be done on employees' last names, you can create an index for the `last_name` column. Last names do not have to be unique. Any kind of index, including primary key, unique key, foreign key, or secondary index, can be a composite key combining multiple columns to form the key value. For primary and unique keys, the individual parts of the composite key do not have to be unique, but the value of the combined columns must be unique.

Column Options

- You can add options to the `CREATE TABLE` statement's column definitions, including `NULL`, `NOT NULL`, `DEFAULT`, or `AUTO_INCREMENT`.
- Example:

```
CREATE TABLE employees6
  (employee_id    INTEGER AUTO_INCREMENT,
   first_name     VARCHAR(20),
   last_name      VARCHAR(25) NOT NULL,
   email          VARCHAR(25) NOT NULL,
   phone_number   VARCHAR(20),
   hire_date      DATE NOT NULL,
   job_id         VARCHAR(10) NOT NULL,
   salary         DECIMAL(8,2),
   commission_pct DECIMAL(2,2),
   manager_id     INTEGER DEFAULT 1,
   department_id  INTEGER DEFAULT 10,
   PRIMARY KEY (employee_id),
   UNIQUE KEY (email),
   INDEX emp6_name_ix (last_name),
   CONSTRAINT emp6_dept_fk
     FOREIGN KEY (department_id)
       REFERENCES departments
         (department_id));
```

The example in this slide creates a table similar to the `employees` table. You can include `NULL` as an option or omit it because it is the default and means that the column can accept `NULL` as a value. Specifying `NOT NULL` indicates that the column must have a value and cannot be set to `NULL`. The `DEFAULT` option specifies the default value to use when a row is inserted. To use the default value, if specifying the list of column names in the `INSERT` statement, omit the column name for the column to be set to its default and don't provide a value for it in the `VALUES` clause. Alternatively, you can use the `DEFAULT` keyword in the `VALUES` clause to set the column to its default value.

The `AUTO_INCREMENT` attribute can be set on an indexed column and sets the value for the column to the next higher value. When inserting a new value, either omit the value from the list of column names and `VALUES` clause or specify the value as `NULL`, and it will be assigned the next higher value.

The following statement would set the `employee_id` to the next higher integer value, set `manager_id` and `department_id` to their default values of 1 and 10, respectively, and set any columns not listed to `NULL`.

```
INSERT INTO employees6 (last_name, email, hire_date, job_id)
VALUES ('jones', 'jones', '2010-01-01', 'AC_MGR');
```

Creating a Table Using a Subquery

- Create a table and insert rows by combining the `CREATE TABLE` statement and the `AS subquery` option.

```
CREATE TABLE table  
  [(column, column...)]  
AS subquery;
```

- Match the number of specified columns to the number of subquery columns.
- Define columns with column names and default values.



A second method for creating a table is to apply the `AS subquery` clause, which both creates the table and inserts rows returned from the subquery.

In the syntax:

- **table:** The name of the table
- **column:** The name of the column, default value, and integrity constraint
- **subquery:** The `SELECT` statement that defines the set of rows to be inserted into the new table

Guidelines

- The table is created with the specified column names, and the rows retrieved by the `SELECT` statement are inserted into the table.
- The column definition can contain only the column name and default value.
- If column specifications are given, the number of columns must equal the number of columns in the subquery `SELECT` list.
- If no column specifications are given, the column names of the table are the same as the column names in the subquery.
- The column data type definitions and the `NOT NULL` constraint are passed to the new table. Note that only the explicit `NOT NULL` constraint will be inherited. The `PRIMARY KEY` column will not pass the `NOT NULL` feature to the new column. Any other constraint rules are not passed to the new table. However, you can add constraints in the column definition.

Creating a Table Using a Subquery: Example

```
CREATE TABLE dept80
AS
SELECT  employee_id, last_name,
        salary*12 ANNSAL,
        hire_date
FROM    employees
WHERE   department_id = 80;
```

```
DESCRIBE dept80;
```

#	Field	Type	Null	Key	Default	Extra
1	employee_id	int(11)	NO		NULL	
2	last_name	varchar(25)	NO		NULL	
3	ANNSAL	decimal(10,2)	YES		NULL	
4	hire_date	date	NO		NULL	

```
SELECT * FROM dept80;
```

#	employee_id	last_name	ANNSAL	hire_date
1	149	Zlotkey	126000.00	2016-01-29
2	174	Abel	132000.00	2012-05-11
3	176	Taylor	103200.00	2014-03-24

The example in the slide creates a table named `dept80`, which contains details of all the employees working in department 80. Notice that the data for the `dept80` table comes from the `employees` table. You can verify the existence of a database table and check the column definitions by using the `DESCRIBE` statement. The `employee_id` column is not set as the primary key, but the `NOT NULL` attributes are maintained. You can provide a column alias when using an expression in the subquery. In the example, the expression `salary*12` is given the alias `ANNSAL`.

ALTER TABLE Statement

Use the `ALTER TABLE` statement to:

- Add a new column
- Modify an existing column definition
- Define a default value for the new column
- Drop a column
- Rename a column
- Add an index or constraint



After you create a table, you may need to change the table structure for any of the following reasons:

- You omitted a column.
- Your column definition or its name needs to be changed.
- You need to remove columns.
- You want to put the table into read-only mode

You can do this by using the `ALTER TABLE` statement.

ALTER TABLE Statement: Add, Modify, or Drop Columns

Use the ALTER TABLE statement to add, modify, or drop columns:

```
ALTER TABLE table
ADD [COLUMN] column datatype [column_options]
    [FIRST | AFTER column_name];
```

```
ALTER TABLE table
MODIFY [COLUMN] column datatype [column_options]
    [FIRST | AFTER column_name];
```

```
ALTER TABLE table
DROP [COLUMN] column;
```

You can add columns to a table, modify columns, and drop columns from a table by using the ALTER TABLE statement.

Adding a Column

- You use the `ADD` clause to add columns:

```
ALTER TABLE dept80
ADD job_id VARCHAR(10) NOT NULL DEFAULT 'ST_CLERK'
AFTER last_name;
```

- The new `job_id` column is added after the `last_name` column, with the values set to `ST_CLERK`:

```
SELECT * FROM dept80;
```

#	employee_id	last_name	job_id	ANNSAL	hire_date
1	149	Zlotkey	ST_CLERK	126000.00	2016-01-29
2	174	Abel	ST_CLERK	132000.00	2012-05-11
3	176	Taylor	ST_CLERK	103200.00	2014-03-24



The example in the slide adds a column named `job_id` to the `dept80` table. Because this table already contains data, to set the `NOT NULL` option, you need to provide a default value, and all the rows are provided with that default value in that column. If the table is empty, you can specify the `NOT NULL` option without providing a default value. If you do not specify the `NOT NULL` or `DEFAULT` options, the values of the new column are set to `NULL`. In the example, the `job_id` column is inserted after the `last_name` column. If you specify `FIRST` rather than `AFTER`, the column is added before the `employee_id` column. If you do not specify either `FIRST` or `AFTER`, by default, the column is added at the end.

Modifying a Column

- You can change a column's data type, size, column options, or position.

```
ALTER TABLE dept80  
MODIFY last_name VARCHAR(30) NOT NULL FIRST;
```

```
DESCRIBE dept80;
```

#	Field	Type	Null	Key	Default	Extra
1	last_name	varchar(30)	NO		NULL	
2	employee_id	int(11)	NO		NULL	
3	job_id	varchar(10)	NO		ST_CLERK	
4	ANNSAL	decimal(10,2)	YES		NULL	
5	hire_date	date	NO		NULL	

- The size of the `last_name` column is modified, and it is moved to the first column.
- The full column definition must be included. If `NOT NULL` had been omitted, the column would have been changed from `NOT NULL` to accepting `NULL` values.
- If a default value is provided, it affects only subsequent insertions to the table.

You can modify a column definition by using the `ALTER TABLE` statement with the `MODIFY` clause. Column modification can include changes to a column's data type, size, or column options like `DEFAULT` and `NOT NULL`. You can also change the position of the column with `FIRST` or `AFTER`. The column remains in its previous position if you omit `FIRST` or `AFTER`.

Dropping a Column

Use the `DROP COLUMN` clause to drop columns that you no longer need from the table:

```
ALTER TABLE dept80  
DROP COLUMN job_id;
```

```
SELECT * FROM dept80;
```

#	last_name	employee_id	ANNSAL	hire_date
1	Zlotkey	149	126000.00	2016-01-29
2	Abel	174	132000.00	2012-05-11
3	Taylor	176	103200.00	2014-03-24



You can drop a column from a table by using the `ALTER TABLE` statement with the `DROP COLUMN` clause. If a table includes only one column, the column cannot be dropped. A MySQL extension to standard SQL permits a single `ALTER TABLE` statement to contain multiple `ADD`, `MODIFY`, or `DROP` clauses, separated by commas. For example, the following statement drops multiple columns:

```
ALTER TABLE dept80  
  
DROP COLUMN ANNSAL,  
  
DROP COLUMN hire_date;
```

ALTER TABLE Statement: Add an Index or Constraint

Normally, you create indexes and constraints by using the `CREATE TABLE` statement when you create the table. Use the `ALTER TABLE` statement to add an index or constraint:

```
ALTER TABLE table
ADD PRIMARY KEY (column_name);
```

```
ALTER TABLE table
ADD [UNIQUE] INDEX (column_name);
```

```
ALTER TABLE table
ADD CONSTRAINT key_name
FOREIGN KEY (child_column_name)
REFERENCES parent_table (parent_column_name);
```

By using an `ALTER TABLE` statement, you can add a primary key (only if the table does not yet have a primary key, a `UNIQUE` index, a secondary index, or a foreign key). If you are adding a primary key or unique index to a table that contains data, the statement is rejected if the values are not unique. If you add a foreign key to a table that already contains data, if the values in the foreign key column in the child table do not have corresponding values in the primary key of the parent table, then the statement is rejected.

ALTER TABLE to Add a Constraint or Index: Example

The following example adds the constraint of `manager_id` being a foreign key in the `employees5` table that references the `employee_id` in the same table.

```
ALTER TABLE employees6  
ADD CONSTRAINT emp6_manager_fk  
FOREIGN KEY (manager_id)  
REFERENCES employees6 (employee_id);
```

The statement in this slide creates a foreign key for the `manager_id` column in the `employees5` table that references the `employee_id` in the same table. Any `manager_id` must be the `employee_id` of some employee in the table.

For Instructor Use Only.
This document should not be distributed.

Creating Indexes by Using the CREATE INDEX Statement

- With the CREATE INDEX statement, you can add a unique key or a secondary index with no constraints to a table.
- Syntax:

```
CREATE [UNIQUE] INDEX index_name  
ON table_name (column_name);
```

- Example:

```
CREATE INDEX emp6_job_ix ON employees6 (job_id);
```

The example in this slide creates a secondary index for `job_id` in the `employees6` table. If you find that many queries use `job_id` as a search criteria, you might find it useful to add this index to speed up searches. Note that `job_id` in the `employees6` table cannot be a unique key because many people might have the same `job_id`. This statement is a shorter version of the following ALTER TABLE statement:

```
ALTER TABLE employees6  
ADD INDEX emp6_job_ix (job_id);
```


Viewing Index Definitions by Using the SHOW INDEX Statement

- General syntax for viewing index definitions:

```
SHOW INDEX FROM table_name [FROM database_name]
```

Example:

```
SHOW INDEX FROM employees6;
```

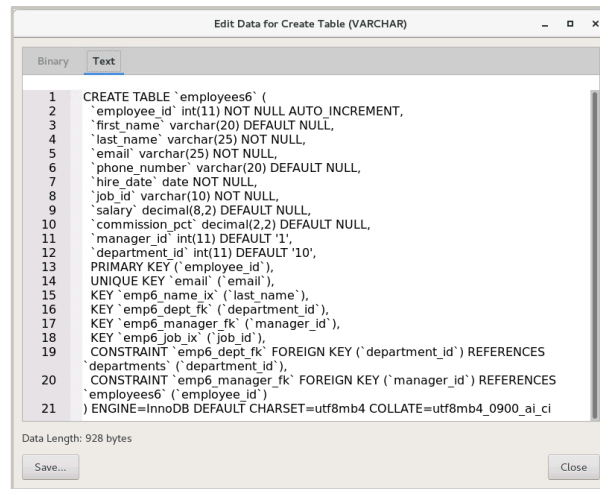
#	Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_commen	Visible
1	employees6	0	PRIMARY	1	employee_id	A	1				BTREE			YES
2	employees6	0	email	1	email	A	1				BTREE			YES
3	employees6	1	emp6_name_ix	1	last_name	A	1				BTREE			YES
4	employees6	1	emp6_dept_fk	1	department_id	A	1			YES	BTREE			YES
5	employees6	1	emp6_manager_fk	1	manager_id	A	1			YES	BTREE			YES
6	employees6	1	emp6_job_ix	1	job_id	A	1				BTREE			YES

If the database containing the table has been set as the default, you can omit the database name from the statement. In the output, the `Non_unique` column displays a 1 if the index does not have the unique constraint and 0 if it is unique. The output displays the name of the index as well as the columns being indexed. The primary key always has the name `PRIMARY`. The `Null` column indicates if the column can be set to `NULL`.

Showing How a Table Was Created with the SHOW CREATE TABLE Statement

View the statement that can be used to create the table. For example:

```
SHOW CREATE TABLE employees6;
```



The `SHOW CREATE TABLE` statement produces a statement that can create a table with the same properties as the table specified. This helps you to understand the structure of a table. You can use it to create a new table with the same structure. It does not show the exact statement that was used to create the table. It includes any modifications to the table from `ALTER` statements and might include more information, for example, about default values, than were specified in any `CREATE TABLE` or `ALTER TABLE` statements. For example, the output shows default values of `NULL` for some columns even though that was not specified in any statement. The `CREATE TABLE` statement also includes some table options at the end even though those were not specified when creating the table.

Note: The results of the statement in MySQL Workbench might be too long to read in the Result Grid. Right-click the output and select **Open Value in Viewer**. Click the **Text** tab. You can save the generated `CREATE TABLE` statement to a file. When you are finished, click the **Close** button.

Dropping a Table

- `DROP TABLE` removes the table definition and all its data entirely.
- Syntax:

```
DROP TABLE [IF EXISTS] table_name [, table_name] ...;
```

The optional `IF EXISTS` clause prevents an error from occurring if the table does not exist. If it is omitted, dropping a table that does not exist generates an error.

Example:

```
DROP TABLE IF EXISTS dept80;
```



You can drop multiple tables in one statement. For example:

```
DROP TABLE IF EXISTS
```

```
employees1, employees2, employees3, employees4, employees5;
```

Summary



In this appendix, you should have learned about the MySQL SQL Syntax.

In this appendix, you have learned to compare the MySQL SQL Syntax to the Oracle SQL Syntax.

For Instructor Use Only.
This document should not be distributed.

For Instructor Use Only.
This document should not be distributed.